

LakeCat

covers lakecat (0.1.0)

An Engine-Close Catalog Foundation for QueryGraph

Alexy Khrabrov

chiefscientist.org

&

Codex with ChatGPT 5.5

Contents

1 LakeCat	3
1.1 Preface	3
1.2 What A Catalog Is	3
1.3 What Iceberg Does	4
1.4 What LakeCat Adds Without Changing Iceberg	5
1.4.1 Standard, Extension, Or Proposal?	6
1.4.2 The Standards Boundary For The Current Release	7
1.4.3 A Claim-By-Claim Concept Ledger	11
1.4.4 Reading The Catalog Concepts Precisely	15
1.4.5 Why Push Work Into Sail	17
1.4.6 Extensions, Proposals, And Local Invention	19
1.4.7 A Detailed Catalog Concept Map	21
1.4.8 Ownership Map For The Catalog Stack	24
1.4.9 Why Work Moves Into Sail	26
1.4.10 From Catalog Concepts To User Workflows	29
1.4.11 A Catalog Concept Guide For LakeCat Readers	30
1.4.12 The Release Vocabulary In One Pass	33
1.4.13 Why Sail Is The Right Engine Boundary	37
1.4.14 A Release-Claim Ledger	39
1.4.15 Reading The Ledger Like A Standards Document	41
1.4.16 Standard Terms, LakeCat Terms, And Proposal Terms	42
1.4.17 The Current Pieces In Plain Terms	44
1.4.18 Four Workflows, One Catalog Boundary	46
1.4.19 Catalog Concepts As A Contract	48
1.4.20 Extensions, Proposals, And Product Boundaries	52
1.4.21 Why Sail Should Carry The Heavy Work	56
1.5 Standard Terms And LakeCat Terms	58
1.5.1 The Concept Status Matrix	62
1.5.2 How To Describe The Concepts To Different Readers	64
1.5.3 Why The Engine Should Carry The Heavy Work	65
1.6 What Exists Today And What It Means	67
1.6.1 Standard Word, LakeCat Mechanism, Future Candidate	73
1.7 A Field Guide To The Catalog Concepts	77
1.8 Reading The Current LakeCat Surface Precisely	80
1.8.1 A Claim-By-Claim Delineation	83
1.8.2 The Rust Service And Catalog Spine	84
1.8.3 Turso As The Durable Local Store	85
1.8.4 Iceberg REST-Compatible Paths	85
1.8.5 Commit CAS, Idempotency, Pointer Logs, Audit, Outbox, Replay	86
1.8.6 Governed Scans And Credential Decisions	87
1.8.7 QueryGraph And QGLake Handoff	87

1.9	Why The Work Belongs In The Engine	88
1.9.1	A PySpark Workflow	89
1.9.2	An Agentic Workflow	90
1.9.3	A QueryGraph Bootstrap Workflow	90
1.10	Extension Or Future Feature?	91
1.10.1	The Standards Position	91
1.10.2	Why The Engine Boundary Matters To Standards	92
1.10.3	A Classification Test For New Catalog Work	93
1.11	The Current Catalog State	94
1.12	What Intermediation Loses	95
1.13	Why Sail Matters	95
1.14	Why Move Catalog Work Closer To The Engine	96
1.15	Catalog Concepts In User Workflows	101
1.16	LakeCat's Thin Boundary	105
1.17	The Read Path	105
1.18	The Commit Path	106
1.19	The Durable Spine	110
1.20	Grust For Graph Concepts	111
1.21	TypeSec For Security	112
1.22	Rust-First Engines And The V3 To V4 Path	113
1.23	OSI, OpenLineage, And Responsible Semantic Handoff	115
1.24	QueryGraph.ai When LakeCat Is Done	116
1.25	Implementation Shape	117
1.26	Standard Compatibility And Extensions	118
1.27	Workflow Examples	119
1.27.1	Starting The Catalog	119
1.27.2	Registering The Warehouse Shape	120
1.27.3	Storage Profiles And Credential Roots	122
1.27.4	A PySpark User Reads Iceberg	127
1.27.5	A Notebook Requests Credentials	128
1.27.6	A View Becomes Part Of The Catalog Story	128
1.27.7	QueryGraph Bootstrap	134
1.27.8	Draining The Outbox	144
1.27.9	An Agentic QGLake Flow	155
1.28	Operating The Book's Example System	156
1.29	Catalog Concepts In Practice	156
1.29.1	A PySpark User Sees Iceberg	157
1.29.2	A Commit Is Standard, The Proof Envelope Is LakeCat	157
1.29.3	Turso Is A Store Choice, Not A Table Feature	159
1.29.4	A Governed Agent Sees Receipts And Bounded Work	159
1.29.5	Credentials Are Exceptions, Not The Default Agent Path	161
1.29.6	QueryGraph Reads The Proof, Not The Storage Layout	161
1.29.7	Why Sail Makes The Proof Credible	162

1.29.8	Extension Or Future Iceberg Proposal?	163
1.29.9	How To Name The New Pieces	164
1.29.10	The Engine-Push Argument	166
1.30	First Release Readiness	168
1.31	What Comes Next	169

1 LakeCat

1.1 Preface

LakeCat is a Rust-native Iceberg catalog foundation for QueryGraph. It starts from a deliberately conservative claim: the ordinary Iceberg REST catalog boundary must keep working for ordinary engines, but the next catalog also needs to become a governed control plane for Rust-first planning, semantic graph handoff, lineage, and agent access.

In this repository the catalog is LakeCat and the engine-side lakehouse implementation is Sail. The idea is not to invent a new table format or to make every client learn a new protocol. It is to keep Iceberg compatibility at the boundary while moving the parts that need deep Iceberg knowledge closer to the Rust engine that can reason about them.

This book starts from first principles. It explains what a catalog is, why Apache Iceberg puts so much responsibility into metadata, what Sail already does with Iceberg metadata, and why a passive catalog is not enough for governed agentic systems. Then it shows the LakeCat shape: a thin Rust catalog that owns identity, tenancy, metadata-pointer state, policy gates, idempotent commits, and integration events, while delegating reusable engine, graph, and security work to Sail, Grust, and TypeSec.

The intended end state is QueryGraph.ai. LakeCat should become the catalog foundation for the next QueryGraph: a place where standard engines still see an Iceberg REST catalog, while QueryGraph can bootstrap Croissant, CDIF, OSI, ODRL, OpenLineage, TypeSec security receipts, and a Grust-backed graph from the same governed source of truth.

1.2 What A Catalog Is

A data catalog is often described as a place that lists datasets. That is true, but too small. A real catalog is the control plane between names, storage, metadata, identity, and intent.

At minimum, a catalog answers four questions:

1. What table does this name mean?
2. Where is its current metadata?
3. Who is allowed to read, write, plan, or administer it?
4. What changed, when, and under whose authority?

In a traditional database, the catalog is embedded inside the database system. The engine owns the table definitions, statistics, indexes, permissions, and transaction log. A client asks the database a question and the same system resolves names, checks permissions, plans the query, and executes it.

A lakehouse splits that system apart. Data files live in object storage. Metadata files live beside them. Multiple engines may read and write the same tables. A catalog becomes the agreement point: it maps a logical table name to the current metadata pointer and arbitrates updates to that pointer.

That pointer is deceptively important. If the catalog points at metadata version 17, the table is version 17. If a writer prepares version 18 and wins the compare-and-swap update, the table becomes version 18. If it loses, the table does not partially change. The catalog is not the table format, but it is the place where table history becomes visible and durable.

For human-scale analytics this can sound like bookkeeping. For agentic systems it becomes a trust boundary. A catalog can know the principal, the warehouse, the namespace, the table, the current snapshot, the requested columns, the row restriction, the storage profile, and the policy receipt. If that information is captured before planning and committed after state changes, the catalog becomes the control plane for governed data access rather than a passive address book.

1.3 What Iceberg Does

Apache Iceberg is a table format for large analytic tables. Its core design is simple: put the table's truth in explicit metadata files, and let engines use that metadata to plan reads and validate writes without relying on directory listing or fragile storage conventions.

An Iceberg table has a current metadata file. That metadata names schemas, partition specs, sort orders, snapshots, properties, and the current snapshot. Snapshots point to manifest lists. Manifest lists point to manifests. Manifests describe data files and delete files. The data files normally use formats such as Parquet, Avro, or ORC.

This layered metadata gives Iceberg its practical power:

- Schema evolution is explicit.
- Partition evolution is explicit.
- Snapshot isolation is explicit.
- Commit conflicts can be checked before the current pointer advances.
- Scan planning can prune manifests and files before touching data.
- Deletes can be represented without rewriting every data file immediately.
- Multiple engines can interoperate because the table state is stored in a shared, documented format.

The catalog role in Iceberg is intentionally narrow. Standard clients need to load table metadata, create namespaces and tables, commit changes, and sometimes receive credentials or scan tasks. The catalog must not require business semantics or proprietary metadata for normal table access. If standard clients have to call a non-standard endpoint to read an ordinary table, compatibility is already broken.

LakeCat preserves that rule. Iceberg metadata stays pristine. Business semantics, policy, graph, lineage, and agent state are derived control-plane or graph data. The table remains an Iceberg table.

1.4 What LakeCat Adds Without Changing Iceberg

LakeCat’s central design promise is compatibility first, evidence second, and semantics above the catalog. That promise lets the system be ambitious without turning into a private table format. A Spark or PyIceberg client should see an Iceberg REST catalog. A QueryGraph or governed-agent client may ask for richer proof. Those two clients can share the same table because LakeCat keeps the portable table truth in Iceberg metadata and puts extra evidence beside it.

The most useful way to read LakeCat is to separate six categories.

Standard Iceberg parlance: These are the words that already belong to Iceberg: catalog, namespace, table identifier, current metadata location, table metadata, snapshot, manifest list, manifest, data file, delete file, schema evolution, partition evolution, optimistic commit, and REST catalog compatibility. LakeCat must implement these faithfully. If LakeCat changes the meaning of these words, it is no longer compatible.

LakeCat implementation: These are choices LakeCat makes to implement a strong catalog: the Rust service spine, the CatalogStore trait, the Turso-backed durable local store, normalized idempotency rows, pointer logs, audit rows, outbox rows, redaction rules, replay validators, and local release gates. These are not Iceberg extensions. They are how LakeCat makes ordinary Iceberg catalog behavior atomic, inspectable, and replayable.

LakeCat optional catalog extensions: These are additive APIs beside the standard REST path: management inventory, commit-history inspection, view proof, credential-root posture, replay verification, OpenLineage projection, and QueryGraph or QGLake bootstrap bundles. They should help operators, agents, and QueryGraph without becoming hidden requirements for standard table access.

TypeSec governance extensions: These are security and authorization concepts: capability decisions, authorization receipts, TypeDID context, ODRL-derived restrictions, secure-agent posture, credential TTL caps, raw-credential exception proof, and receipt evidence that a governed request was narrowed before planning. They are attached to catalog actions, but they do not become custom Iceberg metadata.

QueryGraph application extensions: These are semantic application concepts: Croissant, CDIF, OSI, ODRL application semantics, Grust-backed graph import, QGLake acceptance, agent workflow proof, and user-facing reasoning over catalog facts. QueryGraph consumes LakeCat; it should not be required by ordinary Iceberg clients.

Future Iceberg-adjacent candidates: These are ideas LakeCat can prove in practice before proposing as optional profiles: idempotent commit replay, pointer-history inspection, redacted conflict proof, transactional catalog event streams, OpenLineage or lineage receipt binding, governed credential-vending proof, proof-carrying scan planning, and view lifecycle proof. They should be additive profiles, not mandatory table-format changes.

This distinction answers the question “are these extensions or future proposed Iceberg features?” The answer is deliberately mixed. Rust and Turso are not extensions. CAS is standard catalog behavior, while LakeCat’s idempotency, audit, pointer-log, and replay evidence around it are

implementation and optional management proof. QueryGraph bootstrap is an optional integration extension. TypeSec receipts are governance extensions. Some proof shapes may eventually become Iceberg-adjacent proposals if multiple engines and catalogs need the same optional language.

1.4.1 Standard, Extension, Or Proposal?

LakeCat deliberately uses three different kinds of language. Keeping them separate is how the project can be compatible with Iceberg while still building a new catalog foundation for QueryGraph.

The first language is standard Iceberg parlance. These words already have a portable meaning in the Iceberg ecosystem: catalog, namespace, table identifier, current metadata location, table metadata, snapshot, manifest list, manifest, data file, delete file, schema evolution, partition evolution, sort order, optimistic commit, commit requirements, REST catalog routes, and credential vending as an optional catalog capability. LakeCat should implement these terms faithfully. A standard Iceberg client should not need to learn a LakeCat-specific concept before it can load or commit a normal table.

The second language is LakeCat implementation. These are the choices that make the compatible catalog reliable: the Rust service spine, the CatalogStore trait, Turso-backed local persistence, idempotency rows, pointer logs, audit rows, outbox rows, redaction, replay validation, and local release gates. These concepts are not Iceberg extensions by themselves. They are the internal discipline LakeCat uses to make the standard catalog behavior durable, inspectable, retry-safe, and fit for QueryGraph handoff.

The third language is optional proof and integration. This includes governed scan proof, credential proof, OpenLineage receipt binding, management inventory, view lifecycle proof, QueryGraph bootstrap, QGLake handoff, Croissant, CDIF, OSI, ODRL, TypeDID, Grust graph import, and TypeSec authorization receipts. These are not normal Iceberg table semantics. They are additive surfaces for operators, governed agents, and QueryGraph. They must sit beside the standard path, not in front of it.

That separation gives a practical answer for every feature request:

Question	If yes, the feature is probably...
Must a Spark, PySpark, Flink, Trino, DuckDB, PyIceberg, or Sail client understand it to load or commit an ordinary table?	Standard Iceberg behavior or a compatibility risk.
Is it about the language, process model, database, rows, hashes, replay checks, or local release proof LakeCat uses internally?	LakeCat implementation, not an Iceberg extension.
Is it about policy, capability, TypeDID identity, ODRL restrictions, or secure-agent authorization?	TypeSec governance surface.
Is it about graph taxonomy, graph storage, projection, traversal, or Cypher?	Grust-owned graph surface.
Is it about Croissant, CDIF, OSI, OpenLineage composition, QGLake acceptance, or agent reasoning?	QueryGraph integration surface.
Could many independent catalogs and engines benefit from the same narrow behavior without adopting LakeCat, QueryGraph, TypeSec, Grust, or Turso?	Future Iceberg-adjacent optional profile candidate.

The last row is the standards filter. “LakeCat uses Rust” should not become an Iceberg proposal. “LakeCat uses Turso” should not become an Iceberg proposal. “QueryGraph imports QGLake proof” should not become a requirement for ordinary Iceberg clients. But “catalogs can express exact idempotent retry,” “catalogs can expose pointer-history proof,” “catalog events can have stable replay identity,” “credential vending can carry redacted authorization evidence,” and “scan planning can carry proof of the governed restriction” are small enough to discuss as optional profiles.

This is why LakeCat can be ambitious without forking the table format. Standard clients get standard Iceberg behavior. Operators get durable audit and replay. Governed agents get TypeSec-backed restriction evidence and Sail-planned work. QueryGraph gets proof-bearing catalog anchors for graph, lineage, and semantic handoff. Iceberg remains the portable table contract underneath all of it.

1.4.2 The Standards Boundary For The Current Release

LakeCat should be explicit about which ideas are standard Iceberg, which ideas are LakeCat implementation, which ideas are QueryGraph or TypeSec extensions, and which ideas might eventually be worth proposing back to the Iceberg ecosystem. Without that vocabulary, every strong catalog feature sounds like a table-format change. That would be the wrong message. LakeCat is strongest when ordinary clients see ordinary Iceberg and advanced clients can ask for additional proof beside the standard path.

The Rust service and catalog spine exists, but Rust is not Iceberg parlance. Iceberg specifies catalog behavior: namespace and table resolution, current metadata-location reads, optimistic metadata-pointer commits, and compatibility with the shared table metadata format. It does not prescribe a language, runtime, or process shape. LakeCat chooses Rust because the catalog transaction is dense: one request can carry REST routing, principal identity, warehouse and tenant scope, an expected metadata pointer, an idempotency key, a Sail validation or planning call, TypeSec receipt evidence, audit rows, pointer-log rows, outbox rows, redaction, and replay admission. The portable lesson is not that Iceberg catalogs should be Rust. The portable lesson is that catalogs should be able to prove what action they accepted, rejected, replayed, vended, or emitted.

The Turso-backed local store direction is also LakeCat implementation, not an Iceberg feature. Iceberg needs durable catalog state and atomic movement of the current metadata pointer. LakeCat uses the Rust turso crate as the local durable CatalogStore because it keeps the embedded development and test path Rust-native while still exercising real persistence, transactions, idempotency, and row/content validation. Turso stores projects, warehouses, namespaces, tables, views, storage profiles, policy bindings, idempotency records, pointer logs, audit rows, outbox rows, and soft-delete state. Turso itself should not be an Iceberg proposal. The possible proposal material is narrower: atomic pointer CAS, exact idempotent replay, redacted conflict evidence, pointer history, audit receipts, and transactional catalog event streams.

Iceberg REST-compatible table and namespace paths are standard Iceberg parlance. These are the compatibility floor. PySpark, Spark, Flink, Trino, DuckDB, PyIceberg, and Sail should be able to create namespaces, load tables, commit metadata updates, and discover current table state through the standard catalog surface without learning QueryGraph, QGLake, TypeSec, Grust, Croissant, CDIF, OSI, ODRL, or OpenLineage. LakeCat may record evidence behind those routes and may expose optional management or bootstrap routes beside them, but ordinary table access must not depend on those optional routes. A standard client should benefit from stronger catalog reliability without changing its mental model of Iceberg.

Commit compare-and-swap is standard Iceberg catalog behavior. A writer prepares new metadata and asks the catalog to advance the current metadata location only if the expected requirements still hold. LakeCat does not replace that optimistic commit model. It hardens the envelope around it. Create-only object writes avoid accidental metadata overwrite. Store-level CAS makes pointer movement atomic. Idempotency rows make retries exact or conflicting. Pointer logs record accepted movement. Audit rows record authority. Outbox rows bind graph and lineage delivery to committed catalog state. Replay validation refuses malformed durable evidence before acknowledgement, Grust projection, OpenLineage projection, or QGLake import. The standard term is optimistic commit. The LakeCat terms are idempotency, pointer log, audit, outbox, redaction, and replay validation. The possible future profile is an optional catalog behavior profile for retry semantics, pointer history, conflict proof, and event identity.

Governed scan and credential paths are LakeCat/TypeSec governance extensions around ordinary Iceberg tables. Iceberg metadata already gives engines the information needed for scan planning: schemas, field ids, partition specs, snapshots, manifest lists, manifests, metrics, data files, delete files, and sequence information. LakeCat adds the control-plane prelude. It identifies the principal

and purpose, asks TypeSec for a capability decision, derives allowed columns, mandatory predicates, policy hashes, credential TTL caps, and raw credential posture, and then asks Sail to plan or validate the effective request. Raw object-store credentials are deliberate audited exceptions for trusted principals. Restricted agents should receive governed Sail-planned work instead of broad storage authority. This should remain table-format neutral. If it becomes a wider Iceberg conversation, it should be phrased as optional proof-carrying scan planning or governed credential vending, not as a required TypeSec dependency or custom Iceberg metadata field.

QueryGraph and QGLake handoff, OpenLineage, bootstrap, management, view, credential, and commit proof surfaces are broad because QueryGraph is a richer semantic application layer. They are not standard Iceberg table semantics. LakeCat exports governed catalog truth: warehouses, namespaces, tables, views, current metadata pointers, pointer transitions, view receipt chains, storage profile and policy anchors, management inventory, credential posture, governed scan proof, replay-validation hashes, OpenLineage receipt hashes, and graph anchors. QueryGraph composes those anchors with Croissant, CDIF, OSI, ODRL application meaning, Grust graph import, QGLake acceptance, and agent workflows. The reusable standard candidates are smaller than QueryGraph: catalog event identity, lineage receipt binding, pointer-history inspection, view lifecycle proof, and governed credential proof.

The release vocabulary therefore looks like this:

	Concept	Standard Iceberg?	LakeCat/Query-Graph/TypeSec role	Proposal posture
Rust	service/catalog spine	No. Iceberg specifies behavior, not implementation language.	LakeCat implementation path for REST routing, identity, tenancy, Sail calls, TypeSec receipts, CAS, audit, outbox, and replay admission.	Not a proposal. Deterministic proof behavior may generalize; Rust itself should not.
Turso-backed	local store	No. Iceberg needs durable catalog state and atomic pointer movement, not a specific database.	LakeCat local CatalogStore implementation for catalog state, idempotency, pointer logs, audit, outbox, policy bindings, views, and storage profiles.	Not a database proposal. CAS, idempotency, pointer history, and event behavior may generalize.
Namespaces/table REST paths		Yes. They are the compatibility surface.	LakeCat serves standard Iceberg access under /catalog/v1 and records optional evidence behind it.	Follow Iceberg. Keep management and QueryGraph paths additive.
Commit CAS		Yes. Optimistic metadata-pointer movement is standard catalog behavior.	LakeCat hardens it with create-only writes, exact idempotency, pointer logs, audit, transactional outbox, redaction, and replay validation.	CAS is standard; retry, conflict-proof, pointer-history, and event envelopes may become optional profiles.
Governed scan receipts		No, beyond the metadata engines use to plan scans.	TypeSec decides the effective restriction, LakeCat binds the receipt to the action, and Sail plans the narrowed request.	Candidate optional proof-carrying scan or governed-access profile.
Credential proof		Credential vending exists in catalog ecosystems, but LakeCat's governance proof is additive.	Raw credentials are audited exceptions; restricted principals receive Sail-planned work and receipt evidence.	Candidate optional governed credential-vending profile.
OpenTelemetry/GDPR/OSI, QueryGraph, TypeSec, Audit, Outbox, Replay		No, as QueryEngine being an application and integration layer. ¹⁰	LakeCat can pierce requests through existing distributed systems like Outbox, log identities, lineage graphs, commit policies, and other parts of the catalog.	Deterministic Iceberg plan goals that include security in QueryGraph (e.g., fully auditable above Iceberg).

The important rule is that the standard column in this table is not a judgment about importance. A concept can be non-standard and still be essential to LakeCat. Rust, Turso, TypeSec, Grust, QueryGraph, QGLake, Croissant, CDIF, OSI, ODRL, TypeDID, and OpenLineage are all important to this system, but they are not the portable table contract. They live in implementation, governance, lineage, graph, and application layers around the table. Iceberg’s shared contract stays smaller: table metadata, current pointers, snapshots, manifests, delete files, schemas, partition specs, commit requirements, and catalog routes.

That smaller contract is a strength. It is why multiple engines can share the same table. LakeCat should not weaken that by stuffing business semantics into Iceberg metadata or by making ordinary clients depend on QueryGraph-only routes. The right pattern is additive: ordinary clients use the ordinary catalog, while advanced clients ask for proof, governance, lineage, bootstrap, and graph material beside the standard path.

This also explains which LakeCat ideas are plausible future Iceberg proposals. An implementation choice is not a proposal. “Use Rust” and “use Turso” should not be proposed to Iceberg. An application dependency is not a proposal. “Require QueryGraph” or “require TypeSec” would make Iceberg less portable. The proposal candidates are behavior profiles that many catalogs and engines might want regardless of their implementation: exact idempotent retry, redacted conflict evidence, pointer-history inspection, transactionally emitted catalog events, lineage receipt binding, governed credential-vending proof, and proof-carrying scan planning. Those ideas are narrow enough to improve interoperability without turning LakeCat’s whole stack into a standard.

In practice, the difference looks like this. A PySpark writer sees an Iceberg REST catalog and commits table metadata through normal optimistic commit rules. LakeCat may record the idempotency row, pointer log, audit row, and outbox row, but PySpark does not need to understand those rows. A governed agent sees a different surface: it asks for access, receives a TypeSec-backed decision, and gets Sail-planned bounded work rather than raw storage authority. QueryGraph sees another surface again: it asks LakeCat for proof-bearing catalog anchors that can be imported into Grust-backed graph state and correlated with OpenLineage, Croissant, CDIF, OSI, ODRL, and TypeSec evidence. All three workflows can refer to the same Iceberg table because the base table semantics remain ordinary.

This matrix protects the compatibility story. LakeCat should prove optional catalog behavior without making optional behavior mandatory for ordinary Iceberg clients. Standard clients get standard Iceberg. Operators get durable replay and audit. Governed agents get restricted access proof. QueryGraph gets semantic bootstrap material tied to catalog truth. Future standards work gets small optional profiles rather than a demand to adopt LakeCat, QueryGraph, TypeSec, Grust, or Turso.

1.4.3 A Claim-By-Claim Concept Ledger

The fastest way to avoid confusion is to classify each LakeCat claim by the layer that should own it. The catalog is allowed to be innovative, but the innovation has to be visible as catalog proof or an optional integration surface. It should not silently change what an Iceberg table means.

Rust service/catalog spine exists. This is LakeCat implementation, not Iceberg terminology. Iceberg speaks in catalog behavior: namespaces, table identifiers, table metadata locations, snapshots, manifests, delete files, and optimistic commits. LakeCat speaks in service shape: Axum routes, typed request identity, warehouse routing, store transactions, Sail planning calls, TypeSec decisions, redaction, audit, outbox, and replay validation. The reason Rust matters is that those operations are systems work on the hot path. A catalog request can cross identity, policy, metadata, durable storage, engine planning, and proof emission in one transaction. Rust gives LakeCat a small, explicit, strongly typed spine for that work. That is an implementation advantage, not a standards claim.

Turso-backed local store direction is in place. This is also LakeCat implementation. The standard need is durable catalog state: current table pointers, atomic pointer movement, namespace and table records, and retry behavior that does not corrupt the table. LakeCat’s Turso-backed store is a concrete Rust-native way to exercise that need locally with real persistence, transactions, idempotency records, pointer logs, audit rows, outbox rows, view receipts, policy bindings, storage profiles, and row/content validation. Another store can implement the same CatalogStore contract. Turso should not become an Iceberg proposal. The portable ideas are the invariants the store proves: exact retry, atomic pointer CAS, pointer history, and replayable event identity.

Iceberg REST-compatible table and namespace paths exist. This is standard Iceberg compatibility. A normal client should be able to use standard configuration discovery, namespace routes, table load, table create, and table commit without knowing QueryGraph, TypeSec, Grust, QGLake, Croissant, CDIF, OSI, ODRL, or LakeCat’s internal store. LakeCat may enrich those calls with audit, outbox, pointer-log, and replay evidence, but that evidence lives behind or beside the standard route. If a standard table read requires a QueryGraph-only endpoint, LakeCat has made a compatibility mistake.

Commit CAS, idempotency, pointer logs, audit/outbox, and replay validation are heavily hardened. Commit CAS is the standard Iceberg part: a writer advances the current metadata pointer only if the expected table requirements still hold. The rest is LakeCat’s proof envelope around that standard operation. Idempotency makes client retry exact. Pointer logs make metadata movement inspectable. Audit rows bind accepted actions to principals and receipts. Outbox rows make graph and lineage projection recoverable from committed state. Replay validation rejects malformed durable evidence before it reaches Grust, OpenLineage, or QGLake. The possible Iceberg-adjacent future here is not “adopt LakeCat’s tables.” It is a narrow optional profile for exact retry, redacted conflict proof, pointer history, and catalog event identity.

Governed scan and credential paths have substantial TypeSec-style receipt evidence. This is a governance extension around standard Iceberg tables. The underlying table metadata still belongs to Iceberg: schemas, field ids, partition specs, snapshots, manifest lists, manifests, file metrics, delete files, and sequence information. TypeSec contributes the authorization meaning: principal, purpose, capability, policy hash, allowed columns, mandatory row predicate, credential TTL cap, raw-credential posture, and receipt proof. LakeCat binds that decision to the catalog request and asks Sail to plan or validate the effective work. A future optional Iceberg profile could

describe proof-carrying scan planning or governed credential vending, but it should not require TypeSec or encode QueryGraph semantics into table metadata.

QueryGraph/QGLake handoff, OpenLineage, bootstrap, management, view, credential, and commit proof surfaces are broad. This breadth is intentional because QueryGraph is an application layer, not a standard Iceberg client. QueryGraph wants evidence about warehouses, namespaces, tables, views, storage profiles, policy bindings, credential decisions, governed scans, pointer transitions, OpenLineage receipt hashes, graph anchors, view receipt chains, and accepted bootstrap artifacts. LakeCat should expose those surfaces as additive proof. QueryGraph should then compose them with Croissant, CDIF, OSI, ODRL, TypeDID, secure-agent workflows, and Grust-backed graph import. The standard-candidate pieces are much smaller: catalog event streams, lineage receipt binding, view lifecycle proof, and commit-history inspection.

That classification gives LakeCat a clean standards posture:

Claim	Iceberg parlance	LakeCat/Query- Graph/TypeSec clas- sification	Future proposal shape
Rust service spine	Not standard terminology.	LakeCat implementation.	None, except deterministic proof behavior learned from it.
Turso local store	Not standard terminology.	LakeCat CatalogStore implementation.	None, except durable CAS/idempotency/event invariants.
REST namespaces and tables	Standard catalog compatibility.	LakeCat must preserve it exactly for ordinary clients.	Follow Iceberg REST; keep extra routes additive.
Commit CAS	Standard optimistic catalog behavior.	LakeCat adds idempotency, pointer log, audit, outbox, and replay proof.	Optional retry, pointer-history, conflict-proof, and event profiles.
Governed scans	Standard metadata underneath, additive proof above.	TypeSec decides, LakeCat binds, Sail plans.	Optional proof-carrying scan profile.
Credential governance	Credential vending is catalog-adjacent; proof is additive.	TypeSec-gated exception path, with Sail-planned reads as the default for restricted principals.	Optional governed credential-vending profile.
QueryGraph/ QGLake handoff	Not standard table access.	Application integration and acceptance proof.	Only narrow proof shapes should generalize.
OpenLineage projection	Not Iceberg metadata.	LakeCat outbox projection and QueryGraph correlation surface.	Optional lineage receipt binding.
Croissant, CDIF, OSI, ODRL, TypeDID	Not Iceberg parlance.	QueryGraph and TypeSec semantic/security vocabularies.	Usually external standards; LakeCat may bind receipt hashes.

The difference between an extension and a future proposal is interoperability pressure. A LakeCat extension helps LakeCat, QueryGraph, TypeSec, or local operators. A future Iceberg proposal should help more than LakeCat: it should be small enough for independent engines and catalogs

to adopt without adopting QueryGraph, TypeSec, Grust, Turso, or LakeCat’s service code. That is why proof-carrying scan planning is plausible standards material, while “QGLake handoff” is not. The first describes a portable behavior. The second is a specific application workflow that can consume the behavior.

This is also why pushing work into Sail matters for standards posture. If a proof profile says a scan was narrowed, the narrowing should be expressed in terms an engine understands: field ids, effective projection, residual predicate, snapshot, manifest pruning, delete handling, and scan tasks. If LakeCat computes those facts with catalog-local JSON shortcuts, it proves a catalog approximation. If Sail computes them, LakeCat can prove that a real engine path interpreted the Iceberg table. That makes the proof credible for QueryGraph today and portable enough to discuss with other engines tomorrow.

1.4.4 Reading The Catalog Concepts Precisely

The catalog vocabulary can be confusing because the same request crosses several layers. A single table commit may be a standard Iceberg optimistic commit, a LakeCat Rust service transaction, a Turso row update, a TypeSec authorization receipt, a pointer-log entry, an audit row, an outbox event, an OpenLineage projection, a Grust graph fact, and a QueryGraph handoff proof. The important thing is not to collapse those into one word. The standard table contract is only the Iceberg part. The rest is the proof system LakeCat builds around that contract.

When this book says the Rust service/catalog spine exists, it means LakeCat has a concrete service boundary where Iceberg REST routes, management routes, governed scan routes, credential routes, QueryGraph bootstrap routes, request identity, authorization receipts, catalog store calls, Sail calls, audit recording, outbox recording, replay validation, and redaction meet. That is a LakeCat implementation fact. It is not standard Iceberg parlance. Iceberg does not care whether that service is Rust, Java, Go, Python, or a managed cloud control plane. Iceberg cares that a client can resolve a namespace, load a table, read the current metadata location, and commit a new metadata pointer under the required optimistic rules. LakeCat chooses Rust because the boundary is a high-density systems boundary: every request has to preserve type shape, identity shape, hash shape, failure shape, and replay shape without adding a large dynamic-language gap between the catalog decision and the durable row.

When this book says the Turso-backed local store direction is in place, it means the durable local catalog spine is moving through the Rust turso crate behind the CatalogStore trait. Turso is not an Iceberg feature. It is a pragmatic LakeCat store implementation for local development, embedded deployments, fixture-backed verification, and eventually small production profiles that want a Rust-native durable spine. The useful portability lesson is the store contract, not the database brand: catalog state needs atomic metadata-pointer updates, exact idempotency records, pointer logs, audit rows, outbox rows, soft-delete state, policy bindings, storage-profile anchors, and content validation. Another deployment could use a different durable store and still be LakeCat-compatible if it honors the same store contract and replay invariants.

When this book says Iceberg REST-compatible table and namespace paths exist, it is speaking standard Iceberg language. Namespace create, list, load, and drop operations, table create, load,

commit, delete, and restore-style lifecycle operations, configuration discovery, and warehouse-prefixed catalog routing must behave like Iceberg catalog access. LakeCat can record extra evidence behind those routes, but it should not require a PySpark, Flink, Trino, DuckDB, PyIceberg, or Sail client to call a QueryGraph-only route to read an ordinary table. The optional LakeCat evidence is valuable precisely because it is optional. A normal Iceberg client should experience a better catalog, not a new table format by surprise.

When this book says commit CAS, idempotency, pointer logs, audit/outbox, and replay validation are heavily hardened, it is separating the standard operation from LakeCat’s proof envelope. The standard operation is optimistic commit: the writer proposes a new metadata file and the catalog advances the current metadata pointer only if the table requirements still hold. LakeCat surrounds that with exact retry behavior, create-only metadata writes, request and response hashes, idempotency-key hashes, positive sequence evidence, positive format-version evidence, non-negative snapshot evidence, pointer-log records, principal-aware audit events, transactional outbox events, and replay admission that refuses malformed durable evidence before graph or OpenLineage delivery. Individual table.commit replay also rejects conflicting payload-level warehouse, namespace, or table hints before projection, so pointer-movement proof cannot carry a second contradictory scope beside the durable table identity. That scope binding is LakeCat proof hardening around a standard Iceberg commit, not a change to Iceberg metadata.

When this book says governed scan and credential paths have substantial TypeSec-style receipt evidence, it means LakeCat records why a principal was allowed to perform a governed access path and what restrictions were applied. The standard Iceberg table metadata already contains the information an engine needs to plan scans: schemas, field ids, partition specs, sort orders, snapshots, manifest lists, manifests, data-file metrics, delete-file metadata, and sequence information. TypeSec adds governance meaning: who the principal is, what capability was requested, which policy or ODRL restriction applied, which columns are allowed, what row predicate is mandatory, what credential TTL cap applies, and whether raw credentials are permitted. LakeCat binds that receipt to the catalog action and asks Sail to plan or fetch the narrowed work. The portable future-candidate idea is not “make Iceberg depend on TypeSec.” It is an optional proof-carrying scan or governed credential-vending profile that many engines could understand even if they use a different policy engine.

When this book says QueryGraph/QGLake handoff, OpenLineage, bootstrap, management, view, credential, and commit proof surfaces are broad, it means LakeCat has become a serious source of control-plane evidence for a semantic application. QueryGraph wants more than a current metadata pointer. It wants to know which warehouses, namespaces, tables, views, storage profiles, policy bindings, credential decisions, scan plans, view receipt chains, pointer-log commits, OpenLineage receipts, and graph anchors were accepted. That breadth is appropriate for QueryGraph, but it is not standard Iceberg table semantics. LakeCat should expose this proof as additive management, lineage, replay, and bootstrap surfaces. QueryGraph should then import it into Grust-backed graph state and combine it with Croissant, CDIF, OSI, ODRL, TypeDID, and agent workflow semantics. The Iceberg table remains an Iceberg table underneath.

The extension question should therefore be answered feature by feature. Rust service structure and Turso storage are implementation choices, not Iceberg extensions. Standard REST namespace/table paths and optimistic commit CAS are Iceberg compatibility work. LakeCat idempotency, pointer-log inspection, audit replay, transactional outbox, replay validation, and management inventory are catalog extensions beside the standard path. TypeSec receipts, ODRL-derived restrictions, TypeDID envelopes, secure-agent posture, and credential proof are governance extensions. QueryGraph bootstrap, QGLake acceptance, Croissant, CDIF, OSI, graph import, and application reasoning are integration and application extensions. The pieces worth considering as future Iceberg-adjacent proposals are the narrow portable behaviors: exact idempotent commit replay, pointer-history inspection, redacted conflict proof, transactional catalog event streams, lineage receipt binding, proof-carrying scan planning, governed credential-vending proof, and view lifecycle proof.

1.4.5 Why Push Work Into Sail

The strongest LakeCat design choice is restraint. A catalog should know the current table pointer and the authority under which that pointer moved. It should not become a second Iceberg engine. Iceberg table metadata is rich: schemas use stable field ids, partition specs evolve, sort orders evolve, snapshots form a timeline, manifest lists summarize manifests, manifests carry data-file and delete-file entries, metrics enable pruning, delete files change row visibility, and future v4 work will add more typed behavior. If LakeCat reimplements that logic locally, it risks becoming a shadow engine that is almost correct, hard to optimize, and hard to keep compatible.

The boundary is not “catalog versus engine” as a turf question. It is a proof question. A governed catalog can only prove a safe read if the allowed projection, predicate, snapshot, delete posture, and task set were interpreted by the same kind of logic that execution will use. If LakeCat does field-id matching, manifest-metric decoding, delete association, or v4 metadata interpretation with catalog-local shortcuts, the proof can drift from the plan. The catalog would still produce hashes and receipts, but those receipts would describe a catalog approximation rather than the table work an engine actually performs.

Sail is the right place for that work because Sail is an engine. It can parse and validate Iceberg metadata with execution in mind. It can understand field ids, schema projection, partition transforms, manifest metrics, delete files, scan-task construction, residual filters, metadata tables, and table-format version differences as part of the same planning surface that eventually reads data. It can expose generated Iceberg REST models and typed helpers that LakeCat calls, instead of forcing LakeCat to hand-parse table metadata JSON. That keeps one Rust implementation responsible for reusable Iceberg semantics and lets multiple callers benefit from the same engine work.

The handoff between LakeCat and Sail should therefore be compact and typed:

Work item	LakeCat responsibility	Sail responsibility	Durable proof LakeCat should keep
Table load	Resolve warehouse, namespace, table, principal, and current metadata pointer.	Interpret table metadata and expose table status using reusable Iceberg types.	Table identity, pointer hash, format version, principal, and receipt/action context.
Governed scan	Ask TypeSec for the effective restriction and bind it to the request.	Bind projection and predicates to field ids, prune manifests and files, attach deletes, and produce scan tasks.	Requested and effective projection hashes, predicate hash, snapshot, task count, delete posture, plan hash, and receipt hash.
Fetch scan task	Check that a stateless task fetch belongs to the prior governed plan.	Reapply task interpretation without widening the plan.	Prior plan hash, task token hash, effective stats-field evidence, and receipt hash.
Commit	Own CAS, idempotency, pointer-log, audit, and outbox state.	Validate table metadata, commit requirements, format-version behavior, and future v4 typed semantics.	Expected and new pointer hashes, format version, snapshot evidence, idempotency hash, audit id, and outbox id.
Metadata-as-data	Authorize the request and record who asked.	Expose snapshots, manifests, files, deletes, partitions, and history as engine-shaped metadata views.	Metadata view name, pointer hash, result-shape hash, principal, and receipt hash.

This table is why Sail is a great engine choice for LakeCat rather than merely an implementation convenience. Sail is Rust-native, close to Arrow and DataFusion, close to Iceberg REST model generation, close to table-provider construction, and close to execution planning. A manifest-pruning improvement, delete-file fix, metadata-as-data table, or typed v4 implementation in Sail can serve LakeCat, QueryGraph workflows, and direct Rust lakehouse users at the same time. The catalog gets faster and safer because it calls the shared engine truth instead of maintaining a private planner.

This is especially important for governed reads. A TypeSec decision may say that an agent can read `event_id` and `event_ts` for rows matching a mandatory predicate. LakeCat should bind that

decision to the request and persist the receipt. Sail should decide how that restriction maps to Iceberg schema field ids, manifest pruning, file pruning, delete-file handling, residual filters, and scan tasks. LakeCat should then persist compact evidence that Sail planned the narrowed request: requested projection, effective projection, requested stats fields, effective stats fields, required filters, task counts, hash receipts, and replay anchors. The division is clean: TypeSec decides permission, Sail plans table work, LakeCat records catalog authority and proof.

It is equally important for commits. LakeCat owns the CAS transaction, idempotency, pointer log, audit row, and outbox event. Sail should own reusable commit preparation and validation: interpreting requirements, validating metadata shape, understanding format-version behavior, checking schema and partition evolution, and eventually handling typed v4 semantics. LakeCat can then ask Sail for a validation result and persist a small proof envelope. That is faster, clearer, and safer than having LakeCat grow one-off JSON logic for every Iceberg version.

The same rule applies to metadata-as-data. Operators and QueryGraph want to ask questions about snapshots, manifests, files, delete files, metrics, partitions, and lineage. Those are not mere catalog rows. They are engine-shaped facts derived from Iceberg metadata. Sail can expose them as typed metadata tables or planning outputs. LakeCat can decide who may ask, bind the request to TypeSec receipt evidence, and record the proof that a particular metadata view was served. QueryGraph can import the result through Grust. The catalog stays thin, but the system gets richer.

Sail is also a good performance choice. It keeps table-format parsing, planning, pruning, and scan-task generation close to the Rust execution engine instead of bouncing through remote catalog plugins or language boundaries. A LakeCat request can stay mostly in Rust from REST route to store transaction to Sail planning to replay proof. That matters for agentic systems because the catalog will be hit frequently for small, proof-heavy operations: load this table, plan this restricted scan, fetch this bounded task set, prove this credential decision, list this pointer history, bootstrap this graph. The less work LakeCat duplicates and the more it can call typed Sail APIs, the smaller and faster the trusted catalog boundary becomes.

The design rule is therefore simple. If the work requires interpreting Iceberg table-format semantics, push it into Sail. If it requires graph taxonomy, projection, traversal, or graph query behavior, push it into Grust. If it requires authorization semantics, capability composition, TypeDID envelopes, or secure-agent proof, push it into TypeSec. LakeCat should own identity, tenancy, standard catalog compatibility, pointer state, policy gates, proof persistence, and integration events. QueryGraph should compose the resulting proof into semantic user workflows. That division is what lets LakeCat be both compatible and ambitious.

1.4.6 Extensions, Proposals, And Local Invention

A useful test for any LakeCat feature is: would a Spark, Flink, Trino, PyIceberg, DuckDB, or Sail user need to understand it to read and write a normal Iceberg table? If the answer is yes, the feature is either already part of Iceberg or it risks breaking compatibility. If the answer is no, the feature can be an implementation choice, an optional catalog extension, a governance extension, a QueryGraph application surface, or a future proposal candidate.

The Rust service spine is local invention. It is important, but it is not an Iceberg extension. LakeCat uses Rust because the catalog transaction is a systems transaction: routing, identity, tenancy, metadata-pointer state, idempotency, policy receipt evidence, Sail validation, audit, outbox, redaction, and replay admission all have to agree. A Java catalog, a managed cloud catalog, or a database-backed catalog could implement the same behavioral guarantees. The behavior that might generalize is deterministic catalog proof, not Rust as the mandated language.

The Turso-backed store is also local implementation. It gives LakeCat a Rust-native durable catalog spine for embedded development, local acceptance tests, and single-node deployment. That matters because the hard parts of the catalog are not only HTTP handlers. They are durable invariants: an idempotency row must replay the same request, a table record must match the row that selected it, a pointer log must describe the pointer transition that actually happened, an audit row must bind the actor and action, and an outbox event must not reach QueryGraph or OpenLineage until replay admission proves it still matches the catalog state. Turso is the local vehicle for those invariants. The standardizable idea is the invariant profile: exact retry, pointer history, catalog event identity, and redacted proof.

The Iceberg REST namespace and table paths are not local invention. They are the compatibility surface. A standard client should see a catalog, namespaces, table identifiers, metadata locations, table creation, table loading, and optimistic commits. LakeCat can strengthen those paths internally, but it must not make PySpark learn QGLake, TypeSec, Grust, Croissant, CDIF, OSI, ODRL, or OpenLineage to perform a normal table operation. That is the first rule of the architecture: standard reads and writes stay standard.

Commit CAS sits in both worlds. Optimistic metadata-pointer movement is standard Iceberg catalog behavior. LakeCat's hardening around it is additive: create-only metadata writes, exact idempotency, pointer logs, audit rows, transactional outbox, replay validation, and conflict redaction. Those are not new table semantics. They are a stronger catalog envelope around standard table semantics. If this becomes a standards conversation, it should be framed as an optional catalog reliability profile, not as a LakeCat-specific commit protocol.

Governed scan and credential paths are governance extensions. Iceberg already describes the metadata an engine needs to plan a table: schemas, field ids, partition specs, snapshots, manifests, file metrics, delete files, and sequence numbers. LakeCat adds the question "under whose authority is this plan being requested, and what is the permitted shape of the work?" TypeSec answers the policy question, LakeCat binds that answer to the catalog action, and Sail plans the effective scan. For credentials, the same rule applies: raw storage authority is a deliberately audited exception, while constrained agents should receive governed Sail-planned work. A future optional profile could standardize proof-carrying scan planning or governed credential vending, but it should not require TypeSec or expose LakeCat-only receipt shapes as mandatory Iceberg metadata.

QueryGraph and QGLake handoff surfaces are application extensions. QueryGraph needs semantic graph state, lineage agreement, OpenLineage anchors, management inventory, view receipt chains, storage-profile proof, policy anchors, credential posture, and agent workflow evidence. LakeCat should produce replay-validated catalog facts for that import. Grust should own graph

schema, taxonomy, projection, traversal, and graph query behavior. TypeSec should own capabilities, TypeDID context, policy composition, secure-agent semantics, and authorization receipts. QueryGraph should compose Croissant, CDIF, OSI, ODRL, OpenLineage, QGLake acceptance, Grust graph import, and TypeSec evidence above the portable Iceberg table. Some narrow proof shapes may become optional catalog profiles. QueryGraph itself should not become an Iceberg dependency.

The handoff proof rule follows from that boundary. A QGLake handoff summary must not shrink LakeCat replay evidence into a convenient local shape after LakeCat and QueryGraph have already verified it. Request identity, QueryGraph bootstrap, governed scan, management, view, credential, and commit proofs are allowed to grow as LakeCat hardens the catalog spine. The local acceptance loop should validate those objects and then preserve them, including authorization receipt actions, replay hashes, OpenLineage hashes, policy upsert proof, storage-profile proof, and optional tenant linkage. Otherwise the handoff artifact can prove less than the catalog actually checked, which is exactly the kind of drift QueryGraph and TypeSec are meant to eliminate.

This is why LakeCat keeps pushing work into Sail. The catalog has enough to do without becoming a second engine. It should not decode every manifest metric, implement every partition transform, reason about every delete-file edge case, or infer field-id-safe projections from policy strings. Those jobs need the same semantics the executor uses. If LakeCat reimplements them locally, it creates a drift risk: the catalog can prove one interpretation while the engine executes another. That is the worst kind of proof, because it is precise and wrong.

Sail is the right engine boundary because it is Rust-native, close to Arrow/DataFusion execution, and reusable outside LakeCat. It can own generated Iceberg models, typed metadata interpretation, schema and partition evolution, manifest expansion, metrics decoding, delete planning, metadata-as-data, scan task generation, commit validation, and Iceberg v4 interpretation. LakeCat can then ask Sail for a table-shaped decision and persist compact evidence: snapshot id, format version, plan hash, projected field ids, required predicate hash, delete posture, task count, metadata-table shape, or commit validation result. TypeSec can say what is allowed. Sail can say what that means against the current table. LakeCat can make the decision durable, auditable, and replayable.

That division gives each community a clean artifact. Iceberg keeps portable table semantics. Sail gets reusable engine logic. LakeCat gets a thin, fast, durable catalog spine. TypeSec gets a governance proof boundary. Grust gets graph-native projection and traversal. QueryGraph gets a foundation that can import catalog truth without forcing ordinary Iceberg clients into the QueryGraph application model.

1.4.7 A Detailed Catalog Concept Map

This release is easiest to understand if each catalog concept is placed in one of five buckets: standard Iceberg parlance, LakeCat implementation, LakeCat extension surface, TypeSec or QueryGraph integration surface, and possible future Iceberg-adjacent profile. The buckets matter because LakeCat is trying to do two things at once. It must remain boring where ordinary Iceberg

clients need boring behavior, and it must become much more explicit where governed agents, QueryGraph, and operators need durable proof.

The Rust service and catalog spine belongs in LakeCat implementation. In standard Iceberg language, a catalog is a service or metastore that resolves names and moves metadata pointers under optimistic commit rules. Iceberg does not say that the service must be Rust, must use a particular storage engine, or must expose a particular internal trait. LakeCat chooses Rust because its catalog transaction is now a dense systems boundary. A single table operation can include route parsing, warehouse and tenant scope, authenticated principal identity, namespace validation, expected metadata pointer checks, idempotency keys, metadata-object write posture, TypeSec authorization receipt evidence, Sail validation, pointer-log insertion, audit insertion, outbox insertion, and replay admission. A Rust service spine lets those steps be typed, testable, and close to the Rust engine and security crates that will actually interpret the data. That is not an Iceberg extension. The possible standards lesson is only that catalog actions should be replayable and explainable, regardless of the implementation language.

The Turso-backed local store direction also belongs in LakeCat implementation. Iceberg needs durable catalog state and atomic metadata-pointer movement; it does not require SQLite, Turso, Postgres, FoundationDB, DynamoDB, or any other particular backing store. LakeCat uses the Rust turso crate behind the CatalogStore trait because the local and embedded path should exercise real persistence, transactions, content validation, idempotency, and replay logic without leaving the Rust stack. The important behavior is not “Turso as a standard.” The important behavior is that durable rows are scoped and checked: project rows must match project identity, warehouse rows must match warehouse identity, namespace and table records must match their selected rows, policy and storage-profile records must match their anchors, idempotency responses must match the route and table they replay, and outbox records must be fit for graph and lineage projection. Those are LakeCat reliability properties. The Iceberg-adjacent candidates are much smaller: atomic pointer CAS, exact retry semantics, pointer-history inspection, redacted conflict proof, and durable catalog event identity.

Iceberg REST-compatible table and namespace paths are standard Iceberg parlance. They are the floor, not the innovation. A PySpark, Spark, Flink, Trino, DuckDB, PyIceberg, or Sail user should be able to configure LakeCat as an Iceberg REST catalog and perform normal namespace and table work through the ordinary path. That means the client can create or list namespaces, create or load tables, and commit metadata updates without understanding QueryGraph, QGLake, TypeSec, Grust, OSI, ODRL, Croissant, CDIF, or OpenLineage. LakeCat may record additional evidence behind those requests. It may expose management, bootstrap, credential, replay, and proof surfaces beside those requests. But if a standard table read or commit requires a QueryGraph-only route, LakeCat has moved from compatible catalog to private protocol. The compatibility discipline is simple: standard clients see standard Iceberg behavior; advanced clients can ask for stronger evidence.

Commit CAS is standard Iceberg behavior, while LakeCat’s envelope around it is implementation and optional proof. In Iceberg, a writer does not mutate the table in place. It writes a new metadata file and asks the catalog to advance the current metadata location only if the table still

satisfies the writer’s commit requirements. That optimistic compare-and-swap is essential Iceberg parlance. LakeCat hardens the whole transaction around that standard rule. Metadata objects are treated with create-only discipline. Idempotency rows make retries exact rather than approximate. Pointer logs record accepted transitions with sequence evidence. Audit rows preserve who was allowed to do what. Outbox rows turn committed catalog state into graph and lineage projection work. Replay validators reject malformed durable evidence before LakeCat acknowledges, projects, or hands off proof. The future Iceberg-adjacent shape is not a new commit protocol; it is an optional catalog profile that says how idempotent retries, redacted conflicts, pointer history, and event identities should be represented across catalogs.

Governed scan paths are LakeCat and TypeSec governance surfaces around standard Iceberg metadata. Iceberg already gives engines the material they need to plan: schemas, field ids, partition specs, sort orders, snapshots, manifest lists, manifests, manifest metrics, data files, delete files, sequence numbers, and format-version behavior. LakeCat should not turn those into local ad hoc planner code. Instead, LakeCat identifies the request and policy context, TypeSec decides the allowed action and restriction, and Sail plans the narrowed request against the current table metadata. A governed scan receipt can then say which principal asked, which purpose was claimed, which policy hash applied, which columns and predicates survived, which snapshot and format version Sail planned, and which task or plan hash was accepted. That is new in LakeCat/TypeSec/QueryGraph. It should not be custom Iceberg table metadata. If it proves generally useful, it could become an optional proof-carrying scan-planning profile.

Credential paths follow the same boundary. Catalog ecosystems often include credential vending, but LakeCat treats raw credentials as an exception that must be deliberate, short-lived, audited, and tied to storage-profile evidence. For trusted humans or service principals, LakeCat can record why a credential was vended, which storage root was in scope, which TTL cap was applied, and which policy receipt authorized the exception. For constrained agents, the safer default is governed Sail-planned work, not broad object-store authority. That distinction is a LakeCat/TypeSec security posture, not an Iceberg table requirement. The possible future standards topic is an optional governed credential-vending proof that different catalogs and engines could understand.

Audit, outbox, OpenLineage, and replay validation are LakeCat integration surfaces. Iceberg does not require an outbox table or an OpenLineage emitter to define table semantics. LakeCat uses them because QueryGraph and operators need side effects to be derived from committed catalog truth rather than from best-effort callbacks. The outbox makes graph and lineage projection transactional with catalog state: a table commit, view change, management mutation, credential decision, or scan proof can be retried from durable evidence until it is delivered. Replay validation is the guardrail that keeps bad durable rows from becoming trusted downstream facts. The possible Iceberg-adjacent proposal is a compact event identity and lineage-binding profile. OpenLineage itself remains a lineage ecosystem standard; QueryGraph semantics remain above Iceberg.

QueryGraph and QGLake handoff surfaces are deliberately broad, but they should stay above the standard catalog contract. QueryGraph needs bootstrap bundles, management inventory, view history, credential posture, pointer-history proof, commit proof, governed scan proof,

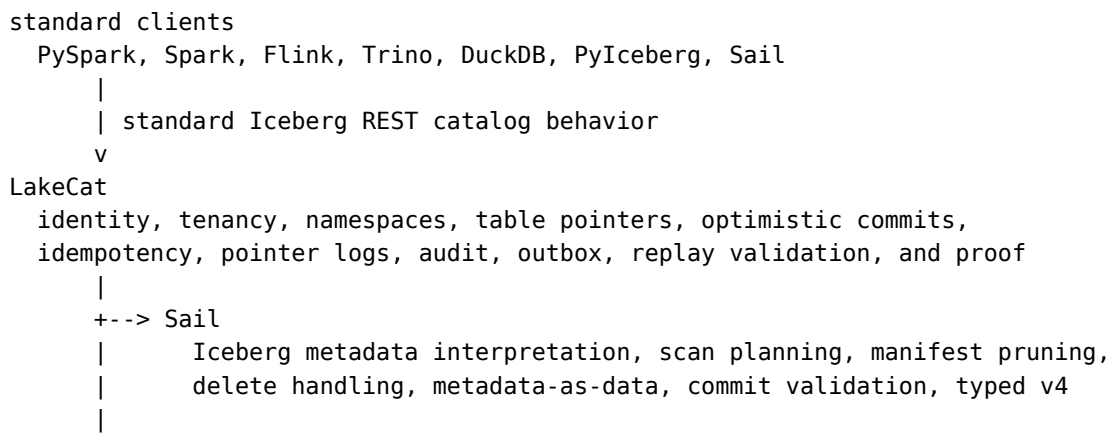
OpenLineage receipt hashes, graph anchors, and import verification. Those are application and integration needs, not ordinary Iceberg table semantics. LakeCat supplies stable catalog anchors and proof hashes. Grust owns graph storage, taxonomy, projection, and traversal. TypeSec owns policy, capability, TypeDID, secure-agent, and authorization semantics. QueryGraph composes Croissant, CDIF, OSI, ODRL, OpenLineage, and agent workflows from that evidence. Some small proof shapes may be candidates for future optional profiles, but QueryGraph itself should remain an application layer, not a hidden dependency of Iceberg clients.

Iceberg v4 work sharpens this boundary. LakeCat can preserve compatibility by accepting metadata it does not yet fully interpret, but long-term typed v4 knowledge should live in Sail. Metadata trees, richer delete semantics, row-lineage-aware planning, new scan-planning rules, metadata-as-data, and future format-version behavior belong in the engine because they require the same interpretation execution will use. LakeCat should store the current metadata pointer, enforce tenancy and policy, call Sail, and persist concise proof of what Sail understood. That makes LakeCat more compatible, not less: the catalog avoids a shadow implementation while still giving QueryGraph and agents strong evidence.

The result is a clean answer to the extension question. Rust service structure and Turso storage are implementation choices. Iceberg REST namespace and table paths, metadata pointers, snapshots, manifests, and optimistic commit are standard Iceberg. Idempotency, pointer logs, audit, outbox, replay validation, management inventory, and proof endpoints are LakeCat implementation or optional catalog extensions. TypeSec receipts, ODRL restrictions, TypeDID context, and secure-agent posture are governance extensions. QGLake bootstrap, QueryGraph import, Croissant, CDIF, OSI, and broad semantic graph workflows are QueryGraph application extensions. The future Iceberg work should be proposed only as small optional profiles where interoperability would clearly improve: retry semantics, pointer-history inspection, redacted conflict evidence, catalog event identity, lineage binding, governed credentials, and proof-carrying scan planning.

1.4.8 Ownership Map For The Catalog Stack

The architecture is easier to reason about if every concept has one primary home. LakeCat can connect the layers, but it should not absorb the whole system. The catalog becomes stronger when each adjacent project owns the domain where it has the best semantics.



```

+--> TypeSec
|   capabilities, TypeDID context, ODRL restrictions, secure-agent
|   semantics, credential posture, authorization receipts
|
+--> Grust
|   graph schema, graph taxonomy, projection mechanics, traversal,
|   graph stores, Cypher-facing behavior
|
+--> OpenLineage and outbox consumers
|   lineage events and replay-validated delivery receipts
|
v
QueryGraph / QGLake
  Croissant, CDIF, OSI, ODRL composition, semantic bootstrap, management
  views, agent workflow proof, graph import, acceptance, and user reasoning

```

The arrows do not mean every request must visit every layer. A standard Spark load should stop at the Iceberg REST-compatible catalog path. A governed agent scan should visit TypeSec and Sail. A QueryGraph bootstrap should consume LakeCat’s replay-validated proof and hand graph mechanics to Grust. The point is that LakeCat is the accountable control plane, not the owner of every domain-specific interpretation.

This ownership map also explains the difference between an extension and a future proposal. A LakeCat extension can depend on LakeCat’s proof rows, TypeSec receipts, Grust graph import, or QueryGraph’s acceptance bundle because it serves this system. A future Iceberg-adjacent proposal should be smaller and less coupled. It should describe behavior that another catalog and another engine could implement without adopting the LakeCat stack.

The following checklist is the standards filter:

Decision question	Keep it where?	Why
Does a normal Iceberg client need it to load or commit a normal table?	Iceberg compatibility path.	Standard clients must not depend on LakeCat-only semantics.
Does it move or inspect the current metadata pointer?	LakeCat store and service spine.	The catalog is the authority for pointer state, CAS, retries, and history.
Does it interpret schemas, field ids, manifests, metrics, deletes, scan tasks, metadata tables, or format-version behavior?	Sail.	Those facts need engine semantics, not catalog-local approximations.
Does it decide who may do what, under which capability, policy, TypeDID, or ODRL restriction?	TypeSec.	Governance semantics should be reusable and independently testable.
Does it define graph taxonomy, projection, traversal, Cypher, or graph storage?	Grust.	Graph mechanics should be reusable outside LakeCat and QueryGraph.
Does it compose Croissant, CDIF, OSI, ODRL, lineage, catalog proof, and agent workflow meaning for users?	QueryGraph.	That is an application layer over the portable table contract.
Could independent catalogs and engines share the behavior without adopting this stack?	Future optional profile candidate.	This is the right shape for Iceberg-adjacent standardization.

The practical consequence is that LakeCat should be strict but narrow. It should reject malformed receipt actions, mismatched principals, weak hashes, scope drift, broken idempotency evidence, and replay-invalid outbox rows because those are catalog proof concerns. It should not become the place where Iceberg scan planning, graph traversal, or policy logic is reinvented. When the proof needs table semantics, LakeCat should call Sail. When the proof needs security semantics, it should call TypeSec. When the proof needs graph semantics, it should call Grust. QueryGraph can then trust LakeCat because the catalog evidence was produced by the right layer and persisted at the right boundary.

1.4.9 Why Work Moves Into Sail

The strongest architectural argument for LakeCat is that the catalog should be engine-close without becoming a shadow engine. Catalogs are the right place for identity, tenancy, names,

metadata pointers, CAS, idempotency, audit, outbox, policy gates, and integration evidence. They are a poor place to reimplement field-id projection, schema evolution, partition transforms, manifest metric decoding, delete planning, scan-task generation, metadata tables, row lineage, or format-version interpretation. Those are data-shaped responsibilities, and they belong in the engine that will execute or expose the work.

Iceberg correctness is not just a string comparison against column names. It is field ids across schema evolution, partition specs across partition evolution, snapshot selection, manifest-list traversal, manifest metrics, lower and upper bounds, null counts, equality deletes, position deletes, sequence numbers, sort orders, metadata tables, v3 row lineage, and future v4 metadata trees. If LakeCat implements those locally, it becomes a partial Iceberg engine with a smaller test surface and a high drift risk. A manifest-metric bug could be fixed in Sail while LakeCat keeps emitting stale governed-scan proof. A delete planner could improve in execution while the catalog still proves the wrong task set. A v4 metadata-tree interpretation could land in the engine while the catalog keeps a JSON-shaped approximation.

Sail is a strong engine boundary because it is Rust-native and close to the actual lakehouse representations LakeCat needs to trust. Sail can own generated Iceberg model handling, catalog-provider integration, table-status conversion, manifest expansion, pruning, delete handling, scan-task generation, metadata-as-data, commit requirement validation, and typed v3/v4 interpretation. LakeCat should call Sail with catalog context and the effective governed request, then persist compact evidence of the Sail decision: snapshot id, format version, projected field ids, restriction hash, task counts, delete posture, metadata-table shape, validation result, or plan hash. That proof says more than “the catalog approved a read.” It says “TypeSec allowed this narrowed request and Sail planned it against the current Iceberg metadata.”

Pushing work into Sail is also a performance strategy. The fast catalog is not the one that parses every manifest and delete file in the control plane. The fast catalog guards identity, tenancy, pointer state, idempotency, and durable evidence, then hands data-shaped work to an engine built for columnar metadata, pruning, statistics, Arrow, DataFusion, and execution planning. LakeCat becomes fast because it stays thin where it should be thin and strict where it must be strict.

The division also keeps cache and locality decisions in the right place. A catalog can remember the current metadata pointer and the proof that a request was allowed, but it should not become the owner of every manifest cache, partition-pruning cache, delete-planning cache, metadata-table scan, or statistics decoder. Those structures are valuable because the engine can use them repeatedly while planning and executing data work. If LakeCat builds a parallel cache, it pays the parsing cost twice and risks proving a plan from different semantics than the engine will execute. If Sail owns those structures, LakeCat can persist small stable facts: which pointer was current, which snapshot was planned, which field ids survived policy narrowing, which delete posture was observed, which manifests or tasks were selected, and which plan hash should be replayed later.

This is especially important for Iceberg v4 compatibility. A v4-compatible catalog must not panic or reject ordinary metadata simply because the catalog does not yet expose every typed helper locally. But a long-lived system should not settle for JSON passthrough as its understanding

of the format. Typed v4 metadata trees, metadata-as-data, row-lineage-aware planning, delete semantics, branch and snapshot behavior, manifest evolution, and future planning rules need one reusable implementation. Sail is the right home because it can serve LakeCat, direct Rust users, QueryGraph workflows, and execution paths with the same interpretation. LakeCat can remain the authority for identity, tenancy, pointer state, and receipts while Sail becomes the authority for what the table metadata means.

A good test for the boundary is simple. If the logic needs to understand field ids, partition transforms, manifests, data files, delete files, row lineage, metadata tables, scan tasks, or format-version-specific behavior, push it into Sail. If the logic needs to understand graph taxonomy, graph stores, projection, traversal, or Cypher, push it into Grust. If the logic needs to understand capabilities, TypeDID, policy composition, secure-agent semantics, or authorization receipts, push it into TypeSec. LakeCat should call those systems and persist their evidence, not absorb their domains.

The same boundary makes governance stronger. A TypeSec receipt saying “allowed columns were narrowed” is only as strong as the interpretation of those columns. If narrowing is catalog-side string matching, schema evolution and field-id behavior can undermine the proof. If narrowing is converted into a Sail-planned scan against the current Iceberg metadata, the proof is tied to the same engine semantics that execution uses. TypeSec owns the authorization meaning. LakeCat owns the durable catalog transaction. Sail owns the data meaning.

This is the shape of the main user workflows:

1. In PySpark, LakeCat should look like a normal Iceberg REST catalog. PySpark creates namespaces, writes metadata, loads tables, and commits through the standard path. LakeCat’s Rust/Turso/CAS/idempotency/audit machinery improves reliability without changing the PySpark model.
2. In a Rust engine workflow, LakeCat can call Sail directly and avoid a JVM-shaped detour. The catalog keeps names, tenancy, policy, and pointers; Sail handles table-format interpretation and planning.
3. In an agentic workflow, LakeCat should refuse broad credentials by default, ask TypeSec for a capability, derive a restriction, and ask Sail for a governed plan. The agent receives bounded work instead of object-store authority.
4. In QueryGraph, LakeCat hands off replay-validated catalog proof while Grust owns graph mechanics and TypeSec owns authorization semantics. QueryGraph imports Croissant, CDIF, OSI, ODRL, OpenLineage, and QGLake meaning from stable catalog anchors rather than guessing from storage layout.

For Iceberg v4, this boundary is decisive. LakeCat can use JSON passthrough as a compatibility bridge when metadata appears ahead of local typed support, but that should remain a bridge. Typed v4 metadata trees, row-lineage-aware planning, richer delete semantics, metadata-as-data, and validation rules belong in Sail. LakeCat should store pointers, authorize actions, call Sail, and preserve receipts. That is how LakeCat stays compatible enough for standard Iceberg clients and strong enough to become QueryGraph’s foundation.

1.4.10 From Catalog Concepts To User Workflows

The catalog concepts are easiest to trust when they are followed through real workflows. The same LakeCat boundary should serve a standard PySpark user, a Rust engine, an agent, and QueryGraph without changing the meaning of an Iceberg table.

For PySpark, the standard vocabulary is enough. The user configures an Iceberg REST catalog, creates a namespace, writes a table, and commits metadata. The visible objects are the Iceberg namespace, table identifier, current metadata location, snapshot, manifest list, manifests, and data files. LakeCat's Rust service spine, Turso rows, idempotency records, pointer logs, audit rows, outbox rows, and replay validators are deliberately behind the curtain. They make the standard path more reliable and explainable, but they are not additional table-format obligations. A PySpark workflow should not need to know whether QueryGraph, TypeSec, Grust, OpenLineage, Croissant, CDIF, OSI, or ODRL exists.

For Sail, the catalog boundary becomes engine-close. LakeCat resolves the warehouse, namespace, table, identity, current metadata pointer, policy context, and commit or scan intent. Sail owns the table-shaped interpretation: schema field ids, partition transforms, snapshot selection, manifest metrics, delete-file semantics, scan task construction, metadata tables, row-lineage behavior, and typed v4 metadata. LakeCat should persist the compact proof that Sail interpreted the current table state, not copy Sail's planner into catalog code. That makes the proof cheaper to maintain and more trustworthy because the same engine semantics serve planning, execution, and validation.

For an agent, the standard Iceberg table still stays standard, but access is not merely "load the table and hand over storage credentials." LakeCat resolves the principal and request purpose, asks TypeSec for a capability decision, derives the effective restriction, and sends that narrowed request to Sail. The returned plan can carry field-id-aware projection evidence, row predicate evidence, snapshot and format-version evidence, policy hashes, TTL caps, and task hashes. If raw credentials are denied, that denial is proof, not a vague failure. If raw credentials are granted to a trusted principal, the grant is a short-lived audited exception tied to storage-profile scope. The extension is the receipt-bearing governance envelope, not a custom Iceberg table.

For QueryGraph, LakeCat becomes the foundation rather than the whole semantic system. LakeCat emits stable catalog anchors: server/project/warehouse scope, namespace and table state, current pointers, commit history, view receipt chains, storage-profile and policy anchors, credential posture, governed scan proof, OpenLineage hashes, and replay-validation hashes. Grust owns graph taxonomy, graph storage, projection mechanics, traversal, and Cypher. TypeSec owns capabilities, TypeDID context, secure-agent posture, policy composition, and authorization receipts. QueryGraph composes Croissant, CDIF, OSI, ODRL, OpenLineage, QGLake acceptance, and agent workflow meaning from those anchors. That keeps the Iceberg table portable while giving QueryGraph a proof-bearing bootstrap path.

The same workflow lens clarifies what could become an Iceberg proposal. Rust, Turso, QueryGraph, TypeSec, and Grust are not proposed table-format requirements. Good candidates are smaller optional profiles: exact idempotent commit replay, pointer-history inspection, redacted conflict proof, transactional catalog event identity, lineage receipt binding, governed credential-

vending proof, and proof-carrying scan planning. Those profiles would help interoperability because they describe behavior at the catalog boundary without requiring every engine to adopt LakeCat’s whole application stack.

1.4.11 A Catalog Concept Guide For LakeCat Readers

The easiest way to misunderstand LakeCat is to treat every visible feature as an Iceberg extension. That makes the system sound more invasive than it is. The second easiest way to misunderstand it is to treat the catalog as only a name server. That makes the system sound weaker than it is. LakeCat sits between those extremes. It keeps the Iceberg table contract ordinary, then makes the catalog transaction more explicit, replayable, and governable.

The Rust service/catalog spine exists. In standard Iceberg vocabulary, that is not a feature. Iceberg does not require Rust, Java, a managed database, an embedded database, or a particular process model. Iceberg requires catalog behavior: resolve a namespace and table identifier, return the current metadata location, accept compatible table creation, and advance metadata pointers only when commit requirements still hold. LakeCat’s Rust spine is the implementation choice that lets those ordinary catalog actions carry stronger evidence. The same request path can resolve identity and tenancy, enforce the Iceberg REST route, check idempotency, call Sail for table-shaped work, ask TypeSec for a decision, persist audit and outbox rows, redact sensitive conflicts, and admit only replay-valid evidence. That is LakeCat implementation, not Iceberg parlance. The possible future standard idea is not “catalogs should be Rust.” It is “catalogs should be able to prove exactly what action they accepted, rejected, replayed, vended, and emitted.”

The Turso-backed local store direction is in place. Turso is also not Iceberg parlance. Standard Iceberg cares that catalog state is durable and that the current metadata pointer changes atomically. LakeCat chooses the Rust turso crate for the local durable catalog spine because it fits an embedded, testable, Rust-native control plane. Behind the `CatalogStore` trait, Turso stores projects, warehouses, namespaces, tables, views, storage profiles, policy bindings, idempotency records, pointer logs, audit rows, outbox rows, and soft-delete state. That does not make Turso an Iceberg feature. The reusable behavior is narrower: compare-and-swap metadata pointer updates, exact idempotent replay, drift rejection, redacted conflict evidence, pointer-history inspection, audit evidence, and transactional catalog events.

Iceberg REST-compatible namespace and table paths exist. This is the standard compatibility layer and should stay boring. A Spark, PySpark, Flink, Trino, DuckDB, PyIceberg, or Sail client should be able to configure LakeCat as an Iceberg REST catalog, create namespaces, create or load tables, and commit metadata updates without learning `QueryGraph`, `QGLake`, `TypeSec`, `Grust`, `Croissant`, `CDIF`, `OSI`, `ODRL`, or `OpenLineage`. LakeCat may record proof behind the route, and it may expose optional management or handoff surfaces beside it, but ordinary table access must not depend on those optional surfaces. If a standard client must call a `QueryGraph` route before it can load a normal table, LakeCat has broken the compatibility contract.

Commit CAS is standard Iceberg catalog behavior. The writer prepares new table metadata and asks the catalog to advance the current metadata location only if the expected table state is still true. LakeCat does not replace that idea. It hardens the envelope around it. Create-only metadata

writes avoid object overwrite surprises. Store-level CAS makes the pointer transition atomic. Idempotency rows make retries exact and detect drift. Pointer logs record accepted movement. Audit rows record authority. Outbox rows bind graph and lineage delivery to committed catalog state. Replay validation refuses malformed durable evidence before acknowledgement, Grust projection, OpenLineage projection, or QGLake import. The Iceberg term is optimistic commit. The LakeCat terms are idempotency, pointer log, audit, outbox, redaction, and replay validation. The future proposal candidate is not “LakeCat commit”; it is an optional catalog profile for retry semantics, pointer history, conflict proof, and catalog event identity. LakeCat also validates retry evidence at the store boundary. A blank or malformed idempotency key, a caller-supplied idempotency request hash without a key, or a request hash that is not full SHA-256 evidence fails before pointer movement, pointer-log insertion, audit, outbox emission, or replay. That keeps Turso and embedded memory behavior aligned with the REST contract and prevents non-REST callers from smuggling weak retry evidence into durable catalog state.

Governed scan and credential paths carry TypeSec-style receipt evidence. The standard Iceberg table already contains the metadata an engine needs for scan planning: schemas, field ids, partition specs, sort orders, snapshots, manifest lists, manifests, metrics, data files, delete files, and sequence information. LakeCat adds a control-plane prelude around that table. It identifies the principal and purpose, asks TypeSec for a capability decision, derives allowed columns, mandatory predicates, TTL caps, policy hashes, raw-credential posture, and receipt evidence, and then asks Sail to plan the effective request. For credential vending, the same posture applies. Broad object-store credentials are deliberate audited exceptions for trusted principals, not the default path for agents or constrained workloads. Restricted principals should receive governed Sail-planned work. This is a LakeCat/TypeSec governance extension. It could inspire a future optional Iceberg-adjacent profile for proof-carrying scan planning or governed credential vending, but it should remain table-format neutral and optional.

QueryGraph and QGLake handoff, OpenLineage, bootstrap, management, view, credential, and commit proof surfaces are broad because QueryGraph needs more than a table pointer. They are not standard Iceberg table semantics. They are LakeCat and QueryGraph integration surfaces that export governed catalog truth: warehouses, namespaces, tables, views, current pointers, pointer transitions, view receipt chains, storage-profile and policy anchors, management inventory, credential posture, governed scan proof, replay-validation hashes, OpenLineage receipt hashes, and graph anchors. QueryGraph composes those facts with Croissant, CDIF, OSI, ODRL application meaning, Grust graph import, QGLake acceptance, and agent workflows. Those application semantics should not become custom Iceberg metadata. Small proof shapes such as catalog event identity, lineage receipt binding, pointer-history inspection, and view lifecycle proof may be worth future standardization as optional profiles. QueryGraph itself is the application layer above Iceberg.

That distinction gives the current catalog concepts stable names:

Concept	Iceberg-standard?	LakeCat/Query-Graph/TypeSec role	Extension or proposal posture
Rust service/catalog spine	No. Iceberg specifies behavior, not implementation language.	LakeCat implementation path for REST routing, identity, tenancy, Sail calls, TypeSec receipts, CAS, audit, outbox, and replay validation.	Not an Iceberg extension. Only deterministic proof behavior may generalize.
Turso-backed local store	No. Iceberg needs durable state and atomic pointer movement, not Turso.	LakeCat durable local CatalogStore implementation.	Not a database proposal. CAS, idempotency, pointer history, and event behavior may generalize.
REST namespace/table paths	Yes.	LakeCat exposes standard Iceberg REST-compatible access under /catalog/v1.	Follow Iceberg. Keep proof and Query-Graph paths additive.
Commit CAS	Yes.	LakeCat implements the optimistic pointer update and adds idempotency, pointer logs, audit, outbox, redaction, and replay validation.	CAS is standard; proof and retry profiles may be future optional catalog profiles.
Governed scan receipts	No, beyond the metadata engines use to plan scans.	TypeSec decides; LakeCat binds the receipt to the catalog action; Sail plans the narrowed request.	LakeCat/TypeSec governance extension; possible proof-carrying scan profile.
Credential proof	Credential vending appears in catalog ecosystems; LakeCat's proof posture is additive.	Raw credentials are audited exceptions; constrained principals get Sail-planned work.	Possible governed credential-vending profile.
OpenLineage and outbox replay	No as table semantics.	LakeCat emits replay-validated committed catalog facts	Possible optional event and lineage-binding profile.
Catalog, Query, Graph, Outbox, TypeSec, and LakeCat	Yes, as Iceberg evaluates.	OpenLineage and graph binding, TypeSec and Query-Graph, LakeCat shows pointer, identity, and replay support, and LakeCat compatibility with typed support is available.	Replay not extensible; only proofs and snapshotting support, and only JSON parsing.

The strongest architectural argument is to push as much table work as possible into the engine. Catalogs are excellent at identity, tenancy, names, metadata pointers, transactions, policy gates, and event evidence. They are poor places to reimplement field-id projection, schema evolution, partition transforms, manifest metric decoding, delete planning, row lineage, scan-task generation, metadata tables, or format-version interpretation. Those are engine-shaped responsibilities because they need the same semantics execution will use.

Sail is a great engine choice for LakeCat because it is Rust-native and close to the actual lakehouse representations. It can share generated Iceberg REST models, Arrow/DataFusion types, table-provider integration, manifest expansion, delete handling, scan-task generation, metadata-as-data, commit validation, and typed v3/v4 interpretation with execution paths instead of leaving LakeCat to build a shadow engine. LakeCat should send Sail the catalog context and the effective governed request, then persist compact evidence of the Sail decision: snapshot id, format version, projected field ids, restriction hash, task counts, delete posture, metadata-table shape, or plan hash. That makes the proof stronger. It says not merely “the catalog approved a read,” but “TypeSec allowed this narrowed request and Sail planned it against the current Iceberg metadata.”

This is how the user workflows line up. In PySpark, LakeCat should look like an ordinary Iceberg REST catalog and hide the proof machinery behind compatible responses. In a Rust engine workflow, LakeCat can call Sail directly and avoid a JVM-shaped detour. In an agentic workflow, LakeCat can refuse broad credentials and return governed Sail-planned work with receipt evidence. In QueryGraph, LakeCat can hand off proof-bearing catalog facts while Grust owns graph mechanics and TypeSec owns authorization meaning. For Iceberg v4, LakeCat can bridge unknown metadata as JSON when necessary, but the target is typed Sail support. That boundary keeps the catalog thin enough to remain compatible and strong enough to become QueryGraph’s trusted foundation.

1.4.12 The Release Vocabulary In One Pass

LakeCat now has enough surface area that a reader can get lost unless the concepts are named with care. The safest vocabulary is not “Iceberg plus extras” but a four-layer contract: standard Iceberg behavior, LakeCat implementation, LakeCat/QueryGraph/TypeSec extension surfaces, and possible future Iceberg-adjacent profiles.

The Rust service and catalog spine exists. That sentence is not an Iceberg feature claim. Iceberg does not care whether a catalog is written in Rust, Java, Go, or run as a managed service. Iceberg cares that the catalog can resolve namespaces and table identifiers, return the current table metadata, and commit compatible metadata updates atomically. LakeCat chooses Rust because the catalog transaction now carries more than a pointer: principal identity, warehouse and tenancy, expected metadata location, idempotency key, Sail validation or planning, TypeSec receipt evidence, audit, outbox, replay admission, and redaction all meet in one request. Rust gives LakeCat a direct, typed path for that dense transaction. The portable lesson is not “Iceberg should use Rust.” The portable lesson is that a catalog should be able to prove what it accepted, rejected, replayed, vended, and emitted.

The Turso-backed local store direction is in place. That is also LakeCat implementation, not Iceberg parlance. The Iceberg terms are catalog, namespace, table identifier, current metadata location, optimistic commit, and table metadata. LakeCat's Turso-backed store is the local durable spine behind those terms. It persists projects, warehouses, namespaces, table records, views, storage profiles, policy bindings, idempotency records, pointer logs, audit rows, outbox rows, and soft-delete state. Turso itself should not be proposed as an Iceberg feature. The future-facing behavior is narrower and more useful: atomic metadata-pointer CAS, exact idempotent replay, drift rejection, redacted conflicts, pointer-history inspection, audit evidence, and transactional catalog events.

Iceberg REST-compatible table and namespace paths exist. This is standard Iceberg compatibility. A PySpark, Spark, Flink, Trino, DuckDB, PyIceberg, or Sail client should be able to use `/catalog/v1` to configure a catalog, create and list namespaces, create and load tables, and commit metadata updates without knowing about QueryGraph, QGLake, Grust, TypeSec, Croissant, CDIF, OSI, ODRL, or OpenLineage. LakeCat can record audit evidence and emit outbox events behind those routes, but those side effects must not become hidden requirements for ordinary table access. If a standard client must call a QueryGraph route before it can read a normal Iceberg table, compatibility has failed.

Commit CAS is standard Iceberg catalog behavior; LakeCat's envelope around it is the hardening. In Iceberg, a writer prepares new metadata and asks the catalog to advance the current metadata location only if the expected table state still holds. LakeCat keeps that optimistic pointer movement, then adds production catalog discipline. Create-only metadata writes avoid overwriting objects. Store CAS makes the pointer transition atomic. Idempotency makes retry exact and drift detectable. Pointer logs preserve accepted movement. Audit records authority. The outbox binds graph and lineage delivery to committed state. Replay validation refuses malformed durable evidence before acknowledgement, Grust projection, or OpenLineage projection. The standard word is commit. The LakeCat words are idempotency, pointer log, audit, outbox, redaction, and replay validation. The future proposal candidates are optional behavior profiles for idempotent commit replay, pointer history, redacted conflict proof, and catalog event streams. The durable store enforces the retry shape as well as the REST edge: malformed idempotency keys, orphaned idempotency request hashes, and short request hashes are rejected before any commit mutation or replay probe can observe catalog state.

Governed scan and credential paths carry substantial TypeSec-style receipt evidence. Standard Iceberg gives engines the metadata required to plan reads: schemas, field ids, partition specs, snapshots, manifest lists, manifests, metrics, data files, delete files, and sequence information. LakeCat adds a governance prelude around that ordinary table: identify the principal and purpose, ask TypeSec for a decision, derive allowed columns, mandatory predicates, policy hashes, TTL caps, and raw-credential posture, then ask Sail to plan the effective request. Credential vending receives the same treatment. Raw object-store credentials are an audited exception for trusted principals, not the default answer for agents. Restricted or untrusted principals should receive governed Sail-planned work instead of broad storage authority. This is a LakeCat/TypeSec gover-

nance extension today. A future Iceberg-adjacent profile could standardize proof-carrying scan planning or governed credential vending, but it should stay optional and table-format neutral.

QueryGraph and QGLake handoff, OpenLineage, bootstrap, management, view, credential, and commit proof surfaces are broad by design. They are not standard Iceberg table semantics. They are optional LakeCat/QueryGraph integration surfaces that let QueryGraph import governed catalog truth instead of scraping storage or inferring meaning from object paths. LakeCat exports catalog facts and proof anchors: warehouses, namespaces, tables, views, current pointers, pointer transitions, view receipt chains, management inventory, credential posture, governed scan proof, replay-validation hashes, OpenLineage receipt hashes, and graph anchors. QueryGraph owns the semantic composition: Croissant, CDIF, OSI, ODRL application meaning, Grust-backed graph import, QGLake acceptance, agent workflows, and user-facing reasoning. Some pieces, such as catalog event identity and lineage receipt binding, may be useful future profiles. QueryGraph itself should remain an application layer above Iceberg.

The difference is easiest to see as a release ledger:

Concept	Standard Iceberg parlance	LakeCat/Query-Graph/TypeSec meaning	Proposal posture
Rust service/catalog spine	No. Iceberg specifies catalog behavior, not implementation language.	LakeCat keeps REST routing, identity, tenancy, Sail calls, TypeSec receipts, CAS, idempotency, audit, outbox, and replay admission in one typed Rust service path.	Not a proposal. The portable idea is deterministic proof, not Rust.
Turso-backed local store	No. Iceberg needs durable catalog state and atomic pointer movement, not a particular database.	Turso backs the local CatalogStore for namespaces, tables, views, pointer logs, idempotency, audit, outbox, policy bindings, and storage profiles.	Not a database proposal. CAS, idempotent replay, pointer history, and event profiles may generalize.
Namespace and table REST paths	Yes. These are core Iceberg REST catalog surfaces.	LakeCat serves them under /catalog/v1 and records optional evidence behind the scenes.	Follow Iceberg. Keep LakeCat management and Query-Graph routes additive.
Commit CAS	Yes. Optimistic metadata-pointer movement is standard catalog behavior.	LakeCat hardens it with create-only writes, exact idempotency, pointer logs, audit, transactional outbox, redaction, and replay validation.	CAS is standard; the retry, proof, conflict, and event envelope may become optional profiles.
Governed scan receipts	No, except for the underlying Iceberg metadata that engines use to plan scans.	TypeSec decides the restriction, LakeCat binds the receipt to the action, and Sail plans the narrowed request.	Candidate optional proof-carrying scan profile.
Credential proof	Catalog credential vending exists in the ecosystem, but LakeCat's proof language is additive.	Raw credentials are audited exceptions; restricted principals receive governed exceptions with receipt evidence, identity, and binding.	Candidate optional governed credential profile. Only a small number of catalog events and linkage profiles generalize.
Open Query-Graph / SQL / Query-Plan / Query-Result	No. as Iceberg table specification and integration layer.	LakeCat exports restricted principals receive governed exceptions with receipt evidence, identity, and binding. LakeCat persists important data	Only a small number of catalog events and linkage profiles generalize.

This ledger is intentionally conservative. It lets LakeCat be ambitious without turning the table into a private format. Standard clients get ordinary Iceberg catalog behavior. Operators get durable replay and audit. Governed agents get restricted access proof. QueryGraph gets a semantic bootstrap with catalog anchors it can trust. A future standards conversation gets small, optional, interoperable profile candidates rather than a demand to adopt LakeCat, QueryGraph, TypeSec, or Turso.

1.4.13 Why Sail Is The Right Engine Boundary

The strongest argument for LakeCat is also a warning: the catalog should bring the engine closer to the data without becoming a second engine. LakeCat sees catalog intent. Sail sees table semantics. TypeSec sees authorization meaning. QueryGraph sees semantic application meaning. Keeping those responsibilities separate is what makes the system fast, compatible, and governable.

Iceberg correctness is field-id correctness, not only column-name correctness. It is schema evolution, partition transforms, snapshot selection, manifest lists, manifest metrics, lower and upper bounds, null counts, equality and position deletes, sequence numbers, sort orders, metadata tables, row lineage, and future v4 metadata trees. Those are engine-shaped problems. If LakeCat implements them locally, it becomes a partial Iceberg engine with a smaller test surface and higher drift risk. A manifest metric bug could be fixed in Sail while LakeCat keeps emitting stale governed-scan proof. A delete-planning bug could be corrected for execution while the catalog still proves the wrong task set. A v4 metadata-tree interpretation could be added to the engine while the catalog keeps a JSON-shaped approximation.

Sail is a strong choice because it is Rust-native and already belongs on the execution side of the lakehouse. It can own generated Iceberg model handling, table provider integration, table-status conversion, manifest expansion, scan-task generation, metadata-as-data, commit requirement validation, delete handling, and typed v3/v4 interpretation. LakeCat should call Sail with the catalog context and persist compact evidence of the Sail decision: snapshot, format version, projection, restriction hash, task counts, delete posture, metadata-table shape, or plan hash. That gives LakeCat proof of a real engine decision instead of a catalog-local approximation.

This is not merely code placement. It is a correctness rule. The engine is the component that has to interpret the table format completely enough to execute or expose data work. Therefore the engine should own the reusable interpretation of schemas, field ids, partition transforms, manifest statistics, data and delete files, sequence numbers, metadata tables, row lineage, and future v4 metadata trees. The catalog should own the authority around those decisions: who asked, which warehouse and table were in scope, which metadata pointer was current, which policy receipt applied, which idempotency key protected the operation, which pointer transition committed, and which replayable facts were emitted.

The difference is easiest to make concrete:

Question	Owner	LakeCat should persist
Which table does <code>warehouse.ns.table</code> mean?	LakeCat	Tenant, warehouse, namespace, table stable id, current metadata location.
Is the caller allowed to plan this read?	TypeSec plus LakeCat	Capability receipt hash, principal, purpose, action, policy ids, policy hashes, TTL cap.
Which columns does a policy name after schema evolution?	Sail, from TypeSec restriction input	Requested projection, effective field ids, rejected or narrowed fields, restriction hash.
Which files can be skipped by manifest metrics?	Sail	Snapshot id, plan hash, task count, selected manifest or task evidence where safe.
How do equality and position deletes affect the scan?	Sail	Delete posture, delete-file/task counts, format-version evidence.
Is a proposed commit valid against the current Iceberg table state?	Sail plus LakeCat CAS	Validation result, expected pointer, new metadata location, request/response hashes.
Did the pointer actually move?	LakeCat	CAS result, sequence, pointer log, audit row, idempotency response, outbox event.
How does QueryGraph learn this fact?	LakeCat to Grust/OpenLineage/QueryGraph	Replay hash, OpenLineage hash, graph anchor, QGLake handoff proof.

That table is the design in miniature. LakeCat should not prove that a policy column survived narrowing by doing string matching against the latest schema. Sail should resolve the policy's table-shaped restriction against field ids and schema evolution. LakeCat should not prove that a manifest can be skipped by decoding lower and upper bounds with its own partial logic. Sail should decode the metrics and choose the tasks. LakeCat should not infer delete behavior from metadata filenames. Sail should understand delete files and sequence numbers. LakeCat should not become the only place that knows how v4 metadata trees work. Sail should expose typed v4 interpretation that LakeCat, QueryGraph workflows, and direct Rust engine users can all reuse.

The payoff is that LakeCat proof becomes smaller and stronger at the same time. Smaller, because LakeCat records concise facts such as `snapshot-id`, `format-version`, `effective-field-ids`, `restriction-hash`, `plan-hash`, `delete-posture`, and `task-count` instead of copying a whole planner into the catalog. Stronger, because those facts came from the same engine semantics that

will plan or execute the table work. A proof generated by a shadow catalog planner is only a proof that the shadow planner believed something. A proof generated through Sail says that the reusable Rust Iceberg engine interpreted the current table metadata under the governed request.

This also keeps future Iceberg work honest. If LakeCat sees a newer metadata shape before typed support has landed, JSON preservation can keep compatibility from breaking. But JSON passthrough should be labeled as a bridge. It is not the final semantics. The target is typed Sail support, because typed support is what lets the catalog, execution engine, QueryGraph import path, and agentic planner agree on the same table meaning.

This matters in a PySpark workflow. PySpark should see ordinary Iceberg REST behavior: configure the catalog, create a namespace, create or load a table, write data, and commit metadata. LakeCat's Rust/Turso/CAS/idempotency/audit machinery runs behind the scenes. If the workflow later feeds QueryGraph, QueryGraph reads pointer logs and outbox proof that came from committed catalog state. The table remains normal Iceberg.

It matters even more in an agentic workflow. An agent should not receive broad object-store credentials just because it can describe a table. LakeCat should ask TypeSec for the capability, derive the restriction, and call Sail to plan the allowed scan against the current Iceberg metadata. The returned work is bounded by field-id-aware projection, predicate binding, manifest pruning, delete handling, TTL, and credential posture. The proof is strong because it connects policy meaning to engine interpretation.

It also matters in QueryGraph. QueryGraph needs bootstrap and QGLake handoff material that can survive scrutiny: which catalog state was current, which principal acted, which view version was accepted, which credential posture was recorded, which scan restriction was enforced, which OpenLineage event was bound to the committed fact, and which graph anchors were imported. LakeCat can provide that evidence because it owns the catalog transaction. Sail makes the evidence data-real because it owns the table-format interpretation.

For Iceberg v4 compatibility, this boundary is decisive. LakeCat can use JSON passthrough as a bridge when newer metadata appears ahead of local typed support, but that should remain a bridge. The long-term home for typed v4 metadata trees, row-lineage-aware planning, richer delete semantics, metadata-as-data, and validation rules is Sail. LakeCat should store pointers, authorize actions, call Sail, and preserve receipts. That is how LakeCat stays thin enough for compatibility and strong enough for QueryGraph.

1.4.14 A Release-Claim Ledger

LakeCat should be precise about release claims because the same word can mean different things to different readers. An Iceberg user hears “catalog” and expects REST compatibility. An operator hears “catalog” and expects durable state, audit, and recovery. A governed-agent designer hears “catalog” and expects a policy decision to constrain what the agent can actually do. A QueryGraph reader hears “catalog” and expects stable semantic anchors. A standards reader asks whether any of this belongs in Iceberg itself.

The current implementation should be described this way.

Claim	Standard	Iceberg?	LakeCat	imple- mentation?	LakeCat/Query- Graph/TypeSec extension?	Future adjacent	Iceberg- candi- date?
Rust ser- vice/catalog spine exists		No		Yes	No		No
Turso- backed local store direc- tion is in place		No		Yes	No		No
REST name- space and table paths exist		Yes		Yes	No		No
Current metadata pointer CAS exists		Yes		Yes	No		No
Idempotent commit re- play	Not required by the table format			Yes	Optional man- agement proof		Possibly
Pointer logs	Related to snap- shot history, but not a standard catalog endpoint			Yes	Optional inspec- tion proof		Possibly
Audit rows		No		Yes	Optional control- plane evidence	Possibly, only as profile language	
Transac- tional out- box		No		Yes	Optional graph/ lineage delivery contract		Possibly
Replay val- idation be- fore graph/ OpenLin- eage deliv- ery		No		Yes	Optional evi- dence hardening		Possibly
Governed scan re- quests		No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	
Key rotation and refresh capabilities on catalog API	Emerging Iceberg REST develop- ments	No	Yes, as catalog gate		TypeSec gover- nance extension	Possibly, as proof-carrying	

1.4.15 Reading The Ledger Like A Standards Document

The release ledger should be read as a compatibility contract, not as a feature checklist. A feature can be essential to LakeCat and still not be an Iceberg feature. A feature can be useful to QueryGraph and still not belong in Iceberg metadata. A feature can be promising for standards work and still need to stay optional until multiple catalogs and engines need the same interoperable shape.

The Rust service spine is the clearest example. It exists, and it matters. It lets LakeCat keep routing, identity, tenancy, commit requirements, idempotency, Sail calls, TypeSec receipts, audit, outbox, and replay admission inside one typed service path. But a Rust spine is not an Iceberg extension. Iceberg should not care whether a compatible catalog is written in Rust, Java, Go, or provided as a managed service. The standardizable part is behavioral: can a catalog prove which action it accepted, rejected, replayed, vended, or emitted?

The Turso-backed local store follows the same rule. It is the right LakeCat implementation direction because it keeps the local durable catalog spine Rust-native and exercises real transactions, row/content validation, CAS, idempotency, pointer logs, audit, outbox, and replay checks. But Turso is not a table-format concept. The portable behavior is not “use this database.” The portable behavior is “make pointer movement atomic, make retries exact, keep pointer history inspectable, redact conflicts, and emit side effects from durable committed state.”

The namespace and table REST paths are different. Those are standard Iceberg surface area. LakeCat’s job is to keep them ordinary. `/catalog/v1` should let standard clients resolve namespaces, load tables, create tables, and commit metadata without a QueryGraph route, TypeSec envelope, Grust graph import, OpenLineage drain, Croissant vocabulary, CDIF document, OSI model, or ODRL policy becoming a hidden prerequisite. LakeCat can record proof behind the route, but that proof is not part of what a standard client must understand to perform a normal Iceberg operation.

Commit CAS is the mixed case. The optimistic current-metadata-pointer update is standard Iceberg. LakeCat’s idempotency records, pointer logs, audit rows, transactional outbox rows, conflict redaction, and replay validation are catalog hardening around the standard commit. This is exactly the kind of place where future Iceberg-adjacent work may be useful. The proposal should not be “adopt LakeCat’s schema.” It should be a narrow optional catalog profile for exact retry, conflict proof, pointer history, and event identity.

Governed scans and credentials are intentionally outside ordinary Iceberg table semantics. The standard table already exposes the metadata engines need to plan reads. LakeCat adds authority: who is asking, why, under which policy, with which restriction, and with what credential posture. TypeSec decides the governance meaning. Sail turns the effective restriction into table-real work. LakeCat records the receipt. That design can inspire optional profiles such as proof-carrying scan planning or governed credential vending, but those profiles should stay table-format neutral and should not require TypeSec as the only policy system.

QueryGraph, QGLake, OpenLineage, Croissant, CDIF, OSI, ODRL, TypeDID, and Grust-backed graph import are broader application and integration surfaces. They are the reason LakeCat needs rich proof, but they are not the normal path for an Iceberg client to load a table. LakeCat

should export stable catalog anchors and receipt hashes. QueryGraph should compose semantic meaning above those anchors. Grust should own graph mechanics. TypeSec should own policy and capability semantics. Iceberg should remain the portable table contract below them.

This gives LakeCat a disciplined proposal rubric:

1. If the feature is needed to understand Iceberg metadata correctly, move the reusable implementation into Sail or the Iceberg ecosystem.
2. If the feature is needed to authorize an action, keep the semantics in TypeSec and persist only catalog-bound receipts in LakeCat.
3. If the feature is graph traversal, taxonomy, projection, or Cypher behavior, push it into Grust and keep LakeCat at the graph-event boundary.
4. If the feature is QueryGraph semantic composition, keep it above LakeCat and import LakeCat proof rather than making standard Iceberg clients depend on QueryGraph.
5. If the feature helps many catalogs interoperate, propose a small optional profile: retry proof, pointer history, conflict redaction, event identity, lineage binding, governed credential proof, or proof-carrying scan planning.

Sail is central to that rubric. It is not merely a dependency that LakeCat can call when convenient. It is the place where Iceberg table meaning should converge for Rust. Field ids, schema evolution, partition transforms, manifest metrics, delete files, row lineage, metadata tables, scan tasks, commit requirements, and v4 metadata interpretation need one reusable implementation. If LakeCat owns a second implementation, the catalog can drift from execution. If Sail owns it, LakeCat proof becomes smaller and stronger: it can store the pointer, request identity, TypeSec receipt hash, effective restriction hash, Sail plan hash, snapshot id, format version, task count, and delete posture without pretending to be the engine.

That is the core LakeCat argument. The catalog should be thin in table-format semantics and thick in authority, durability, and replay evidence. Sail should be thick in Iceberg semantics. QueryGraph should be thick in semantic composition. TypeSec should be thick in security semantics. Grust should be thick in graph semantics. Keeping those centers of gravity separate is what lets LakeCat remain compatible with ordinary Iceberg while still becoming the foundation for governed, agentic QueryGraph workflows.

1.4.16 Standard Terms, LakeCat Terms, And Proposal Terms

The safest way to explain LakeCat is to keep three questions separate.

First: is the word already standard Iceberg parlance? If the word is catalog, namespace, table identifier, current metadata location, table metadata, snapshot, manifest list, manifest, data file, delete file, schema evolution, partition evolution, optimistic commit, or REST catalog compatibility, the answer is yes. LakeCat should use those words in the ordinary Iceberg sense. They are not marketing words and they are not QueryGraph words. They are the shared contract that lets Spark, PySpark, Flink, Trino, DuckDB, Sail, and future Rust engines reason about the same table.

Second: is the word LakeCat implementation? Rust service spine, Turso-backed store, CatalogStore, normalized idempotency row, pointer log, audit row, outbox row, redaction rule,

replay validator, local release gate, and QGLake fixture are LakeCat implementation or product surfaces. They may be critical to reliability, but they are not Iceberg table semantics. A standard client should benefit from them without being forced to understand them.

Third: is the word an optional extension or a future standard candidate? QueryGraph bootstrap, QGLake handoff, view receipt chains, OpenLineage projection, credential-root posture, management proof, TypeSec receipts, TypeDID context, ODRL-derived restrictions, secure-agent proof, Croissant, CDIF, OSI, and Grust graph import are optional LakeCat, QueryGraph, TypeSec, or Grust surfaces beside Iceberg. They should remain optional unless a smaller portable behavior proves useful beyond LakeCat. A future Iceberg-adjacent proposal should be phrased as a behavior profile, not as a product dependency: idempotent commit replay, pointer-history inspection, redacted conflict proof, transactional catalog event streams, governed credential proof, lineage receipt binding, view lifecycle proof, and proof-carrying scan planning are plausible profiles. “Use QueryGraph” or “store TypeSec receipts in Iceberg metadata” is not the right proposal shape.

That distinction gives the current release claims clear names:

- The Rust service/catalog spine exists. This is LakeCat implementation. It keeps REST routing, identity, tenancy, CAS, idempotency, Sail calls, TypeSec receipts, audit, outbox, and replay validation in one typed Rust path.
- The Turso-backed local store direction is in place. This is LakeCat implementation behind a portable store contract. Turso is not an Iceberg feature; atomic CAS, exact idempotency, pointer history, redaction, audit, and transactional outbox behavior are the reusable ideas.
- Iceberg REST-compatible table and namespace paths exist. This is standard Iceberg compatibility. Optional management, proof, and QueryGraph paths must stay beside the `/catalog/v1` path, not in front of it.
- Commit CAS is standard catalog behavior. LakeCat’s idempotency rows, pointer logs, audit rows, outbox rows, redacted conflict evidence, and replay validators are hardening around that standard behavior.
- Governed scan and credential paths are LakeCat/TypeSec governance extensions. They bind a principal, purpose, restriction, TTL, policy hash, credential posture, and receipt to the catalog action, then rely on Sail to make the restriction true against Iceberg metadata.
- QueryGraph and QGLake handoff are application integration extensions. LakeCat exports catalog facts and proof anchors; QueryGraph composes Croissant, CDIF, OSI, ODRL, OpenLineage, graph import, and agent workflows.

The most important standards sentence is therefore: LakeCat should prove optional catalog behavior without making optional behavior mandatory for ordinary Iceberg clients. Standard clients get ordinary Iceberg. Operators, governed agents, and QueryGraph get stronger proof. Future proposals should lift only the small pieces that multiple catalogs and engines need to interoperate.

That table is intentionally conservative. It keeps the Iceberg compatibility claim clean. LakeCat should not say “Iceberg has TypeSec receipts” or “Iceberg has QGLake handoff.” It should say that LakeCat is an Iceberg-compatible catalog that adds optional proof surfaces around catalog

actions. Those proof surfaces may produce evidence useful for future standards work, but they do not change what a standard client must do to read or write a table.

The important standards argument is not “make LakeCat mandatory.” It is “prove small, portable optional profiles.” Idempotent commit replay, redacted conflict proof, pointer-history inspection, transactional catalog event streams, OpenLineage receipt binding, governed credential-vending evidence, and proof-carrying scan planning are all candidates for that treatment. They are not table-format semantics. They are interoperable catalog-adjacent behaviors that become valuable when more than one engine, catalog, or governance system needs to compare evidence.

1.4.17 The Current Pieces In Plain Terms

The Rust service/catalog spine exists. Iceberg does not prescribe an implementation language, but LakeCat chooses Rust because the catalog request has become a dense transaction. One request can carry an HTTP route, a principal, a warehouse, a namespace, a table, an expected metadata pointer, an idempotency key, a TypeSec receipt, a Sail validation or planning call, an audit row, and an outbox row. Keeping those relationships in one Rust path reduces adapter drift and lets LakeCat persist compact proof without making the standard client see a new protocol.

The Turso-backed local store direction is in place. Turso is not Iceberg parlance and should not be proposed as an Iceberg feature. The Iceberg idea is the current metadata location and the atomic update of that location. LakeCat’s Turso store is the local durable implementation behind that behavior: projects, warehouses, namespaces, tables, views, storage profiles, policy bindings, idempotency records, pointer logs, audit rows, outbox rows, and soft-delete state. The reusable lesson is the store contract: atomic CAS, exact replay, drift rejection, redacted conflicts, durable pointer history, and transactional event emission.

Iceberg REST-compatible table and namespace paths exist. This is the standard compatibility surface. A PySpark workflow should be able to create a namespace, create or load a table, write data, and commit new Iceberg metadata through `/catalog/v1` without knowing that QueryGraph, TypeSec, Grust, OpenLineage, or QGLake exist. LakeCat may record evidence and emit events behind that path, but the client should still experience normal Iceberg catalog behavior.

Because namespace routes are standard Iceberg surface area, LakeCat treats the durable namespace row as part of the compatibility contract. A Turso row selected as warehouse `local` and namespace path `default` must decode to that same namespace before LakeCat lists it, loads it, or drops it. That row/content check is not an Iceberg extension; it is LakeCat’s local-store guard that keeps standard namespace responses and later QueryGraph bootstrap proof from trusting spliced durable JSON.

Commit CAS, idempotency, pointer logs, audit/outbox, and replay validation are heavily hardened. The standard Iceberg concept is the optimistic pointer update: advance the current metadata location only when the expected requirements still hold. LakeCat keeps that behavior and surrounds it with production catalog discipline. Idempotency makes retries exact or conflicting. Pointer logs record accepted movement. Audit records authority. The outbox ties graph and lineage delivery to committed catalog state. Replay validation refuses malformed durable evidence before acknowledgement, graph projection, or OpenLineage projection. Only the optimistic

commit is standard Iceberg; the proof envelope is LakeCat hardening and a possible source for optional future profiles.

Governed scan and credential paths carry TypeSec-style receipt evidence. A standard Iceberg scan is engine work over schemas, field ids, partition specs, snapshots, manifests, metrics, data files, and delete files. LakeCat adds a governed control-plane prelude: identify the principal and purpose, ask TypeSec for a decision, derive allowed columns, mandatory predicates, TTL caps, policy hashes, and raw-credential posture, then ask Sail to plan the effective request. Credential vending follows the same philosophy. Raw credentials are a deliberate audited exception; untrusted or restricted principals should receive Sail-planned work instead of broad object-store power. That is a LakeCat/TypeSec extension around a normal Iceberg table.

QueryGraph and QGLake handoff surfaces are broad by design. LakeCat can export bootstrap, management, view, credential, commit-history, OpenLineage, and replay proof material so QueryGraph can import governed catalog truth. That handoff should include catalog facts and proof anchors: warehouses, namespaces, tables, views, current pointers, commit sequences, view receipt chains, credential posture, governed scan restrictions, lineage hashes, and management inventory. QueryGraph owns the semantic meaning: Croissant, CDIF, OSI, ODRL application alignment, graph import, and agent workflows. The handoff is an optional LakeCat/QueryGraph extension, not a replacement for Iceberg REST.

Management roots need the same evidence discipline. Server and warehouse upserts may contain operationally sensitive endpoint URLs or storage roots. New LakeCat producers persist those values as hash evidence in audit/outbox payloads, and replay admission requires `endpoint-url-hash` or `storage-root-hash` whenever raw endpoint or root material is present in older durable events. That behavior is not Iceberg parlance. It is LakeCat's redaction and replay contract for management state that will later feed Grust, OpenLineage, and QueryGraph.

The durable management rows themselves also need row/content proof. Iceberg does not define servers, projects, or the QueryGraph tenant spine; those are LakeCat/QueryGraph control-plane concepts. But once LakeCat uses those rows to bootstrap QueryGraph or prove management inventory, a row selected as `server-a`, `project-a`, or `warehouse local` must not decode into a different tenant root. Turso therefore binds decoded `server`, `project`, and `warehouse record_json` back to the selecting row identity before returning lists, warehouse loads, or project warehouse inventories.

Policy roots need content proof too. Listing a policy binding proves that a policy id was visible, but it does not prove which ODRL material the catalog recorded for that id. LakeCat therefore treats `policy-binding.upserted` as a separate evidence event: the producer records an `odrl-hash` over the captured ODRL policy document, service replay admission requires that hash to match the captured policy material, raw lineage replay must carry the same policy id and ODRL hash, and compact QGLake `managementProof.policyUpsertProof` preserves the policy id, `odrlHash`, principal subject/kind, authorization receipt hash, the `policy-manage` action, graph event count, replay hash, and OpenLineage hash. The Turso store also binds decoded `binding_json` back to the row/query warehouse and policy id before listing policies or matching policies for a table,

so a durable row for one policy cannot carry ODRL evidence for another policy id and still feed governed scans or QGLake proof. That is not standard Iceberg. It is LakeCat/TypeSec/QueryGraph governance evidence around a standard catalog that happens to serve Iceberg tables.

Storage-profile roots now follow the same row/content proof rule. Iceberg clients still see ordinary table metadata locations and catalog credentials, but LakeCat's Turso store refuses to list a storage profile or match a table to a credential root unless the decoded `profile_json` agrees with the selected warehouse row and profile id. A durable row for one credential root therefore cannot carry the location prefix, issuance mode, provider, or secret-reference posture from another profile and still become QGLake proof.

1.4.18 Four Workflows, One Catalog Boundary

The cleanest way to see the boundary is to follow a request through the system. The catalog participates in every workflow, but it does not play the same role in every workflow. In a standard engine workflow, LakeCat should disappear behind normal Iceberg REST behavior. In a governed workflow, LakeCat should become the proof-producing control point. In a QueryGraph workflow, LakeCat should become the source of replayable catalog truth.

A PySpark writer uses LakeCat as an ordinary Iceberg REST catalog:

1. PySpark resolves the configured catalog endpoint and namespace.
2. LakeCat serves the Iceberg REST namespace and table path under `/catalog/v1`.
3. PySpark writes new Iceberg metadata and asks the catalog to commit it.
4. LakeCat checks the expected metadata pointer and requirements.
5. The store performs compare-and-swap on the current metadata location.
6. LakeCat records idempotency, pointer-log, audit, and outbox evidence.
7. PySpark receives an ordinary Iceberg-compatible response.

The standard Iceberg portion is namespace/table resolution, metadata loading, and optimistic commit. The LakeCat implementation portion is Rust routing, Turso-backed persistence, idempotency, pointer logs, audit, and outbox. The future proposal candidates are not "PySpark should learn LakeCat"; they are small optional profiles such as idempotent commit replay, pointer-history inspection, and redacted conflict proof. A PySpark user should not have to know whether QueryGraph, TypeSec, Grust, QGLake, or OpenLineage exist.

A governed analyst or service account uses the same catalog boundary, but the request has more proof attached:

1. The client asks to read a table for a stated purpose.
2. LakeCat identifies the principal, warehouse, namespace, table, requested columns, and requested filters.
3. TypeSec evaluates the capability and returns an authorization receipt.
4. LakeCat turns the receipt into an effective read restriction: allowed columns, mandatory predicates, TTL caps, policy hashes, and credential posture.
5. LakeCat asks Sail to plan the effective Iceberg scan.

6. Sail binds field ids, interprets schemas and partition specs, prunes using manifest metrics, accounts for delete files, and returns task evidence.
7. LakeCat records receipt, restriction, plan, audit, and outbox evidence.

The standard Iceberg portion is the table metadata that makes planning possible. The LakeCat/TypeSec extension is the receipt proving why the request was narrowed. The Sail portion is the engine interpretation that makes the restriction real against data, rather than just text in a catalog. A future Iceberg-adjacent proposal could define a proof-carrying scan-planning profile, but the profile should be optional and should not require TypeSec or QueryGraph as specific implementations.

A credential workflow is the same trust problem with a sharper edge. A catalog that vends raw object-store credentials gives the caller broad power. LakeCat therefore treats raw credential vending as an audited exception:

1. The client requests access material for a table or warehouse scope.
2. LakeCat resolves the storage profile and principal.
3. TypeSec decides whether raw credentials are allowed for that principal, purpose, scope, and TTL.
4. If raw credentials are allowed, LakeCat records the credential posture, secret-reference evidence, TTL cap, authorization receipt, and redacted proof.
5. If raw credentials are not appropriate, LakeCat steers the caller toward a governed Sail-planned read instead of object-store authority.
6. Replay validation later rejects malformed credential posture before graph or OpenLineage delivery.

Catalog-mediated credential vending exists in the Iceberg ecosystem, but LakeCat's proof language is additive. The strongest portable idea is not "standardize LakeCat's secret store." It is an optional governed credential profile that can prove whether credentials were issued, denied, narrowed, or replaced by planned work.

A QueryGraph bootstrap workflow is broader still:

1. QueryGraph asks LakeCat for bootstrap or QGLake handoff material.
2. LakeCat drains only replay-validated catalog events.
3. The handoff includes namespaces, tables, views, current metadata pointers, commit-history proof, view receipt chains, management inventory, credential posture, governed scan proof, OpenLineage hashes, and graph anchors.
4. Grust owns graph schema, projection mechanics, storage, traversal, and Cypher behavior.
5. TypeSec owns policy meaning, TypeDID context, capabilities, secure-agent semantics, and ODRL interpretation.
6. QueryGraph composes Croissant, CDIF, OSI, ODRL application meaning, OpenLineage, QGLake acceptance, and agent workflows from those inputs.

None of that makes QueryGraph a required Iceberg client path. It is a semantic application workflow above the catalog. LakeCat's job is to make sure the catalog facts QueryGraph imports are tied to committed state, replay validation, and durable proof. The pieces that may generalize

are smaller than QueryGraph: catalog event identity, lineage receipt binding, view lifecycle proof, credential proof, and pointer-history profiles.

1.4.19 Catalog Concepts As A Contract

The easiest mistake to make with LakeCat is to call every useful surface an Iceberg extension. That is too broad. LakeCat should use stricter language because different audiences need different guarantees. A Spark user wants to know whether the catalog is a normal Iceberg REST catalog. An operator wants to know whether commits, retries, lineage, and credentials are replayable. A QueryGraph user wants to know whether semantic graph import is bound to real catalog state. A standards discussion wants to know which ideas are mature enough to become optional, interoperable profiles.

The Rust service/catalog spine exists as an implementation of the catalog contract, not as a proposed Iceberg feature. Iceberg says what the catalog must do for tables: resolve namespaces and identifiers, return the current metadata location, accept compatible commits, and preserve the table-format contract. It does not say the catalog should be Rust, Java, a managed service, or an embedded process. LakeCat chooses Rust because the catalog transaction is becoming richer than a pointer lookup. A single request can carry a principal, warehouse, namespace, table, expected pointer, idempotency key, policy receipt, Sail validation, audit row, outbox row, and redacted proof. Keeping that in one typed Rust path reduces adapter drift and gives QueryGraph a foundation whose evidence was produced at the same place as the state transition.

The Turso-backed local store direction is also implementation, not Iceberg parlance. Iceberg needs atomic catalog state. LakeCat needs a durable local spine while the system is embedded, testable, and easy to run. The Rust turso crate gives LakeCat a local transactional database behind the portable CatalogStore trait. The important catalog behavior is not “Turso is the database.” The important behavior is that namespaces, tables, views, storage profiles, policy bindings, pointer logs, idempotency rows, audit rows, and outbox rows can be updated with the same atomicity as the metadata-pointer decision. If LakeCat later gains another durable store, the user-facing contract should remain the same: exact idempotency replay, CAS conflicts on drift, durable pointer history, and transactionally emitted side effects.

Iceberg REST-compatible table and namespace paths are standard Iceberg parlance. These are the compatibility line. A client should be able to create a namespace, create or load a table, commit a metadata update, and discover the current table state through `/catalog/v1` without knowing that QueryGraph, TypeSec, Grust, OpenLineage, ODRL, or QGLake exist. LakeCat may attach audit and outbox evidence behind that request, but those attachments cannot become hidden prerequisites for ordinary table access. The standard response must remain familiar enough that engines can keep their normal Iceberg behavior. That compatibility rule should be read literally. If a PySpark job creates `analytics.events`, the standard Iceberg concept is the namespace `analytics`, the table identifier `analytics.events`, the current metadata location, the metadata JSON, the snapshots, manifests, partition specs, schemas, and delete files that an engine reads. LakeCat may additionally know the warehouse’s tenant, the caller’s principal, the idempotency key hash, the pointer-log sequence, the audit receipt, and the outbox event id. Those extra facts are catalog evidence. They do not redefine the Iceberg table.

Commit CAS is standard catalog behavior; LakeCat's proof envelope around it is catalog hardening. The Iceberg idea is optimistic pointer movement: the writer prepared new metadata, the catalog checked the requirements, and the pointer advanced only if the requirements still held. LakeCat adds production discipline around that moment. Idempotency rows make retries exact or conflicting. Pointer logs preserve accepted movement. Audit rows preserve the principal and action. Outbox rows make graph and lineage delivery a durable consequence of committed catalog state. Replay validators reject malformed evidence before acknowledgement, Grust projection, or OpenLineage projection. Only the optimistic pointer update is standard Iceberg. The surrounding ledger is LakeCat implementation today and a possible source of future optional REST catalog profiles. This split is important for retries. The standard catalog question is whether the expected pointer can move to the new metadata pointer. LakeCat's stronger question is whether a retry is exactly the same request, whether the stored response hash matches, whether the pointer log already proves the transition, whether the audit row names the same principal and action, and whether any queued graph or OpenLineage side effect can be replayed without inventing new evidence. That retry and replay envelope is not table metadata. It is the control-plane contract that makes the catalog trustworthy after failures.

Governed scan and credential paths are LakeCat/TypeSec extensions around standard Iceberg tables. Standard Iceberg gives engines the metadata needed to plan scans: schemas, field ids, partition specs, snapshots, manifests, metrics, data files, and delete files. LakeCat adds a governed prelude for principals and agents: ask TypeSec for a decision, derive allowed columns, mandatory predicates, purpose, TTL, policy hashes, and credential posture, then ask Sail to plan or validate the effective request. A trusted human may receive audited raw credentials when policy allows that exception. A restricted agent should receive a Sail-planned task set instead of broad object-store authority. That is not standard Iceberg today, but it is a strong future Iceberg-adjacent candidate if expressed as an optional proof-carrying access profile. The same distinction applies to credential roots and storage profiles. Iceberg clients care that a table can be read and written through compatible catalog and object-store behavior. LakeCat additionally records which storage profile selected the credential root, which provider and issuance mode were configured, which redacted storage-scope hash was used, whether a secret reference was present, and which TypeSec authorization receipt allowed management of that profile. Those fields are LakeCat/TypeSec governance evidence. They are useful for QueryGraph and operators, but they should never become required custom Iceberg table metadata.

QueryGraph and QGLake handoff are application integration surfaces, not table format features. LakeCat can export bootstrap, management, view, credential, commit-history, replay, OpenLineage, and import proof because QueryGraph needs to build semantic workflows from catalog truth. QueryGraph owns Croissant, CDIF, OSI, ODRL application alignment, Grust-backed graph import, QGLake acceptance, and agent-facing reasoning. LakeCat should provide stable evidence anchors and replay validators, but it should not make QueryGraph semantics part of standard table access. Some pieces may generalize, such as catalog event identity, lineage receipt hashes, view lifecycle proof, and pointer-history profiles. QueryGraph's semantic graph itself should stay above Iceberg.

That gives LakeCat a conservative standards position:

1. Implementation details such as Rust, Turso, local release gates, and exact process layout are not Iceberg extensions.
2. Standard Iceberg concepts such as namespaces, table identifiers, metadata locations, manifests, snapshots, delete files, and optimistic commits remain standard and should be preserved exactly.
3. LakeCat control-plane APIs such as management inventory, replay verification, pointer-history inspection, OpenLineage projection, and QueryGraph bootstrap are optional extensions beside the standard path.
4. TypeSec-backed authorization receipts, ODRL-derived restrictions, credential posture, Type-DID envelopes, and secure-agent proof are governance extensions around catalog actions, not custom table metadata.
5. Future Iceberg proposals should be small optional profiles proven by repeated interoperability need: idempotent commit replay, catalog event streams, redacted conflict proof, governed credential proof, proof-carrying scan planning, pointer-history inspection, and view lifecycle proof.

The rule is simple enough to use during development. If a feature changes what a normal Iceberg client must know to read or write an ordinary table, it is suspect. If it adds replayable proof beside the standard path and can be ignored by clients that do not need it, it is a LakeCat extension. If several engines and catalogs would benefit from the same optional proof shape, then it may be a future Iceberg-adjacent proposal. Until then, the table remains Iceberg, the governance proof remains catalog-adjacent, and QueryGraph remains the integration layer above the catalog.

The following matrix is the practical reading guide for the current release shape. It deliberately separates the user-facing concept from the standards bucket and the LakeCat implementation state.

Concept	Standard Iceberg par- lance?	LakeCat/Query- Graph/TypeSec meaning	Future Iceberg-adj- acent candidate?
Rust service and cat- alog spine	No. Iceberg does not prescribe implementa- tion language or process shape.	LakeCat uses a typed Rust service so iden- tity, tenancy, REST routing, CAS, idem- potency, audit, out- box, and replay vali- dation happen in one coherent path.	No. The implementa- tion language should remain a deployment choice.
Turso-backed local store	No. Iceberg needs atomic catalog state, not a specific embed- ded database.	Turso is LakeCat's durable local im- plementation behind CatalogStore, cover- ing catalog state, pointer logs, au- dit, idempotency, and outbox rows.	No. The portable idea is the store contract, not Turso itself.
Namespace and table REST paths	Yes. These are the compatibility surface engines expect.	LakeCat serves them under /catalog/v1 and records evidence behind the scenes without making or- dinary clients learn QueryGraph.	Mostly already stan- dard. LakeCat should follow the Iceberg REST contract rather than propose a com- peting path.
Commit CAS	Yes. Optimistic pointer movement is central to Iceberg cat- alog behavior.	LakeCat hardens CAS with exact idempotency, pointer logs, redacted con- flicts, audit, and transactional outbox records.	Yes, but only the sur- rounding proof and retry profile, not the CAS idea itself.
Idempotency records	Partly. Retry safety is operationally nec- essary, but concrete cross-catalog profiles are not yet the table format.	LakeCat stores re- quest and response hashes so exact re- tries replay and drifted retries fail without duplicating side effects.	Strong candidate for an optional REST cat- alog profile.
Pointer logs and exceptions	Partly. Iceberg meta- data has history; cat- alog pointer history is a control-plane view. language.	LakeCat records ac- cepted pointer move- ment with compact hashes, sequence, ev- idence, principal/ac- tion proof, and idem- potency profile.	Strong candidate for optional catalog event or pointer his- tory profiles. Highly portable libraries should remain

This table matters because it keeps the release honest. LakeCat can be more ambitious than a passive catalog without pretending every ambitious piece is an Iceberg feature. It also protects users from accidental lock-in. A PySpark writer can stay in the standard column. A platform operator can use the LakeCat evidence column. A standards conversation can mine the fourth column for small, optional, interoperable proposals.

1.4.20 Extensions, Proposals, And Product Boundaries

The phrase “Iceberg extension” should be used sparingly. It should mean an additive behavior that ordinary Iceberg clients can ignore and that other catalogs or engines could implement without adopting QueryGraph. LakeCat has many useful surfaces, but only some deserve that label.

A LakeCat implementation choice is not an Iceberg extension. Rust is an implementation choice. Turso is a local durability choice. The exact crate layout, release gate, fixture script, and replay test harness are LakeCat engineering decisions. They matter because they make the catalog reliable, but they are not standards proposals.

A LakeCat control-plane extension is an additive catalog surface beside the standard REST path. Pointer-history inspection, management inventory, compact replay verification, storage-profile posture, credential-root evidence, and outbox delivery state fit this category. They should be versioned, documented, and safe for operators and agents, but standard Iceberg table access must not depend on them.

A TypeSec governance extension is a proof that an action was authorized and narrowed. Authorization receipt hashes, capability decisions, TypeDID context, ODRL-derived restrictions, policy hashes, purpose, credential TTL caps, and raw-credential exception evidence live here. They are attached to catalog actions and governed reads. They should never become custom required fields in Iceberg table metadata.

A QueryGraph integration extension is a handoff for semantic applications. Croissant, CDIF, OSI, ODRL application semantics, Grust graph import, QGLake acceptance, agent workflow proof, and QueryGraph bootstrap belong above LakeCat. LakeCat can emit stable catalog facts and proof anchors; QueryGraph interprets them as a semantic graph and agent workflow.

A future Iceberg-adjacent proposal should be smaller than LakeCat. The best candidate proposals are not “QueryGraph support” or “TypeSec support.” They are narrow interoperable profiles that multiple catalogs and engines could use: idempotency-key replay, redacted commit-conflict proof, pointer-history reads, transactional catalog event streams, lineage receipt binding, governed credential-vending proof, proof-carrying scan planning, and view lifecycle receipts. LakeCat should prove those ideas locally first, then propose the portable subset only after it is clear which fields need to interoperate. Even then, the proposal should name behavior, not products. “A catalog event stream carries ordered, replayable, redacted commit receipts” is a plausible portable profile. “An Iceberg catalog must emit QueryGraph bootstrap bundles” is not. “A credential-vending response can carry governed-access proof” may become portable. “An Iceberg table metadata file must contain TypeDID or ODRL fields” would be the wrong layer. This restraint lets LakeCat be a proving ground without turning every local integration into a standards demand.

This gives each current release claim a clear home:

Claim	Standard today	LakeCat/Query-Graph/TypeSec to-day	Proposal posture
Rust service/catalog spine exists	Not a standard concept.	A LakeCat implementation choice that keeps REST routing, identity, tenancy, CAS, idempotency, audit, outbox, and replay validation in one typed service path.	Not a proposal. Other catalogs should choose their own implementation language.
Turso-backed local store direction is in place	Not a standard concept.	A local durable CatalogStore implementation for metadata pointers, pointer logs, idempotency, audit, outbox, policy bindings, storage profiles, and views.	The proposal-worthy part is the store contract, not Turso.
Iceberg REST-compatible table and namespace paths exist	Yes.	LakeCat serves the standard catalog path and records evidence behind it.	Follow Iceberg REST; do not invent a competing table path.
Commit CAS is hardened	CAS is standard catalog behavior.	Idempotency, request/response hashes, pointer logs, audit, transactional outbox, replay validation, and redacted conflict evidence surround the standard pointer update.	Strong candidate for optional REST retry, conflict-proof, event, and pointer-history profiles.
Governed scan receipts exist	Scan planning metadata is standard; receipt proof is not.	TypeSec decides the capability, LakeCat binds the decision to the catalog action, and Sail plans the narrowed request.	Candidate for optional proof-carrying scan and governed-access profiles.
Governed credential proof exists Storage/Query/Graph/DDL LakeCat proof is replayable	Credential vending exists in catalog ecosystems; management is not standard. LakeCat's proof language is additive. mat parlance.	Raw credentials are an audited exception; LakeCat proof exports restricted agents bootstrap profiles; receive Sail-planned metadata view, standard work and compact record decompose, replay, and receipt evidence. OpenLineage-referenced proof, Query-	Candidate for optional governed credential-vending, provided as credential identity for shape might-handling should be considered for broader profiles.

The boundary can be tested with a simple question: can a normal Iceberg engine ignore this surface and still read or write the table correctly? If the answer is no, the feature is either standard Iceberg work or a compatibility problem. If the answer is yes, the feature may be a LakeCat extension. If the same extension would be useful outside LakeCat and QueryGraph, it may become a future Iceberg-adjacent profile.

The same distinctions show up in ordinary workflows.

A PySpark workflow should look conventional. The user configures an Iceberg REST catalog endpoint, creates a namespace, creates or writes a table, and commits a new metadata file. LakeCat resolves the namespace and table, checks the expected metadata pointer, advances it atomically when requirements hold, and returns an ordinary catalog response. Behind that response LakeCat can persist an idempotency row, pointer log, audit row, and outbox event. Those records help operators and QueryGraph, but the PySpark job should not need to know they exist.

A Rust engine workflow should lean harder into Sail. A Rust client or local service path can ask LakeCat to validate a commit, plan a scan, or fetch governed tasks. LakeCat should not parse every manifest and delete file itself. It should assemble the request context, ask TypeSec for the policy decision when needed, call Sail for Iceberg-aware validation or planning, and persist only the compact evidence that proves what happened. That gives Rust users a fast local path while avoiding a second Iceberg implementation inside the catalog.

An operator workflow uses LakeCat as a replayable control plane. The operator can inspect pointer history, idempotency outcomes, redacted conflicts, view proof, management inventory, credential posture, and delivery state. These are not required for standard Iceberg reads, but they are what make a catalog operable under incident pressure. If an outbox event cannot be projected to OpenLineage or Grust, LakeCat keeps the event pending. If replay evidence is malformed, LakeCat rejects it before acknowledgement. That is the difference between “we emitted a best-effort notification” and “catalog truth has a durable, replayable integration record.”

A governed-agent workflow uses the catalog as a narrowing point rather than a credential dispenser. The agent presents identity, purpose, and requested data. TypeSec decides the capability and records a receipt. LakeCat binds that receipt to the catalog action, checks that replay evidence still matches the event type, and asks Sail to produce the effective scan or task plan. The agent receives bounded work: allowed columns, required predicates, TTL, credential posture, and plan evidence. This is the path that makes raw credentials an audited exception instead of the normal answer.

A QueryGraph workflow uses LakeCat as the foundation for semantic import. The graph should not guess whether a table exists, which pointer is current, which principal committed a change, or whether a view lifecycle proof is internally consistent. LakeCat exports bootstrap and QGLake handoff material after replay validation. Grust owns graph storage and traversal. QueryGraph owns Croissant, CDIF, OSI, ODRL application meaning, agent workflows, and user-facing reasoning. LakeCat remains the source of catalog truth; QueryGraph becomes the semantic system that can trust that truth.

1.4.21 Why Sail Should Carry The Heavy Work

The catalog should be close to the engine, but it should not become the engine. That is the most important architectural line in LakeCat. Iceberg correctness is not just a pointer comparison. Real correctness involves field ids, schema evolution, partition transforms, manifest metrics, lower and upper bounds, delete-file association, sequence numbers, snapshot selection, sort orders, metadata tables, v3 row lineage, and v4 metadata trees. Those are engine-shaped responsibilities.

The deeper argument is that a catalog sees intent while an engine sees data. LakeCat sees “principal X wants table Y at snapshot Z under policy P.” Sail can see the schemas, field ids, manifests, delete files, metrics, partition transforms, object locations, and execution representation that determine what that request actually means. A governed catalog that tries to answer data-shaped questions without the engine will drift toward approximation. A governed catalog that asks the engine for the data-shaped answer can persist compact proof of a real plan.

If LakeCat implements those details locally, it becomes a second partial Iceberg engine. A manifest-metric bug would need one fix in Sail and another fix in LakeCat. A delete-file planning bug could be fixed for execution while the catalog still produced stale governed proof. A future v4 metadata-tree model would have to be parsed twice. That is not thin; it is duplicated semantics with a smaller test surface.

Sail is a strong engine choice because it is Rust-native, close to Arrow and DataFusion, and already shaped around generated Iceberg REST models, catalog provider seams, table-status conversion, manifest expansion, scan planning, write plumbing, and format-version checks. LakeCat can call Sail with typed request state, receive typed plan or validation evidence, and persist compact hashes and counts. That keeps ordinary REST clients portable while giving governed paths stronger proof.

This also makes Sail a good home for Iceberg v4 compatibility. V4 table features should not arrive in LakeCat as an expanding set of catalog-side JSON special cases. LakeCat can carry a JSON passthrough bridge when compatibility requires it, but the durable target is typed Sail support for the format details. V4 metadata trees, row-lineage-aware planning, richer delete handling, metadata-as-data, and future validation rules are engine work. LakeCat should store the current pointer, call the engine for interpretation, and preserve the receipt that proves which engine decision was applied.

The split should remain simple:

- Sail owns Iceberg semantics, planning, pruning, delete handling, metadata-as-data, commit requirement validation, and v3/v4 table-format interpretation.
- LakeCat owns identity, tenancy, metadata pointers, CAS, idempotency, audit, outbox, replay validation, and optional catalog proof.
- TypeSec owns authorization semantics, capability composition, TypeDID envelopes, ODRL meaning, secure-agent posture, and credential decisions.
- Grust owns graph taxonomy, projection mechanics, storage, traversal, and Cypher.
- QueryGraph owns semantic application import, OSI/Croissant/CDIF/ODRL alignment, QGLake acceptance, and end-to-end agent workflows.

This is why pushing work into Sail is not only a performance optimization. It is a compatibility strategy. A PySpark job can keep using standard Iceberg REST. A governed agent can receive a Sail-planned task set instead of raw storage credentials. QueryGraph can bootstrap from LakeCat evidence whose table facts were interpreted by the same engine path that future Rust workflows will use. The standard path stays portable, and the advanced path gets stronger because it is built on real engine semantics rather than catalog-side approximation. It is also a governance strategy. Policy proof is only as strong as the data semantics it constrains. A receipt saying “allowed columns were narrowed” is weak if the narrowing was performed by catalog string matching that ignores field ids or schema evolution. A receipt saying “Sail planned this narrowed scan against the current Iceberg metadata” is much stronger. The policy remains TypeSec’s responsibility, but the interpretation of columns, predicates, manifests, deletes, and snapshots should come from the engine that understands the table.

Sail is a particularly good fit for this split because the hard problems are already engine-shaped. Manifest pruning depends on Iceberg metrics, null counts, lower and upper bounds, field ids, partition transforms, and delete semantics. Commit validation depends on requirements, current metadata, snapshot ancestry, sequence numbers, and format-version behavior. Metadata tables and v4 metadata trees are naturally queried and planned as data, not as ad hoc catalog strings. Sail sits close to Arrow/DataFusion execution and Rust Iceberg model code, so it can turn those details into reusable typed APIs instead of one-off LakeCat validators.

That gives LakeCat a better failure model. If Sail does not yet expose a typed operation, LakeCat can use a narrow JSON passthrough as a compatibility bridge and record that bridge explicitly. The long-term target remains typed Sail support. When a manifest metric decoder, delete planner, metadata-table reader, or v4 parser improves in Sail, LakeCat and QueryGraph both benefit without a catalog rewrite. The engine learns more about the data; the catalog persists better proof about what the engine decided.

Pushing work into Sail also keeps performance honest. The fastest catalog is not the one that re-parses every metadata file in a control-plane transaction. The fast path is a catalog that guards identity, tenancy, pointer state, and durable evidence, then hands data-shaped questions to an engine built for columnar metadata, pruning, statistics, and execution. LakeCat should be fast because it is thin where it should be thin, strict where it must be strict, and directly connected to a Rust engine that can do the expensive work without adapter indirection.

This is not only about CPU time. It is also about where correctness can be tested once. Manifest metrics, field-id projection, equality deletes, position deletes, sequence numbers, snapshot selection, metadata tables, row lineage, and v4 metadata trees all interact. If LakeCat implements a small local version of each one, every improvement in Sail has to be mirrored by a catalog patch, and every mismatch becomes a governance bug. A TypeSec receipt that says “column access was narrowed” is only strong if the narrowing follows Iceberg field ids and schema evolution. A QGLake proof that says “this scan was planned” is only strong if delete files, manifests, and snapshot state were interpreted by the same engine path that will execute or expose the work.

Sail is a strong default because it gives LakeCat a Rust-native engine boundary instead of an adapter maze. LakeCat can assemble the catalog context, TypeSec can decide the security context, and Sail can answer the data question:

1. Which snapshot and metadata tree are current for this request?
2. Which schema fields and field ids survive the requested and policy-derived projection?
3. Which partition specs, manifest metrics, lower and upper bounds, and null counts can prune the plan?
4. Which equality or position deletes must be honored?
5. Which scan tasks, metadata-as-data rows, or validation results should be exposed back to the catalog as compact proof?

That division gives LakeCat a better long-term v4 posture. JSON passthrough is a compatibility bridge when a model is ahead of the local typed surface. It is not the design center. The durable design center is typed Sail support for the new Iceberg semantics, with LakeCat storing only the pointer state, request context, authorization receipt, plan hash, and replayable evidence that says which Sail decision was used.

The same split makes the proof stronger. A replay artifact that says “the policy allowed columns A and B” is useful. A replay artifact that also proves Sail planned the scan with field-id-aware projection, predicate narrowing, delete-file accounting, manifest pruning, and format-version validation is much stronger. QueryGraph can then import a catalog fact with confidence that the fact was not merely text supplied by a control plane. It was the compact trace of an engine decision made against Iceberg metadata.

That is why the LakeCat release should keep moving reusable work upstream into Sail: manifest metric decoding, delete planning, scan-task generation, metadata-table reads, table-status conversion, commit requirement validation, and typed v4 interpretation. LakeCat should expose the governed catalog contract; Sail should make that contract true against the data.

1.5 Standard Terms And LakeCat Terms

LakeCat is easiest to understand when the words are separated into three layers: standard Iceberg vocabulary, LakeCat implementation choices, and QueryGraph/TypeSec control-plane vocabulary. Mixing those layers is where catalog designs become confusing. LakeCat deliberately keeps the ordinary Iceberg words ordinary, then adds stronger evidence and governance around them.

The following concept map is the working contract. Each entry separates the standard Iceberg meaning from LakeCat’s implementation and from the QueryGraph/TypeSec extension surface.

Catalog: In standard Iceberg, a catalog resolves table identifiers, namespaces, and current table metadata locations. It is the control point that lets engines load table metadata and make optimistic updates without agreeing on one execution engine. In LakeCat, the catalog is also a Rust service spine under `/catalog/v1` with management and QueryGraph surfaces beside it. That spine owns tenancy, request identity, durable catalog state, and replayable evidence. The Rust implementation is not an Iceberg extension. It is how LakeCat makes a standard catalog reliable enough to become QueryGraph’s foundation.

Namespace: In standard Iceberg, a namespace groups tables and views. In LakeCat, a namespace is also a governed resource under a warehouse. Namespace creation, listing, and lifecycle events can be authorized, audited, replay-validated, and projected as graph anchors. The grouping is standard Iceberg parlance. The authorization receipt, outbox record, and QueryGraph graph anchor are optional control-plane evidence.

Table identifier: In standard Iceberg, a table identifier is the catalog-relative table name, usually namespace plus table name. In LakeCat, that same identity becomes the root key for store rows, audit events, outbox messages, TypeSec contexts, and QueryGraph handoff bundles. The name is standard. The durable evidence envelope around it is LakeCat.

Current metadata location: In standard Iceberg, the current metadata location is the pointer to the active Iceberg table metadata JSON file. In LakeCat, that pointer is the central compare-and-swap value in the store, with pointer-log history and redacted hash evidence. Pointer history is useful enough to be a future Iceberg REST management extension, but the table itself is still defined by the standard metadata pointer.

Table metadata: In standard Iceberg, table metadata is the JSON metadata file containing schemas, snapshots, partition specs, sort orders, properties, and related table state. LakeCat stores that file in object storage and keeps it pristine. It validates and references the metadata, but it does not use custom business fields to carry policy, graph, lineage, or agent state. QueryGraph derives semantic and governance facts beside the metadata, not inside it.

Snapshots, manifest lists, manifests, data files, and delete files: These are standard Iceberg metadata layers used by engines to plan reads and validate table state. LakeCat delegates their interpretation to Sail for scan planning, manifest expansion, pruning, delete-file handling, and metadata-as-data work. They are engine responsibilities. Pushing them into Sail avoids a second partial Iceberg engine inside the catalog.

Iceberg REST paths: The standard Iceberg REST catalog path is the compatibility boundary. LakeCat serves table and namespace operations under `/catalog/v1` so ordinary engines can create namespaces, load tables, commit metadata updates, and use compatible scan or credential flows without learning QueryGraph. Management paths such as `/management/v1` and bootstrap paths such as `/querygraph/v1/bootstrap` are LakeCat/QueryGraph additions beside the standard surface. They must never become prerequisites for ordinary table access.

Commit: In standard Iceberg, a commit is an optimistic update that advances the current metadata pointer when requirements still hold. In LakeCat, that standard commit is wrapped in a catalog transaction: request normalization, TypeSec receipt capture, Sail validation, create-only metadata writes, compare-and-swap, idempotency records, audit, pointer logs, and outbox events. The optimistic commit is standard. The receipt, pointer-log, and outbox evidence are LakeCat extensions around the standard commit.

Compare-and-swap: Compare-and-swap is the catalog-side atomicity rule behind optimistic Iceberg commits. A writer can advance the pointer only if the current metadata location still matches the expected state and the update requirements remain valid. In LakeCat, CAS is hardened by the store contract and by audit-safe conflict evidence: failed races expose hashes and structured error

classes rather than raw storage paths. CAS itself is standard catalog behavior. LakeCat's redacted proof envelope is an implementation and governance extension.

Idempotency: Idempotency is the retry discipline that prevents duplicate effects. LakeCat makes it a hardened store contract: exact replay returns the stored response, different bodies under the same key conflict, and replay cannot emit duplicate outbox events. Iceberg can benefit from stronger cross-catalog idempotency conventions, but LakeCat treats its concrete key rules as catalog implementation behavior today and as a possible future REST profile rather than a table-format change.

Pointer log: A pointer log is LakeCat's compact history of accepted metadata-pointer movement. Iceberg tables already have snapshot and metadata history, but a catalog pointer log answers a different operational question: which catalog transaction advanced which pointer under which principal, request hash, response hash, policy hash, and idempotency key hash? This is not standard Iceberg today. It is an optional management surface and a strong candidate for future Iceberg REST or OpenLineage-adjacent catalog event conventions.

Audit: Audit is not an Iceberg table-format concept. LakeCat writes audit records for governed catalog actions so operators can reconstruct who did what, which authority was used, and which redacted evidence was captured. Audit belongs to the catalog control plane. It should stay outside Iceberg metadata, because a portable table should not be forced to carry one deployment's governance log.

Outbox: The outbox is LakeCat's transactional delivery buffer for committed catalog facts. LakeCat writes outbox events with catalog transactions, validates replay evidence before delivery, projects to Grust and OpenLineage, and acknowledges only after projection succeeds. The pattern is not standard Iceberg, but it is one of LakeCat's most important catalog extensions. It is also one of the best places to propose future interoperability: event identity, replay ordering, lineage binding, and redaction shape could become common catalog-event language without changing table metadata.

Replay validation: Replay validation is LakeCat's rule that durable evidence must be internally consistent before it can be acknowledged, projected to graph, or emitted as lineage. For example, governed scan events must preserve matching read-restriction evidence in the top-level payload and the TypeSec receipt, and commit events must carry full request, response, idempotency, principal, and optional policy hashes. This is LakeCat control-plane hardening. The future standardization question is not whether Iceberg should require LakeCat's exact validators, but whether catalogs should share proof-carrying event profiles.

Credential vending: Iceberg REST catalogs may provide storage credentials or access material for table operations. LakeCat makes that path fail closed. Raw credentials are an audited exception; governed Sail-planned reads are the default for agents and untrusted principals. TypeSec receipts and credential-root proofs are LakeCat/TypeSec extensions. They are candidates for future governed-access conventions, not current Iceberg requirements.

Governed scan: Standard engines use Iceberg metadata to produce file tasks, apply projection, prune manifests and files, and account for deletes. LakeCat asks TypeSec for restrictions, passes

effective projection, mandatory filters, purpose, and TTL constraints to Sail, records receipt evidence, and returns compatible plan/task shapes. Governed planning evidence is a LakeCat/TypeSec extension. The underlying pruning and delete semantics belong in Sail and the Iceberg engine layer.

Management surfaces: LakeCat management APIs expose warehouses, storage profiles, policy bindings, commit logs, view receipt chains, credential-root posture, and operational state. Iceberg REST does not standardize all of those surfaces today. Some are implementation administration APIs. Some are QueryGraph handoff APIs. Some, especially commit-history, credential, and event replay profiles, may be worth proposing as optional Iceberg REST management extensions.

View proof: Iceberg has view concepts and REST view endpoints, but LakeCat adds proof surfaces around view lifecycle, list evidence, and receipt chains so QueryGraph can verify that a view import corresponds to governed catalog state. The view itself should remain standard where the standard applies. The receipt chain is LakeCat/QueryGraph evidence beside it.

OpenLineage projection: OpenLineage is not Iceberg, but it is a natural consumer of catalog events. LakeCat projects committed namespace, table, scan, credential, view, and management events into OpenLineage from the durable outbox. That makes lineage reflect committed catalog state instead of handler-side best effort. This is an extension around Iceberg and a likely interoperability point, not a replacement for Iceberg metadata.

Graph projection: Graph projection is not an Iceberg table-format feature. LakeCat emits catalog-facing graph facts only at the boundary; graph taxonomy, storage, traversal, and query behavior live in Grust. QueryGraph builds the semantic graph from these anchors. The graph is an extension around Iceberg, not an alternative to Iceberg.

Policy receipt: Policy receipts are not an Iceberg table-format feature. LakeCat persists TypeSec-style authorization receipts and checks replay evidence before admitting outbox delivery. This belongs to TypeSec and governed catalog protocols. It may inspire future Iceberg governance extensions, but it should not be mandatory for ordinary table access.

Bootstrap bundle: A QueryGraph bootstrap bundle is not part of standard Iceberg. It is a handoff contract that packages catalog, table, namespace, view, lineage, management, credential, and commit proof surfaces. This is QueryGraph-specific integration. Standard clients should never need it.

Rust service spine: LakeCat's Rust service/catalog spine is an implementation choice with architectural consequences. It lets one process own request identity, Iceberg-compatible routing, Turso-backed catalog state, Sail calls, TypeSec receipts, and outbox projection without turning every boundary into a remote adapter. This is not a proposed Iceberg feature. The feature that might matter to Iceberg is the behavior it enables: deterministic commits, replayable catalog events, and engine-close governed planning.

Turso-backed local store: The Turso-backed store is LakeCat's durable local spine. It persists projects, warehouses, namespaces, tables, storage profiles, pointer logs, idempotency records, audit rows, and outbox rows through the Rust turso crate. This is not Iceberg parlance and not

an Iceberg extension. It is LakeCat’s chosen local implementation of the catalog-state contract, kept behind CatalogStore so the higher-level catalog behavior remains portable.

This separation answers an important design question: are LakeCat’s additions Iceberg extensions, future Iceberg features, or something else?

Some additions are not extensions at all. The Rust service spine, Turso-backed local store, normalized idempotency table, pointer log, and replay validators are implementation choices behind standard catalog behavior. A Spark, Trino, Flink, or PyIceberg client does not need to know that LakeCat uses Rust, Turso, or a particular hash discipline to make a commit safe. Those details matter because they make the catalog reliable, portable, and inspectable, but they do not change what an Iceberg table is.

Some additions are catalog extensions. QueryGraph bootstrap, management replay, credential-root proofs, view proof chains, commit-history inspection, and OpenLineage projection are useful APIs beside the Iceberg REST surface. They should remain optional. A standard Iceberg client should be able to ignore them and still create, load, update, and read tables through the normal catalog paths.

Some additions are future Iceberg-adjacent candidates. The community may eventually want common language for idempotent commit replay, governed credential vending, catalog event streams, lineage receipts, policy-bound scan planning, or proof-carrying table/view management operations. LakeCat should be a good laboratory for those ideas, but it should not force them into Iceberg metadata or make them prerequisites for ordinary compatibility. The right shape is additive: standard REST stays stable, advanced governance evidence is discoverable, and engines that do not understand it keep working.

The clean line is this: implementation details make LakeCat reliable, optional extensions make QueryGraph rich, and future proposals should be phrased as additive catalog profiles rather than mandatory table-format changes. Iceberg metadata remains the shared table truth. LakeCat’s proof surfaces explain how a catalog transaction, scan plan, credential decision, or semantic handoff happened around that table truth.

That is also why LakeCat is careful with the phrase “Iceberg v4 compatible.” For the catalog, compatibility means preserving the standard contract while being ready for newer format metadata and REST models as Sail exposes typed support. It does not mean guessing future semantics in LakeCat or stuffing custom control-plane state into table metadata. JSON passthrough can keep the catalog tolerant during transition, but typed behavior should move into Sail as soon as Sail has the reusable model.

1.5.1 The Concept Status Matrix

The following matrix is the safest way to explain LakeCat without accidentally turning implementation details into table-format claims. It names the concept, classifies it, explains what exists in LakeCat, and says whether it is a future Iceberg candidate.

Concept	Standard Iceberg?	LakeCat/Query-Graph status	Future Iceberg candidate?
Catalog	Yes. The catalog resolves namespaces, table identifiers, metadata locations, and commits.	LakeCat implements the catalog as a Rust service spine and adds management, audit, outbox, and QueryGraph handoff beside the standard REST surface.	The service implementation is not a candidate; stronger optional catalog proof profiles are.
Namespace and table paths	Yes. REST namespace/table routes are the compatibility boundary.	LakeCat serves Iceberg REST-compatible namespace and table paths under /catalog/v1, with warehouse-aware tenancy and standard responses.	No proposal needed for the base path; optional management discovery may be useful later.
Current metadata location	Yes. It is the active table metadata pointer.	LakeCat stores and advances it through CatalogStore CAS, records pointer movement, and exposes redacted proof.	Pointer-history inspection is a good optional catalog-management candidate.
Table metadata, snapshots, manifests, delete files	Yes. They are the portable Iceberg table truth.	LakeCat keeps them pristine and asks Sail to interpret them for planning, table status, pruning, delete association, and metadata-as-data.	v4 behavior belongs in Iceberg and Sail. LakeCat should not invent it.
Rust service/catalog spine	No. Iceberg does not prescribe implementation language.	LakeCat uses Rust to keep routing, identity, Turso state, Sail calls, TypeSec receipts, and outbox admission in one typed path.	Not a feature proposal. The reusable lesson is deterministic proof surfaces, not Rust itself.
Turso-backed local GC, store, QGLake handoff	No. Iceberg does not prescribe a catalog database. 63 not TypeSec receipt proof.	LakeCat uses the Rust turso crate behind CatalogStore for durable local projects, warehouses, tables, idempotency, and pointer logs. 64	Not a feature proposal. Atomic CAS and idempotent replay behavior may be applicable to table metadata not be

The table also clarifies release language. “Implemented in LakeCat” does not mean “standard Iceberg.” “Useful to QueryGraph” does not mean “mandatory for every Iceberg engine.” “A good future proposal” does not mean “LakeCat should force it today.” The standard path must remain boring, portable, and familiar. The advanced path can be richer because it is additive.

1.5.2 How To Describe The Concepts To Different Readers

LakeCat needs to explain the same machinery to several audiences without changing the architecture for each one. The language should shift, but the boundary should not.

For a standard Iceberg reader: LakeCat is an Iceberg REST-compatible catalog. It resolves namespaces and tables, returns current metadata locations, and advances metadata pointers with optimistic commit requirements. The Rust implementation, Turso store, audit tables, outbox, TypeSec receipts, and QueryGraph bundles are not required to understand a normal table. They are catalog-side machinery around a standard table.

For an operator: LakeCat is a durable catalog control plane. The important words are CAS, idempotency, pointer log, audit, outbox, replay validation, redaction, and local release gates. Those words explain why a retry does not duplicate a commit, why a race is rejected, why graph and lineage are emitted from committed state, and why replayed evidence cannot be malformed without being stopped before delivery.

For a governed-agent designer: LakeCat is the gate between intention and data access. The important words are principal, purpose, TypeSec receipt, ODRL-derived restriction, allowed columns, mandatory predicate, TTL cap, credential posture, and Sail-planned read. The catalog should make the narrow governed path easier than the broad credential path. Raw credentials are exceptional; planned work is the default for restricted principals.

For a QueryGraph reader: LakeCat is the trusted substrate for semantic import. The important words are bootstrap bundle, QGLake handoff, OpenLineage receipt, graph anchor, management inventory, view proof, credential proof, commit-history proof, and replay agreement. QueryGraph should receive catalog facts that are already tied to committed state and validated receipts, then compose them with Croissant, CDIF, OSI, ODRL, and Grust graph semantics.

For a standards reader: LakeCat is a laboratory for optional catalog profiles, not a request to change the table format. The likely candidates are idempotent commit replay, pointer-history inspection, redacted conflict proof, transactional catalog event streams, governed credential-vending proof, proof-carrying scan planning, view lifecycle proof, and lineage receipt binding. The non-candidates are implementation details: Rust, Turso, crate layout, LakeCat’s exact internal tables, and QueryGraph application semantics.

The following rule keeps the explanation honest:

Claim	What to say	What not to say
Rust service spine exists	LakeCat implements the Iceberg catalog boundary in Rust so request identity, state transition, Sail calls, receipts, audit, and outbox evidence stay in one typed control path.	Rust is an Iceberg feature.
Turso-backed store exists	LakeCat uses the Rust turso crate behind CatalogStore for durable local catalog state and replayable tests.	Iceberg should standardize on Turso.
REST namespace/table paths exist	LakeCat speaks the standard catalog language for normal clients and keeps optional management/Query-Graph APIs beside it.	A client must use Query-Graph to read an Iceberg table.
Commit CAS is hardened	Optimistic pointer movement is standard; idempotency, pointer logs, audit, outbox, and replay validation harden the catalog transaction.	LakeCat's proof rows are part of Iceberg table metadata.
Governed scans are receipt-backed	TypeSec narrows the request and Sail plans the effective scan against Iceberg metadata.	The catalog can safely approximate all engine planning locally.
Credential paths are governed	Raw credential vending is an audited exception; restricted agents should receive Sail-planned work.	Credential proof is mandatory for every Iceberg client today.
QGLake handoff is broad	LakeCat exports optional proof surfaces so Query-Graph can import governed catalog truth.	QueryGraph semantics are future Iceberg table-format semantics.

That table is intentionally conservative. It lets LakeCat be ambitious in the control plane while remaining ordinary at the Iceberg boundary.

1.5.3 Why The Engine Should Carry The Heavy Work

The most important technical argument in LakeCat is not simply that Rust is fast. It is that the system should avoid making the catalog a second, incomplete Iceberg engine. Iceberg's hard work

lives in metadata interpretation: schema ids, field ids, partition specs, manifest metrics, delete files, sequence numbers, snapshot selection, sort orders, row lineage, and newer metadata trees. A catalog can remember the current pointer, but it should not independently become the authority on every detail described by that pointer.

Pushing this work into Sail gives LakeCat one reusable engine truth. Sail is the place where Iceberg metadata can be parsed, validated, planned, and exposed as execution- and metadata-as-data structures. LakeCat can then own the trust boundary: principal, warehouse, namespace, table, policy, idempotency, pointer CAS, audit, outbox, replay validation, and handoff proof. The split is precise: Sail answers table-format questions, LakeCat answers catalog-transaction questions.

A PySpark workflow shows why the split matters. Spark should use LakeCat like a normal Iceberg REST catalog. It creates a namespace, loads a table, commits new metadata, and reads the current pointer. LakeCat should not require Spark to understand QueryGraph or TypeSec. But when the commit touches Iceberg metadata, LakeCat should lean on Sail-shaped validation and response models rather than duplicate them locally. Spark sees a standard catalog. LakeCat records durable evidence. Sail remains the reusable table-format implementation.

A governed agent workflow shows the same principle from the opposite side. The agent should not receive broad object-store credentials when policy only allows a narrow task. LakeCat resolves the agent and asks TypeSec for a decision. TypeSec returns a receipt with allowed columns, a mandatory predicate, purpose, TTL, and raw-credential posture. LakeCat then asks Sail to plan the effective request against the current Iceberg metadata. Sail binds fields, prunes manifests, respects delete files, and returns task evidence. LakeCat records the proof and can later replay it into QueryGraph and OpenLineage. The agent receives a governed plan, not a private table semantics invented in the catalog.

This is why Sail is a particularly strong engine choice for LakeCat:

1. It is Rust-native, so LakeCat can call the engine path directly and keep typed evidence close to the catalog transaction.
2. It sits near Arrow and DataFusion, so table planning can produce structures that are natural for execution, metadata-as-data, and future QueryGraph workflows.
3. It already has Iceberg REST models, catalog/provider seams, table-status conversion, manifest/planning paths, and format-version checks that LakeCat should reuse rather than fork.
4. It lets fixes land once. A manifest-metric decoding fix, delete-file association fix, or v4 metadata-tree implementation should benefit LakeCat, QueryGraph, and other Rust lakehouse tools together.
5. It keeps LakeCat standard at the boundary. LakeCat can answer normal REST clients while still producing richer governed proof for clients that opt in.

The design rule follows directly. If a feature needs Iceberg metadata semantics, move it toward Sail. If it needs catalog atomicity and durable evidence, keep it in LakeCat. If it needs graph projection mechanics, move it to Grust. If it needs authorization semantics, move it to TypeSec. If it needs semantic application workflow, let QueryGraph compose it. That is how LakeCat can

be foundational without becoming a warehouse, graph database, policy engine, and planner all at once.

1.6 What Exists Today And What It Means

LakeCat is not only a design sketch. The current implementation already has a Rust service/catalog spine, a Turso-backed local store direction, Iceberg REST-compatible namespace and table paths, hardened commit and replay evidence, governed scan and credential proof, and broad QueryGraph/QGLake handoff surfaces. Those pieces are easy to over-describe as “Iceberg extensions,” but that phrase hides the most important distinction: some pieces are ordinary catalog implementation, some are optional LakeCat/QueryGraph APIs, some are TypeSec governance proof, and only some are plausible future standardization candidates.

The safest release language is therefore claim by claim, not slogan by slogan. When LakeCat says the Rust service/catalog spine exists, it is not claiming a new Iceberg feature. It is saying the standard catalog boundary now has a Rust control plane underneath it. When LakeCat says Turso-backed local storage is in place, it is not proposing Turso for Iceberg. It is saying the local catalog has a durable transaction spine for pointer state, idempotency, audit, and outbox. When LakeCat says Iceberg REST-compatible table and namespace paths exist, that is standard Iceberg catalog language. When LakeCat says commit CAS, pointer logs, replay validation, governed scan receipts, credential proof, and QGLake handoff exist, those are additional proof and integration surfaces around the standard table. They are valuable because they are beside Iceberg, not because they replace Iceberg.

That distinction matters for how the work should be proposed upstream. A future Iceberg proposal should not be “LakeCat’s architecture.” It should be a small, optional, interoperable profile that multiple catalogs and engines can adopt. For example, “a catalog MAY expose an idempotent commit-replay profile with these conflict and response-hash fields” is a better proposal shape than “a catalog MUST implement LakeCat’s store.” “A catalog event stream MAY bind a committed table operation to a lineage receipt hash” is better than “QueryGraph bootstrap is Iceberg.” “A governed credential response MAY carry proof that raw credentials were denied or narrowed” is better than “TypeSec receipts are part of every Iceberg table.” LakeCat can be an implementation laboratory, but the portable Iceberg contribution should be the smallest shared behavior that solves a real interoperability problem.

The following release ledger is the rule this book uses when it describes the current system.

Release claim	What standard Iceberg already says	What LakeCat has built	Extension or future proposal?
Rust service/catalog spine exists	Iceberg requires compatible catalog behavior, not a language or framework.	LakeCat binds REST routing, identity, tenancy, Sail calls, TypeSec receipts, Turso state, audit, outbox, and replay admission in one Rust service path.	Implementation detail. The reusable future idea is proof-carrying catalog behavior, not Rust itself.
Turso-backed local store direction is in place	Iceberg needs durable catalog state and atomic metadata-pointer updates, but does not choose a database.	LakeCat uses the Rust turso crate behind CatalogStore for local warehouses, namespaces, tables, views, pointer logs, idempotency, audit, and outbox rows.	Implementation detail. Atomic CAS, idempotent replay, redacted conflicts, and pointer-history profiles may be future candidates.
Iceberg REST-compatible table and namespace paths exist	Namespaces, table identifiers, load-table, create-table, and commit routes are core REST catalog compatibility.	LakeCat serves those paths under /catalog/v1 while recording optional evidence behind the scenes.	Standard Iceberg surface. LakeCat-specific management and bootstrap routes stay additive.
Commit CAS is hardened	Optimistic pointer advancement is standard catalog behavior.	LakeCat adds create-only metadata writes, exact idempotency replay, drift conflicts, redacted proof, audit rows, pointer logs, and transactional outbox emission.	CAS is standard. The hardening envelope is LakeCat implementation and a candidate optional REST profile.
Idempotency is durable	Iceberg clients need retry safety, but table metadata does not define exact idempotency-key behavior.	LakeCat stores request/response hashes, returns exact replay, rejects drift under the same key, and prevents duplicate graph or lineage side effects.	LakeCat implementation today; strong future optional catalog profile candidate.
Outbox log/receipts @LakeToSailReceiptEnvelope	Iceberg defines table metadata, QueryGraph, and other catalog artifacts, but does not specify receipt handling or replay logic.	LakeCat adds a typed outbox log, receipt ID, and replay logic to handle drift and ensure consistency across retries.	Optional extension. LakeCat could expose a managed replay API or standardized receipt format for future catalogs.

The most important word in that table is “beside.” LakeCat should put new capability beside the Iceberg REST path, not in front of it. A standard client should never have to present a TypeDID, parse a QueryGraph bundle, understand a Grust graph edge, or inspect an OpenLineage receipt before it can load a normal table. That is the compatibility promise. At the same time, a governed platform cannot stop at the standard path. It needs to know whether the commit was retried safely, whether the pointer moved under the same principal that authorized it, whether a credential response was narrowed or denied, whether a scan was planned against the current metadata, and whether the evidence later imported by QueryGraph came from committed catalog state. LakeCat’s additions live in that second space.

This gives each workflow a different view of the same catalog:

- A PySpark user sees ordinary Iceberg. The user configures a REST catalog, creates a namespace, writes a table, and later loads the table. LakeCat may persist pointer-log, audit, and outbox rows, but those rows are invisible to the user’s table semantics.
- A platform operator sees a hardened catalog transaction log. The operator can inspect idempotency outcomes, pointer movement, redacted conflict proof, storage-profile posture, credential-root state, pending outbox delivery, and replay-validation failures without reading Iceberg metadata by hand.
- A governed agent sees a narrowed access path. The agent asks for data, TypeSec decides the capability, LakeCat binds that receipt to the catalog action, Sail plans the effective Iceberg scan, and the agent receives bounded work rather than broad storage credentials.
- A QueryGraph importer sees a proof-bearing bootstrap. The importer can verify that table, view, management, credential, scan, commit-history, OpenLineage, and graph-import anchors line up before accepting the semantic layer.
- A standards discussion sees candidate profiles. The interesting portable pieces are not “LakeCat uses Turso” or “QueryGraph imports a bundle.” They are smaller shapes such as idempotent commit replay, catalog pointer history, governed credential vending, proof-carrying scan planning, catalog event streams, redacted conflict evidence, and lineage receipt binding.

That same separation keeps the v4 story disciplined. An Iceberg v4-compatible LakeCat should not mean “LakeCat locally guesses every future table-format semantic.” It should mean the catalog preserves compatibility at the REST boundary, keeps metadata pristine, tolerates newer metadata while typed support is landing, and pushes reusable interpretation into Sail. A JSON passthrough can be a bridge when the model is ahead of the local type system. The final home for v4 semantics should be Sail, because v4 metadata trees, row lineage, delete behavior, manifest interpretation, and metadata-as-data access are engine concerns. LakeCat should record which typed Sail decision it relied on, not become the second place where those semantics are reimplemented.

The standards posture is therefore additive and evidence-driven:

1. Keep the base Iceberg REST behavior strict and boring.
2. Keep LakeCat proof surfaces optional for clients that need them.
3. Keep TypeSec and QueryGraph semantics out of Iceberg metadata.
4. Use Sail for reusable table-format and planning semantics.

5. Promote only small, interoperable proof profiles once real workflows prove that multiple engines and catalogs need them.

That posture is conservative in the right way. It protects standard Iceberg users from LakeCat-specific lock-in, while giving QueryGraph enough trusted evidence to build agentic and semantic workflows over the same tables.

This classification also keeps the social contract clear for users. A PySpark or PyIceberg user should not have to read this whole ledger before doing normal Iceberg work. They should configure a catalog, create a namespace, write a table, and load it later. The advanced proof path is there when the user is an operator, lineage system, governed agent, or QueryGraph importer. The standard path should feel ordinary. The optional path should be explicit and verifiable.

The harder engineering argument is where to put the work. LakeCat should own catalog authority, not table-format interpretation. Iceberg scan correctness depends on details that belong to an engine: field ids rather than column names, schema evolution, partition transforms, manifest metrics, lower and upper bounds, equality and position deletes, sequence numbers, snapshot selection, metadata tables, row lineage, and future v4 metadata trees. If a catalog reimplements those details, it becomes a smaller, less-tested engine. The danger is not only performance. The danger is semantic split-brain: Sail could fix a delete-file association bug while LakeCat's local approximation keeps emitting stale governed scan proof.

Sail is the right place for that heavy work because it is Rust-native and already sits on the execution side of the lakehouse. It can turn Iceberg metadata into typed planning structures, Arrow/DataFusion-friendly execution inputs, metadata-as-data views, and future v4-aware table state. LakeCat can then persist compact evidence about the Sail decision: which snapshot was planned, which files or tasks were exposed, which restrictions were applied, which delete semantics were honored, and which hashes bind the plan to the catalog request. The catalog records trust. The engine interprets the table.

This makes governed workflows cleaner. In a PySpark workflow, the user stays on the standard REST path and LakeCat behaves like a normal catalog while recording durable proof. In an agentic workflow, LakeCat resolves the principal, TypeSec narrows the request, Sail plans the effective Iceberg scan, and QueryGraph later imports the resulting evidence. In a management workflow, LakeCat can show pointer history and replay proof without pretending pointer history is table metadata. In a lineage workflow, OpenLineage receives committed catalog facts from the outbox rather than best-effort handler observations. In all four workflows, Sail prevents the catalog from becoming a shadow engine, and LakeCat prevents the engine from becoming the governance system.

The Rust service/catalog spine exists today. Its job is to receive standard catalog traffic, normalize request identity, bind that request to warehouse and table state, call the local store, call Sail, call TypeSec, and write durable audit/outbox evidence. None of that is standard Iceberg vocabulary. Iceberg does not care whether a catalog is implemented in Rust, Java, Go, or as a managed cloud service. It cares that clients can create namespaces, load table metadata, commit compatible updates, and receive compatible responses. The Rust spine is therefore an implementation choice,

not an Iceberg extension. It matters because it makes LakeCat fast enough and direct enough to be a QueryGraph foundation: request identity, commit CAS, policy receipts, Sail planning, and outbox projection can be kept in one strongly typed transaction path instead of being scattered across adapters.

The Turso-backed local store direction is in place. Turso is not part of Iceberg parlance either. It is LakeCat's durable local catalog database behind the portable CatalogStore trait. The standard Iceberg concept is the current metadata location and the optimistic update of that location. LakeCat's Turso store persists the things a catalog must remember around that pointer: warehouses, namespaces, table records, storage profiles, idempotency rows, pointer logs, audit rows, outbox rows, view records, policy bindings, and soft-delete state. The future-facing idea is not "Iceberg should use Turso." The future-facing idea is that Iceberg REST catalogs may benefit from clearer profiles for durable idempotency, pointer-history inspection, and event replay. Turso is LakeCat's Rust-local implementation of those behaviors.

Iceberg REST-compatible table and namespace paths exist. This is the standard surface. A normal client should be able to reach the catalog through /catalog/v1, create and list namespaces, load tables, commit table changes, and use compatible credential or scan flows without understanding QueryGraph. That compatibility boundary is non-negotiable. LakeCat-specific paths such as management APIs, QueryGraph bootstrap, QGLake handoff, replay verification, and proof inspection live beside the standard surface. They are optional control plane APIs. They can be useful to QueryGraph, operators, agents, and lineage systems, but they must not become hidden requirements for ordinary table access.

Commit CAS, idempotency, pointer logs, audit/outbox, and replay validation are heavily hardened in LakeCat. The standard Iceberg idea is simple and powerful: a commit advances the current metadata pointer only if requirements still hold. LakeCat keeps that idea, then surrounds it with production catalog discipline. CAS is performed by the store. Idempotency rejects different bodies under the same key and replays exact stored responses without duplicating side effects. Pointer logs preserve accepted pointer movement. Audit records preserve who and what authorized the operation. The outbox keeps graph and OpenLineage delivery transactional with catalog state. Replay validation checks that durable event evidence still agrees with the receipt, table identity, hash shape, counts, policy restrictions, credential posture, and principal before any projection is acknowledged. In Iceberg terms, only the optimistic pointer update is standard. The surrounding evidence system is LakeCat's catalog hardening. The standard candidate is an optional catalog-event or management profile, not a change to Iceberg metadata files.

Governed scan and credential paths carry substantial TypeSec-style receipt evidence. Standard Iceberg metadata gives engines enough information to plan files, prune manifests, and handle deletes. Standard REST catalogs can also vend credentials. LakeCat adds a governed path for principals, agents, and policy-bearing requests: it asks TypeSec for a decision, turns that decision into an effective read restriction, passes the narrowed request to Sail, records the receipt, and validates replay before graph or OpenLineage projection. Credential vending follows the same posture. Raw credentials are a trusted, audited exception; untrusted agents should receive Sail-planned work instead of broad storage power. This is not standard Iceberg today. It is a LakeCat/

TypeSec governance extension and a strong candidate for future governed-access conventions. If standardized, it should remain additive: clients that understand the proof can verify it, and clients that do not can still use the standard catalog path.

QueryGraph and QGLake handoff surfaces are broad. LakeCat can expose bootstrap, management, view, credential, commit-history, OpenLineage, and replay proof material so QueryGraph can import a governed catalog state rather than scrape a database or infer semantics from object paths. That handoff is intentionally not Iceberg itself. QueryGraph owns Croissant, CDIF, OSI, ODRL interpretation at the semantic application layer, graph import semantics, agent workflow meaning, and higher-level business vocabulary. LakeCat supplies the governed catalog facts and proof anchors: which namespace, which table, which current pointer, which commit sequence, which credential decision, which view receipt, which scan restriction, which lineage hashes. QueryGraph decides what those facts mean in a semantic application.

The clean classification is:

- Standard Iceberg parlance: catalog, namespace, table identifier, current metadata location, table metadata, snapshots, manifests, data files, delete files, optimistic commit requirements, and REST catalog compatibility.
- LakeCat implementation: Rust service spine, Turso local store, internal store traits, request normalization, durable idempotency, redaction rules, pointer-log rows, audit rows, outbox rows, replay validators, and local release gates.
- LakeCat optional catalog extensions: management APIs, pointer-history reads, view receipt chains, storage-profile management, replay verification, QueryGraph bootstrap generation, QGLake compact handoff, and OpenLineage projection.
- TypeSec governance extensions: authorization receipts, capability decisions, TypeDID/agent identity context, ODRL-derived read restrictions, credential-root proof, governed-read evidence, and raw-credential exception receipts.
- QueryGraph application extensions: Croissant/CDIF/OSI/ODRL semantic import, graph-model bootstrap, agentic workflow proof, business semantic alignment, and QueryGraph import verification.
- Future Iceberg-adjacent candidates: optional profiles for idempotent commit replay, catalog event streams, governed credential vending, proof-carrying scan planning, pointer-history inspection, view lifecycle proof, and lineage receipt binding.

The last category should be handled carefully. LakeCat should not try to declare a private future Iceberg. It should implement useful behavior, keep it cleanly additive, publish precise proof shapes, and learn from real QueryGraph/QGLake workflows. Good future proposals come from proven interoperability pressure: several engines, catalogs, lineage systems, and governance systems discovering that they need the same optional evidence. Until then, the standard table remains standard, the extension remains optional, and the implementation remains replaceable.

The practical test is whether a normal Iceberg client can still succeed while ignoring the extra surface. If Spark loads a table, reads the current metadata location, and commits a compatible metadata update through `/catalog/v1`, it is using standard Iceberg catalog behavior. If Query-

Graph later asks LakeCat for a bootstrap bundle containing graph anchors, OpenLineage receipts, view receipt chains, credential posture, and commit-history proof, it is using an optional LakeCat/QueryGraph extension. If an agent asks LakeCat for a governed plan and LakeCat asks TypeSec for a receipt before calling Sail, that is a governance extension around a standard table. None of those flows require LakeCat to add private fields to Iceberg metadata or require an ordinary client to understand QueryGraph semantics.

That distinction gives each concept a status:

1. **Required for Iceberg compatibility.** Namespaces, table identifiers, current metadata locations, table metadata, snapshots, manifests, delete files, optimistic commits, and REST-compatible routes must behave like Iceberg expects.
2. **Required for LakeCat reliability.** The Rust service spine, Turso store, CAS discipline, idempotency records, pointer logs, audit rows, outbox rows, replay validators, redaction rules, and local release gates make LakeCat a trustworthy implementation, but they are invisible to ordinary clients.
3. **Optional LakeCat/QueryGraph extensions.** Management routes, view proof, bootstrap bundles, QGLake compact handoff, OpenLineage projection, credential-root posture, and replay-verification endpoints enrich the catalog for operators and QueryGraph without changing table truth.
4. **Governance extensions owned with TypeSec.** Policy decisions, capability receipts, Type-DID context, ODRL-derived restrictions, secure-agent proof, and raw-credential exception evidence belong beside catalog operations, not inside table metadata.
5. **Possible future Iceberg proposals.** Only repeated interoperability need should turn LakeCat behavior into a proposal. Good candidates are optional catalog profiles for idempotent commit replay, pointer-history inspection, governed credential vending, catalog event streams, lineage receipt binding, and proof-carrying scan planning.

The word “extension” should therefore be used narrowly. Turso is not an Iceberg extension. Rust is not an Iceberg extension. A replay validator is not an Iceberg extension. Those are implementation details that make standard behavior stronger. QueryGraph bootstrap is an extension because it is a new surface beside Iceberg REST. Governed scan proof is an extension because it adds policy and receipt semantics around scan planning. A future Iceberg proposal should come only after those extensions prove useful enough that other catalogs and engines would benefit from a shared optional profile.

1.6.1 Standard Word, LakeCat Mechanism, Future Candidate

The distinction is easiest to keep honest by asking four questions for each concept:

1. Is this already Iceberg parlance?
2. Is this LakeCat’s implementation of a standard catalog obligation?
3. Is this an optional LakeCat, QueryGraph, or TypeSec surface beside Iceberg?
4. Is this mature enough to become a future Iceberg proposal?

The Rust service spine answers the second question. Iceberg says a catalog must resolve identifiers, serve metadata, and commit compatible updates. It does not say the catalog must be written

in a particular language. LakeCat chooses Rust so request identity, store transactions, Sail engine calls, TypeSec receipts, replay validation, and outbox projection can be expressed in one strongly typed service path. That is not an Iceberg extension. It is an implementation stance. The future Iceberg-relevant lesson is not “catalogs should be Rust,” but “catalog APIs should make it possible to produce deterministic proof for commits, scans, credentials, and events.”

The Turso-backed local store also answers the second question. Standard Iceberg needs atomic pointer movement and durable catalog state. LakeCat’s Turso store is how the local Rust catalog persists that state without reintroducing a separate SQLx/SQLite abstraction. The interesting general idea is the store contract: CAS must be atomic, idempotency must reject drift, pointer logs must be replayable, audit and outbox rows must be transactionally tied to catalog state, and redaction must be stable. Turso is LakeCat’s local durable spine. The possible future proposal is a catalog-state behavior profile, not a database choice.

Iceberg REST-compatible table and namespace paths answer the first question. They are standard catalog parlance. LakeCat’s `/catalog/v1` surface must let ordinary clients create namespaces, list namespaces, load tables, commit table metadata, and interact with compatible scan or credential routes. LakeCat can add management, bootstrap, replay, and handoff APIs beside that surface, but those additions must never become prerequisites for a normal table read or commit. If a PySpark or PyIceberg workflow succeeds by using only the standard catalog surface, compatibility is real. If it secretly depends on a QueryGraph-only route, compatibility has leaked.

Commit CAS is partly first-question and partly second-question. Optimistic catalog commits are central Iceberg behavior. LakeCat’s CAS hardening is the implementation of that behavior under production pressure: object writes are create-only, pointer movement is atomic, conflict evidence is redacted, request and response hashes are preserved, and idempotency prevents duplicate side effects. The future Iceberg-adjacent candidate is a shared way to express idempotent commit replay and pointer-history proof. It should not change the table metadata format. It should describe how a catalog can prove a compatible commit happened once.

Pointer logs, audit, outbox, and replay validation answer the third question. They are not ordinary Iceberg table terms, but they are natural catalog-control terms. A pointer log records accepted pointer movement. Audit records preserve authority and request evidence. The outbox records durable facts that must be projected to graph and lineage. Replay validation proves that the durable fact still matches identity, table state, hashes, counts, policy restrictions, credential posture, and principal shape before delivery. These are LakeCat extensions around a standard catalog. They are also plausible future interoperability candidates because every serious catalog eventually has to answer the same operational question: how do I prove which catalog fact I emitted, under which authority, and from which committed state?

Governed scan and credential proof answer the third and fourth questions. The standard Iceberg world already has metadata, manifests, delete files, scan planning, and catalog credential vending. LakeCat adds TypeSec-style proof: which principal asked, which capability or TypeDID context applied, which ODRL or policy rule produced the restriction, which columns and rows survived, what purpose and TTL were allowed, and whether raw credentials were blocked or audited. That

is not required for standard clients today. It is a governance extension. It may become a future Iceberg-adjacent profile only if the community wants a common governed-access shape that multiple catalogs and engines can verify.

QueryGraph and QGLake handoff answer the third question. They are intentionally application-facing. QueryGraph needs Croissant, CDIF, OSI, ODRL, OpenLineage, graph import, view proof, credential posture, management inventory, bootstrap artifacts, and commit-history proof. LakeCat should provide the catalog facts and evidence anchors. QueryGraph should compose the semantic graph and user workflow. Those handoff surfaces are optional LakeCat/QueryGraph extensions, not future mandatory Iceberg behavior. Parts of them may inform future standard profiles, especially lineage receipt binding and catalog event streams, but the QueryGraph semantic import itself belongs above the catalog.

This gives a simple reader rule. If a concept describes the table's portable metadata truth, call it Iceberg. If it describes how LakeCat makes that truth durable, atomic, redacted, and replayable, call it LakeCat implementation. If it describes policy receipts, secure agents, ODRL interpretation, or raw credential exceptions, call it TypeSec governance. If it describes semantic graph import, OSI/Croissant/CDIF alignment, or agentic application workflow, call it QueryGraph. Only after an optional surface proves useful across catalogs and engines should it be described as a future Iceberg proposal.

The same rule can be applied as a release ledger.

Rust service/catalog spine: This is LakeCat implementation. Standard Iceberg does not prescribe a runtime, language, process layout, or dependency graph. The standard concern is that the catalog can resolve namespaces and tables, return metadata, and commit compatible updates. LakeCat's Rust spine exists so the standard request can be bound to identity, tenancy, store CAS, Sail planning, TypeSec receipt capture, audit, outbox, and replay validation in one local control path. It is not an Iceberg extension. A future Iceberg proposal should not say "use Rust"; it could say that a catalog may expose deterministic commit, scan, credential, and event proof produced by whatever implementation it uses.

Turso-backed local store: This is LakeCat implementation behind a portable store contract. Iceberg parlance talks about a catalog, a current metadata location, and atomic pointer updates. It does not require one durable database. LakeCat uses the Rust turso crate for the local spine because projects, warehouses, storage profiles, namespaces, tables, idempotency records, pointer logs, audits, and outbox rows need durable local transactions. Turso is therefore not an Iceberg extension and not a future Iceberg feature. The candidate future feature is a behavioral profile for catalog-state durability: atomic CAS, idempotent retry, redacted conflict evidence, pointer-history reads, and transactional event emission.

Iceberg REST-compatible table and namespace paths: These are standard Iceberg catalog parlance. A Spark, Flink, Trino, PyIceberg, or PySpark workflow should be able to create namespaces, load tables, and commit metadata through the standard catalog path without knowing that QueryGraph exists. LakeCat's standard path is the compatibility promise. The LakeCat and QueryGraph

additions begin only where the client asks for management proof, replay verification, OpenLineage projection, governed scans, credential posture, or bootstrap bundles.

Commit CAS: This is standard catalog behavior with LakeCat hardening. Iceberg expects an optimistic commit to advance the metadata pointer only when the expected requirements still hold. LakeCat makes that atomic movement redacted, idempotent, audited, and replayable. The standard word is commit. The LakeCat mechanism is create-only metadata writes, store CAS, normalized idempotency, pointer-log evidence, audit rows, and outbox emission. The future candidate is an optional REST profile for idempotent commit replay and pointer-history proof.

Idempotency: This is LakeCat implementation today and a plausible future catalog profile. Iceberg clients retry commits, but the table format does not define every catalog's idempotency-key semantics. LakeCat treats idempotency as part of catalog safety: exact retry returns the stored response, drift under the same key conflicts, and replay does not duplicate graph or lineage side effects. It should not be written into table metadata. If standardized later, it should be an optional catalog behavior profile that ordinary clients can adopt without changing table files.

Pointer logs: This is LakeCat management evidence around standard pointer movement. Iceberg metadata already records table snapshots and metadata history, but a catalog pointer log records the catalog transaction that made one pointer current: principal, request hash, response hash, idempotency-key hash, optional policy hash, and sequence. That is not standard Iceberg parlance today. It is an optional LakeCat management surface and a strong future candidate for interoperable catalog-history inspection.

Audit, outbox, and replay validation: These are LakeCat control-plane extensions. Audit explains authority. Outbox ties committed catalog facts to graph and OpenLineage delivery. Replay validation refuses to acknowledge or project malformed durable evidence. None of this belongs inside Iceberg metadata. The future Iceberg-adjacent idea is an optional catalog event stream with stable event identity, redaction rules, replay ordering, and lineage binding.

Governed scan planning: This is a LakeCat/TypeSec governance extension around standard Iceberg scan planning. Standard Iceberg gives engines the metadata needed to plan files. LakeCat adds policy-derived restrictions before planning: allowed columns, mandatory predicates, purpose, TTL, principal, and receipt proof. Sail should perform the Iceberg planning because it understands schemas, field ids, manifests, metrics, deletes, and format evolution. A future proposal might define an optional proof-carrying scan profile, but the base table remains a normal Iceberg table.

Credential paths: Credential vending exists in catalog practice and Iceberg REST discussions, but LakeCat deliberately narrows it. Trusted principals can receive audited raw credential exceptions when policy allows them. Agents and fine-grained restricted principals should be steered to Sail-planned reads instead of broad object-store access. The standard idea is catalog-mediated access. The LakeCat/TypeSec addition is credential-root proof, block reason proof, and receipt evidence. The future candidate is a governed credential-vending profile that standardizes how catalogs prove why credentials were issued or withheld.

QueryGraph and QGLake handoff: This is not standard Iceberg. It is an optional LakeCat/Query-Graph integration surface over standard catalog facts. LakeCat exports the evidence anchors: namespaces, tables, views, current pointers, commit proof, scan proof, credential posture, management inventory, OpenLineage hashes, and replay verification. QueryGraph owns Croissant, CDIF, OSI, ODRL, semantic graph import, agent workflows, and user-facing reasoning. Some components, especially lineage receipt binding and catalog event streams, may become future interoperability profiles. The QueryGraph semantic bundle itself should remain an application extension.

1.7 A Field Guide To The Catalog Concepts

The first confusion in any advanced catalog design is vocabulary. Some words come from Iceberg. Some describe how a particular catalog is implemented. Some describe governance and semantic systems that sit beside the catalog. If those categories blur, the architecture becomes either too timid or too invasive. A timid catalog becomes a passive pointer map. An invasive catalog starts inventing private table semantics and breaks the ecosystem it is trying to serve.

LakeCat uses a stricter rule. A concept belongs to standard Iceberg when it is part of the portable table or REST catalog contract. A concept belongs to LakeCat implementation when it is how this Rust catalog satisfies that contract reliably. A concept is a LakeCat or QueryGraph extension when it adds optional control-plane value beside the standard path. A concept belongs to TypeSec when it describes authorization semantics, capability proof, secure-agent posture, or policy-derived restrictions. A concept becomes a possible future Iceberg proposal only after it has proven useful as an optional, interoperable profile that other catalogs and engines could adopt without changing the table format.

That gives each current LakeCat claim a precise status.

Rust service/catalog spine exists: This is LakeCat implementation. Iceberg does not prescribe Rust, async Rust, Axum, a crate layout, or an in-process planning path. It prescribes catalog behavior: resolve namespaces and tables, return metadata, enforce optimistic commit requirements, and speak compatible REST shapes. LakeCat chooses Rust because the catalog needs to keep request identity, tenancy, CAS, idempotency, Sail planning calls, TypeSec receipts, audit writes, outbox writes, and replay validation in a single strongly typed control path. That is a performance and correctness decision, not an Iceberg extension. The future Iceberg-adjacent lesson is behavioral: catalogs should be able to prove what they committed, planned, vended, and emitted, regardless of implementation language.

Turso-backed local store direction is in place: This is also LakeCat implementation. The Iceberg word is not “Turso”; the Iceberg words are catalog, table identifier, metadata location, and atomic commit. LakeCat uses the Rust turso crate as the durable local spine behind CatalogStore because the catalog needs local transactions for warehouses, storage profiles, namespaces, tables, views, idempotency records, pointer logs, audit rows, and outbox rows. Turso should not be proposed as an Iceberg feature. The portable part is the store contract: atomic compare-and-swap, exact idempotency replay, conflict rejection for drift, stable redaction, durable pointer history, audit evidence, and transactional event emission.

Iceberg REST-compatible table and namespace paths exist: This is standard Iceberg catalog parlance. It is the compatibility line. A Spark, PySpark, Trino, Flink, or PyIceberg client should be able to use the standard catalog surface to create namespaces, load tables, and commit metadata updates without knowing that QueryGraph exists. LakeCat can add management, bootstrap, replay, OpenLineage, graph, and credential-proof APIs beside that surface, but those APIs must not be hidden prerequisites for ordinary table access. If a client needs QueryGraph semantics to perform a normal Iceberg read, LakeCat has crossed the wrong boundary.

Commit CAS is heavily hardened: The standard Iceberg concept is the optimistic catalog commit. A writer has a base table state, writes new metadata, and asks the catalog to advance the current metadata pointer only if the expected requirements still hold. LakeCat keeps that behavior and hardens the implementation around it. The store does the atomic compare-and-swap. The metadata write path is create-only. The idempotency record ensures exact retry has one effect and drift has none. The pointer log records the accepted transition. Audit records authority. The outbox records the durable fact for graph and lineage. Replay validation checks that committed evidence still agrees before delivery. The Iceberg term is commit. The LakeCat terms are idempotency, pointer log, audit, outbox, replay validation, and redacted proof. A future Iceberg proposal could define an optional idempotent commit-replay profile, but it should not change the table metadata file.

Idempotency is a catalog reliability feature: Iceberg engines retry. Networks fail. Clients may not know whether a commit response was lost after the catalog accepted the pointer movement. LakeCat treats idempotency as a first-class safety contract. A repeated request under the same idempotency key and same body receives the stored response. A different body under the same key is rejected. A replayed success does not emit a second graph event, OpenLineage event, audit story, or pointer-log transition. This is not standard Iceberg table metadata. It is implementation behavior that could become an optional REST catalog convention.

Pointer logs are catalog history, not table history: Iceberg metadata already contains snapshots and metadata history. That history answers what the table says about itself. LakeCat's pointer log answers what the catalog did: which transaction advanced which pointer, under which principal, with which idempotency-key hash, request hash, response hash, policy hash, and sequence. Those two histories complement each other. The table history remains portable Iceberg metadata. The pointer log is LakeCat management evidence. A future optional catalog-history profile could standardize the shape without requiring every table metadata file to embed deployment-specific audit state.

Audit, outbox, and replay validation are hardened: These are LakeCat control-plane mechanisms. Audit records who or what acted. The outbox makes graph and OpenLineage side effects depend on committed catalog state rather than best-effort handler callbacks. Replay validation checks the durable payload before acknowledgement, including principal shape, table identity, hash form, counts, read restrictions, credential posture, management actor evidence, and receipt context. These mechanisms are intentionally outside Iceberg metadata. They are excellent candidates for future optional catalog event profiles because many catalogs need trustworthy event emission, but

the standard table should remain readable by clients that know nothing about LakeCat's replay validators.

Governed scan paths carry TypeSec-style receipt evidence: The standard Iceberg work is scan planning over metadata: bind expressions, apply projection, prune manifests and files, attach delete files, and produce tasks for execution. LakeCat adds a governed prelude. It identifies the principal and purpose, asks TypeSec for a decision, derives allowed columns, mandatory predicates, TTL caps, and raw-credential posture, and asks Sail to plan the effective request. LakeCat then records both the user request and the allowed request as receipt evidence. The scan remains a scan over a normal Iceberg table. The proof that the scan was narrowed by policy is a LakeCat/TypeSec governance extension. A future Iceberg-adjacent proposal could standardize proof-carrying scan planning, but only as an additive profile.

Credential paths are governed access decisions: A catalog may help clients obtain storage access. LakeCat narrows the meaning of that power. Trusted principals can receive audited raw credentials when policy allows it. Agents and restricted principals should normally receive Sail-planned work instead of broad object-store power. The TypeSec receipt explains whether raw credentials were allowed, blocked, or replaced by a governed plan, and LakeCat validates that evidence before projecting it to graph or OpenLineage. The standard idea is catalog-mediated access. The LakeCat/TypeSec addition is proof of why access was issued, withheld, or converted into a governed Sail plan.

QueryGraph and QGLake handoff surfaces are broad: These are optional LakeCat/QueryGraph extension surfaces. QueryGraph needs a rich bootstrap story: warehouses, namespaces, tables, views, current pointers, commit history, view receipt chains, credential posture, scan proof, OpenLineage receipts, management inventory, Croissant/CDIF/OSI/ODRL context, and replay-verification results. LakeCat should not make those concepts part of the Iceberg table. LakeCat should export catalog facts and proof anchors. Grust should own graph taxonomy, storage, traversal, and query behavior. TypeSec should own security semantics. QueryGraph should compose those facts into the semantic application graph and agent workflow. Some handoff pieces, especially lineage receipt binding and catalog event identity, may later inform standards work. The QueryGraph semantic bundle itself is an application extension.

OpenLineage projection is a consumer of catalog truth: OpenLineage is not Iceberg, but it becomes much more useful when it is emitted from the catalog's durable outbox. A table commit, governed scan, credential decision, view change, storage-profile update, or management mutation should not become lineage merely because a handler tried to emit an event. It should become lineage because the catalog transaction committed and the replay validator accepted the durable evidence. That is LakeCat's extension. A future interoperability profile could describe how catalog events bind to OpenLineage receipts, but ordinary Iceberg metadata should stay untouched.

Management and view proof surfaces are operational extensions: Warehouses, projects, storage profiles, policy bindings, servers, view receipt chains, soft-delete state, and compact management inventories are not the core portable Iceberg table format. They are the operational reality around a catalog. LakeCat exposes them as management and proof surfaces, validates their replay

payloads, and lets QueryGraph import them as governed catalog context. The future standards question is selective: view lifecycle proof, pointer-history inspection, and event-stream identity may deserve optional profiles. Project naming, local storage-profile rows, and the exact management schema are LakeCat implementation choices.

The most important architectural implication is that LakeCat should not turn standard compatibility into local reimplementations. Anything that understands Iceberg table structure belongs as far down in Sail as possible. Manifest metrics, delete-file association, schema and partition evolution, field-id binding, metadata-as-data, v3 row lineage, v4 metadata trees, branch and snapshot selection, and commit requirement validation are engine-shaped. LakeCat should initiate those actions from a governed catalog transaction, but Sail should perform the reusable table-format work.

That is why Sail is a strong engine choice. It is Rust-native, close to Arrow and DataFusion, already shaped around generated Iceberg REST models and table providers, and reusable by more than LakeCat. A governed PySpark workflow can still see a normal REST catalog while LakeCat validates and records the transaction. A governed agent can be denied raw credentials and receive a Sail-planned task set. A QueryGraph bootstrap can consume catalog evidence whose table facts were interpreted by the same engine path that will serve future Rust lakehouse execution. Sail lets LakeCat stay thin without staying blind.

The extension rule is therefore conservative:

1. Do not call implementation details Iceberg extensions.
2. Do not store LakeCat, QueryGraph, TypeSec, graph, or OpenLineage state inside Iceberg table metadata when it can live beside the table.
3. Keep optional LakeCat/QueryGraph APIs additive and discoverable.
4. Promote a feature toward an Iceberg proposal only when the behavior is useful across catalogs and engines and can be standardized as an optional profile.
5. Push reusable table-format work into Sail before adding a LakeCat-local parser, planner, or validator.

1.8 Reading The Current LakeCat Surface Precisely

The current LakeCat surface can sound broad because it touches standard cataloging, commit safety, policy, lineage, graph handoff, and agent access. The important thing is not to flatten those into one category. LakeCat is a catalog foundation, not a new table format. It is allowed to be strong around the catalog transaction, but it should remain humble about Iceberg table semantics and defer those semantics to Sail.

The release-facing vocabulary is:

- **Standard Iceberg parlance** names the portable table and catalog contract: catalog, namespace, table identifier, current metadata location, table metadata, snapshots, manifests, delete files, optimistic commit requirements, REST namespaces, REST tables, REST config, and compatible credential or scan responses.

- **LakeCat implementation** names how this Rust catalog satisfies and hardens that contract: the service spine, CatalogStore, Turso local persistence, request normalization, CAS, idempotency, pointer logs, audit records, outbox rows, redaction, replay validators, and local release gates.
- **LakeCat optional extensions** name additive catalog-control APIs beside the standard path: management inventory, commit-history inspection, view receipt chains, credential-root posture, replay verification, OpenLineage projection, and QueryGraph/QGLake bootstrap bundles.
- **TypeSec governance extensions** name security and policy proof: capabilities, TypeDID context, authorization receipts, secure-agent evidence, ODRL-derived restrictions, credential TTL caps, raw-credential exception receipts, and proof that a governed request was narrowed before planning.
- **QueryGraph application extensions** name semantic composition: Croissant, CDIF, OSI, ODRL application semantics, Grust-backed graph import, QGLake acceptance, agent workflow proof, and user-facing reasoning over the catalog facts.
- **Future Iceberg-adjacent candidates** are not private LakeCat standards. They are optional profiles that may be worth proposing only after repeated interoperability need: idempotent commit replay, pointer-history inspection, catalog event streams, governed credential vending, proof-carrying scan planning, view lifecycle proof, and lineage receipt binding.

This section walks the major current claims through that vocabulary.

The following ledger is the quick way to classify the current catalog concepts. It is intentionally strict: when a concept is standard Iceberg, the book uses Iceberg words; when it is LakeCat, QueryGraph, or TypeSec innovation, the book says so directly; when something could become standards work, the candidate is narrowed to the portable behavior rather than the LakeCat implementation.

Current concept	Standard Iceberg parlance	LakeCat/QG/TypeSec meaning	Extension or proposal	Why the work moves toward Sail
Rust service/catalog spine	Catalog service, REST routes, namespace and table identifier resolution, current metadata location, optimistic table commit.	LakeCat's service binds identity, tenancy, metadata-pointer state, TypeSec receipts, Sail calls, idempotency, audit, outbox, and replay admission in one typed control plane.	Rust is not an Iceberg extension. Proof-carrying catalog actions could become an optional profile if the proof shape is portable.	Sail should own any reusable interpretation of table metadata reached from that service, so the Rust catalog does not become a second engine.
Turso-backed local store	Durable catalog state and atomic current metadata pointer update. Iceberg does not prescribe the storage engine.	The turso crate backs LakeCat's local CatalogStore spine for warehouses, namespaces, tables, views, policies, storage profiles, idempotency, pointer logs, audit, outbox, and soft-delete state.	Turso is not an Iceberg extension. Atomic pointer proof, exact idempotent replay, and transactional event identity are possible future optional profiles.	The store records what happened; Sail should determine whether table-format facts such as manifests, deletes, snapshots, and field ids were interpreted correctly.
Iceberg REST-compatible namespace and table paths	REST config, namespaces, table create/load/drop where supported, and table commit through optimistic requirements.	LakeCat adds warehouse-aware routing, request identity, policy gates, audit, replay proof, and optional management surfaces behind or beside the standard path.	This is standard compatibility. Extra proof is optional catalog-control behavior, not a hidden dependency for Spark or PyIceberg.	Sail should provide generated models and table-status conversion where possible so the REST surface stays aligned with engine-facing Iceberg semantics.
Commit CAS, QG, L, path, DRF, Dependency	Optimistic commit and atomic current metadata pointer state improvement, stateful dependency, stateful dependency	LakeCat receives and implements a CAS through CatalogStore, records pointer movement, and	QG itself is standard catalog behavior, pointer history proof is a future optional profile	Commit validation that depends on table metadata for mature versions or requires

This ledger is also a discipline for future work. If the concept changes how ordinary table access works, it is Iceberg compatibility work. If the concept records who acted, what pointer moved, which receipt authorized it, and which event can be replayed, it is LakeCat catalog work. If the concept interprets columns, predicates, manifests, delete files, snapshots, branches, or metadata tables, it should move into Sail. If it decides authority, it belongs in TypeSec. If it becomes reusable graph structure, it belongs in Grust. If it turns catalog proof into semantic product behavior, it belongs in QueryGraph.

1.8.1 A Claim-By-Claim Delineation

The current release has seven headline claims. Each claim should be read at the right layer, because the layer determines whether the concept is standard Iceberg, a LakeCat implementation choice, a LakeCat/QueryGraph/TypeSec extension, or a possible future Iceberg-adjacent proposal.

The Rust service/catalog spine exists. In Iceberg vocabulary, the relevant standard promise is not Rust. It is compatible catalog behavior: resolve namespaces and table identifiers, serve current table metadata, accept optimistic commits, and keep table metadata portable. LakeCat's Rust service spine is the implementation that makes that promise strong enough for governed systems. It keeps request identity, tenancy, table state, Sail calls, TypeSec receipts, idempotency, audit, outbox, redaction, and replay admission close together in one typed control plane. That is a LakeCat implementation choice, not an Iceberg extension. The possible future proposal is not "Rust catalogs." It is a narrower optional profile for proof-carrying catalog actions.

The Turso-backed local store direction is in place. Iceberg needs durable catalog state and atomic metadata-pointer movement, but it does not choose a database. LakeCat uses the Rust turso crate behind CatalogStore so local development, embedded tests, and early deployments exercise a real durable spine instead of an in-memory sketch. Turso rows hold LakeCat control-plane state: warehouses, namespaces, tables, views, storage profiles, policies, idempotency records, pointer logs, audit records, outbox rows, and soft-delete markers. None of those rows are Iceberg table metadata. The reusable idea is the behavior around those rows: exact replay, drift rejection, atomic pointer CAS, transactional audit/outbox emission, redacted diagnostics, and durable history.

Iceberg REST-compatible table and namespace paths exist. This is the main standard surface. A PySpark, Spark, Flink, Trino, DuckDB, PyIceberg, or Sail client should be able to use normal namespace and table routes without learning QueryGraph, QGLake, TypeSec, Grust, OpenLineage, Croissant, CDIF, OSI, or ODRL. LakeCat can record more evidence behind those routes, and it can expose richer management or bootstrap routes beside them, but ordinary table access must stay ordinary. The standard term is Iceberg REST compatibility. The LakeCat terms are tenant routing, identity binding, receipt capture, audit, and replay proof.

Commit CAS, idempotency, pointer logs, audit/outbox, and replay validation are heavily hardened. The standard Iceberg part is optimistic commit: a writer advances the current metadata pointer only if the catalog requirements still hold. LakeCat hardens the envelope around that standard rule. Idempotency ensures a retry returns the same response or conflicts if the request changed. Pointer logs record accepted movement of the metadata pointer. Audit records bind the change to principal and authority. The outbox turns graph and lineage delivery into a committed

catalog side effect rather than a request-handler callback. Replay validation refuses malformed or drifted durable evidence before acknowledgement, Grust projection, OpenLineage projection, or QGLake handoff. CAS is standard. The hardening envelope is LakeCat implementation and optional catalog-control surface. Future Iceberg proposals should be limited to optional profiles for idempotent commit replay, pointer-history inspection, redacted conflicts, event identity, and lineage binding.

Governed scan paths carry substantial TypeSec-style receipt evidence. Standard Iceberg metadata gives an engine enough information to plan: schema fields and ids, partition specs, snapshots, manifest lists, manifests, metrics, data files, delete files, sequence numbers, and current metadata. LakeCat adds the governed prelude. It identifies the principal, purpose, table, requested columns, and filters; asks TypeSec for a decision; derives the effective restriction; calls Sail to plan against Iceberg metadata; and records receipt evidence that can be replayed. This is not a new table format. It is a LakeCat/TypeSec governance extension around a normal Iceberg table. The future candidate is an optional proof-carrying scan profile that other engines and catalogs could understand without adopting TypeSec wholesale.

Credential paths carry receipt evidence as well. Standard catalogs may vend credentials, but raw storage authority is too broad for many agentic workflows. LakeCat treats raw credentials as deliberate audited exceptions for principals allowed to hold them. Restricted agents should receive governed Sail-planned work instead of broad object-store reach. Receipt evidence explains why a credential was issued, narrowed, capped by TTL, denied, or replaced by a plan. That is LakeCat/TypeSec governance behavior today. A future Iceberg-adjacent profile could standardize the proof shape for governed credential vending, but it should remain optional and table-format neutral.

QueryGraph and QGLake handoff, OpenLineage, bootstrap, management, view, credential, and commit proof surfaces are broad because they serve the semantic application layer. They are not standard Iceberg table semantics. LakeCat exports catalog facts and proof anchors: table and view identities, current pointers, pointer histories, view receipt chains, storage-profile posture, credential decisions, governed scan proof, replay-validation hashes, and OpenLineage receipt hashes. QueryGraph composes those anchors with Croissant, CDIF, OSI, ODRL application semantics, Grust graph import, QGLake acceptance, and agent workflows. The broad handoff is a LakeCat/QueryGraph extension. The possible future standard pieces are smaller: catalog event identity, lineage receipt binding, view lifecycle proof, and commit-history proof.

The short rule is: standard clients get standard Iceberg; governed clients get optional proof; reusable table-format interpretation goes to Sail; reusable security semantics go to TypeSec; reusable graph behavior goes to Grust; and QueryGraph composes the semantic application.

1.8.2 The Rust Service And Catalog Spine

LakeCat has a Rust service/catalog spine today. In standard Iceberg terms, the catalog must resolve identifiers, serve metadata, and commit compatible table updates. Iceberg does not prescribe Rust, async Rust, Axum, a crate layout, or whether planning is in-process. Those are implementation choices.

LakeCat chooses Rust because the catalog transaction is now dense. A single request can involve an HTTP route, a principal, a warehouse, a namespace, a table record, an expected metadata pointer, a Sail validation call, a TypeSec authorization receipt, a durable idempotency record, an audit row, and an outbox row. Rust lets those relationships stay typed and local. That avoids a chain of loosely typed adapters where a policy decision happens in one service, metadata validation happens in another, idempotency happens in a third, and lineage is emitted as a best-effort callback.

The Iceberg-compatible claim is therefore behavioral, not linguistic. A normal client should see ordinary REST catalog behavior. LakeCat's Rust spine is the machinery that makes the ordinary behavior reliable enough for QueryGraph to trust it.

This is not an Iceberg extension and should not be proposed as one. A future Iceberg-adjacent proposal might say that catalogs can expose deterministic proof for commits, scan planning, credential decisions, and event delivery. It should not say anything about implementation language.

1.8.3 Turso As The Durable Local Store

LakeCat's Turso-backed local store direction is in place. The standard Iceberg language is current metadata location and optimistic pointer update. The LakeCat implementation language is a durable local CatalogStore backed by the Rust turso crate.

The store persists catalog state that surrounds the Iceberg pointer: warehouses, namespaces, table records, views, storage profiles, policy bindings, idempotency records, pointer logs, audit records, outbox records, soft-delete state, and management inventory. Those rows are not Iceberg table metadata. They are the local catalog spine.

That distinction matters. "Turso-backed" does not mean "Iceberg should adopt Turso." It means LakeCat has a Rust-local way to make catalog state durable while keeping the store contract portable. The portable part is the behavior: atomic CAS, exact idempotent replay, drift rejection, create-only metadata writes, redacted conflict evidence, durable pointer movement, transactional audit/outbox emission, and replay validation before delivery.

If any of this becomes future Iceberg work, the candidate is an optional catalog durability and proof profile. It would describe what a catalog can prove about pointer movement and side effects. It would not prescribe a database.

1.8.4 Iceberg REST-Compatible Paths

LakeCat has Iceberg REST-compatible table and namespace paths. This is the main standard surface. A PySpark, Spark, Trino, Flink, or PyIceberg client should be able to use the catalog without learning QueryGraph.

The standard path should let a client:

1. Discover catalog config.
2. Create, list, load, and drop namespaces where supported.
3. Create and load tables.
4. Commit table metadata updates through optimistic requirements.

5. Use compatible scan or credential flows where the client and catalog agree.

LakeCat can add warehouse-aware tenancy, identity binding, policy gates, audit, and replay evidence behind those routes. It can also expose management and QueryGraph routes beside them. But the extra routes must not become hidden requirements for ordinary Iceberg access. If a PySpark job has to call `/querygraph/v1/bootstrap` to read a normal Iceberg table, the design is wrong.

This path is standard Iceberg parlance. LakeCat's additional proof around the path is optional catalog control-plane behavior.

1.8.5 Commit CAS, Idempotency, Pointer Logs, Audit, Outbox, Replay

The standard Iceberg commit concept is optimistic pointer movement. A writer starts from a known table state, prepares new metadata, and asks the catalog to advance the current metadata pointer only if the requirements still hold. The catalog must make that update atomic.

LakeCat hardens the whole commit envelope:

- **CAS** is the standard catalog atomicity rule, implemented through the store. It prevents two writers from both believing they advanced the same table from the same previous state.
- **Create-only metadata writes** keep the new metadata object from being silently overwritten once written.
- **Idempotency** makes retries safe. Exact replay returns the stored response; drift under the same key conflicts; successful replay does not duplicate outbox, graph, lineage, audit, or pointer-log effects.
- **Pointer logs** record accepted catalog pointer movement with audit-safe hashes and sequence evidence.
- **Audit** records who acted, which authority was used, and which redacted request facts were captured.
- **Outbox** records committed catalog facts transactionally so graph and OpenLineage projection are not best-effort request-handler side effects.
- **Replay validation** refuses malformed durable evidence before LakeCat acknowledges, projects to Grust, or emits OpenLineage.

Only part of that list is standard Iceberg. The optimistic commit and pointer CAS are standard catalog behavior. Idempotency, pointer logs, audit/outbox, and replay validators are LakeCat implementation and optional management proof. They are deliberately outside Iceberg metadata.

The future standards question is narrow and practical. Iceberg catalogs could benefit from optional shared profiles for idempotent commit replay, pointer history, redacted conflict proof, event ordering, and lineage binding. Those profiles should remain additive. They should let stronger catalogs interoperate without making every existing table or engine understand LakeCat's exact internal rows.

1.8.6 Governed Scans And Credential Decisions

Standard Iceberg gives engines the metadata required to plan reads: schemas, field ids, partition specs, snapshots, manifest lists, manifests, data files, delete files, metrics, and sequence information. LakeCat should not duplicate that engine logic.

LakeCat's governed scan path adds a control-plane envelope:

1. Identify the principal, warehouse, namespace, table, purpose, and requested projection/filter.
2. Ask TypeSec for an authorization decision.
3. Convert the decision into an effective read restriction: allowed columns, mandatory predicates, policy hashes, TTL caps, and raw-credential posture.
4. Ask Sail to plan the narrowed request against the current Iceberg metadata.
5. Record receipt evidence, plan summary, read restriction, and replayable hashes in audit/outbox.
6. Validate that evidence again before graph or OpenLineage projection.

The scan remains a scan over a normal Iceberg table. The governance proof is LakeCat/TypeSec extension material around the scan. That proof is valuable because an agent can later show not just that it read a table, but that it read the allowed columns and rows for an allowed purpose under a specific policy.

Credential vending follows the same posture. Raw storage credentials are powerful. LakeCat treats them as a deliberate, audited exception for trusted principals, not as the default for agents. For untrusted or restricted principals, the preferred path is a governed Sail-planned read. The catalog does not hand over broad storage power when the policy only authorizes a narrow task.

This is not standard Iceberg today. It is a governance extension. The future Iceberg-adjacent candidate is an optional governed-access profile that lets catalogs prove why a credential was issued, withheld, narrowed, or replaced by a planned read.

1.8.7 QueryGraph And QGLake Handoff

QueryGraph and QGLake handoff surfaces are broad because QueryGraph needs to start from governed catalog truth, not from a private scrape of object storage. LakeCat can expose bootstrap bundles, management inventory, view proof, credential posture, commit history, scan proof, OpenLineage receipts, replay validation summaries, and graph anchors. Those surfaces are optional LakeCat/QueryGraph extensions.

The boundary is:

- LakeCat supplies catalog facts and proof anchors.
- Sail supplies reusable Iceberg metadata interpretation and planning.
- Grust supplies graph schema, projection logic, storage, traversal, and query behavior.
- TypeSec supplies policy, capability, TypeDID, secure-agent, ODRL, and authorization semantics.
- QueryGraph composes Croissant, CDIF, OSI, ODRL, OpenLineage, graph import, agent workflows, and user-facing reasoning.

LakeCat should not import QueryGraph. QueryGraph may consume LakeCat's standard REST surface, management APIs, bootstrap bundle, OpenLineage projection, and outbox replay. That keeps LakeCat a catalog foundation instead of an application server.

Some of the handoff ideas may become general. Catalog event identity, lineage receipt binding, view lifecycle proof, and credential proof could interest other systems. QueryGraph's semantic model itself should not be presented as future Iceberg. It is an application layer that benefits from strong catalog evidence.

1.9 Why The Work Belongs In The Engine

The most dangerous failure mode for a smart catalog is becoming a partial engine. It begins innocently: the catalog needs to validate a schema, inspect a manifest metric, expand a manifest list, check a delete file, or reason about a format-version field. Each small parser looks cheaper than an engine call. Over time the catalog grows a second Iceberg implementation with weaker tests, fewer execution users, and subtle drift from real planning behavior.

LakeCat should avoid that trap. The catalog owns the transaction. Sail owns the table semantics.

Sail is a strong engine choice for this architecture because it is Rust-native and already close to the structures LakeCat needs: generated Iceberg REST models, provider seams, table-status conversion, Arrow/DataFusion-shaped execution paths, local catalog integration, and reusable scan planning. When a feature needs field-id binding, schema evolution, partition transforms, manifest metrics, delete association, v3 row lineage, v4 metadata trees, branch selection, snapshot selection, or metadata-as-data, it should move toward Sail.

That gives LakeCat several concrete benefits.

First, compatibility improves. A standard client sees standard REST behavior while the same engine-shaped code interprets the table metadata that execution will use. LakeCat is not inventing one interpretation for governance and letting another engine execute a different one.

Second, performance improves. Rust-to-Rust service and engine paths avoid a large amount of marshaling and indirection. The catalog can make a local typed call to plan or validate instead of serializing table state through multiple remote services. The result is not just lower latency. It is fewer places for policy, metadata, and request identity to drift.

Third, correctness compounds. A Sail fix for manifest metrics, delete files, format-version handling, or table-status conversion benefits LakeCat, QueryGraph, and any other Rust lakehouse code that uses Sail. The alternative is a LakeCat-local helper that must be remembered, tested, and eventually deleted.

Fourth, governance becomes more honest. TypeSec can authorize a restriction, but only an engine can faithfully apply that restriction to Iceberg metadata. Allowed columns need field-id binding. Row predicates need expression binding. Manifest pruning needs metrics. Delete handling needs table-format semantics. LakeCat should record that the restriction existed and that Sail planned under it. Sail should do the planning.

Fifth, QueryGraph receives better evidence. A QueryGraph bootstrap bundle is more useful when its table facts were interpreted by the same engine path that will plan reads and expose metadata-as-data. The semantic graph can then reason over catalog evidence without pretending the catalog is a graph engine or a query engine.

Sail is a particularly good fit because it is not merely a helper crate beside LakeCat. It is the Rust lakehouse engine path LakeCat wants to stand next to. It can own the pieces that are expensive and correctness-sensitive:

- generated Iceberg REST models and response conversion;
- schema, field-id, partition, snapshot, and branch interpretation;
- manifest-list and manifest reading;
- metrics-based pruning;
- equality and position delete handling;
- metadata-as-data access for table, snapshot, manifest, and file facts;
- scan-task generation and fetch-task revalidation;
- commit requirement validation and table-status conversion;
- v3 and v4 table-format interpretation as those surfaces mature.

Those tasks all have one property in common: the right answer depends on the table format, not on catalog bookkeeping. Keeping them in Sail means one engine-grade implementation can serve LakeCat, QueryGraph, and future Rust lakehouse users. LakeCat then persists compact proof of what Sail decided: which metadata pointer was current, which snapshot or branch was planned, which restriction was applied, which task set was returned, which delete semantics were honored, and which receipt hashes bind the plan to the request.

That split also makes standards work cleaner. If LakeCat later proposes an optional proof-carrying scan profile, the proof can say “this restriction was planned by the engine against this Iceberg table state” instead of encoding a catalog-local parser’s private interpretation of manifests and deletes. If LakeCat proposes a commit-history profile, the commit proof can rely on engine-aligned validation rather than a second implementation hidden in the catalog. Sail therefore protects both performance and portability: fewer remote adapters, fewer duplicated parsers, and fewer places where governance evidence can drift from execution behavior.

1.9.1 A PySpark Workflow

A PySpark user should configure LakeCat as an Iceberg REST catalog and use it in an ordinary way:

1. Create or select a namespace.
2. Create a table or load an existing table.
3. Write data and commit new Iceberg metadata.
4. Retry safely if the client loses the response.
5. Read the table through the engine’s normal Iceberg path.

In this workflow the user should not need QueryGraph, QGLake, TypeSec, Grust, OpenLineage, or LakeCat management APIs. Those systems may observe and prove what happened, but they

do not redefine the table. LakeCat's standard surface preserves Iceberg compatibility; the Rust/Turso/CAS/idempotency/audit/outbox machinery makes that compatibility dependable.

If the PySpark commit races with another writer, LakeCat's store CAS rejects the losing update. If the client retries the exact same commit with the same idempotency key, LakeCat returns the stored response. If the retry drifts, LakeCat rejects it. If QueryGraph later imports the catalog, it can see a pointer-log and outbox trail that explains what committed. Spark still saw a normal catalog.

1.9.2 An Agentic Workflow

An agent workflow has different risk. The agent may ask for a table, a purpose, a set of columns, and a task. The wrong design would give the agent broad object-store credentials and hope it behaves. LakeCat's governed design should make the narrow path easier than the broad path.

The flow is:

1. LakeCat identifies the agent principal and table scope.
2. TypeSec evaluates the request and returns a receipt.
3. The receipt yields a read restriction: columns, predicates, purpose, policy hashes, TTL, and raw-credential posture.
4. LakeCat asks Sail to plan under that restriction.
5. Sail interprets the Iceberg metadata and returns compatible task evidence.
6. LakeCat writes audit/outbox evidence and can replay it to QueryGraph and OpenLineage.

This is where pushing work into Sail matters most. The agent does not need a catalog-authored approximation of the table. It needs a governed plan produced by an engine that understands the table. LakeCat remains the authority on who asked, what was allowed, what was committed to the evidence log, and what can be replayed.

1.9.3 A QueryGraph Bootstrap Workflow

QueryGraph needs more than a table pointer. It needs a trustworthy substrate for semantic import and agent reasoning. LakeCat can provide that substrate without becoming QueryGraph.

A bootstrap can collect:

- namespaces, tables, views, and warehouse scope;
- current metadata pointers and commit-history proof;
- view lifecycle and receipt-chain proof;
- credential posture and storage-profile proof;
- governed scan proof and raw-credential exception evidence;
- OpenLineage event hashes and replay validation summaries;
- catalog graph anchors suitable for Grust import;
- Croissant, CDIF, OSI, ODRL, and QueryGraph manifest artifacts.

The standard Iceberg portion is still the table and catalog state. The QueryGraph portion is the semantic interpretation and application workflow. LakeCat's role is to bind the two with

replayable evidence. That makes the catalog the foundation for QueryGraph without turning LakeCat into a semantic application.

1.10 Extension Or Future Feature?

The right answer depends on which layer the feature belongs to.

Implementation choices should stay implementation choices. Rust, Turso, crate layout, local feature gates, release scripts, and replay-validator internals should not be proposed as Iceberg features. They are valuable because they make LakeCat reliable.

Optional catalog surfaces should stay additive. Management inventory, commit-history proof, view proof, credential-root posture, OpenLineage projection, replay verification, and QueryGraph bootstrap should be available to clients that need them and invisible to clients that do not.

Governance proof should remain outside table metadata. TypeSec receipts, TypeDID context, ODRL restrictions, secure-agent posture, and raw-credential exception evidence can be linked to catalog actions, but the table metadata should not become a policy log.

Future Iceberg proposals should be behavior profiles, not private semantics. The strongest candidates are:

- idempotent commit replay;
- pointer-history inspection;
- redacted conflict proof;
- transactional catalog event streams;
- OpenLineage or lineage-receipt binding;
- governed credential vending proof;
- proof-carrying scan planning;
- view lifecycle and receipt-chain proof.

Those proposals should be optional, compatible, and table-format neutral. A catalog that supports them can be more governable. A catalog that does not support them can still be a valid Iceberg catalog. That is the balance LakeCat should preserve.

1.10.1 The Standards Position

LakeCat should be precise when it talks about “extensions.” The word can mean three different things, and only one of them is a standards proposal.

The first meaning is an implementation extension. LakeCat’s Rust service spine, Turso-backed local store, crate layout, trait boundaries, local release gates, and replay-validator internals extend the implementation, not Iceberg. They make a standard catalog more durable and observable. They should be documented for LakeCat operators, but they should not be described as Iceberg features. Nobody should have to run Rust or Turso to interoperate with LakeCat as an Iceberg catalog.

The second meaning is a catalog-control extension. QueryGraph bootstrap, QGLake handoff, management inventory, commit-history proof, view receipt chains, credential-root posture, replay verification, and OpenLineage projection are additive APIs beside the standard REST

catalog path. They are real LakeCat features. They are also optional. A Spark, PySpark, Flink, Trino, or PyIceberg client can ignore them and still use the table through ordinary Iceberg catalog operations.

The third meaning is a future Iceberg-adjacent profile. A LakeCat feature should move into this category only when it satisfies four tests:

1. It is useful across more than one catalog or engine.
2. It can be expressed without private table metadata.
3. A client that does not understand it can still use the standard catalog.
4. Its proof shape is stable enough to test independently of LakeCat.

That test eliminates some tempting overreach. Rust is not a future Iceberg proposal. Turso is not a future Iceberg proposal. QueryGraph’s semantic graph is not a future Iceberg proposal. TypeSec’s complete policy model is not a future Iceberg proposal. Those are valuable system choices, but they are not portable table-format or REST-catalog obligations.

The strongest proposal candidates are narrower:

- idempotent commit replay, because clients need retry safety without duplicate side effects;
- pointer-history inspection, because operators need to explain which catalog transaction advanced a metadata pointer;
- redacted conflict proof, because failed commits should be debuggable without leaking storage paths or credentials;
- transactional catalog event streams, because graph and lineage consumers need committed facts rather than best-effort handler callbacks;
- lineage receipt binding, because OpenLineage and similar consumers need to know which catalog state produced an event;
- governed credential vending proof, because catalogs increasingly mediate storage access for humans, services, and agents;
- proof-carrying scan planning, because policy-restricted scans need a way to prove the effective projection, predicate, purpose, and TTL;
- view lifecycle proof, because views are catalog objects and QueryGraph-style import needs to verify their version and deletion history.

Each candidate should remain a profile. A profile can say, “if a catalog supports this proof, here is the interoperable shape.” It should not say, “every Iceberg table must carry this governance log.” That is the architectural line LakeCat has to defend. The Iceberg table remains the portable truth. The catalog profile explains how the control plane acted around that truth.

1.10.2 Why The Engine Boundary Matters To Standards

The standards position depends on the Sail boundary. If LakeCat implements its own manifest parser, delete-file planner, field-id binder, partition evaluator, and v4 metadata-tree logic, then any proof it emits risks becoming LakeCat-specific table semantics. A future proposal based on that proof would be suspect because it would encode the behavior of a partial catalog engine.

If Sail owns that work, the proof has a better foundation. LakeCat can say: TypeSec authorized this principal, these columns, this predicate, this purpose, and this TTL; Sail planned that effective request against the current Iceberg metadata; LakeCat committed this receipt, pointer state, audit row, and outbox event. The proof is then layered. Iceberg supplies the portable metadata model. Sail supplies reusable engine interpretation. TypeSec supplies policy meaning. LakeCat supplies transaction and replay evidence. QueryGraph supplies semantic application import.

That layering is why Sail is not just an implementation convenience. It is what keeps LakeCat's extensions credible. A proof-carrying scan profile is useful only if the scan proof came from an engine path that understands the table. A governed credential profile is useful only if the alternative to raw credentials is a real engine-planned task set. A commit-history profile is useful only if commit validation follows the same Iceberg semantics that execution will respect. Sail gives LakeCat that shared Rust engine truth.

1.10.3 A Classification Test For New Catalog Work

LakeCat will keep growing, so the book needs a durable way to classify new ideas before they become architecture. The test is deliberately practical: which system would be wrong if it did not understand the feature?

If a standard Iceberg client would be wrong without the feature, the feature is either already part of Iceberg or it is a compatibility risk. Namespace resolution, table identifiers, current metadata pointers, metadata file content, snapshots, manifests, delete files, commit requirements, and REST catalog routes live here. LakeCat must implement those faithfully. It should not rename them, hide them behind QueryGraph, or reinterpret them with LakeCat-only semantics.

If the catalog transaction would be wrong without the feature, the feature belongs in LakeCat. This includes identity binding, tenancy, durable pointer state, optimistic CAS, exact idempotency, pointer logs, audit rows, outbox rows, redaction, replay validation, management inventory, and proof endpoints. Those features are not Iceberg table metadata. They are the durable envelope around catalog actions. They can be optional extension surfaces or future profile candidates, but their first job is to make LakeCat's own state changes correct and explainable.

If the answer depends on table-format semantics, the feature belongs in Sail. Allowed-column enforcement is not only a list of names; it has to survive schema evolution and field-id binding. Row filtering is not only a JSON predicate; it has to be bound against the table schema and current metadata. Manifest pruning is not only reading a file; it has to understand partition evolution, metrics, null counts, lower and upper bounds, sequence numbers, and delete semantics. Iceberg v4 compatibility is not only accepting an unknown JSON object; it is typed interpretation of the format as engines adopt it. Those are engine responsibilities. LakeCat should call Sail and persist what Sail proved, not grow a second planner.

If the answer depends on authorization, policy composition, secure-agent posture, TypeDID context, capabilities, ODRL interpretation, or proof that an actor was allowed to perform a catalog action, the feature belongs in TypeSec. LakeCat can carry a TypeSec-style receipt, enforce the decision at the catalog boundary, and persist the evidence. It should not become the long-term

owner of policy language semantics. That is how a governed scan can stay precise: TypeSec says what is allowed, Sail says what that means for the table, and LakeCat records the action and proof.

If the answer depends on graph shape, traversal, taxonomy, graph storage, or Cypher behavior, the feature belongs in Grust. LakeCat should emit bounded catalog-domain facts: this table exists, this snapshot was observed, this policy was attached, this scan was planned, this credential decision was made. Grust should own how those facts become graph structure and how QueryGraph asks questions of that structure.

If the answer depends on Croissant, CDIF, OSI, broad ODRL application meaning, OpenLineage composition, QGLake acceptance, agent workflow explanation, or user-facing semantic reasoning, the feature belongs in QueryGraph. LakeCat should provide trustworthy anchors and replayable proof. QueryGraph should compose those anchors into the application model. That is the difference between a catalog foundation and a semantic product.

The proposal question should be asked last, not first. A behavior becomes a candidate Iceberg-adjacent proposal only after it has proven useful beyond LakeCat and can be described without requiring LakeCat, QueryGraph, TypeSec, Grust, Sail, or Turso. Exact idempotent retry, pointer-history inspection, redacted conflict proof, catalog event identity, lineage receipt binding, governed credential-vending proof, proof-carrying scan planning, and view lifecycle proof are plausible because they describe portable catalog behavior. The full QGLake handoff is not a proposal. Rust is not a proposal. Turso is not a proposal. TypeSec's entire policy model is not a proposal. They are important parts of this system, but the standardizable material is the smaller interoperability contract that other catalogs and engines could implement in their own way.

The result is a simple engineering habit. Start with standard Iceberg. Keep the Iceberg metadata pristine. Put durable catalog state and receipts in LakeCat. Push data-shaped semantics into Sail. Push policy-shaped semantics into TypeSec. Push graph-shaped semantics into Grust. Let QueryGraph compose the application. Promote only the smallest portable behavior when the ecosystem needs a shared profile.

1.11 The Current Catalog State

The current lakehouse catalog market is converging on a simple fact: the catalog is no longer a sidecar. It is the place where table identity, tenancy, commits, credentials, governance, and interoperability are negotiated.

Hive Metastore gave early lakehouses a familiar namespace and table registry, but it was born for a different format era. Cloud warehouses built proprietary catalogs around their own engines. Nessie explored Git-like table references and branching. Unity Catalog turned governance and sharing into a central product surface. Lakekeeper has shown how a modern open Iceberg REST catalog can model warehouses, projects, credentials, soft deletion, and management APIs without polluting table metadata. Polaris has made the strongest standards-centered move: an open Iceberg REST catalog with vendor gravity, clear governance ambition, and a credible route for many engines to meet at the same catalog boundary.

That is why Polaris is a winner. It is not because it is the last word in catalog architecture. It is winning because it occupies the obvious shared surface at the right moment:

- It speaks Iceberg REST instead of asking every engine to learn a new table protocol.
- It treats the catalog as a security and governance surface, not merely a pointer map.
- It gives enterprises an open center of gravity around Iceberg while the warehouse, query-engine, and cloud-storage layers remain plural.
- It can be adopted incrementally: engines can keep reading tables, while operators gain a real control plane.

LakeCat should learn from that. The winning move is not to reject Polaris-style compatibility. The winning move is to keep that compatibility and then ask what a Rust-native, QueryGraph-facing catalog can do when it is allowed to plan near Sail, emit graph through Grust, and ask TypeSec for authorization proofs.

1.12 What Intermediation Loses

Catalogs win by sitting between engines and data. That same position can also flatten the system.

When the catalog is only an intermediary, it often sees the table name, the current metadata pointer, the caller, and perhaps a credential request. The engine sees the schema, manifests, statistics, deletes, partition evolution, scan filters, and physical plan. The governance system sees policy. The lineage system sees an event after the fact. The semantic layer sees a separate model. Each system receives a shard of the truth.

The loss is not merely elegance. It is operational:

- Policy can be checked before access but not carried into scan planning.
- Credentials can be vended without proving why a raw credential exception was allowed.
- Lineage can describe that something happened but not bind to the exact governed plan, snapshot, policy, and table metadata.
- Semantic layers can drift from the physical tables they describe.
- Agents can receive broad storage access when they should have received a governed plan and short-lived, narrow task set.
- Engines can optimize with file statistics while catalogs remain blind to the consequences of those choices.

The catalog should not become a replacement engine. But it should not be a blind intermediary either. LakeCat's thesis is that the catalog can stay thin and standard while still becoming engine-close at the moments where correctness, security, and semantics depend on planning evidence.

1.13 Why Sail Matters

Sail is the Rust lakehouse engine path. It already contains the pieces that should understand Iceberg deeply: catalog abstractions, generated Iceberg REST models, DataFusion table providers, manifest pruning, metadata-as-data paths, write and commit plumbing, and format-version checks.

That matters because scan planning and commit validation are not generic catalog chores. They require knowledge of Iceberg schemas, projections, partition specs, sort orders, snapshots, manifests, data files, delete files, statistics, and expression models. If LakeCat reimplements those details, it becomes a second Iceberg engine. The same bug class appears twice, and the catalog begins to drift away from the planner that actually executes work.

LakeCat takes the opposite path. Put reusable Iceberg and planning behavior in Sail. Let LakeCat call Sail for the parts that require engine-grade knowledge. Keep LakeCat responsible for the catalog boundary: identity, tenancy, durable state, policy gates, idempotency, audit, outbox, and standard REST compatibility.

The current LakeCat architecture follows this line. The service exposes an Iceberg REST-compatible surface under `/catalog/v1`. The Sail-facing engine path handles scan planning, table-status conversion, metadata preparation, manifest expansion, and standard response validation. LakeCat keeps only the additional context it must own: which principal asked, which policy narrowed the read, which metadata pointer was current, which commit won, and which event should be replayed to graph and lineage sinks.

1.14 Why Move Catalog Work Closer To The Engine

A passive catalog returns metadata locations and lets the client plan. That is compatible, but it is not enough for the next generation of governed systems. Agents, notebooks, services, and model pipelines need stronger guarantees:

- A policy should be enforced before a scan is planned.
- Column restrictions should narrow the projection before file tasks are produced.
- Row predicates derived from policy should become mandatory scan filters.
- A stateless `fetchScanTasks` call should not widen a prior governed plan.
- Credentials should be short-lived, scoped, and audited.
- Commit retries should be idempotent and should not replay a different request.
- Graph and lineage side effects should reflect committed catalog state, not best-effort handler side effects.

Those guarantees sit between catalog state and engine planning. If the catalog is too far from the engine, it can check a policy and then hand the client a metadata pointer, hoping the client preserves the restriction. If the engine is too far from the catalog, it can plan efficiently but may not know the governance receipt or durable identity context.

LakeCat fuses those two moments. LakeCat authorizes the request, derives the effective restriction, and asks Sail to plan or validate against the current metadata pointer. Sail performs the Iceberg work. LakeCat returns the standard shape and records the governance evidence.

The result is still an Iceberg catalog. The difference is that governed reads and writes are planned through the same Rust implementation that `QueryGraph` will use downstream.

The deeper reason is that Iceberg correctness is engine-shaped. A catalog can store a metadata pointer and compare it during commit, but the hard questions are about the table described by that pointer:

- Which schema id is current, and how do projected field ids map through evolution?
- Which partition spec applies to a manifest, and can a predicate be evaluated against its lower and upper bounds?
- Which delete files must travel with a data file task?
- Which statistics are absent, stale, truncated, or encoded with format-specific rules?
- Which snapshot or branch should a read bind to?
- Which update requirements conflict with the current metadata?
- Which metadata tables expose the right diagnostic view without forcing a client to parse every object manually?

Those are not good catalog-only questions. They are Iceberg engine questions. If LakeCat answers them locally, it must reimplement expression binding, schema projection, manifest reading, metrics decoding, delete semantics, format-version checks, and table-status conversion. The catalog would slowly become a second planner with a smaller test surface than the real engine. That is exactly the shape LakeCat should avoid.

Sail is the better place for that work for six practical reasons.

1. Sail is already Rust-native. LakeCat can call it without crossing a JVM service boundary, serializing every intermediate object through a foreign adapter, or hiding engine evidence behind opaque text blobs.
2. Sail sits in the Arrow and DataFusion ecosystem. Planning can produce structures that are natural for columnar execution, metadata-as-data queries, table providers, and future in-process QueryGraph workflows.
3. Sail already carries Iceberg-specific code paths: generated REST models, catalog provider seams, table-status conversion, manifest expansion, pruning helpers, write plumbing, and format-version checks.
4. Sail can be tested once and reused by more than LakeCat. If manifest metric decoding, delete-file indexing, or v4 metadata-tree support is improved in Sail, LakeCat, QueryGraph, and other Rust lakehouse tools all benefit.
5. Sail is close enough to execution to understand cost and shape. A catalog can know that a policy narrowed a scan, but the engine knows how that projection changes files, tasks, columns, delete handling, and downstream execution.
6. Sail lets LakeCat keep the compatibility promise. LakeCat can expose normal REST responses while asking Sail for the engine-grade validation and plan evidence required by governed reads and commits.

That gives LakeCat a concrete rule for future work. If a feature has to understand Iceberg metadata structure, expression binding, file statistics, delete semantics, snapshot selection, schema evolution, partition evolution, sort order, row lineage, metadata tables, or physical task shape, LakeCat should first try to push it into Sail. The catalog may initiate the work and persist the receipt, but the reusable implementation should live where the engine can use it too.

The following responsibilities are Sail-shaped:

- decoding manifest metrics and using them for pruning;

- interpreting lower and upper bounds across Iceberg physical encodings;
- expanding manifest lists into file scan tasks;
- attaching positional and equality delete files to the data tasks they affect;
- binding REST expressions to table schemas and field ids;
- preserving nested field projection through schema evolution;
- validating commit requirements against current table metadata;
- preparing new metadata JSON for commits, creates, deletes, and restores;
- exposing metadata-as-data tables for snapshots, manifests, files, history, and partitions;
- carrying v3 row-lineage and v4 metadata-tree behavior once those models are typed in Sail;
- converting Iceberg REST and table metadata into engine table status and provider objects.

The matching LakeCat responsibilities are catalog-shaped:

- decide which principal, warehouse, namespace, table, and policy context apply;
- ask TypeSec for an authorization decision and receipt;
- derive the effective read restriction from policy and request context;
- call Sail with the narrowed projection, mandatory filters, purpose, and current metadata pointer;
- persist the metadata pointer only after the catalog transaction wins;
- store idempotency, audit, pointer-log, and outbox evidence;
- reject replay evidence that no longer matches the durable receipt;
- publish optional QueryGraph, Grust, and OpenLineage handoff surfaces.

That separation is important for both speed and correctness. A Rust catalog can call a Rust engine path directly, without a JVM service hop or a pile of language-adapter objects, but the benefit is deeper than latency. It means the same implementation that plans a governed task set for LakeCat can also be used by notebooks, QueryGraph ingestion, maintenance jobs, and future Rust execution surfaces. Every manifest-pruning fix, delete-file fix, or v4 metadata fix lands once in Sail and becomes available to the whole stack.

This division also improves security. A policy decision is not useful if it is only an annotation on a request. LakeCat should turn a TypeSec decision into an effective projection, mandatory predicate, purpose, and receipt. Sail should plan from that effective request, not from the client's wider request. LakeCat then records both sides: what was asked for, what was allowed, what Sail planned, which metadata pointer was current, and which receipt authorized the result. That is the evidence QueryGraph needs, and it is stronger than handing an agent an object-store credential and hoping the client behaves.

The same division improves performance. Pushing pruning, manifest expansion, delete handling, and metadata-as-data into Sail means LakeCat does not need to load and reinterpret Iceberg structures just to prove a governed task set. When the service and engine are both Rust, the catalog can make a local call, reuse typed objects, and persist compact proof evidence rather than copying large metadata bodies through multiple services. The fast path remains the standard Iceberg path for ordinary engines. The governed path becomes richer without becoming a compatibility tax.

Consider a governed read. A user or agent asks for a table, a projection, and perhaps a filter. LakeCat can identify the principal, warehouse, namespace, table, request purpose, and policy context.

TypeSec can decide that the request is allowed only for certain columns, with a mandatory row predicate and a credential TTL cap. At that point LakeCat should not parse every manifest and invent file tasks itself. It should call Sail with the effective request. Sail can bind fields by Iceberg field id, preserve nested projection, evaluate partition statistics, attach delete files, and return task evidence that corresponds to the real table metadata. LakeCat records the receipt and hashes of that plan. QueryGraph can later verify that the agent saw a governed plan, not a broad pointer and a polite suggestion.

Consider a commit. LakeCat owns request identity, idempotency, compare-and-swap, audit, pointer logs, and outbox delivery. Sail should own the table-format part: checking update requirements against current metadata, preparing or validating new metadata JSON, understanding format-version behavior, and returning the standard response shape. If LakeCat writes the pointer only after Sail validates the table-format work, the catalog transaction remains authoritative without duplicating the engine. If the writer retries, LakeCat's idempotency record decides whether it is the same request. If the pointer moved, LakeCat returns a redacted conflict. If the commit wins, the outbox can project graph and lineage from a durable catalog fact.

Consider a QueryGraph bootstrap. LakeCat should provide catalog facts: warehouses, namespaces, tables, views, current pointers, commit history, credential posture, scan receipts, view receipt chains, and OpenLineage hashes. Sail should provide the engine facts behind those catalog facts: what the metadata describes, what a plan contains, what delete files apply, and how newer Iceberg versions should be interpreted. Grust should own graph taxonomy and traversal. TypeSec should own policy meaning and secure-agent receipts. QueryGraph should own Croissant, CDIF, OSI, ODRL application semantics and the final graph import. That is how LakeCat can become foundational without becoming a warehouse, a policy engine, a graph database, and a semantic app at the same time.

In short: LakeCat should own trust, identity, pointers, transactions, and evidence. Sail should own Iceberg semantics, planning, pruning, metadata interpretation, and engine-facing execution shape. QueryGraph should consume the resulting evidence as a semantic graph. That is the architecture that keeps LakeCat thin without making it weak.

Sail is a particularly good engine choice because it is close to every object LakeCat should not duplicate. A governed read is not just a list of files. It is a bound expression over an evolved schema, a projection over field ids, a snapshot selection, a partition pruning problem, a manifest-metric decoding problem, and often a delete-file association problem. A governed commit is not just a pointer write. It is an update-requirement check, metadata JSON preparation problem, object-write plan, and response-shape validation problem. Those are precisely engine-shaped responsibilities.

If LakeCat implemented those details locally, each standard improvement would have to land twice. A new Iceberg metadata version would require catalog-side parsing and engine-side parsing. A metrics-decoding bug would need one fix in the planner and one fix in the catalog proof path. A delete-file attachment bug could be fixed for query execution while the catalog still

produced stale governed task proof. That is not a thin catalog; it is an accidental second engine. Sail prevents that drift by becoming the reusable Rust place where the Iceberg semantics live.

The performance argument points the same way. LakeCat should be able to call a Rust Sail API with typed request state, receive typed plan or validation evidence, and persist compact hashes and counts. It should not shell out to a separate planner, round-trip through a JVM bridge, or translate every metadata structure through an untyped JSON corridor just to prove that a policy narrowed a scan. For ordinary engines, LakeCat can still return ordinary REST responses. For governed QueryGraph and agent paths, the same request can carry stronger proof because Sail has already done the real planning work.

Sail is also the right place because the user workflow begins long before QueryGraph sees a graph. A PySpark user may create a table and commit metadata through an Iceberg REST catalog. A notebook may ask for a scan. A maintenance job may inspect manifests. An agent may ask for a governed subset of the same table. QueryGraph may later import the table as a semantic asset. All of those flows depend on the same Iceberg facts: schemas, snapshots, manifests, partition specs, delete files, statistics, and metadata evolution. If each surface implements those facts independently, the system becomes inconsistent by construction. If Sail owns them, LakeCat can expose different catalog surfaces while depending on one reusable engine truth.

The PySpark path illustrates the point. Spark should see a normal Iceberg REST catalog. It should not care about QueryGraph bootstrap or TypeSec receipts when it performs ordinary table work. But LakeCat still benefits from Sail because the catalog can validate standard request and response shapes against the same Iceberg model that the Rust engine uses. When a commit succeeds, LakeCat owns the transaction and evidence. Sail owns the table-format understanding. Spark gets compatibility, and LakeCat gets a trustworthy proof trail.

The governed-agent path is different but uses the same engine core. The agent should not receive the broad storage credential merely because a standard catalog can vend one. LakeCat asks TypeSec for a decision, derives an effective projection and row restriction, and asks Sail to plan the narrowed request. Sail can prune manifests, attach delete files, and shape scan tasks from the actual table metadata. LakeCat records what was requested, what was allowed, what Sail planned, which credential posture was chosen, and which receipt authorized it. QueryGraph can then verify the evidence without trusting the agent or re-planning the lake by itself.

The management path also benefits. Operators need to inspect pointer history, storage-profile posture, view receipt chains, credential decisions, and replay delivery status. Those are catalog facts, so LakeCat owns the API and store contract. But when an operator asks why a governed scan saw only a subset of files, the answer depends on engine facts: partition pruning, manifest metrics, delete handling, and snapshot selection. Sail gives LakeCat a way to explain that outcome without duplicating the implementation that produced it.

This is why pushing work into Sail is not merely an optimization. It is the mechanism that lets LakeCat stay both standard and ambitious. The catalog can remain narrow at the Iceberg boundary while becoming rich in evidence. The engine can remain reusable while carrying the hard table semantics. QueryGraph can build a semantic graph on top of proof rather than inference.

TypeSec can make security decisions that affect real plans rather than annotations. Each part gets stronger because the table-format work is concentrated where it can be tested, optimized, and reused.

Sail is a strong engine choice for LakeCat because it matches the shape of the problem. Iceberg is metadata-heavy, columnar, versioned, and planner-driven. Rust is a good fit for a catalog that must be fast, explicit about ownership, careful with redaction, and comfortable passing typed objects between the service, store, and engine layers. Sail adds the engine side of that same discipline: generated Iceberg REST models, Arrow/DataFusion-native execution objects, catalog/provider seams, manifest and metadata paths, and a natural place to grow v3 and v4 table-format support. LakeCat can therefore keep the catalog transaction small while still asking a real engine to answer engine-shaped questions.

That choice changes user workflows without breaking them. A PySpark user still sees a normal Iceberg REST catalog. A Rust service can call the same catalog and receive evidence-rich governed planning. An agent can be denied raw credentials and handed a Sail-planned task set instead. An operator can inspect pointer logs and replay proof while knowing that file pruning and delete-file attachment came from the reusable engine path. QueryGraph can import a semantic graph whose catalog facts were produced by durable LakeCat state and whose table facts were interpreted by Sail. That is the practical meaning of pushing work into the engine: the standard path stays portable, and the advanced path gets stronger because it is built on the same Iceberg semantics rather than a catalog-side approximation.

The design rule is therefore operational, not philosophical:

1. If the work needs table-format semantics, push it into Sail.
2. If the work needs catalog atomicity or durable request evidence, keep it in LakeCat.
3. If the work needs graph taxonomy, traversal, projection storage, or Cypher, push it into Grust.
4. If the work needs authorization semantics, capability composition, ODRL meaning, TypeDID envelopes, or secure-agent proof, push it into TypeSec.
5. If the work needs semantic application import, OSI/Croissant/CDIF alignment, or end-to-end graph acceptance, make it QueryGraph's responsibility and let LakeCat provide the catalog facts.

That rule makes first-release scope clearer. LakeCat needs enough Sail integration to prove that reads and commits are planned and validated through the engine path. It needs enough TypeSec integration to prove authorization receipts and governed restrictions. It needs enough Grust/OpenLineage output to prove replayable side effects. It needs enough QueryGraph handoff to prove that the next system can bootstrap from LakeCat without inventing a private import shortcut. It does not need to own every future semantic model, graph query, policy language feature, or Iceberg parser. The strongest catalog is the one that knows exactly when to stop and call the right sibling.

1.15 Catalog Concepts In User Workflows

The concept boundary becomes clearest when it is traced through ordinary user workflows. A catalog concept should not be classified by how advanced it sounds. It should be classified by the

role it plays in the request. The same table can be touched by a PySpark job, a Rust service, a governed agent, an operator, a lineage consumer, and QueryGraph. Each client may see a different surface, but the portable table truth should remain Iceberg metadata and the engine-shaped table work should remain Sail.

The PySpark workflow is the compatibility baseline. A Spark user configures an Iceberg REST catalog, creates a namespace, creates a table, writes data, and later loads the table through the catalog. In standard Iceberg terms, that workflow uses namespaces, table identifiers, table metadata, snapshots, manifests, data files, delete files, current metadata locations, and optimistic commits. LakeCat should look like an ordinary Iceberg REST catalog for that flow. The user should not need to know about QueryGraph bootstrap, TypeSec receipts, Grust projection, OpenLineage receipt hashes, or Turso rows.

LakeCat still does real work during that ordinary PySpark flow. The Rust spine normalizes the request, resolves tenancy, persists namespace and table state, performs compare-and-swap on the metadata pointer, records idempotency when the client supplies a key, writes pointer-log and audit evidence, and enqueues committed events. Those are LakeCat implementation details around standard Iceberg behavior. They are not Iceberg extensions, because the PySpark client does not change its table model or call a private endpoint. They are the discipline that lets a standard catalog become inspectable and replayable.

Sail enters the PySpark story where the catalog would otherwise need table-format knowledge. If a commit needs metadata preparation or validation, or if LakeCat needs to validate standard response shape against Iceberg models, the reusable implementation belongs in Sail. The catalog should not grow its own partial manifest reader or metadata-version validator simply because the client happened to be Spark. Spark gets the standard surface. LakeCat gets the durable transaction. Sail keeps the table-format semantics reusable.

A notebook or data-service workflow is slightly richer. A Python or Rust service may ask the catalog to plan a read, fetch tasks, inspect metadata, or obtain short-lived access. In standard Iceberg terms, the engine still needs schemas, projections, partition specs, snapshots, manifests, statistics, delete files, and scan tasks. LakeCat can add request identity, purpose, allowed columns, required row predicates, credential TTL caps, and receipt evidence. The first group is Iceberg and engine work. The second group is LakeCat/TypeSec control-plane work.

That distinction matters because a governed notebook should not rely on a client promise to behave. If policy narrows the request to five columns and a mandatory row predicate, LakeCat records the requested projection and the effective projection, asks TypeSec for the receipt, and calls Sail with the effective request. Sail binds fields by Iceberg field id, handles schema evolution, evaluates partition and manifest statistics conservatively, attaches delete files, and returns a plan shape the catalog can expose. LakeCat then stores proof of the narrowed plan. The standard table is unchanged. The governed evidence is additive.

The governed agent workflow is the reason LakeCat cannot be only a passive pointer map. An agent may ask for data in order to answer a question, train a model, enrich a graph, or take an action. Giving that agent raw object-store credentials is often the wrong default. The safer path

is to treat raw credential vending as an audited exception and treat Sail-planned reads as the normal governed path. LakeCat identifies the agent, asks TypeSec for a decision, derives a read restriction, asks Sail to plan against the current metadata pointer, and returns only the narrowed work or credential posture the policy allows.

In standard Iceberg parlance, the agent workflow still touches a normal table. The scan still comes from Iceberg metadata. The delete files still mean what Iceberg says they mean. The snapshots and manifests are not QueryGraph objects. The LakeCat/TypeSec additions are the receipt, purpose, TTL, allowed-column set, row predicate, raw-credential exception proof, block reason, and replay validation. Those additions are extensions around catalog access, not custom table metadata. They are good candidates for future optional governed-access profiles precisely because they can be described without changing the table.

An operator workflow exercises a different surface. Operators need to know which metadata pointer is current, which commits won, which retries were exact, which idempotency keys drifted, which credentials were issued or withheld, and which outbox events have reached graph and lineage sinks. Iceberg table metadata answers part of this story: it records table snapshots and metadata history. LakeCat pointer logs answer the catalog part: which transaction advanced the pointer, under which principal, request hash, response hash, idempotency-key hash, policy hash, and sequence. Audit and outbox rows answer authority and delivery questions.

Those operator surfaces should be viewed as LakeCat management extensions. They are not prerequisites for a standard client. They are not private fields inside metadata JSON. They are the catalog's operational memory. Some of them could become future Iceberg-adjacent proposals. Pointer-history inspection, idempotent replay profiles, and catalog event streams are broadly useful. The proposal shape should be optional REST or event profiles, not a requirement that every Iceberg table embed one deployment's audit vocabulary.

The lineage workflow shows why the outbox matters. If a table commit, scan, credential decision, or view update emits OpenLineage directly from the HTTP handler, lineage becomes best effort. The handler may fail after committing. The sink may be down. A retry may emit duplicates. LakeCat instead persists the catalog fact and outbox row in the transaction, validates the durable evidence on replay, then projects to OpenLineage. OpenLineage is not Iceberg, and it is not QueryGraph, but it becomes more trustworthy when it is bound to committed catalog state.

The graph workflow follows the same rule. LakeCat should not become a graph database. It should emit catalog-facing graph facts at the boundary: warehouses, namespaces, tables, views, commits, scans, credentials, management changes, and receipt anchors. Grust should own graph taxonomy, storage, projection logic, traversal, Cypher support, and graph query behavior. QueryGraph should compose the semantic application graph. In that workflow, Iceberg supplies the table truth, Sail supplies the engine interpretation, LakeCat supplies durable catalog evidence, Grust supplies graph mechanics, and QueryGraph supplies meaning.

The QueryGraph bootstrap workflow is therefore an integration flow, not a replacement catalog protocol. QueryGraph can ask LakeCat for a bootstrap bundle or QGLake handoff that includes warehouses, namespaces, tables, views, current pointers, commit proof, scan proof, credential

posture, view receipt chains, management inventory, OpenLineage receipt hashes, and replay-verification results. QueryGraph can then align those facts with Croissant, CDIF, OSI, ODRL, and application semantics. That handoff is broad by design, but it remains an optional LakeCat/QueryGraph extension. Standard clients do not need it, and standard Iceberg metadata does not carry it.

This workflow view also answers whether LakeCat's ideas should be called extensions or future Iceberg features. The answer is intentionally split:

1. Rust service spine and Turso local store are implementation choices. They make LakeCat fast, durable, and inspectable, but they are not Iceberg extensions and should not become Iceberg features.
2. Iceberg REST namespace and table paths, current metadata pointers, manifests, delete files, snapshots, and optimistic commits are standard Iceberg. LakeCat should implement them faithfully.
3. Idempotency records, pointer logs, audit rows, outbox rows, replay validation, and management proof are LakeCat control-plane mechanisms. They are optional extensions around the catalog.
4. TypeSec receipts, secure-agent decisions, ODRL-derived restrictions, governed credential posture, and raw-credential exception proof are governance extensions.
5. QueryGraph bootstrap, Croissant/CDIF/OSI semantic import, Grust graph projection, and QGLake handoff are application and integration extensions.
6. Future Iceberg-adjacent proposals should be limited to behavior that is useful across catalogs and engines: idempotent commit replay, pointer history, catalog event streams, governed credential vending, proof-carrying scan planning, view lifecycle proof, and lineage receipt binding.

The common thread is that every extension stays additive. A PySpark job should keep working through the normal catalog path. A governed agent can opt into the proof-carrying path. An operator can inspect management evidence. QueryGraph can bootstrap a graph. The Iceberg table remains portable, and the strongest future proposals emerge from evidence that has already worked in real workflows.

That is the practical argument for pushing work into the engine. If LakeCat plans scans itself, it risks creating private semantics for every workflow. The PySpark path and the governed-agent path might disagree about field ids, partition pruning, delete files, v4 metadata interpretation, or expression binding. If Sail owns those semantics, the ordinary and governed paths share one engine truth. LakeCat can add identity, policy, receipts, transactions, audit, outbox, graph, and lineage without becoming a second table engine.

Sail is a particularly strong engine choice for that role because it is already the Rust side of the lakehouse. It is close to generated Iceberg REST models, Arrow/DataFusion execution objects, metadata-as-data, manifest handling, provider abstractions, and future v3/v4 format support. It can be improved once and reused by LakeCat, QueryGraph, notebooks, maintenance tools, and any other Rust lakehouse surface. That is how LakeCat can be ambitious without becoming inva-

sive: the catalog owns trust and state, Sail owns table semantics, TypeSec owns policy meaning, Grust owns graph behavior, and QueryGraph owns the end-to-end semantic application.

1.16 LakeCat's Thin Boundary

LakeCat should be thin, but thin does not mean trivial. It owns the durable catalog state that must be correct even when external sinks are unavailable.

The core LakeCat responsibilities are:

- Serve the Iceberg REST Catalog API for standard clients.
- Model projects, warehouses, namespaces, tables, views, and storage profiles.
- Persist metadata pointers and compare-and-swap commit history.
- Validate idempotency keys and replay only matching commit bodies.
- Resolve request identity from headers, bearer tokens, agents, and TypeDID envelopes.
- Ask TypeSec for authorization decisions and persist receipts.
- Route scan planning and commit preparation through Sail.
- Record audit and outbox events inside the catalog transaction.
- Drain committed events to Grust and OpenLineage sinks.
- Publish a QueryGraph bootstrap bundle.

The deliberately excluded responsibilities are just as important:

- LakeCat does not invent a table format.
- LakeCat does not fork Iceberg manifest pruning.
- LakeCat does not own graph traversal or graph query behavior.
- LakeCat does not own security semantics or agent trust semantics.
- LakeCat does not author QueryGraph's final business semantic model.

That boundary gives each sibling project a clear job. Sail owns reusable Iceberg and planning behavior. Grust owns graph schema, storage, and query mechanics. TypeSec owns policy, capabilities, TypeDID envelopes, secure agents, and authorization semantics. QueryGraph owns the semantic application built on top.

1.17 The Read Path

The LakeCat read path begins like a standard catalog request and ends with a governed Sail plan.

1. A client asks to load or plan a table through the Iceberg REST surface.
2. LakeCat resolves the warehouse, namespace, table, and current metadata pointer.
3. LakeCat resolves the request identity.
4. LakeCat asks TypeSec whether the principal can perform the requested action.
5. TypeSec returns a decision and any enforced restrictions.
6. LakeCat turns those restrictions into a ReadRestriction: allowed columns, required row predicate, purpose, policy hash, and audit context.
7. LakeCat asks Sail to plan against the current metadata pointer with the effective projection and filters.

8. Sail validates Iceberg expressions against generated REST models and table schema, expands manifests, applies conservative file-bound pruning when metrics are present, and carries delete-file references into file scan tasks.
9. LakeCat returns Iceberg-compatible plan and task responses.
10. LakeCat records audit and outbox events that can later be projected into graph and lineage.

The important detail is that the policy restriction becomes part of planning, not a note beside it. An empty client projection under a column restriction means the allowed columns. A client projection can narrow further, but cannot widen. LakeCat records both the client's requested projection and the effective projection that survived the server-derived column restriction in the durable scan-planned replay evidence. The same rule applies to stats-field requests: LakeCat records both the client's requested stats fields and the effective stats fields that survived the restriction, while the compatibility `stats-fields` extension remains the narrowed effective set. The default REST path is tested at the Sail boundary: Sail receives only the effective projection and mandatory policy filters, while LakeCat keeps the broader request and narrowed result as replay evidence. During `fetchScanTasks`, LakeCat recomputes the current restriction and sends Sail the required projection and mandatory filters again; the response extension and audit outbox record the same proof. A stale or legacy token cannot silently expand back to all columns. Outbox admission also checks that governed planned/fetched scan replay carries the same `read-restriction` in the top-level payload and in `authorization-receipt.context.read-restriction`, so replay cannot claim policy narrowing that the durable receipt did not capture. The same admission boundary requires governed scan replay to keep a nonblank purpose and a positive `max-credential-ttl-seconds` value before graph or OpenLineage projection, so a QGLake handoff cannot learn task evidence whose purpose or credential TTL cap was lost before replay.

1.18 The Commit Path

The write path follows the same principle: LakeCat owns the catalog transaction, Sail owns reusable Iceberg validation and metadata preparation.

1. A client sends an Iceberg commit request, optionally with an idempotency key.
2. LakeCat validates the request shape and the idempotency key.
3. LakeCat resolves identity and asks TypeSec for the commit capability.
4. If the idempotency key already has an exact stored response, LakeCat returns that response before Sail validation or metadata-object writes.
5. LakeCat loads the current metadata pointer from the store.
6. LakeCat delegates Iceberg update validation and metadata assembly to Sail.
7. LakeCat rejects metadata-object writes that target the table's current metadata pointer, so the current metadata file cannot be overwritten before the store commit has won.
8. LakeCat rejects metadata-write plans that do not carry a concrete new metadata location.
9. LakeCat rejects metadata-object locations outside the table's matched storage profile prefix, and also rejects the storage-profile root itself because metadata commits must create a child object.

10. LakeCat rejects literal or percent-encoded dot path segments in metadata object locations, so commit plans cannot rely on traversal-like spelling to address anything other than a plain child object.
11. LakeCat writes the new metadata object through the warehouse storage profile with create-only object-store semantics.
12. LakeCat advances the table pointer with compare-and-swap.
13. LakeCat persists idempotency, audit, pointer-log, and outbox records.
14. If the store rejects the commit after a local metadata write, LakeCat cleans up the uncommitted metadata object when it can do so safely.
15. Outbox draining projects the committed event to graph and lineage sinks.

The Turso-backed store binds decoded table JSON back to the selected table identity on this path. A row selected for `local.default.events` cannot return or replay `record_json` or idempotency `response_json` that claims another table. LakeCat rejects that drift before loading a table, listing standard catalog tables, replaying an idempotent commit response, committing over the row, soft-deleting it, or restoring it. That is not an Iceberg extension; it is durable-store hygiene around standard Iceberg table access.

The cleanup path is deliberately secondary to the commit result. If metadata cleanup fails after the store rejects a commit, LakeCat preserves the original store or compare-and-swap error class and appends cleanup context. A stale pointer conflict still looks like a conflict to an Iceberg client, but the message carries SHA-256 hashes of the expected and actual metadata locations so operators can diagnose the race without exposing raw object paths. The service-level regression for this path checks the API response and the filesystem side effect together: the rejected metadata object is gone, while the conflict body contains only hashed pointer evidence. True cleanup failures use the same redaction discipline: the cleanup context identifies the uncommitted metadata object by `metadata-location-hash=sha256:...`, not by the raw object path. When that cleanup failure is appended to the preserved commit conflict, LakeCat keeps only `error-detail-hash=sha256:...` evidence for the cleanup detail, so raw backend text cannot leak through the combined conflict message. If cleanup discovers the uncommitted object is already absent, LakeCat treats that as successful cleanup rather than turning a resolved orphan into an internal error. Cleanup also refuses to delete the previous committed metadata pointer if a future plan accidentally reports it as the staged write; the committed metadata object remains the table's current state, not an orphan. The same audit-safe shape applies before the write: current-pointer overwrite, existing-object overwrite, unsupported object-store configuration, and storage-profile-prefix failures report metadata-location hashes, and prefix mismatches also report a storage-profile-prefix hash rather than raw object paths. LakeCat also keeps the storage-profile id out of this error text, so tenant or profile naming conventions do not leak when a planned metadata object falls outside the selected root. A root-targeted metadata write uses the same redacted error shape: the operator sees that the plan did not name a child metadata object without receiving the raw table or storage root. Dot-segment failures use the same style: literal `..` and percent-encoded `%2e%2e` paths fail before object-store writes and expose only the metadata-location hash. Decorated metadata object locations with URI query strings, fragments, or URI userinfo are rejected at the same pre-write

boundary, so a commit plan cannot smuggle version selectors, backend hints, fragment markers, or embedded credentials into what should be a plain metadata object address.

Idempotency is part of correctness. Reusing the same key for the same commit can return the stored response even after the table has advanced beyond the original commit requirements. Reusing the same key for a different body must conflict. LakeCat persists a normalized request hash and stores only audit-safe evidence, not raw secrets or raw idempotency keys. REST idempotency keys are intentionally narrow: `x-lakecat-idempotency-key` must be 1 to 128 ASCII characters and may use only letters, digits, `-`, `_`, `.`, or `:`. Invalid keys, including non-ASCII or invalid header bytes, fail before LakeCat performs authorization, Sail validation, table loading, or metadata-object writes. When a reused key is attached to a different commit body, the conflict response also stays redacted: it does not echo the raw key or the mismatched metadata object location. The Turso spine pins the same redaction for both commit-time reused-key conflicts and explicit idempotency replay probes: the raw key, mismatched request hash, and mismatched metadata object location are not operator-facing error text. The service regression for this path proves the replay happens before metadata-object writes: an exact retry returns the stored response without touching the already committed metadata object. The same regression pins the outbox side effect: exact replay and mismatched reused-key conflicts leave only the original `table.commit` outbox event, so `QueryGraph` and `OpenLineage` replay do not see duplicate commit work from retry traffic. Another regression sends a commit whose requested metadata location is the table's current pointer and verifies that LakeCat returns a bad request without touching the existing metadata file. Another sends a commit to a different metadata location that already exists and verifies that LakeCat returns a conflict without overwriting that non-current object. The same guard fails closed if a future Sail plan asks LakeCat to write metadata but does not provide a new object location, or if it tries to use the storage profile root as the new metadata object. A companion regression rejects both literal and percent-encoded dot path segments in a planned metadata location. When a backend object store fails setup, create-only write, or cleanup, LakeCat keeps the metadata location hash and adds hash evidence instead of returning raw backend text. Invalid metadata URI parsing and unsupported backend setup failures use `backend-error-hash=sha256:...`, making the setup-admission boundary explicit. Create-only write and cleanup failures keep `error-detail-hash=sha256:...` because those happen after setup. In every case, the response names the hashed metadata location and hashed failure detail, not the submitted path, object name, scheme, or parser/backend diagnostic. That route-level promise is pinned by a commit regression for decorated metadata locations with raw query-token material. It matters for local files, cloud bucket keys, and credential-provider diagnostics: operators can correlate a failure without copying sensitive storage topology into API responses or logs.

The embedded in-memory store follows the same commit evidence contract as the Turso path. A successful commit emits one `table.commit` audit/outbox event with the compact commit record, authorization receipt, response hash, and redacted idempotency-key hash. An idempotent replay returns the stored response without adding a second outbox event, so tests and local embedded deployments exercise the same outbox invariant as the durable spine.

Commit records also carry a response hash over the stored table response. That pair matters: the request hash proves which commit body won or replayed, while the response hash proves which metadata pointer and table body LakeCat returned to clients and later projected through graph and lineage replay.

The same commit record includes compact summary evidence: Iceberg format version, current snapshot id, and the policy hash from the authorization receipt when one exists. Memory and Turso commit producers now require positive Iceberg format-version evidence before table or commit metadata can produce a durable commit record. If the table metadata has no current snapshot, the producer emits explicit `snapshot_id: 0` evidence instead of omitting the field, preserving no-snapshot Iceberg states without creating an undrainable `table.commit` event. QueryGraph can inspect those fields from the pointer-log/outbox stream without parsing full table metadata for every catalog audit question. Before a `table.commit` outbox event is projected or acknowledged, LakeCat now checks that it carries a commit object, an unsigned sequence number, a decodable root table identity, matching nested commit-table identity when present, both the commit principal and authorization receipt principal with matching values, positive format-version evidence, non-negative snapshot-id evidence, and full sha256:-prefixed 64-hex request, response, idempotency-key, and present policy hashes. A prefix-shaped placeholder, contradictory commit identity, missing receipt principal, missing table-format evidence, or drifted principal cannot become delivered commit replay evidence.

Operators and QueryGraph can read that pointer-log evidence through a governed management endpoint:

```
curl -s \
  -H 'x-lakecat-principal: operator@example.com' \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/tables/
events/commits
```

The response contains compact commit records: sequence number, previous and new metadata locations, request hash, response hash, idempotency-key hash, Iceberg format version, current snapshot id, policy hash, principal, and a commit hash. The read itself enters the durable outbox as `table.commits-listed` and drains as lineage evidence plus catalog-facing Commit graph anchors keyed by table and sequence number. That gives audit tools and QueryGraph a Grust-visible pointer-log inspection trail without requiring direct access to the Turso catalog database or making LakeCat a graph query engine. The outbox payload also carries `principal-subject` and `principal-kind`, and service replay admission requires those fields to match the authorization receipt principal before graph or OpenLineage projection. QGLake acceptance now exercises this path directly: the fixture issues an idempotent no-op commit probe, reads the compact commit-history endpoint, verifies that the record preserves the table's Iceberg format-version and current snapshot summary, requires the compact request, response, idempotency-key, commit, and optional policy hashes to be full sha256:-prefixed 64-hex digests, and then requires the lineage drain to replay `table.commits-listed` receipt hashes plus compact commit count, sequence-number, and commit-hash summary fields before the QueryGraph handoff is accepted.

1.19 The Durable Spine

LakeCat's durable local spine uses the Rust `turso` crate behind the `turso-local` feature. The store contract remains portable, but the local foundation is not SQLx. The important tables are not an application afterthought; they define the catalog's control-plane memory:

- projects and warehouses;
- storage profiles;
- namespaces and tables;
- metadata pointer log;
- idempotency records;
- soft deletes;
- policy bindings;
- audit events;
- outbox events.

Object storage remains the source of Iceberg metadata files. Turso stores the atomic pointer, management state, idempotency evidence, and event record. This mirrors the Iceberg catalog contract: metadata files describe the table; catalog state decides which metadata file is current.

Outbox draining is intentionally strict. LakeCat projects a batch to graph and lineage sinks first, then acknowledges the whole projected batch in the store. Embedded and Turso stores select the same pending prefix by sorting on `created_at`, `event_id` before applying the drain limit, so a small batch means the same replay set in either durable backend. The drain response and delivery acknowledgement follow that ordered prefix, leaving later pending events for a future drain. If projection fails, nothing is acknowledged. If the store reports that fewer events were acknowledged than LakeCat projected, the drain fails with an acknowledgement mismatch instead of returning a quiet partial success. That keeps retry and operator evidence honest when a concurrent drain or backend anomaly interferes with delivery accounting. The regression suite covers the uncomfortable middle case too: if the first event in a multi-event batch already projected to graph and lineage but a later event fails during lineage projection, LakeCat still acknowledges none of the events. Recovery starts from the committed outbox batch instead of from a half-delivered response. The drain also refuses unknown event types before any projection happens. A future or custom event stays pending until LakeCat knows how to project it, instead of disappearing behind an empty graph/lineage receipt. The drain also validates governed-read evidence before projection. If a pending event contains a `read-restriction.policy-hashes` array, it must be non-empty and each entry must already be a full `sha256:-prefixed 64-hex` digest. A readable placeholder such as `sha256:policy-name`, or an empty policy anchor array, fails the drain before graph or lineage sinks run and before the store can mark the event delivered, keeping malformed source evidence available for retry or operator repair instead of promoting it into a QGLake handoff. LakeCat now applies that same admission rule to `authorization-receipt.context.read-restriction.policy-hashes`, so the receipt kept for later proof cannot preserve an empty or placeholder policy anchor while the top-level scan event looks valid. LakeCat also rejects both planned and fetched scan replay when the top-level `read-restriction` differs from `authorization-receipt.context.read-restriction`, so graph and OpenLineage

evidence cannot drift from the TypeSec receipt that authorized the narrowed read. Planned and fetched scan replay must also carry nonblank purpose evidence and a positive policy-derived credential TTL cap before the outbox event can be acknowledged or projected. Table commit events receive the same treatment for compact commit receipt evidence: `request_hash`, `response_hash`, `idempotency_key_sha256`, and any present `policy_hash` must be full digests before the event can be projected or acknowledged.

1.20 Grust For Graph Concepts

Catalog events naturally form a graph. A server contains projects. A project contains warehouses. A warehouse contains namespaces and storage profiles that define credential roots. A namespace contains tables. A table has columns, snapshots, manifests, files, policies, commits, scan plans, principals, and lineage runs. QueryGraph needs that graph, but LakeCat should not become a graph database.

Grust owns the graph layer. It is the place for reusable graph taxonomy, projection builders, graph stores, traversal indexes, Cypher support, and typed or untyped graph operations. LakeCat's responsibility is narrower: translate committed catalog events into a bounded envelope and pass it through a catalog-facing sink.

In practice this means LakeCat can emit graph events for stable catalog facts:

```
Server CONTAINS Project
Project CONTAINS Warehouse
Warehouse CONTAINS Namespace
Namespace CONTAINS Table
Table HAS_COLUMN Column
Table GOVERNED_BY Policy
Warehouse HAS_STORAGE_PROFILE StorageProfile
Principal CAN_PLAN ScanPlan
Commit DERIVED_FROM Snapshot
LineageRun USED_BY QueryGraphModel
```

High-cardinality file and manifest facts should stay queryable through Iceberg/Sail metadata-as-data unless Grust provides a reusable taxonomy and storage strategy for them. The graph should be powerful, but the catalog must not smuggle a second lakehouse engine into its event sink.

The current local direction already proves the boundary: LakeCat's `grust-local` sink calls Grust-owned LakeCat projection helpers, and the Grust Cypher boundary test verifies catalog graph projection without making LakeCat own Cypher parsing, traversal, or graph execution. The current boundary test writes table-adjacent `Column`, `Snapshot`, and `Commit` events plus `Principal`, `ScanPlan`, tenant-root `Server`, and credential-root `StorageProfile` catalog events through Grust, then matches catalog-event labels through Grust Cypher. Storage profile replay uses redacted evidence such as `secret-ref-present` and the `secret-reference` provider, never the full `secret-store` URI. Credential-vend attempts replay through that same thin boundary as `StorageProfile` graph events keyed by the redacted credential-root anchor, so QueryGraph can see a principal attempted credential-root access without seeing a secret reference or raw credential material.

That proves QueryGraph can discover the semantic anchors LakeCat emits while the richer node/edge materialization remains reusable Grust work.

1.21 TypeSec For Security

LakeCat is a policy enforcement point, not the author of security semantics. TypeSec owns the semantics: RBAC, ODRL-style policy composition, typed capabilities, TypeDID envelopes, secure agents, credential issuance decisions, and authorization proofs.

Every externally meaningful action should pass through TypeSec:

- catalog configuration reads;
- namespace creation, listing, loading, and dropping;
- table creation, load, scan planning, commit, drop, and restore;
- credential vending;
- policy management;
- graph and lineage reads.

LakeCat gathers the request context and asks TypeSec for a decision. The context can include principal DID, agent DID, bearer-derived subject, warehouse, namespace, table, columns, snapshot, requested credential duration, purpose, and active policy bindings. TypeSec returns a decision and receipt. LakeCat persists the receipt with audit-safe hashes and applies the resulting restrictions before Sail plans.

This is where ODRL becomes operational. An ODRL-style policy can say that a principal may read only certain columns, only for a purpose, or only with an enforced row predicate. LakeCat parses the minimal enforceable subset it needs to narrow the plan, but policy composition and authorization semantics belong to TypeSec. LakeCat should ask TypeSec, not grow a parallel security language. When that subset is expressed through ODRL constraints, LakeCat accepts only operators that actually mean “use this as the allowed or narrowing value”; missing or deny-shaped operators fail closed instead of being treated as governed read permission. The bounded parser accepts camel-case, kebab-case, and prefixed JSON-LD operand keys such as `odrl:leftOperand` and `odrl:rightOperand`. It also accepts compact JSON-LD term objects such as `{"@id": "odrl:eq"}` for constraint operands and operators, plus JSON-LD `@value` and `@list` right operands for bounded allowed-column, purpose, and credential-TTL values. Malformed JSON-LD lists still fail closed, and the parser does not turn LakeCat into a full ODRL reasoner.

Namespace events follow the same receipt discipline as table, view, and management events. A namespace list proves namespace-list; creation proves namespace-create; loading proves namespace-load; dropping proves namespace-drop. Replay admission rejects action drift before graph or OpenLineage projection, so standard Iceberg namespace behavior cannot become QueryGraph evidence under the wrong TypeSec-style authority. Recognized constraint operands must also include a right operand; otherwise LakeCat rejects the policy material instead of silently dropping an allowed-column, row-predicate, purpose, or credential-TTL restriction. The derived restriction also rejects empty or blank allowed-column lists and blank purposes before they can reach credential issuance or governed Sail planning and fetch paths. The service route pins this behavior too: a table scan with a malformed active ODRL restriction, including malformed

JSON-LD allowed-column lists, fails before `Sail` planning and before `table.scan-planned` replay evidence is emitted. A `fetchScanTasks` call with the same malformed JSON-LD active policy fails before `Sail` fetch execution and before `table.scan-tasks-fetched` replay evidence is emitted, and a credential request with the same malformed JSON-LD active policy fails before issuer dispatch and before `credentials.vend-attempted` replay evidence is emitted. Purpose is composed the same way: every purpose source in the active policy material must agree. If one binding says a read is for `resilience-demo` and another says `training`, LakeCat rejects the restriction instead of guessing which purpose should follow the agent into `Sail` planning, credential TTL proof, and `QueryGraph` handoff evidence.

Credential vending follows the same rule. Raw credential vending is an audited exception. Governed `Sail`-planned reads are the default path for agents and untrusted principals. When credentials must be issued, `TypeSec` checks the `credentials.issue` capability for the exact secret reference and LakeCat returns only scoped, short-lived credential configuration. If policy carries a `max-credential-ttl-seconds` restriction, LakeCat passes that cap to the credential issuer and annotates each returned credential with `lakecat.max-credential-ttl-seconds`, so the exception path has an auditable duration bound. If the cap appears in multiple supported ODRL locations in the same policy document, or across multiple active policy bindings, LakeCat keeps the tightest value before asking the credential issuer. If an issuer returns that LakeCat TTL key itself, LakeCat normalizes the response to one TTL entry per credential and keeps the stricter valid TTL, so duplicate backend-supplied entries cannot widen or confuse the policy cap. The same response boundary owns the rest of the LakeCat evidence: issuer-supplied values for `lakecat.storage-profile-id`, `lakecat.storage-provider`, `lakecat.credential-mode`, `lakecat.authorization-principal`, `lakecat.governed-read-required`, and `lakecat.secret-ref-provider` are removed and replaced with catalog-derived values before the response is returned. The REST credential-vending regressions exercise this at the public response boundary: a backend can return multiple TTL entries or forged catalog evidence, but `loadCredentials` exposes one canonical proof while preserving issuer-owned credential details such as credential kind and provider session tokens. LakeCat records the same decision shape in audit/outbox evidence without copying raw credentials: each vended credential gets a hashed prefix, canonical LakeCat evidence values, and a hash of issuer-owned config. Replay can prove the response posture, but it does not inherit cloud session tokens. If the credential event carries a governed read restriction, outbox admission requires the top-level `read-restriction` to match `authorization-receipt.context.read-restriction`, keeping TTL and blocked-read evidence inside the durable receipt. Raw credential exceptions follow the same rule: the top-level `lakecat:raw-credential-exception` object must match `authorization-receipt.context.lakecat:raw-credential-exception` exactly, so trusted-human exceptions and blocked-agent denials cannot drift during replay.

1.22 Rust-First Engines And The V3 To V4 Path

The new Rust-first engine path matters because it changes where catalog intelligence can live. `Sail` is not a Java service with Rust bindings bolted on the side. It is a Rust engine path built around

Arrow, DataFusion, generated Iceberg REST models, catalog provider traits, manifest pruning, metadata-as-data scans, and table-status conversion.

That shape gives LakeCat a better option than reimplementation. LakeCat can ask Sail for typed Iceberg behavior instead of parsing just enough JSON to survive a request. That matters for Iceberg v3 and the emerging v4 work. Format v3 already pushes catalogs and engines toward richer metadata, row lineage, deletion semantics, and better interoperability around advanced table state. Format v4 is still settling, but it is plainly moving the lakehouse toward more adaptive and structured metadata rather than less.

LakeCat should evolve under three rules:

1. Conform to Iceberg v3 for ordinary clients.
2. Preserve unknown and emerging v4 metadata without claiming settled semantics.
3. Prefer typed Sail support as soon as Sail exposes it, using JSON passthrough only as a compatibility bridge.

That gives LakeCat room to support v4-ready capability flags, round-trip tests, metadata extension preservation, and future metadata-tree planning without forking Iceberg. The catalog can become more intelligent while the table remains portable. The standard path stays boring. The governed Sail path becomes richer.

The reason to push work into the engine is not architectural tidiness. It is correctness. Iceberg semantics are field-id semantics, snapshot semantics, manifest semantics, delete semantics, and metrics semantics. A catalog can guard the pointer, but it cannot safely become a second planner without reimplementing the engine. The moment LakeCat starts doing its own file pruning, delete application, partition tuple decoding, field-id projection, residual filter evaluation, or v4 metadata interpretation, it risks drifting from the engine that will actually read the files.

Sail is a strong engine choice because it is already close to the representation LakeCat needs to trust. It is Rust-native, it speaks Arrow and DataFusion, it has Iceberg REST model generation and catalog-provider seams, and it can expose metadata-as-data without routing everything through a JVM adapter. That means LakeCat can keep the catalog transaction small and ask Sail questions that belong to an engine:

- Which Iceberg field ids satisfy this requested projection?
- Which required filters are enforceable at planning time?
- Which manifests and files survive partition and statistics pruning?
- Which delete files must accompany the selected data files?
- Which manifest metrics are trustworthy enough for stats-field proof?
- Which scan tasks are children of a governed parent plan?
- Which v4 fields are known, which are preserved as passthrough, and which are not yet safe to interpret?

Those answers should come from Sail because they require table-format knowledge and execution-plan discipline. LakeCat should persist the request, the TypeSec decision, the effective restriction, the plan/fetch receipts, and the replay evidence. Sail should own the reusable mechan-

ics that turn current Iceberg metadata into tasks and validation. QueryGraph should consume the proof and project it into graph, lineage, and agent workflows.

This division also makes standards work easier. A future optional Iceberg profile for proof-carrying scan planning should be engine-shaped: field ids, snapshot ids, manifest-list anchors, projection evidence, filter evidence, delete-file evidence, and task lineage. If LakeCat proves that profile by calling Sail, another Rust engine can reuse the same semantics. If LakeCat hand-rolls it, the proof becomes a LakeCat-specific story.

The current v4 bridge is intentionally narrow and tested as such. When LakeCat sees `format-version: 4`, it does not pretend that Sail already has a settled typed v4 model. Instead, `lakecat-sail` extracts the stable JSON envelope fields that remain useful across versions: table UUID, location, schema id, snapshot id, sequence number, manifest-list path, default spec, and field names. It can plan a governed manifest-list scan task from that envelope and validate the signed plan task again during `fetchScanTasks` so a stateless fetch cannot drift to a different manifest list or widen the governed projection and filters. It also validates stable commit requirements such as table UUID, current schema id, main snapshot id, last assigned field id, and default spec id. Pruning and typed metadata-tree semantics wait for Sail-owned v4 support.

1.23 OSI, OpenLineage, And Responsible Semantic Handoff

QueryGraph needs more than physical table access. It needs a semantic picture: datasets, fields, policies, lineage, graph relationships, and anchors that can survive import. LakeCat should publish that picture without pretending to be QueryGraph.

The LakeCat bootstrap bundle contains:

- Semantic Croissant and CDIF projections for dataset and field discovery.
- An OSI handoff with stable dataset and field anchors.
- ODRL policy artifacts and TypeSec policy context.
- OpenLineage events for catalog changes, scan plans, commits, and maintenance.
- A Grust-ready graph envelope.
- A manifest that hashes each emitted artifact.

The OSI boundary is deliberately careful. LakeCat should not author rich business metrics, dimensions, joins, ontology claims, or authoritative semantic names. It can publish stable anchors and governed source metadata. QueryGraph owns the final semantic model.

This is the Responsible Semantic Layer boundary. Semantic Croissant and CDIF make datasets and fields discoverable and exchangeable. OSI gives QueryGraph a stable handoff for semantic anchors without forcing LakeCat to own business meaning. OpenLineage records how catalog and planning events happened. Together they let the semantic layer be responsible because it can be traced back to catalog state, policy, and lineage, not just to a hand-authored model file.

OpenLineage fits the catalog outbox. A committed table create, scan plan, commit, soft delete, restore, or maintenance action can become a lineage event. Because the event is drained from a durable outbox after the catalog transaction, lineage reflects committed state rather than a handler's best-effort side effect. Drains process pending events in `created_at,event_id` order

before projection, response summarization, and delivery acknowledgement. That stable order is part of the replay contract QueryGraph and OpenLineage consume, and it holds even when a custom store implementation returns a pending batch in another order.

1.24 QueryGraph.ai When LakeCat Is Done

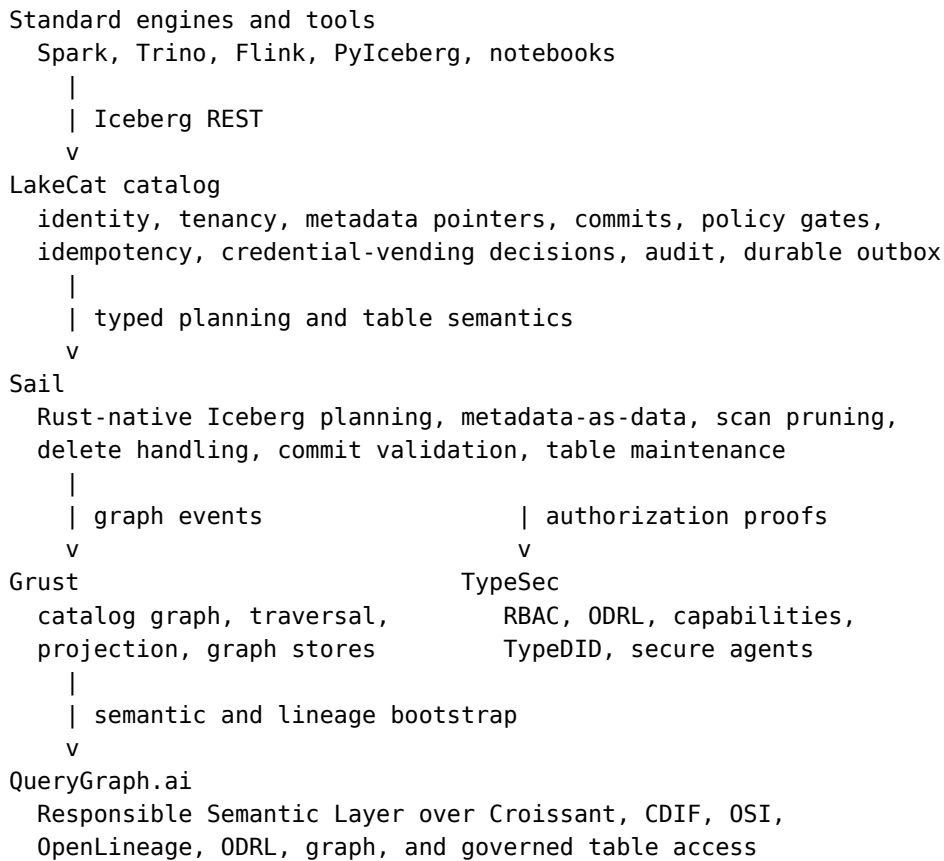
QueryGraph.ai is the enterprise lakehouse this work is pointing toward. QueryGraph needs the catalog to be more than a storage address book. It wants to answer questions over data, metadata, semantics, policy, and lineage as one governed graph. That requires a catalog foundation that can speak to ordinary Iceberg clients and also publish trustworthy control-plane facts.

LakeCat supports that foundation by exposing a QueryGraph bootstrap endpoint:

```
/querygraph/v1/bootstrap
```

The bundle gives QueryGraph an import contract. QueryGraph can verify hashes, load catalog graph envelopes through Grust, inspect policy artifacts through TypeSec, and attach semantic modeling work to stable dataset and field anchors. The import path should prove that LakeCat is the substrate, not a standalone demo.

When LakeCat is done, the QueryGraph.ai architecture looks like this:



Basic catalog use remains optional and standard. A normal Iceberg engine can load and commit tables without knowing QueryGraph exists. The enhanced path is there for enterprises that want

more: governed Sail-planned reads, TypeSec authorization receipts, ODRL rights, TypeDID agent identity, OpenLineage replay, Semantic Croissant/CDIF publication, OSI handoff, and Grust graph loading.

That is the core motivation for LakeCat. The next QueryGraph.ai should not bolt governance, graph, and lineage onto tables after the fact. It should begin from a catalog that already records governed state transitions and exposes standard, engine-close planning.

1.25 Implementation Shape

The current workspace shape expresses the architecture directly:

```
crates/
lakecat-core      stable IDs, errors, time, config, content hashes
lakecat-api       Iceberg REST request/response adapters
lakecat-store     catalog state traits and Turso-backed implementation
lakecat-sail      Sail provider bridge and privileged planning client
lakecat-graph     catalog-facing Grust sink/adapters
lakecat-security  TypeSec integration and authorization receipts
lakecat-lineage   OpenLineage projection and event receipts
lakecat-querygraph Croissant/CDIF/OSI/ODRL/OpenLineage bootstrap projection
lakecat-service   axum service, middleware, auth, routing
lakecat-cli       admin, local demo, conformance, bootstrap export
```

Feature gates keep integrations honest:

```
sail-local    use local Sail APIs for planning and provider integration
typesec-local use local TypeSec APIs for governance and TypeDID verification
grust-local   use local Grust APIs for catalog graph projection
turso-local   use the Turso-backed durable store
```

Embedded defaults stay safe for tests. Real integrations are explicit. That matters because LakeCat is a foundation, not a pile of optional demos. A test that only uses memory stores should not accidentally depend on a sibling repo. A test that claims to validate TypeSec should enable `typesec-local` and call TypeSec.

The dependency contract is executable because LakeCat still has one active sibling bridge. Grust and TypeSec now resolve from the published `grust-graph 0.9.0`, `grust-cypher 0.9.0`, and `typesec 0.8.0` crates, so the `grust-local` and `typesec-local` features no longer require sibling checkouts merely to compile. That makes the graph and governance boundaries reproducible outside this machine while still keeping their reusable behavior in Grust and TypeSec.

Sail is different today: LakeCat still uses local Sail paths plus a checked-in patch bridge for helper APIs that are not yet published. Before pushing a slice that touches integration features, run:

[scripts/check-local-dependency-contract.sh](#)

The script checks the manual-only CI trigger, scans every GitHub workflow file for forbidden automatic cloud triggers, verifies crates.io resolution for the published Grust graph/Cypher and TypeSec versions, the local Sail path bridge, the Sail patch files manual CI applies, and the concrete Sail helper API surface LakeCat uses: generated Iceberg REST models, typed metadata inputs,

planning result helpers, fetchScanTasks result helpers, and table-status conversion. It also checks the local QueryGraph Rust importer for the LakeCat view receipt-chain contract: receipt-chain-hash must be preserved in view receipt evidence and missing receipt-chain evidence must fail closed. Manual-only means no automatic push, pull-request, pull-request-target, merge-queue, repository-dispatch, scheduled, workflow-run, or reusable-workflow cloud runs; the local audit fails if any of those triggers appear before the local gates are proven stable, including compact scalar triggers, inline lists and maps, block lists and maps, quoted YAML forms such as "on": ["push"], and quoted event names in trigger blocks. The guard looks specifically at the workflow on: declaration, so a harmless job id such as jobs.push is allowed. The focused workflow-trigger self-test exists so this guard can be checked without running the full dependency audit. It is not a substitute for upstreaming the Sail helper APIs or re-enabling automatic CI; it is a guard that makes drift visible while LakeCat still depends on unpublished Sail helper work and a local QueryGraph acceptance target.

For the first release, LakeCat has one local release gate:

[scripts/check-release-readiness.sh](#)

The full gate runs the dependency contract, the workspace formatting matrix, default workspace tests, QGLake fixture coverage, Turso store tests, Sail, TypeSec, and Grust integration feature tests, an explicit all-features CLI test, the all-features workspace library test, the book build, and the QGLake handoff proof. The default workspace test still covers ordinary doc-tests; the feature matrix targets package unit tests so an empty rustdoc phase cannot hang after the actual Turso/Sail/TypeSec/Grust coverage has passed. The --quick mode keeps script syntax, dependency-contract, formatting, and diff checks cheap enough to run inside narrow implementation slices. Cloud CI remains manual-only until this local gate is boringly green.

As of the current local reconciliation, the Sail helper work is not an anonymous dirty tree. /Users/alex/src/sail has scoped local commits on codex/graph for exposing Iceberg REST models to LakeCat, preserving Iceberg manifest lower and upper bounds in Avro, and adding Sail's Cypher graph query extension. That Cypher extension is a Sail SQL/analyzer/planning surface; the catalog graph taxonomy, projection helpers, traversal, and stores remain Grust responsibilities. The only remaining Sail working-tree entries are untracked artifact/book directories, and pushing the Sail branch upstream is blocked by HTTPS GitHub authentication rather than by local test failures.

1.26 Standard Compatibility And Extensions

LakeCat must be boring where standards require boring behavior. Standard Iceberg clients should be able to use the catalog without knowing QueryGraph exists. Business semantics and agent state must not be required custom Iceberg metadata.

Extensions belong beside the standard path:

- /catalog/v1 serves Iceberg REST compatibility.
- management APIs handle warehouses, policies, profiles, and operational state.
- /querygraph/v1/bootstrap publishes QueryGraph import artifacts.

- feature-gated Sail paths provide governed scan planning and local provider integration.

Format v4 work should follow the same rule. Prefer typed Sail support when available. JSON passthrough can bridge compatibility, but it is not the desired long-term implementation. Round-trip tests should prove LakeCat preserves unknown or evolving metadata without claiming settled semantics too early.

1.27 Workflow Examples

The catalog is easiest to understand by watching it participate in ordinary work. LakeCat should not ask users to think about graph, lineage, security, and Sail every time they read a table. Those systems should appear when they matter: at the boundary where a name is resolved, a policy is enforced, a plan is created, credentials are withheld or issued, and a durable event is replayed.

The examples below use one table, `local.default.events`, but the pattern is the same for larger warehouses. The important point is not the exact sample data. It is the catalog role in each workflow.

1.27.1 Starting The Catalog

A local operator starts LakeCat as an Iceberg REST catalog plus management surface:

```
cargo run -p lakecat-service --features sail-local,turso-local,typesec-local,grust-local
```

The standard catalog path is still `/catalog/v1`. The management and QueryGraph surfaces sit beside it:

```
/catalog/v1
/management/v1
/querygraph/v1/bootstrap
```

A simple health-oriented configuration read shows the split. Standard engines care about the Iceberg endpoints. Operators and QueryGraph care about the management and bootstrap endpoints.

```
curl -s http://127.0.0.1:3000/catalog/v1/config
```

The defaults intentionally separate compatibility from future capability:

```
{
  "defaults": [
    {"key": "lakecat.compatibility", "value": "iceberg-rest"},
    {"key": "lakecat.format.baseline", "value": "iceberg-v1-v3"},
    {"key": "lakecat.format.v4", "value": "extension-ready"},
    {"key": "lakecat.format.v4.bridge", "value": "json-passthrough"},
    {"key": "lakecat.format.v4.typed-sail", "value": "unavailable"}
  ]
}
```

That means LakeCat can preserve and replay emerging v4 metadata through the Sail JSON bridge, but it is not claiming typed Sail v4 semantics yet. The same defaults are stored in catalog config-read replay evidence, and malformed replay that omits the v4 bridge posture is rejected before graph or OpenLineage projection. The replay defaults must also

be ordinary string key/value entries with duplicate-free keys, so a saved outbox event cannot say both `lakecat.format.v4.typed-sail=unavailable` and `lakecat.format.v4.typed-sail=available`. LakeCat also rejects unsupported extra `lakecat.format.v4*` defaults, such as preview typed-Sail keys, because those would make the bridge posture sound more settled than the current Sail-owned typed v4 surface proves. Config overrides are held to the same honesty rule for v4 posture: until typed Sail v4 support is available, replay evidence cannot use an override to claim `lakecat.format.v4.typed-sail=available` or introduce another v4 bridge key. Catalog config replay now also preserves the advertised endpoint list. That is not a new protocol requirement for standard clients; it is proof that the configuration LakeCat projected to graph and OpenLineage still contained the ordinary Iceberg REST surface. Replay validation requires the config endpoint, namespace list/create endpoints, table create endpoint, table load endpoint, and table commit endpoint for both the default and warehouse-prefixed catalog routes before the config read can become compatibility evidence. Replay validation also requires LakeCat's governed access endpoints: plan, fetch-scan-tasks, and credentials. Those routes are not a new table format and not a QueryGraph dependency for ordinary reads. They are additive catalog APIs that let governed clients ask LakeCat, TypeSec, and Sail for proof-carrying plans, task fetches, or audited credential decisions over the same standard Iceberg tables. Replay validation also preserves the additive integration surfaces that make LakeCat useful as the QueryGraph foundation: `/querygraph/v1/bootstrap` and `/management/v1/lineage/drain`. These are not standard Iceberg REST table operations and they are not required for PySpark or another ordinary Iceberg client to load a table. They are LakeCat/QueryGraph/OpenLineage control-plane endpoints. Their presence in config evidence proves that a QGLake import, OpenLineage replay, or agentic management workflow saw the same integration contract that LakeCat later projects into graph and lineage systems.

The bridge is intentionally conservative, but it should not reject Iceberg metadata that Sail has already decoded. Manifest expansion now emits null partition slots as JSON `null` and recursively encodes nested Sail partition literals into JSON objects, arrays, and explicit key/value map entries. That keeps standard Iceberg REST fetch responses usable for richer partition tuples without pretending LakeCat owns a full typed v4 implementation.

At this point the catalog is already doing more than route HTTP. It has a warehouse identity, a store, a governance engine, a Sail planning seam, a graph sink, and a lineage sink. Embedded defaults keep the local loop small, but the same trait boundaries can point to Turso, TypeSec, Grust, and Sail.

1.27.2 Registering The Warehouse Shape

An operator usually starts with management objects. A server groups projects. A project groups warehouses. A warehouse owns namespaces, tables, views, storage profiles, policy bindings, and the metadata pointer state that standard engines see through Iceberg REST.

```
curl -s -X PUT http://127.0.0.1:3000/management/v1/servers/prod \
  -H 'content-type: application/json' \
  -d '{
    "display-name": "Production LakeCat",
```

```

    "endpoint-url": "https://lakecat.example.com",
    "properties": {
      "owner": "platform"
    }
  }'

curl -s -X PUT http://127.0.0.1:3000/management/v1/projects/resilience \
-H 'content-type: application/json' \
-d '{
  "display-name": "Resilience Desk",
  "server-id": "prod",
  "properties": {
    "environment": "demo"
  }
}'

curl -s -X PUT http://127.0.0.1:3000/management/v1/projects/resilience/
warehouses/local \
-H 'content-type: application/json' \
-d '{
  "display-name": "Local QGLake Warehouse",
  "storage-root": "file:///tmp/lakecat/qglake",
  "properties": {
    "querygraph": "enabled"
  }
}'

```

These writes are not Iceberg table metadata. They are catalog control-plane state. LakeCat persists them durably, records authorization receipts, and writes outbox events. When the outbox drains, server, project, warehouse, and storage-profile and policy-binding changes become catalog graph events; the same management changes also become OpenLineage receipts. QueryGraph can later learn the management shape without requiring every Iceberg client to understand it. Project, server, and warehouse tenant-root replay is checked before projection: project evidence must carry a matching project id, optional valid server scope, and string-map public properties; server evidence must carry a valid server id, optional valid endpoint URL or full endpoint-url-hash, and string-map properties; warehouse evidence must carry a valid warehouse, project id, optional valid storage root or full storage-root-hash, and string-map properties. Policy-binding upsert replay is checked before projection too: the evidence must carry a valid policy id, warehouse, optional namespace/table scope, an enforcement flag, the captured ODRL material, and an odr1-hash that matches that material. LakeCat does not reason over that ODRL during replay, but malformed binding shape or drifted ODRL content proof fails closed before the policy anchor can be delivered to graph or lineage sinks. Those management upserts must also carry a valid authorization receipt principal, so the catalog graph and OpenLineage stream never accept actorless tenant-root, storage-profile, or policy mutations. Namespace lifecycle replay is checked before projection as well: create, load, and drop events must carry a valid warehouse and either a valid namespace path or non-empty namespace component array. A malformed namespace lifecycle event stays pending and reaches neither the Grust-facing graph sink nor OpenLineage. Catalog

read replay has the same fail-closed shape: `catalog.config-read` events must carry a valid warehouse, and `namespace.listed` events must carry both a valid warehouse and an unsigned namespace count before the read evidence can be projected. These standard catalog reads and namespace lifecycle events must also carry a valid authorization receipt principal before delivery, so Iceberg-compatible control-plane replay remains attributable. Management-list replay is checked before delivery too: policy-binding, project, server, storage-profile, and warehouse list events must carry unsigned counts, warehouse-scoped lists must carry a valid warehouse, and optional project scope on warehouse-list replay must be a non-empty, syntactically valid project identifier before those reads can become replay evidence. View replay is checked at the same boundary: view list events must carry valid warehouse, namespace, and count evidence, while view create/load/drop evidence must carry a valid warehouse, namespace, and non-empty view name before graph or OpenLineage projection. View list and lifecycle replay must also carry a valid authorization receipt principal before delivery, preserving actor evidence for QueryGraph view proofs. A view list is read-side catalog evidence, so the service requires its authorization receipt action to be `view-load`; a `view-manage` receipt is valid for mutations but not for replaying `view.listed`. View lifecycle replay is action-bound too: `view.upserted` requires `view-manage`, `view.loaded` requires `view-load`, and `view.dropped` requires `view-drop` before LakeCat emits graph or OpenLineage evidence. Table lifecycle replay now follows the same rule:

Active view state is protected before replay as well. A Turso row selected as warehouse `local`, namespace `default`, and view `active_customers` must decode to that same view before LakeCat returns it, lists it, updates it, or drops it. The memory store applies the same check to keyed active-view reads. This is not an Iceberg view extension; it is LakeCat's durable row/content guard around the control-plane view state that later produces view receipt chains and QGLake proof. create, load, delete, and restore events must carry a valid root table identity, and any payload warehouse, namespace, table-name, or soft-delete table evidence must agree with that identity before the event can be acknowledged. Their authorization receipts must also carry the matching lifecycle action: `table-create`, `table-load`, `table-drop`, or `table-restore`, along with an allow decision, engine, and checked-at timestamp. Server endpoint URLs are operator-visible management metadata, so LakeCat keeps them deliberately plain: they must be absolute `http` or `https` URLs, and they cannot include query strings, fragments, or URI userinfo. Rejected submissions return `server-endpoint-url-hash=sha256:... evidence` rather than echoing the submitted endpoint. Warehouse replay does not forward the raw storage root to graph or lineage consumers. The drained payload replaces `storage-root` with `storage-root-hash`, so QueryGraph can bind tenant evidence to a configured root without receiving the local filesystem path or bucket URI.

1.27.3 Storage Profiles And Credential Roots

Storage profiles bind a warehouse to physical storage roots and credential issuance policy. A local profile can return scoped local file configuration. A remote profile should usually reference a secret store and require TypeSec to authorize issuance before any resolver sees the secret reference. Warehouse storage roots are validated before memory or Turso persistence: query strings, fragments, URI userinfo, and literal or percent-encoded dot path segments fail with `warehouse-storage-root-hash=sha256:... evidence` rather than echoing the submitted root.

LakeCat rejects profiles whose declared provider conflicts with the URI scheme of the location prefix, so a credential root cannot claim to be local while pointing at an S3 prefix. Those provider/location mismatch errors follow the same redaction rule as replay: they name provider labels and a `storage-profile-prefix-hash=sha256:...`, not the raw storage root. When multiple profiles in the same warehouse could match a table, LakeCat uses the longest matching location prefix. If two profiles tie on that longest prefix, LakeCat fails closed rather than guessing which credential root or metadata-object boundary should apply. The ambiguity error reports the competing profile ids and `location-prefix-hash=sha256:...` evidence, not the raw storage root. The location prefix itself must be plainly addressed: LakeCat rejects literal and percent-encoded dot path segments, query strings, fragments, and URI userinfo before the profile can reach memory or Turso persistence. Traversal-shaped or decorated storage-profile prefixes fail with `storage-profile-prefix-hash=sha256:...` evidence rather than echoing the raw prefix, token-like query value, or embedded userinfo. The management route pins the same operator-facing behavior, so a rejected storage-profile upsert does not leak the submitted decorated prefix. It also rejects unsafe issuance-mode combinations: `local-file-no-secret` is for file storage only, while `short-lived-secret-ref` is for configured remote providers such as S3, GCS, and Azure. Those mismatches fail with the same `storage-profile-prefix-hash=sha256:...` anchor and without echoing the raw storage prefix or submitted `secret-ref`, so operators can correlate the credential-root error without turning the management API into a credential leak. The public-config map is only for non-secret routing hints such as region, endpoint labels, and operational purpose. LakeCat rejects secret-looking public keys and values, so raw tokens, passwords, access keys, and credential query parameters must move behind `secret-ref` and the TypeSec-authorized resolver path. That rule is enforced both when a profile is built from a management request and when a storage profile is revalidated before memory or Turso persistence, so deserialized control-plane records cannot bypass the public-config guard. Public-config validation failures also use `public-config-key-hash=sha256:...` evidence rather than echoing the submitted key or value, because even a rejected key name may contain a secret-looking identifier. LakeCat also reserves credential-evidence keys such as `lakecat.storage-profile-id`, `lakecat.storage-provider`, `lakecat.credential-mode`, and `lakecat.max-credential-ttl-seconds`; operators may still publish non-secret hints such as `lakecat.endpoint`, but they cannot shadow catalog-owned proof in the eventual credential response. The `secret-ref` field itself must remain a clean external secret-store locator: LakeCat rejects query strings, URI fragments, and userinfo before persisting a storage profile, so token-like material cannot hide inside a decorated secret URI. It also rejects literal and percent-encoded dot path segments, so a credential root cannot rely on traversal-like spelling before a resolver sees it. Unsupported credential-root schemes and malformed secret-root paths are rejected with `secret-ref-hash=sha256:...` evidence instead of echoing the submitted secret reference. The same hash-only rule applies to invalid `secret-ref` URI syntax, decorated URI forms, and embedded secret-like material such as password or token assignments. Management upsert and list responses follow the same redaction rule. They do not echo the raw `secret-ref`; they return `secret-ref-present`, `secret-ref-provider`, and `secret-ref-hash` so operators and QueryGraph can verify that a credential root exists and correlate it without learning the secret-store path. When LakeCat selects the storage profile for a table, the location prefix is also matched on a storage-root boundary. A profile for `s3://lakecat/events` applies to that exact root and

to children such as `s3://lakecat/events/tenant-a/table`, but it does not apply to a sibling path such as `s3://lakecat/events-shadow/table`. That keeps credential roots from accidentally governing more storage than their configured prefix describes; if no stored profile matches, LakeCat falls back to an inferred governed-read profile for the table location. The same check runs after the credential issuer returns: `loadCredentials` rejects any returned prefix broader than the selected profile before LakeCat attaches canonical response evidence, so a custom issuer cannot widen catalog-owned storage scope. The rejection exposes only `credential-prefix-hash` and `storage-profile-prefix-hash` evidence, and LakeCat records no `credentials.vend-attempted` replay event for that failed issuer response.

```
curl -s -X PUT \
  http://127.0.0.1:3000/management/v1/warehouses/local/storage-profiles/local-
events \
  -H 'content-type: application/json' \
  -d '{
    "location-prefix": "file:///tmp/lakecat/qglake/events",
    "provider": "file",
    "issuance-mode": "local-file-no-secret",
    "public-config": {
      "lakecat.purpose": "developer-loop"
    }
  }'
```

```
curl -s -X PUT \
  http://127.0.0.1:3000/management/v1/warehouses/local/storage-profiles/s3-events
\
  -H 'content-type: application/json' \
  -d '{
    "location-prefix": "s3://lakecat/events",
    "provider": "s3",
    "issuance-mode": "short-lived-secret-ref",
    "secret-ref": "vault://kv/lakecat/events",
    "public-config": {
      "lakecat.region": "us-west-2",
      "lakecat.purpose": "production-events"
    }
  }'
```

The catalog row stores the public profile and secret reference, not raw cloud keys. A later credential request is checked against TypeSec and against the effective read restriction for the target table. Agents with fine-grained table restrictions are steered to governed Sail-planned reads instead of raw credentials. Trusted humans can receive audited standard credentials only when policy allows the exception. For production secret managers, LakeCat keeps a provider-dispatch seam rather than hard-coding credentials into catalog state. `vault://` can resolve through the built-in Vault HTTP backend when Vault environment configuration is present. `aws-sm://`, `gcp-sm://`, and `azure-kv://` can dispatch to explicitly configured provider backends after TypeSec authorizes the exact `secret-ref` resource. They can also use LakeCat's built-in file-backed

provider roots for local or single-node deployments: LAKECAT_AWS_SECRETS_MANAGER_FILE_DIR, LAKECAT_GCP_SECRET_MANAGER_FILE_DIR, and LAKECAT_AZURE_KEY_VAULT_FILE_DIR. Each directory contains JSON credential config files named as the full SHA-256 digest of the exact secret reference, without the sha256: prefix, plus .json. For example, gcp-sm://lakecat/events is authorized as that exact TypeSec resource and then resolved from a hash-named JSON file under the configured GCP root. If no backend is configured, those providers fail closed with an operator-readable not-configured error, and denied TypeSec decisions do not call the backend or read the file at all. Configured provider backends receive the same policy-derived max-credential-ttl-seconds cap that LakeCat records in the read restriction, and returned credentials must preserve that cap in lakecat.max-credential-ttl-seconds. LakeCat rewrites duplicate TTL config entries into one effective value before returning credentials, preserving a stricter issuer TTL when it is valid and otherwise falling back to the policy cap. It also rewrites LakeCat-owned profile, provider, mode, principal, and governed-read-required evidence after issuance. For secret-ref-backed profiles it also derives lakecat.secret-ref-provider and lakecat.secret-ref-hash from the selected storage profile, so a cloud secret backend cannot make the response look like a different catalog decision, secret-provider path, or secret-reference anchor. Replay admission treats that evidence as structural too: secret-ref providers and hashes must be nonblank when secret-ref-present is true, and provider/hash fields must be absent when secret-ref-present is false, no matter how a corrupted pending event encodes them. The service tests for the REST credential endpoint prove this response shape directly, not just through helper functions. LakeCat also rejects any credential whose returned prefix is outside the storage profile's location-prefix, so a misconfigured cloud secret backend cannot widen a table's storage scope after TypeSec has authorized the secret reference. That failure remains hash-only and stops before credential-vend replay evidence is recorded. The audit event for the credential attempt records redacted credential-response-evidence: the response prefix is hashed, LakeCat-owned proof fields are kept as canonical values, and issuer-owned config is hashed rather than copied. That keeps OpenLineage and QueryGraph replay useful without turning lineage into a credential leak. For secret-ref-backed profiles the redacted response evidence includes the catalog-derived lakecat.secret-ref-provider and lakecat.secret-ref-hash, while the storage-profile replay evidence includes secret-ref-provider and a full secret-ref-hash; outbox admission rejects any credential response whose provider or hash proof drifts from the selected profile before graph or OpenLineage projection. The nested storage-profile proof is still checked even when no credentials are returned: provider and issuance mode must be compatible, and secret-reference presence must match the mode. That keeps blocked credential attempts from projecting a weaker credential-root proof than storage-profile management would accept. The storage-profile and credential-vend service tests pin that producer-side location-prefix-hash evidence is already a full SHA-256 digest before QGLake receives the compact locationPrefixHash proof. The trusted-human credential-vending route test pins that the committed outbox payload contains this redacted proof for the audited raw-credential exception path. The blocked-agent route pins the other side of the same contract: when Sail-planned reads are required and no raw credentials are returned, the outbox records an explicit empty credential-response-evidence array rather than leaving replay to infer why no credential proof exists. A not-configured resolver error reports the provider label and a secret-ref-hash=sha256:... value, not the raw secret URI, so the operator can correlate

configuration without leaking the credential root. Resolver validation errors for malformed Vault and TypeSec environment references follow the same rule: wrong schemes, missing Vault mounts or paths, and invalid environment-variable names produce hash evidence instead of echoing the malformed secret reference. Generic provider detection and resolver URI parsing follow that rule too, including unsupported provider schemes, so malformed credential-root strings cannot leak through production resolver diagnostics. Once a configured resolver is authorized to run, backend lookup and secret payload parse failures still stay hash-only: LakeCat returns the secret-reference hash and an error-detail hash instead of the environment variable name, Vault path, token, namespace, backend exception text, cloud secret-manager ARN or account path, or malformed secret fields. That rule applies both to the built-in Vault and environment resolvers and to explicitly configured AWS Secrets Manager, GCP Secret Manager, and Azure Key Vault style backend seams, including the file-backed provider roots. The file-backed roots are not a claim that LakeCat has cloud SDK support for those providers; they are a redacted built-in backend that lets the same production-shaped secret-ref dispatch run locally while SDK resolvers are added later. Secret payload parsing also rejects malformed credential configuration before issuance, including blank config keys in either object-shaped secrets, ConfigEntry-array secrets, or Vault's nested data object. That keeps secret-manager output from turning into ambiguous Iceberg client credential configuration. When storage-profile changes replay into lineage/OpenLineage evidence, LakeCat does not forward the full secret-store URI or raw storage root. The committed audit/outbox payload keeps `secret-ref-present` and `secret-ref-provider` so QueryGraph can verify that a production credential root exists without learning the Vault, cloud secret manager, or TypeSec environment path. It also records a full `location-prefix-hash` instead of raw `location-prefix`, so replayed evidence can bind a credential root to a storage scope without exposing the bucket, path, or local filesystem root to downstream consumers. Warehouse replay follows the same shape: `storage-root` becomes `storage-root-hash` before graph and lineage projection, keeping the tenant root replayable without exposing the raw root itself. The drain also rejects unsafe storage-profile upsert evidence before delivery: `storage-profile.upserted` must carry a full `location-prefix-hash` and must not carry raw `location-prefix` or raw `secret-ref` values. If `secret-reference` presence is true, the replay evidence must carry a provider and full `secret-ref-hash`; if presence is false, provider and hash evidence must be absent. The same admission check validates the credential-root identity before projection: `profile id` must be non-empty, the nested warehouse must be valid and match any top-level warehouse field, and provider plus issuance mode must use LakeCat's supported storage-profile vocabulary. Provider and issuance-mode compatibility is replay-checked as well: `local-file-no-secret` is only valid for the file provider, and `short-lived-secret-ref` is only valid for cloud object providers. Secret-reference presence must also agree with issuance mode: short-lived secret-ref profiles must carry redacted secret-reference proof, while governed and no-secret profiles cannot carry secret-reference proof. The drain summary lifts the same proof into compact fields alongside the profile id and provider. QGLake replay verification requires that compact storage-profile upsert evidence, which means a saved handoff can prove the credential root was configured without handing the next system a secret path.

1.27.4 A PySpark User Reads Iceberg

A PySpark user should not need to know about QueryGraph. They configure Spark's Iceberg REST catalog and point it at LakeCat:

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
        .appName("lakecat-events")
        .config("spark.sql.catalog.lakecat", "org.apache.iceberg.spark.SparkCatalog")
        .config("spark.sql.catalog.lakecat.type", "rest")
        .config("spark.sql.catalog.lakecat.uri", "http://127.0.0.1:3000/catalog/v1")
        .config("spark.sql.defaultCatalog", "lakecat")
        .getOrCreate()
)

events = spark.table("default.events")
events.select("event_id", "severity").where("severity = 'critical']").show()
```

For an unrestricted principal, the flow looks like a normal Iceberg read:

1. Spark asks LakeCat to resolve default.events.
2. LakeCat checks the principal and table capability.
3. LakeCat returns an Iceberg-compatible table response.
4. Spark plans the read using Iceberg metadata.

For a governed principal, the more interesting path is server-side planning. The user request still looks ordinary, but LakeCat derives the mandatory restriction before Sail sees the plan. If policy allows only event_id and severity, then a wider client projection is narrowed:

```
client asks:    event_id, severity, raw_payload
policy allows:  event_id, severity
Sail receives:  event_id, severity
```

The catalog does not trust the client to remember that. The restriction is re-applied when scan tasks are fetched, and the audit payload records the policy hash, narrowed columns, row predicate, storage location, metadata location, principal, requested stats fields, and effective stats fields. The fetch response also carries LakeCat extension evidence for the exact required-projection and required-filters derived from the authorized capability. That makes a stateless fetchScanTasks replay prove the restriction was re-applied, not merely that the original policy object was echoed. The QGLake fixture verifier checks those fields directly when it fetches scan tasks, so a local acceptance run fails if the response drops either the narrowed projection or the mandatory row predicate proof. The same verifier now checks that the exported policy binding, scan planning extension, and fetch extension all preserve the server-derived purpose and policy-derived max-credential-ttl-seconds cap before compact replay proof can be accepted. The scan-planned replay proof also carries plannedRequestedStatsFields and plannedEffectiveStatsFields, so QGLake can prove a broader stats request, the server-derived narrowing, and the final effective stats fields that match the allowed columns. Saved handoff

summaries and captured replay output are rejected if those fields disappear, widen, or drift apart. It also cross-checks the embedded ODRL policy projection against the structured binding so QueryGraph cannot import a stale copy while LakeCat verifies a different one.

1.27.5 A Notebook Requests Credentials

Credential vending is deliberately different from scan planning. Returning storage credentials gives the client broader power than returning a governed task list, so LakeCat treats it as an exception path.

```
curl -s \
  -H 'x-lakecat-principal: agent:trriage' \
  http://127.0.0.1:3000/catalog/v1/local/namespaces/default/tables/events/
credentials
```

For an agent bound by a fine-grained restriction, LakeCat should fail closed:

```
{
  "credentials": [],
  "lakecat:credential-block-reason": "fine-grained read restriction requires
Sail-planned reads"
}
```

That empty credential response is not a missing feature. It is the intended agentic posture. The agent should ask LakeCat to plan the read through Sail, not receive raw storage reach. The audit event records the decision and the lineage outbox can replay the credential-vend attempt with the same block reason.

For a trusted human principal, policy can allow an audited raw credential exception. LakeCat still records the same context:

```
principal: analyst:maya
principal-kind: human
table: local.default.events
decision: raw credential exception allowed
reason: trusted human principal
restriction: present in receipt
lineage: credential vend attempted
```

The contrast matters for operators. They can prove that agents were kept on the governed path while a human exception was explicit, policy-backed, and replayable.

1.27.6 A View Becomes Part Of The Catalog Story

Views are catalog objects too. LakeCat stores durable view records with SQL, dialect, schema version, typed columns, properties, creator, and warehouse scope. They can be managed through the management API or through warehouse-prefixed catalog routes.

```
curl -s -X PUT \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/views/
events_view \
  -H 'content-type: application/json' \
```

```
-d '{
  "sql": "select event_id, severity from default.events where severity =
'\''critical'\''",
  "dialect": "spark-sql",
  "schema-version": 1,
  "columns": [
    {
      "name": "event_id",
      "data-type": {"type": "long"},
      "nullable": false,
      "comment": "Stable event identifier"
    },
    {
      "name": "severity",
      "data-type": {"type": "string"},
      "nullable": false,
      "comment": "Operational severity"
    }
  ],
  "properties": {
    "owner": "resilience-desk"
  }
}'
```

This is not Iceberg business metadata glued onto a table. It is catalog state about a view object. LakeCat records view.listed, view.upserted, view.loaded, and view.dropped audit events. Outbox replay projects listing reads to namespace-scoped graph and OpenLineage evidence, and projects single-view changes and reads to catalog-facing View graph events plus LakeCat OpenLineage view dataset receipts. QueryGraph bootstrap can then include views with OSI hashes, store-assigned view versions, view-aware graph edges, and OpenLineage view counts. The lineage-drain summary also carries compact view replay identity:

```
{
  "name": "events_view",
  "view-version": 1,
  "schema-version": 1,
  "dialect": "sql"
}
```

That view-version is assigned by the durable store on each upsert, not by the caller. It is the first compatibility bridge toward Iceberg view commit history: QueryGraph can compare a bootstrap view artifact with the catalog's current view version today. A caller that wants optimistic commit behavior can include expected-view-version on the next upsert:

```
curl -s -X PUT \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/views/
events_view \
  -H 'content-type: application/json' \
  -d '{
```

```

    "sql": "select event_id from default.events where severity =
  \"'critical'\",
    "dialect": "spark-sql",
    "schema-version": 2,
    "expected-view-version": 1
  }'

```

If another writer has already advanced the view, LakeCat returns a conflict before it replaces the current view or appends a receipt. Omitting the field keeps the compatibility behavior: the store assigns the next version. The guard must be a positive store-assigned version. `expected-view-version=0` is rejected as a bad request before LakeCat changes the active view or appends any view-version receipt. The same guard can protect deletion:

```

curl -s -X DELETE \
  'http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/views/
events_view?expected-view-version=2'

```

If the current view is no longer version 2, LakeCat returns a conflict before it removes the view or appends a tombstone receipt. Accepted guarded mutations also carry their `expected-view-version` into the audit/outbox payload. During lineage drain, LakeCat turns that into compact replay evidence, so QueryGraph can distinguish “the replacement happened at version 2” from “the replacement was guarded by version 1 and then produced version 2.” Replay admission rejects view lifecycle evidence that omits the positive store-assigned `view-version` or carries a non-positive guarded `expected-view-version`, so graph and OpenLineage sinks never observe a versionless view lifecycle fact. Replay admission also checks the authorization receipt action: `upsert` uses `view-manage`, `load` uses `view-load`, and `drop` uses `view-drop`, so a valid view artifact cannot be replayed under a weaker or unrelated catalog action.

LakeCat also writes a compact view-version receipt in the durable store. The receipt records the stable view id, assigned version, previous version, previous receipt hash, content hash, principal, operation, and timestamp. That makes the compact receipt list a hash chain: version 2 points at the version 1 receipt hash, and a later tombstone points at the last upsert receipt hash. Fuller version-log semantics remain a Sail-aligned implementation target. When a view is dropped, LakeCat appends a compact tombstone receipt instead of inventing a new view version: the receipt keeps `view-version` at the last durable version, sets operation to `drop`, links to the previous receipt, and preserves the last content hash so QueryGraph or an operator can prove which catalog view state was removed. If the same view name is later recreated, the new upsert continues after the latest receipt in that durable chain. A create, drop, and recreate sequence therefore looks like version 1 upsert, version 1 drop tombstone, version 2 upsert linked to the tombstone receipt, not two unrelated version-1 chains for the same stable view id.

```

{
  "event-type": "view.upserted",
  "view-warehouse": "local",
  "view-namespace": ["default"],
  "view-name": "events_view",
  "view-stable-id": "lakecat:view:local:default:events_view",

```

```

    "view-version": 2,
    "expected-view-version": 1,
    "graph-events": 2,
    "lineage-events": 1,
    "replay-event-hashes": ["sha256:..."],
    "replay-open-lineage-hashes": ["sha256:..."]
}

```

A `view.listed` replay is intentionally namespace-scoped. It records the warehouse, namespace, view count, graph and lineage projection counts, and receipt hashes for the list read without fabricating a single `view-stable-id`.

QGLake acceptance compares both that `view-stable-id` and `view-version` with the accepted QueryGraph bootstrap view artifacts. That closes a small but important gap: the bootstrap bundle may say a view was exported, and the drain evidence can now prove the corresponding view catalog event was replayed at the same durable catalog version with graph and lineage receipts. When the event was guarded, QGLake replay JSON also includes `expectedViewVersion`, preserving the optimistic version that LakeCat checked before accepting the mutation. The live QGLake fixture uses that path for deletion: after QueryGraph accepts the transient view in the bootstrap bundle, the fixture drops it with `expected-view-version` equal to the accepted durable view version.

When QueryGraph bootstrap is replayed through the outbox, LakeCat includes only compact receipt hashes:

```

{
  "event-type": "querygraph.bootstrap",
  "view-artifact-count": 1,
  "view-version-receipt-hashes": ["sha256:..."]
}

```

QGLake acceptance requires one non-empty receipt hash for each accepted view artifact. That keeps normal Iceberg view access standard, but gives the QueryGraph handoff a durable proof that the exported view version came from LakeCat's catalog spine. Compact handoff verification also binds those bootstrap receipt hashes to `viewReceiptChainProof.views[].acceptedReceiptHash` exactly, so a saved summary cannot splice a valid-looking QueryGraph bootstrap receipt array from another accepted view proof. The fixture also exercises the deletion side of the same workflow: it creates a transient view, accepts a QueryGraph bootstrap that contains that view, drops the view, reads the receipt chain through the governed management endpoints by view name and namespace, and then requires lineage-drain replay to include `view.dropped`, `view.version-receipts-listed`, and `view.version-receipt-chains-listed` evidence with non-empty tombstone receipt hashes and namespace chain hashes. The service route tests pin those produced `receipt-hash`, `view-hash`, and `chain-hash` fields as full SHA-256 digest evidence before the QGLake verifier consumes them. LakeCat also validates the ordered `previous-receipt-hash` links before marking a namespace chain as `chain-verified`, so QueryGraph can reject a replay that contains hashes but not a coherent chain. The lower store layer now applies the same structural guard when view receipts are read: both memory and Turso-backed receipt reads reject

forged previous-receipt-hash links before service replay, OpenLineage projection, or QGLake handoff can consume the chain. The mutation path uses that same guard before extending history. A guarded or unguarded view upsert/drop first validates the existing durable receipt chain, then computes the latest receipt hash, and only then appends the next receipt. If a stored receipt has a forged previous-receipt-hash, LakeCat rejects the new mutation before changing the active view record or writing another receipt. For the Turso-backed store, LakeCat also compares the decoded receipt JSON to the row/query scope. A receipt row selected for one warehouse, namespace, and view cannot carry JSON that claims another view and still be returned, grouped into a namespace chain, or used as the latest receipt for the next mutation.

The verifier is fail-closed on version progression too. The first receipt must be a version-1 upsert with no previous version or receipt hash; zero-version chains, first-receipt tombstones, and first receipts with forged previous-link fields fail before the chain can be marked verified. Later upserts must point at the previous receipt hash and advance exactly one durable view version. Drop tombstones must point at the previous receipt hash, cite the previous view version, and keep view-version equal to the accepted version that was deleted. Unsupported operations and forged previous-receipt-hash links fail the same check. That lets QueryGraph reject a chain that is cryptographically linked but lies about how the catalog view advanced.

QueryGraph and operators can also read the compact receipt chain directly from the governed management surface:

```
curl -s \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/views/
events_view/version-receipts \
  -H 'x-lakecat-agent-did: did:example:resilience-agent'
{
  "receipts": [
    {
      "stable-id": "lakecat:view:local:default:events_view",
      "view-version": 1,
      "previous-view-version": null,
      "operation": "upsert",
      "view-hash": "sha256:...",
      "receipt-hash": "sha256:..."
    },
    {
      "stable-id": "lakecat:view:local:default:events_view",
      "view-version": 1,
      "previous-view-version": 1,
      "previous-receipt-hash": "sha256:...",
      "operation": "drop",
      "view-hash": "sha256:...",
      "receipt-hash": "sha256:..."
    },
    {
      "stable-id": "lakecat:view:local:default:events_view",
```

```

    "view-version": 2,
    "previous-view-version": 1,
    "previous-receipt-hash": "sha256:...",
    "operation": "upsert",
    "view-hash": "sha256:...",
    "receipt-hash": "sha256:..."
  }
]
}

```

The response is catalog evidence, not Iceberg table metadata. It lets QueryGraph verify the version chain, including tombstones after the current view row is gone, while keeping the richer view history model available for a future Sail-owned implementation.

When the caller needs discovery rather than a known view name, the management surface can return all view receipt chains in a namespace, including chains for views that no longer appear in the active view list:

```

curl -s \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/view-
  version-receipt-chains \
  -H 'x-lakecat-agent-did: did:example:resilience-agent'
{
  "chains": [
    {
      "stable-id": "lakecat:view:local:default:events_view",
      "chain-hash": "sha256:...",
      "chain-verified": true,
      "latest-view-version": 1,
      "latest-operation": "drop",
      "tombstoned": true,
      "receipt-count": 2,
      "receipts": ["..."]
    }
  ]
}

```

That tombstone read is replayable too. LakeCat projects `view.version-receipts-listed` and `view.version-receipt-chains-listed` as lineage evidence, not as graph topology. The graph taxonomy stays in Grust; LakeCat only proves that the governed read saw the tombstone receipt needed to explain why a previously accepted view is now deleted. The namespace response also carries a deterministic chain-hash over the chain identity and ordered receipt hashes, and lineage-drain summaries replay that value as `view-version-receipt-chain-hashes` together with a verified-chain count. The QGLake fixture now fails if the namespace-level receipt-chain read, its chain hash, or its verified-chain evidence is absent from lineage-drain replay, so QueryGraph acceptance can depend on compact chain evidence without scraping store internals.

1.27.7 QueryGraph Bootstrap

QueryGraph should import LakeCat facts through a verified handoff, not by scraping service internals. The bootstrap endpoint publishes a bundle with artifact hashes:

The exported graph includes a tenant spine:

```
Catalog HAS_SERVER Server
Server HAS_PROJECT Project
Project HAS_WAREHOUSE Warehouse
Warehouse HAS_NAMESPACE Namespace
```

When LakeCat has durable management rows, those graph nodes come from the stored `ServerRecord`, `ProjectRecord`, and `WarehouseRecord`. That means a `QueryGraph` import can see the real server id, project id, warehouse id, display names, and tenant relationships that operators manage through LakeCat's management API. Replay evidence deliberately redacts storage roots and server endpoint URLs into hashes, so `QueryGraph` can prove which management state was observed without inheriting local paths, bucket roots, query tokens, URI fragments, or userinfo. In the bootstrap graph, `Server` nodes carry `endpointUrlHash` and `Warehouse` nodes carry `storageRootHash`; they do not carry the raw endpoint URL or storage root. The projection code and service route tests both pin those emitted fields as full SHA-256 digest evidence, so the producer and verifier agree on the shape before `QueryGraph` import. Authorized management responses can still show the configured endpoint URL or storage root to an operator; graph and lineage replay receive hash-only evidence. When those records do not exist yet, LakeCat falls back to the old deterministic default anchors so bootstrap remains compatible with minimal embedded tests and older import flows.

LakeCat also keeps the older `Catalog HAS_NAMESPACE Namespace` edge in the bundle so existing `QueryGraph` importers can keep working while newer flows read the tenant path. The tenant anchors and warehouse-to-namespace edges are part of the manifest-covered graph hash, so an importer or handoff verifier can reject a bundle whose namespace is silently detached from the warehouse or rebound to a different durable tenant chain. The local `QGLake` verifier enforces that shape: an accepted bootstrap must prove `Catalog -> Server -> Project -> Warehouse -> Namespace -> Table`, not merely that a table node exists somewhere in the graph. It also rejects bundles whose tenant graph nodes expose raw `endpointUrl` or `storageRoot` properties, even if the graph hash and bundle hash have been recomputed around those raw values. Accepted handoffs must use hash-only `endpointUrlHash` and `storageRootHash` evidence for those roots. Those fields are not merely labels with a `sha256:` prefix: the `QGLake` verifier requires a full 64-hex SHA-256 digest, so an importer cannot accept a rewritten bundle that replaces raw roots with placeholder hash text after the fact. The saved handoff verifier repeats that check against the archived `lakecat-bootstrap.json` artifact, so a later replay cannot pass with a compact summary whose bundle file has lost the tenant path or drifted from the summary hashes.

```
curl -s \
  -H 'x-lakecat-principal: agent:querygraph-importer' \
  http://127.0.0.1:3000/querygraph/v1/bootstrap \
  -o target/qglake/lakecat-bootstrap.json
```

The bundle contains catalog tables, views, policy bindings, graph artifacts, OpenLineage artifacts, Croissant/CDIF/OSI/ODRL projections, and a manifest that hashes what was emitted. The manifest is the import contract. QueryGraph can refuse a bundle whose graph hash, OpenLineage hash, table artifact hash, view artifact hash, or QueryGraph import-compatibility hash does not match. For view-bearing bundles, the import contract also carries compact receipt evidence for each exported view version:

```
{
  "querygraph-import": {
    "schema-version": "lakecat.querygraph.import-compat.v1",
    "view-count": 1,
    "view-receipt-evidence": [
      {
        "stable-id": "lakecat:view:local:default:events_view",
        "view-version": 1,
        "receipt-hash": "sha256:...",
        "receipt-chain-hash": "sha256:..."
      }
    ],
    "view-receipt-evidence-hash": "sha256:..."
  }
}
```

That gives QueryGraph a manifest-covered way to reject a view bootstrap that lost the accepted catalog receipt or detached the ordered receipt chain before the richer graph import begins.

LakeCat also enforces the same binding when a `querygraph.bootstrap` outbox event is replayed. The authorization receipt must carry a valid principal and the `graph-read` action, table artifact stable IDs must match the `verified-tables` manifest exactly, view artifact stable IDs must match `verified-views`, and view-version receipt stable IDs must match `verified-views`. A saved replay event that keeps valid-looking hashes while dropping actor proof, drifting to a `lineage-read` action, swapping in another table artifact, or borrowing another view's receipt evidence fails before graph or OpenLineage projection.

The QueryGraph side should verify the same bundle before importing it:

```
cd /Users/alexysrc/querygraph/qg-rust
```

```
cargo run -- lakecat-verify \
  --bundle /Users/alexysrc/lakecat/target/qglake/lakecat-bootstrap.json
```

```
cargo run -- lakecat-import \
  --bundle /Users/alexysrc/lakecat/target/qglake/lakecat-bootstrap.json \
  --output .querygraph/lakecat/import-plan.json
```

The importer checks the outer bundle hash, the manifest hashes, the QueryGraph-import compatibility hash, the graph hash, and view receipt evidence, including the accepted receipt hash and receipt-chain hash for each exported view. The graph envelope must be valid as a graph, not just valid JSON: for example, a table and a view in the same namespace must share one namespace

node, not emit duplicate vertex ids. That validation belongs on the QueryGraph/Grust side, while LakeCat is responsible for producing a clean catalog-facing graph projection.

For the full local handoff, LakeCat carries a script that runs both sides without writing generated artifacts into the QueryGraph checkout:

```
scripts/qglake-handoff-local.sh
```

The script starts LakeCat on 127.0.0.1:18181, uses a Turso-backed local store under `target/qglake-handoff/`, generates the paired QGLake bootstrap bundle and lineage-drain response, runs `qglake-verify-replay`, then runs QueryGraph's `lakecat-verify` and `lakecat-import` against the same bundle. The resulting import plan is written to `target/qglake-handoff/querygraph-import-plan.json`. The same run also writes `target/qglake-handoff/handoff-summary.json`, a compact machine-readable record under the `lakecat.qglake.handoff-summary.v1` schema. It records the catalog URL, principal, table scope, LakeCat replay status from `lakecat.qglake.replay-verification.v1`, QueryGraph-verified table/view counts, and semantic bundle/graph/OpenLineage/import hashes plus standards accepted only after LakeCat replay, `lakecat-verify`, and `lakecat-import` agree. Before writing that compact summary, the harness also requires the governed scan replay evidence to include planned requested/effective projection, planned requested/effective stats fields, fetched required/effective projection, and fetched filters, with effective projection matching the server-derived read restriction. The compact verifier requires the catalog URL to be an absolute HTTP(S) endpoint and requires warehouse, namespace, and table scope to be present before accepting the summary. It rejects captured QueryGraph verify/import output whose warehouse no longer matches the summary. The lineage-drain replay summaries are bound back to the drain-level eventTypes manifest as well. A saved handoff cannot add a compact replay summary for `storage-profile.upserted`, `querygraph.bootstrap`, or any other catalog event type unless the drain itself declared that event type as delivered. LakeCat checks this as a multiset rather than a simple set: repeated event types such as credential vending or scan-task fetching must appear in the same multiplicity in eventTypes and in the replay summary array. It also checks order: `eventTypes[i]` must name the same event type as replay summary events[i]. That makes the manifest a compact replay sequence proof instead of a loose inventory that could be reordered after the fact. It also embeds `querygraphVerification.verifiedTables` and `verifiedViews` directly in the compact summary. `verifiedTables` must include the stable LakeCat table id derived from that scope, such as `lakecat:table:local:default:events`; `verifiedViews` must include every accepted stable view id from LakeCat replay, such as `lakecat:view:local:default:active_customers_view`; and both arrays must match the QueryGraph table/view counts. LakeCat rejects duplicate entries in those manifests at outbox admission before graph or OpenLineage projection, and the compact handoff verifier repeats the check for archived summaries. Captured QueryGraph verify/import output must match those compact arrays exactly, which keeps a verified artifact set from being replayed against the wrong catalog tenant, table, or view. The `querygraphImportVerification` object is also a compact proof, not only a boolean: it repeats the QueryGraph import table/view ids, counts, bundle hash, graph hash, OpenLineage hash, QueryGraph import hash, and standards, and LakeCat rejects a summary unless those fields are

SHA-256-shaped and match both querygraphVerification and the captured lakecat-import output. It also records structured request-identity, scan, management, credential, table-commit, and view replay evidence, plus compact requestIdentityProof, queryGraphBootstrapProof, governedScanProof, tableCommitHistoryProof, viewReceiptChainProof, managementProof, storageProfileUpsertProof, and credentialVendingProof objects that lift the replay principal proof, QueryGraph bootstrap/import proof, governed scan counts, pointer-log read proof, view version and receipt-chain proof, management-list counts, redacted credential-root proof, and credential-vending decision out of the full replay tree. The identity proof shows the principal subject and kind used for the replay, the request-identity source and state, the authorization receipt hash, and sanitized TypeDID envelope/proof hashes when a TypeDID envelope is present. The local QGLake fixture currently records the agent-header source with null TypeDID hash slots; a future TypeDID-envelope run can fill those slots without changing the handoff schema. The QueryGraph bootstrap proof shows the accepted bundle, graph, OpenLineage, and QueryGraph import hashes, the table and view artifact counts, standards, policy-binding count, agent delegation and summary signature hashes, view-version receipt hashes, and replay/OpenLineage sink hashes. Those core bootstrap hash anchors must also be SHA-256-shaped before the verifier compares them with QueryGraph's verify/import proof. In the compact handoff verifier, the QueryGraph bundle, graph, OpenLineage, import, bootstrap replay, and bootstrap OpenLineage anchors must be full sha256:-prefixed 64-hex digests, so saved summaries cannot use prefix-shaped placeholders as QueryGraph acceptance evidence. The scan proof shows LakeCat planned and fetched scan tasks through the governed path, including file, delete-file, and child plan-task counts with replay and OpenLineage hashes. The commit-history proof shows the catalog pointer log was read back with commit count, sequence numbers, commit hashes, positive graph event evidence, and replay/OpenLineage hashes. The view receipt-chain proof shows QueryGraph's accepted view versions together with accepted receipt hashes, accepted expectedViewVersion guard evidence when a mutation was guarded, tombstone receipt hashes, namespace chain hashes, verified-chain counts, positive graph event evidence for accepted view replay, and replay/OpenLineage hashes. The credential proof shows the restricted agent was blocked onto Sail-planned reads while the trusted human path used the audited raw-credential exception. The summary also records artifact paths, raw file hashes, captured LakeCat replay output, captured LakeCat handoff-summary verification output, captured QueryGraph verification output, captured QueryGraph import output, captured-output hashes for the LakeCat replay and QueryGraph verify/import JSON files, and service log path. The handoff verifier does not stop at byte hashes: it parses the saved LakeCat replay JSON and QueryGraph verify/import JSON captures and checks their replay schema/status, table and view counts, warehouse, verified table ids, bundle hash, graph hash, OpenLineage hash, QueryGraph import hash, and standards against the compact summary. It compares those standards across sections and independently requires the full QGLake standards set, so a compact handoff cannot omit ODRL or another required contract simply by making every section omit it consistently. It compares the captured LakeCat replay-evidence.requestIdentity and replay-evidence.queryGraphBootstrap objects with the compact request-identity and bootstrap proofs, including the principal, authorization hash, TypeDID hash slots, delegation and summary signature hashes, artifact counts, standards, replay hashes, and the accepted bundle, graph, OpenLineage, and QueryGraph import hashes. The com-

compact verifier depends on the bootstrap manifest verifier having already rejected duplicate stable IDs across table projections, table artifact manifests, view projections, and view artifact manifests. That duplicate-free rule is not an Iceberg REST requirement for ordinary table reads; it is LakeCat/QGLake import proof. It prevents a semantic bundle from satisfying table or view counts by repeating the same stable ID, then letting QueryGraph believe it verified more catalog objects than the manifest uniquely proved. The compact verifier also requires the bootstrap proof to carry the same request-identity source and verification state as requestIdentityProof. The authorization receipt hashes are intentionally distinct proof slots: requestIdentityProof records the lineage-drain read receipt, while queryGraphBootstrapProof records the original bootstrap event receipt. The verifier requires both hashes to be full sha256:-prefixed 64-hex digests and requires their actions to keep the correct meaning: compact requestIdentityProof must be lineage-read, and compact queryGraphBootstrapProof must be graph-read. Those values are then bound back to their captured replay sections rather than forcing the receipt hashes to be equal. The same full-digest rule applies to the required agent delegation and agent summary-signature hashes in the bootstrap proof, so a saved handoff cannot replace those proof anchors with short readable placeholders. The compact verifier also validates the TypeDID hash-slot shape directly: envelope and proof slots must be null or full sha256:-prefixed 64-hex digests, and a TypeDID proof hash cannot appear without the paired envelope hash. As with authorization receipts, the request and bootstrap TypeDID hash slots are independently shaped replay evidence because they may come from different requests in the captured workflow. That keeps the compact handoff self-describing without moving TypeDID trust semantics out of TypeSec. Live request parsing now enforces the same boundary earlier: a caller that sends x-lakecat-typedid-proof without x-lakecat-typedid-envelope receives a hash-only typedid-proof-hash rejection, and a caller that sends agent delegation or agent summary proof headers without an agent-shaped identity receives only the matching proof hash. Those failures happen before governance or capability receipt creation, so raw proof material cannot become either policy context or operator-facing diagnostics. TypeDID verifier failures follow the same rule: malformed or rejected envelopes report only the envelope hash and error-detail hash, and a verified-subject mismatch reports only hashes of the verified and supplied principals before governance dispatch. LakeCat applies that redaction at the verifier trait boundary, so a custom TypeDID verifier can choose the error class without leaking raw envelope, DID, gateway, or payload text into the catalog response. The JSON output from lakecat-cli qglake-verify-handoff also carries the accepted lineage-drain identity source, identity state, and TypeDID envelope/proof hash slots in lineageDrainArtifactSemantics, so QueryGraph can index the verified drain boundary without reparsing the raw drain artifact. Before that boundary is accepted, the verifier reconciles the drain artifact's top-level delivered count, eventTypes list, graph event count, and lineage event count against the replay summary array. If a saved lakecatHandoffVerifyOutput artifact is present, LakeCat binds those saved drain identity semantics back to the compact requestIdentityProof, so a rehash cannot disguise drift in principal, authorization receipt, source/state, or TypeDID hash-slot evidence. The same self-verification pass compares the saved verifier output's delivered count, eventTypes, graph event count, and lineage event count with the archived lineage-drain artifact, so a rehashed verifier output cannot rewrite the drain manifest while keeping the artifact hash set intact. It compares captured replay-evidence.scan with governedScanProof, requiring positive plan task, scan-

plan graph event, file task, delete file, and child plan task counts plus the planned and fetched read-restriction objects and the fetch-side required projection/filter evidence. The verifier rejects a summary if the fetched restriction drifts from the planned restriction, so the compact handoff proves the narrowed allowed columns, row predicate, and policy hashes alongside the planned/fetched replay and OpenLineage hashes that prove the Sail-planned read path. The compact Rust verifier requires both planned and fetched replay/OpenLineage arrays to contain full sha256:-prefixed 64-hex digests, so automation can reject incomplete or placeholder scan lineage without falling back to the shell harness. Captured scan replay-line recomputation also reuses the governed read-restriction guard, so an archived replay artifact cannot make empty planned or fetched allowed-columns look like a readable operator summary. It also compares the captured replay-evidence.tableCommitHistory object with tableCommitHistoryProof, including the commit count, sequence numbers, commit hashes, replay principal subject/kind, authorization receipt hash/action, graph event count, replay hashes, and OpenLineage hashes that prove the pointer-log commit history was not rewritten between replay and summary and that the commit-history replay projected catalog graph evidence for the accepted actor. The compact verifier also requires the commit-history principal subject and kind to match the accepted QGLake handoff principal, requires the authorization receipt hash to be a full SHA-256 digest, requires the authorization action to be the read-side table-load action for table.commits-listed, requires the commit count to match the sequence-number and commit-hash arrays, requires every sequence number to be positive and strictly increasing, requires commit hashes to be duplicate-free, and requires positive graph event evidence plus replay and OpenLineage receipt hashes. Captured raw lineage-drain regressions cover both missing and drifted commit-history principal subject, principal kind, and authorization action, so actor and action attribution must survive before the compact handoff proof exists. The service admission layer now rejects table.commits-listed source replay whose authorization receipt principal is missing or malformed, whose top-level principal-subject or principal-kind is missing, or whose top-level actor summary drifts from the receipt before acknowledgement, catalog graph projection, or OpenLineage projection, so the raw lineage-drain summary is never built from an actorless pointer-log read. It also binds the replayed warehouse, namespace, and table evidence to the durable outbox table identity before projection, so a source replay cannot project one table's pointer log as another table's history. Captured LakeCat replay-line recomputation enforces the same sequence invariant even when the captured replay JSON and compact summary agree on malformed sequence evidence, so operator-readable table-commit-history-replay text cannot launder zero, duplicated, or reordered pointer-log proof. It compares the captured replay-evidence.views object with viewReceiptChainProof, including accepted view receipts, accepted-view graph event counts, expected-version guard evidence, tombstone receipts, namespace receipt-chain hashes, and their replay/OpenLineage hashes, so durable view history stays tied to the saved LakeCat replay artifact. It also compares the tombstone branch's expectedViewVersion with the accepted view version, so a handoff cannot claim a governed deletion unless the saved LakeCat replay proves that deletion used the catalog's optimistic version guard; the standalone qglake-verify-handoff command enforces this match even when a summary is checked outside the local shell harness. The same standalone verifier now requires the compact view proof to keep viewCount consistent with the accepted view list, carry stable warehouse/namespace/name identity, prove viewVersion == acceptedViewVersion, and

carry accepted receipt hashes, accepted receipt-chain hashes, positive accepted-view graph event evidence, tombstone receipt hashes, positive verified-chain counts, receipt-chain warehouse/namespace identity, namespace chain hashes, and replay/OpenLineage hashes. It also derives the expected lakecat:view:<warehouse>:<namespace>:<name> stable ID from each accepted view's warehouse, namespace, and view name, so a saved handoff cannot keep a verified stable ID while drifting the visible component fields. Tombstone receipt entries use the same component binding before their expectedViewVersion guard evidence is accepted, so deletion proof cannot drift from the stable view identity either. The verifier also checks that each namespace receipt-chain summary's verifiedChainCount equals the number of structural chains[] entries and that every chain entry is covered by chainHashes. The compact proof carries chains[] entries inside each namespace receipt-chain summary. Each chain entry keeps only catalog-facing evidence: stable view identity, the chain hash, the verified flag, latest view version, latest operation, tombstone state, receipt count, and per-receipt version, operation, view hash, principal subject, principal kind, recorded timestamp, receipt hash, and previous-link fields. The chain warehouse and namespace must match the enclosing namespace receipt-chain summary, and every receipt's stable ID, warehouse, namespace, and view name must match the chain identity. Each structural chain stable ID is also checked against its own warehouse, namespace, and view-name components, so compact evidence cannot splice receipts across views or namespaces while preserving hash-shaped fields. Structural chain bodies cannot repeat a chainHash. The enclosing namespace chainHashes and receiptHashes arrays are duplicate-free summaries of those same structural chains, and receiptHashes must match the structural per-receipt hashes exactly, so a compact proof cannot hide extra receipt hashes or omit receipts from the ordered chain bodies. Each structural chainHash is also recomputed from the same content-derived digest LakeCat service uses for view receipt chains: stable view identity, latest version, latest operation, tombstone state, and the ordered receipt hashes. A compact proof therefore cannot pair an accepted receipt-chain hash with a different ordered receipt body. LakeCat enforces the duplicate-free part before compact proof generation as well: outbox replay rejects duplicate receipt-hashes, drop-receipt-hashes, or chain-hashes in view receipt-list and receipt-chain events before graph or OpenLineage projection. Each structural receiptHash is recomputed too, using the same content-derived view-version receipt digest LakeCat service emits over stable view identity, version, previous-link fields, operation, view hash, principal, and recorded timestamp. That closes the gap between a valid-looking chain over opaque receipt hashes and a chain whose individual receipt bodies are themselves durable catalog facts. qglake-verify-handoff rejects a chain whose first receipt is not version 1 upsert, whose previous links do not point to the prior receipt, whose upsert skips a version, whose drop advances the durable version, whose operation is unsupported, or whose latest receipt does not match the chain head. In the compact verifier, accepted-view receipt hashes, accepted receipt-chain hashes, tombstone receipts, namespace receipt/chain hashes, per-receipt hashes, and view replay/OpenLineage hashes must all be full sha256:-prefixed 64-hex digests, so a saved handoff cannot use readable placeholder strings as view acceptance evidence. It also requires queryGraphBootstrapProof.viewVersionReceiptHashes to match the accepted view receipt hashes exactly, so the compact summary cannot combine bootstrap view receipt evidence from one run with accepted-view proof from another. Accepted receipt-chain hashes and tombstone receipt hashes must be covered by

structural chains[] evidence for the same stable view, not merely by another chain in the same namespace receipt-chain summary. A tombstoned view therefore carries both the accepted pre-drop chain and a structural drop chain whose final receipt is the tombstone. Tombstoned views must also include tombstone receipt evidence whose `expectedViewVersion` preserves the accepted view version. A consumer can reject a handoff whose view history claim lacks identity, accepted-version, count-aligned hash-chain evidence, duplicate-free exact structural receipt-hash coverage, content-derived chain-hash agreement, same-view accepted-chain coverage, tombstone guard evidence, same-view tombstone receipt coverage, or replay evidence before parsing the full replay tree. It compares captured LakeCat replay `replay-evidence.management` list counts with compact `lakecatReplayVerification.managementProof`, requiring positive server, project, warehouse, and storage-profile counts, positive graph event counts for server, project, warehouse, policy-binding, and storage-profile list replay, and a policy-binding count that matches the `QueryGraph` bootstrap proof. Those management-list counts are receipt-backed: the compact proof and captured replay must also agree on replay and OpenLineage hash arrays for `server.listed`, `project.listed`, `warehouse.listed`, `policy-binding.listed`, and `storage-profile.listed`. It also compares the captured LakeCat replay `replay-evidence.management.storageProfileUpsert` object with the compact `lakecatReplayVerification.storageProfileUpsertProof`, including the profile id, provider, issuance mode, location-prefix hash, secret-reference presence/provider/hash, replay hashes, and OpenLineage hashes. The compact verifier also requires those replay and OpenLineage arrays to contain full `sha256:-prefixed 64-hex` digests. It requires that location-prefix value to be a full `sha256:-prefixed 64-hex` digest and requires a redacted secret-reference provider plus full-digest `secretRefHash` whenever the proof says a secret reference is present. If the proof says no secret reference is present, the provider and hash may be omitted or null, but any other provider/hash value is rejected. Source replay enforces the same full-digest secret-reference rule before compact proof generation, so the saved summary cannot launder short placeholder credential-root hashes through the lineage-drain artifact. The positive QGLake acceptance fixture covers the production-shaped case too: when the storage profile uses a secret reference, the compact handoff accepts the proof only when the storage-profile branch and both credential branches agree on the redacted provider and full `secretRefHash`, and the operator replay line carries `secret_ref_hash=sha256:...` rather than a raw secret URI. Those operator-facing management and credential replay lines also fail closed when secret-backed evidence has only a prefix-shaped or placeholder hash, so the human-readable proof cannot be weaker than the structured verifier. It also compares the captured `replay-evidence.credentials` restricted-agent and trusted-human branches with the compact `credentialVendingProof`, so a saved handoff cannot claim that agents were blocked onto Sail-planned reads or that humans used an audited exception unless the captured LakeCat replay proves the same decision. That equality includes `credentialPrefixHashes`, `authorizationReceiptHash`, and `authorizationReceiptAction`, which closes the gap where a captured replay artifact could report a different redacted returned-credential set or authorization action while the compact summary still looked valid. Both credential branches must carry a full authorization receipt hash, the `credentials-vend` authorization action, and replay/OpenLineage arrays whose entries are full `sha256:-prefixed 64-hex` digests, so the compact proof cannot replace credential receipt evidence with prefix-shaped placeholders or a different catalog action. They

also carry `credentialPrefixHashes`: the restricted-agent branch must prove the array is empty when zero credentials were returned, while the trusted-human branch must prove the array length matches `credentialCount`, every entry is a full SHA-256 digest, and no prefix hash is repeated. The verifier also binds the operator-facing replay text back to the same proof fields. The captured top-level scan-replay line is recomputed from `governedScanProof`, including plan/fetch task counts, the policy-derived purpose, and the TTL cap. The captured top-level credential-replay line is recomputed from `credentialVendingProof`, including the restricted-agent block, trusted-human exception, TTL caps, and redacted storage-profile anchors. The captured management-replay line is also recomputed from `managementProof` and `storageProfileUpsertProof`, while `table-commit-history-replay` is recomputed from `tableCommitHistoryProof`. A saved artifact therefore cannot keep valid structured proof while presenting a different terminal transcript or principal-attribution story to an operator. Each credential branch carries the same redacted storage-scope anchor as the storage-profile upsert proof: `locationPrefixHash` binds the credential-vend attempt to the configured storage root without replaying the raw prefix. That anchor must be a full sha256:-prefixed 64-hex digest in both the storage-profile proof and each credential branch. LakeCat checks the storage-scope hash at lineage-drain replay time before the compact handoff summary is accepted, and the operator-readable credential replay line prints the same hash so captured terminal output cannot look complete while omitting the credential-root boundary. Source replay validates secret-reference shape on the credential branches themselves: if a credential-root proof says a secret reference is present, it must carry a non-empty provider and full sha256:-prefixed 64-hex `secretRefHash`; if it says no secret reference is present, provider and hash evidence must be absent. LakeCat now applies the same discipline before the outbox event is delivered: `credentials.vend-attempted` must carry a `credential-count` that matches its credential-response evidence, full SHA-256 prefix and issuer-config hashes for each returned credential, a full storage-profile `location-prefix-hash`, and non-contradictory secret-reference state. The event must also carry a valid authorization receipt principal before delivery, including blocked zero-credential attempts where no returned credential entry exists to repeat actor evidence. The top-level `storage-profile-id` must match the nested `storage-profile.profile-id`, even when no raw credentials were returned, and the nested `storage-profile.warehouse` must match the event table warehouse. The replay payload's table hint must also match the durable outbox table identity before acknowledgement, graph projection, or `OpenLineage` projection, so a credential decision for one table cannot be replayed as another table's credential-root evidence. If the top-level `secret-ref-present` field is missing, non-boolean, or different from `storage-profile.secret-ref-present`, the replay event is rejected before delivery. That duplicate field is small, but it keeps compact credential proof from omitting whether the selected credential root depends on an external secret reference. Each returned credential entry must also agree with the catalog-derived storage-profile id, catalog profile id, storage provider, credential mode, authorization principal, receipt principal, governed-read marker, and any policy-derived TTL cap. Returned credential entries must be duplicate-free by prefix-hash, so a replay event cannot count the same redacted credential twice. LakeCat carries those redacted prefix hashes into the raw lineage-drain summary as `credentialPrefixHashes`, and the QGLake verifier rejects the drain before compact proof generation if the prefix hashes are missing, count-drifted, short, or duplicated. A malformed credential replay event therefore remains pending instead of becoming

graph or OpenLineage evidence. Credential replay also rejects a governed read-restriction that is missing from, or different from, the authorization receipt context, so credential TTL and blocked-agent evidence cannot drift away from the receipt that authorized the decision. The credential replay boundary now also treats a governed restriction without a nonblank purpose or a positive `max-credential-ttl-seconds` cap as malformed before acknowledgement, graph projection, or OpenLineage projection. That keeps a blocked zero-credential agent decision from becoming QGLake evidence unless it still proves the policy purpose and TTL cap that made raw credential vending inappropriate. Source replay and compact handoff verification both reserve `rawCredentialExceptionReason` for the audited trusted-human path; a restricted agent proof must be blocked with `blockReason` and cannot carry a raw exception reason. The local handoff harness now preserves the lineage-drain artifact before replay verification, so if any of these proof checks fail the operator still has the exact failed drain JSON for diagnosis. The compact handoff verifier also validates that credential proof directly: the restricted branch must name the accepted agent principal, carry the Sail-planned-read block reason, prove zero credentials, carry the policy-derived `maxCredentialTtlSeconds` cap, explicitly set `rawCredentialExceptionAllowed` to false, reject any non-null `rawCredentialExceptionReason`, and include replay/OpenLineage hashes; the trusted-human branch must name a human principal, prove a positive credential count, carry the same policy-derived TTL cap, carry the exact audited raw-credential exception reason, prove `blockReason` is null, and include replay/OpenLineage hashes. Both branches must also carry count-aligned, duplicate-free `credentialPrefixHashes`, with an empty array for the blocked branch and full SHA-256 returned-prefix hashes for any issued credential. The compact verifier has direct negative coverage for the credential-branch secret-reference rules too: blank providers are rejected when a secret ref is present, and non-null provider/hash evidence is rejected when no secret ref is present. That way a handoff cannot hide malformed provider or hash evidence behind an otherwise matching storage-profile upsert proof. The local handoff harness now preserves those replay fields while building the compact summary, rather than relying on the nested raw replay blob alone: scan graph events and fetch-side projection/filter requirements, management graph events, storage-profile graph events, credential block/exception fields, and table commit-history graph events all move into the verifier-facing proof sections before `qglake-verify-handoff` runs. That makes the handoff repeatable from the LakeCat repo while keeping QueryGraph responsible for graph validation and import semantics. The handoff script refuses to write the summary unless LakeCat replay JSON contains request-identity evidence for the expected agent principal, an explicit identity source/state, an authorization receipt hash, and explicit TypeDID envelope/proof hash slots that are either null or full `sha256:-`-prefixed 64-hex digests. A proof hash is valid only when the matching envelope hash is present. It also refuses to write the summary unless LakeCat replay JSON contains redacted `storageProfileUpsert` evidence with replay and OpenLineage hashes, and the accepted summary repeats that evidence as `lakecatReplayVerification.storageProfileUpsertProof`. QueryGraph gets proof that the credential root was configured, including the provider, issuance mode, the configured location-prefix hash, and a full `sha256:-`-prefixed 64-hex digest of the secret reference, without receiving the underlying secret-store URI or full storage prefix in the compact proof. That compact proof must also name the principal subject and kind, carry a full authorization receipt hash, and prove the receipt action was `storage-profile-manage`. Captured LakeCat replay and the compact QGLake

summary are compared field by field, so a saved handoff cannot turn an authorized storage-profile management action into an actorless credential-root fact or replay it under a weaker catalog action. The operator-readable management replay line now prints the same storage-scope hash and redacted secret-reference state, so a captured transcript cannot describe the credential root only by provider while omitting its redacted storage scope or secret-reference boundary. The saved handoff verifier recomputes that management line from compact proof fields before accepting captured output. The script also refuses to write a summary unless LakeCat replay proves both sides of credential vending: untrusted agents get no raw credentials, trusted humans receive only the audited standard exception, and both branches preserve the `max-credential-ttl-seconds` restriction as `maxCredentialTtlSeconds` in compact evidence. For reads, the summary similarly refuses to omit proof that scan planning and scan-task fetch both replayed with full digest-shaped sink receipt hashes. The compact scan proof must preserve the server-derived read restriction as a full restriction, not only as columns and filters: allowed columns, row predicate, purpose, full `sha256:-prefixed` policy-hash evidence, and `max-credential-ttl-seconds` must be present, and the planned and fetched restrictions must agree. Short readable policy anchors such as `sha256:policy-name` are rejected before QueryGraph receives the compact handoff proof. The fetched required filters must also be exactly the mandatory row predicate evidence, not a prefix with extra unverified filters appended. For catalog state, it refuses to omit proof that the table commit-history read replayed with sequence-number and commit-hash evidence. For views, it refuses to omit proof that accepted view versions line up with their receipt hashes and that the namespace-level tombstone and receipt-chain reads replayed with chain hashes and verified-chain counts. The service-side lineage-drain summary preserves the full receipt hash set from receipt-list reads and nested namespace receipt-chain payloads, so the QGLake verifier can prove that both upsert and tombstone receipts are covered by the replayed namespace chain.

This gives the semantic layer a responsible starting point. LakeCat says:

```
Here are the governed catalog objects.
Here are the policies that shaped planning.
Here are stable dataset and field anchors.
Here is the graph envelope.
Here is the OpenLineage replay evidence.
Here are the hashes that bind the handoff.
```

QueryGraph then owns the richer semantic work: metrics, dimensions, joins, business names, multi-dataset reasoning, and agent-facing synthesis.

1.27.8 Draining The Outbox

LakeCat records side effects as durable outbox events. Draining the outbox is what turns committed catalog facts into graph and lineage receipts:

```
curl -s -X POST \
  -H 'x-lakecat-principal: agent:lineage-drainer' \
  http://127.0.0.1:3000/management/v1/lineage/drain \
  -o target/qglake/lineage-drain.json
```

A useful drain response includes delivered event types, graph projection counts, lineage projection counts, receipt hashes, and the authorization proof for the drain request itself. That last part is easy to overlook. Reading the replay stream is also privileged, so LakeCat records that the drainer was allowed to read lineage evidence. Before LakeCat returns that raw lineage-drain summary or acknowledges delivery, it also checks the projection receipt arrays produced by the graph/lineage sink boundary: replay hashes and OpenLineage hashes must be count-aligned with lineage events, full SHA-256-shaped, and duplicate-free. That keeps a malformed sink receipt from inflating the proof QGLake later archives. Standard catalog reads replay too: `catalog.config-read` records a warehouse-scoped graph/OpenLineage fact for the Iceberg REST configuration endpoint; `namespace.listed` records the namespace listing at the warehouse; and `namespace.loaded` records the specific namespace resolved through the standard catalog route. For view events, the response includes the warehouse, namespace, view name, and QueryGraph-compatible stable id, so downstream acceptance can check replay identity without parsing the full audit payload. Table restores replay as table graph evidence plus a restore OpenLineage receipt, so a soft-deleted table returning to service is visible to QueryGraph without forcing LakeCat to invent restore-specific graph taxonomy. Management list reads for policy bindings, projects, servers, storage profiles, and warehouses replay as OpenLineage receipts too. They intentionally do not create list-specific graph nodes in LakeCat; Grust owns the reusable hierarchy and traversal model. The durable audit/outbox payload carries only the redacted stable ID arrays beside the counts: policy ids, project ids, server ids, storage-profile ids, and warehouse names. Before projection, LakeCat rejects a management-list event when the required ID array is missing, malformed, contains an invalid identifier, repeats an identifier, or no longer matches the recorded count. LakeCat also requires the authorization receipt to carry a valid principal, the event-matching catalog action, an affirmative allow decision, a non-empty engine, and an RFC3339 checked-at timestamp before acknowledging the event, so QueryGraph never has to accept actorless or action-drifted management inventory replay. The drain response lifts their counts, ID arrays, and management scope into compact fields, so QueryGraph can verify the control-plane read evidence without opening the raw lineage payload. It also carries replay and OpenLineage hash arrays for those management-list reads, so a compact handoff cannot prove only that the right number of management records existed while losing the identities or receipt evidence for the reads. The lineage-drain verifier rejects those source replay events when the ID arrays are missing, empty, duplicate-inflated, count-drifted, or when the receipt arrays are empty or not full SHA-256-shaped, or when the event no longer preserves nonblank principal subject/kind evidence and a full authorization receipt hash, so the compact managementProof starts from verified replay evidence rather than untrusted text. This is deliberately LakeCat/QGLake/TypeSec control-plane proof: the underlying namespace, table, warehouse, policy, and storage-profile inventory remains standard catalog state, while the actor, authorization, replay, and OpenLineage receipts prove how that state was read and projected for QueryGraph. The compact handoff verifier repeats that check with the stricter full sha256:-prefixed 64-hex digest shape for every management replay and OpenLineage array, and it verifies that `serverIds`, `projectIds`, `warehouseNames`, `policyIds`, and `storageProfileIds` match their recorded counts and are duplicate-free. Saved summaries therefore cannot preserve only prefix-shaped placeholders for control-plane read receipts, inflate a count with repeated valid identities, or normalize malformed management identities later. Captured replay agreement checks the same

ID arrays against the saved compact managementProof, so a handoff cannot keep valid artifact hashes while swapping the server, project, warehouse, policy, or storage-profile identities between source replay and the summary. When an operator preserves the verifier output as lakecat-handoff-verify.json, LakeCat re-checks the saved capturedOutputSemantics.lakecatReplay proof against the compact summary for every replay section, including management IDs, governed scans, commit history, view receipt chains, storage-profile evidence, and credential-vending proof. That makes the verifier output a replayable audit artifact instead of a second place where compact proof drift can hide. It also rejects management-list source replay without catalog graph projection evidence, keeping the durable server/project/warehouse, policy, and storage-profile facts visible to QueryGraph through Grust-facing graph events. Compact managementProof carries those graph event counts too, and captured replay agreement checks them, so the graph evidence cannot disappear between source replay and handoff verification. Policy binding upserts add a content anchor to that management proof. A policy list can prove that agent-columns was listed; it cannot prove which ODRL document agent-columns meant. LakeCat now carries a compact policyUpsertProof with policyId, odrHash, graph event count, replay hashes, OpenLineage hashes, principal subject/kind, a full authorization receipt hash, and the policy-manage action. The raw lineage-drain verifier requires a matching policy-binding.upserted replay event, requires the policy id to be present in the policy list, requires the ODRL hash to be a full SHA-256 digest, and rejects action drift away from policy-manage. Captured replay agreement compares the same object against the saved summary. That keeps QueryGraph from accepting a management proof that preserved the policy name but lost or swapped the policy document anchor or the authority under which it was recorded. Tenant-root upserts get the same hash-binding treatment. When a server replay event carries an endpoint-url, LakeCat recomputes endpoint-url-hash from that value before projection. When a warehouse replay event carries a storage-root, LakeCat recomputes storage-root-hash from that value before projection. This is not an Iceberg table-access rule; it is LakeCat/QGLake management proof. It keeps a replay event from pairing one raw endpoint or storage root with a valid-looking hash for another value, then asking Grust, OpenLineage, or QueryGraph to trust the mismatched tenant-root evidence. The QGLake acceptance workflow now establishes its server/project/warehouse tenant spine, performs governed server, project, warehouse, policy-list, policy-upsert, storage-profile-list, scan-planning, scan-task-fetch, and table commit-history reads before bootstrap, and rejects a drain that does not replay matching server.listed, project.listed, warehouse.listed, policy-binding.listed, policy-binding.upserted, storage-profile.listed, table.scan-planned, table.scan-tasks-fetched, and table.commits-listed evidence. Request-identity and bootstrap replay are checked before any compact handoff proof is built: the drain authorization, bootstrap authorization, QueryGraph bundle/import hashes, agent delegation hash, agent summary signature hash, and TypeDID envelope/proof hashes must be full sha256:-prefixed 64-hex digests, and a TypeDID proof hash cannot appear without its paired envelope hash. For scan replay, the typed drain summary carries scan-plan task counts, scan-plan graph event evidence, fetched file-scan, delete-file, and child-plan task counts, along with planned and fetched OpenLineage receipt hashes. Source replay validation now also requires planned/fetched replay and OpenLineage receipt arrays to be full SHA-256 digests, and the compact handoff verifier repeats that full-digest check for the saved governedScanProof arrays. The scan read restriction itself is part of

that proof: both source replay and compact `plannedReadRestriction/fetchedReadRestriction` evidence require `policy-hashes` to be non-empty full `sha256:-prefixed 64-hex` digests, so a self-consistent handoff cannot smuggle placeholder policy names or empty policy anchors through a field that later readers treat as integrity evidence. The outbox drain checks the same digest shape and non-empty requirement before acknowledging any pending event that carries `read-restriction.policy-hashes`, including the copy embedded in the authorization receipt context, so malformed source evidence is stopped before it becomes delivered replay material. Scan replay also requires the top-level read restriction to match the authorization receipt context exactly before delivery, so policy narrowing cannot be asserted in one replay field and absent from the durable receipt. Scan replay also requires the authorization receipt itself to be complete before delivery: a valid principal, the event-matching `table-plan-scan` catalog action, an affirmative allowed decision, a non-empty receipt engine, and an RFC3339 `checked_at` timestamp. Valid-but-wrong actions such as `table load` or `commit` actions are rejected before governed scan proof reaches graph or lineage sinks. This is LakeCat replay admission, not Sail planning logic; Sail remains responsible for producing reusable `table-format` and `scan-planning` behavior, while LakeCat refuses to turn actorless or action-drifted scan evidence into graph or lineage proof. QGLake preserves the same actor and action evidence in compact handoff proof: `planned` and `fetch`d scan proof carry principal subject/kind, full authorization receipt hashes, and `table-plan-scan` actions, and captured LakeCat replay must match those fields. That keeps archived handoffs from retaining only the restriction and task counts while dropping who was authorized to perform the governed scan. Scan replay now gets the same drain-side admission check before Grust or OpenLineage projection: `planned-scan` events must carry matching table identity, unsigned task counts, requested/effective projection arrays, and requested/effective stats-field arrays; `fetch`d-task events must carry matching table identity, `fetch`d file/delete/child-plan counts, required filters, and required/effective projection arrays. Those scan proof arrays must be non-empty, non-blank, and duplicate-free; present-but-empty projection or stats evidence is malformed, not an implicit unrestricted read. `Fetch`d-task `required-filters` must also exactly preserve the governed row predicate at service admission, so an event with empty or drifted `fetch`d filter proof is rejected before graph or OpenLineage projection. When a governed read restriction is present, the effective projection and effective stats fields must remain inside the allowed columns, empty allowed-column arrays fail closed for both `planned` and `fetch`d replay, and explicit effective projection or stats evidence cannot widen beyond the caller-requested or server-required evidence it claims to preserve. QueryGraph bootstrap replay is also checked at the drain boundary before it becomes accepted handoff material: the event must carry a valid warehouse, `table/view` counts matching the verified ids and artifact arrays, full SHA-256 bundle/graph/OpenLineage/import hashes, full `table/view` artifact hashes, view receipt and receipt-chain hashes for accepted views, the expected standards list, and full optional TypeDID or agent proof hashes when those slots are present. View receipt replay follows the same fail-closed rule at the drain boundary. A `view.version-receipts-listed` event is not acknowledged unless its warehouse, namespace, view, and authorization receipt principal are valid, its authorization receipt action is `view-load`, its receipt-count matches full SHA-256 receipt hashes, and every drop receipt hash is included in the listed receipts. A `verified view.version-receipt-chains-listed` event is not acknowledged unless its warehouse, namespace, authorization receipt principal, read-side

view-load authorization action, chain count, receipt count, and tombstone count are valid and count-aligned, each verified chain and receipt carries full SHA-256 digest evidence, the first receipt is a version 1 upsert without previous links, and every later upsert or drop links to the previous receipt with the expected view-version transition. That keeps malformed view-history evidence out of both graph projection and OpenLineage replay before QueryGraph ever sees a compact handoff. The verifier also requires table-commit replay to be internally consistent before delivery: `table.commit` must carry a commit object, unsigned sequence number, stable table identity, matching nested commit-table identity, a valid commit principal and a valid authorization receipt principal with matching values, an authorization receipt whose action is a known LakeCat catalog action matching the `table.commit` event, whose allowed decision is true, whose engine is non-empty, whose `checked_at` timestamp is RFC3339, an RFC3339 commit `committed_at` timestamp, and commit hash evidence that is full SHA-256 before graph or OpenLineage projection can start. The action, decision, engine, and timestamp fields are deliberately small but important: replay evidence must prove which catalog action was authorized, prove the catalog acted under an affirmative authorization decision, say which engine made that decision, preserve when the authorization check happened, and preserve when the catalog accepted the pointer transition. Local/default receipts identify the local allow-all compatibility engine, while real TypeSec-backed receipts identify TypeSec. That keeps replay evidence from becoming actorful but action-less, decision-less, engine-less, or timeless proof. The compact QGLake handoff now preserves the same action proof instead of collapsing it into a receipt hash. The lineage-drain response carries `authorizationReceiptAction` for the drain read itself and for each replayed event summary. The QGLake verifier requires the drain read to prove `lineage-read`, requires each replayed event summary to carry a non-empty action, and rejects valid-but-wrong action drift, such as `table-commit` attached to `table.commits-listed`. Captured replay agreement checks the same field in saved handoff artifacts, so an archive cannot keep valid hash-shaped proof while silently changing what catalog action was authorized. The compact handoff summary now makes the same requirement before captured-output comparison: request identity proves `lineage-read`, and QueryGraph bootstrap proves `graph-read`. The saved self-verifier sidecar carries top-level copies of both compact proof objects, and artifact verification rejects those copies if either authorization action drifts from the summary. The saved self-verifier sidecar repeats that binding for `lineageDrainArtifactSemantics`: its drain-read `authorizationReceiptAction` must still match the compact request-identity proof, so a rehashed `lakecat-handoff-verify.json` cannot describe a different lineage-read action. Commit-history replay has the same shape: `table.commits-listed` event must carry a `commit-count` that matches both full SHA-256 commit hashes and unsigned sequence numbers, plus `principal-subject` and `principal-kind` fields that match the authorization receipt principal, a known authorization receipt action matching `table.commits-listed`, specifically the read-side `table-load` action rather than a mutation action such as `table-commit`, an affirmative authorization receipt decision, and a non-empty authorization receipt engine with an RFC3339 `checked_at` timestamp; compact QGLake proof also binds that pointer-log replay to the accepted principal subject/kind, a full authorization receipt hash, and the `table-load` action. The raw QGLake lineage-drain verifier checks the same accepted-principal, agent kind, receipt hash, and action before compact handoff proof is generated, so malformed, denied, actor-drifted, action-drifted, action-less, decision-less, engine-less, or timeless pointer-log sum-

maries cannot become delivered replay evidence. Individual `table.commit` replay is held to the same commit envelope before graph or OpenLineage delivery: it must include a positive sequence number, non-empty new metadata pointer evidence, non-blank previous pointer evidence when present, matching commit and authorization principals, the `table-commit` receipt action, positive Iceberg format-version evidence, non-negative snapshot-id evidence, and full SHA-256 request, response, and idempotency-key hashes. The store path now supplies explicit `snapshot_id: 0` proof for metadata with no current snapshot, keeping empty-table or schema-only commits compatible with the replay contract. The policy hash is the only optional hash in that envelope, because some standard commits do not pass through a policy binding. Credential-vend replay gets the same treatment: `credentials.vend-attempted` must carry a matching credential count, full duplicate-free credential-response prefix hashes, a full redacted storage-profile location hash, a valid authorization receipt principal, a full authorization receipt hash, the `credentials-vend` authorization action that matches the outbox event type, internally consistent secret-reference presence/provider/hash fields, a top-level storage-profile id that agrees with nested storage-profile evidence, a nested storage-profile warehouse that agrees with the event table warehouse, required top-level secret-reference presence evidence that agrees with nested storage-profile evidence, and credential-response metadata that agrees with the selected storage profile and authorization receipt before delivery. Storage-profile upsert replay must likewise reject raw secret references and contradictory secret-reference-state evidence before delivery. Policy-binding upsert replay must carry valid catalog scope evidence before delivery, including policy id, warehouse, optional namespace/table scope, enforcement state, captured ODRL material, and a matching `odrl-hash`. Namespace lifecycle replay must carry a valid warehouse and namespace path or component array before create/load/drop events can be delivered. Catalog config and namespace-list read replay must likewise carry a valid warehouse, and namespace listing must preserve both an unsigned namespace count and count-aligned namespace-paths evidence. Those paths are parsed as namespace identities and must be non-empty and duplicate-free before standard catalog reads become delivered graph/OpenLineage evidence. Catalog config, namespace list, namespace lifecycle, view list, and view lifecycle replay must also carry valid authorization receipt principals, so saved replay cannot turn standard Iceberg control-plane activity into actorless QueryGraph facts. Management-list read replay applies the same rule to operational discovery: policy, project, server, storage-profile, and warehouse list events must carry unsigned counts, valid warehouse scope when warehouse-scoped, and valid optional project scope before delivery. The warehouse-list project scope is parsed as a project identifier, not accepted as an arbitrary string, so compact management proof cannot smuggle malformed project filters into QueryGraph or OpenLineage. QGLake preserves that scope as `warehouseProjectId` in compact managementProof, and the verifier requires it to match one of the compact `projectIds`. A saved handoff therefore cannot pair a project-filtered warehouse inventory with an unrelated or malformed project identity. View list and lifecycle replay must carry valid warehouse, namespace, view name, count, and receipt principal evidence before those view events become graph/OpenLineage material for QueryGraph. View-list replay also carries count-aligned view-names evidence. Each name must parse as a valid catalog view/table name and the array must be duplicate-free, so an archived `view.listed` event cannot inflate view discovery by repeating or forging view identities. The receipt action must be `view-load`, matching the compact QGLake action contract; `view-manage` remains mutation proof

for `view.upserted`, while `view.loaded` uses `view-load` and `view.dropped` uses `view-drop`. View lifecycle replay with a drifted action is rejected before graph or OpenLineage projection, so QueryGraph cannot accept a view mutation or read under the wrong catalog permission. Table lifecycle replay applies the same identity discipline before delivery: `table.created`, `table.loaded`, `table.deleted`, and `table.restored` must carry a decodable table identity, optional payload scope hints must match it, delete replay must carry soft-delete evidence that points at the same table with a positive unsigned version, and the authorization receipt must carry a valid principal plus the matching lifecycle action. Create, load, and restore replay must also carry the unsigned table version emitted by the catalog producer plus positive Iceberg format-version evidence. Delete replay carries the same generation and table-format evidence inside the required positive soft-delete record, accepting the durable Rust record spelling `format_version` and the Iceberg-style proof spelling `format-version`. When those lifecycle events carry `table metadata-location`, `table location`, or `soft-delete metadata-location` evidence, the values must be non-empty before the event is acknowledged or projected. The Iceberg table operation remains the standard catalog action; the stricter non-empty replay evidence is LakeCat's control-plane proof that QueryGraph and OpenLineage did not accept an empty pointer, table-format, or storage-location claim from a corrupted outbox record. Project, server, and warehouse upsert replay must carry valid tenant-root evidence too: project ids, server scopes, endpoint URLs, storage roots, identifiers, properties, and pre-redacted hash anchors are checked before delivery. Policy-binding, project, server, storage-profile, and warehouse upsert replay must also carry a valid authorization receipt principal, an event-matching catalog action, an affirmative allow decision, a non-empty receipt engine, and an RFC3339 checked-at timestamp before delivery, so compact QueryGraph proof can attribute management mutations to the actor and TypeSec-style action accepted by LakeCat. It also requires planned and fetched read restrictions to match before compact proof generation, requires both requested/effective projection and requested/effective stats-field evidence, requires effective projection to be a narrowed subset of the requested projection and to match the planned allowed columns, and requires effective stats fields to be both inside the planned allowed columns and a narrowed subset of the requested stats fields in source replay and compact handoff proof. Empty allowed-column evidence is rejected in planned and fetched replay instead of being interpreted as unrestricted replay. It also requires the fetched projection and filter requirements to exactly preserve the fetched allowed columns and row predicate. A fetched response that omits required-filter proof is rejected just like one that widens or changes that proof, and the compact handoff summary applies the same missing-proof check before accepting governed scan evidence. Credential replay applies the same policy-proof discipline to the two credential branches: the restricted-agent denial and trusted-human audited raw-credential exception must both carry a complete read restriction, and those restrictions must match before credential proof can feed the compact handoff summary. For commit-history replay, the typed drain summary carries the commit count, committed sequence numbers, commit hashes, replay hashes, and OpenLineage hashes. The handoff verifier rejects compact scan proofs without those OpenLineage hashes and compact commit-history proofs whose counts, sequences, or hash arrays do not align. It also requires the compact commit, replay, and OpenLineage arrays to contain full `sha256:-`-prefixed 64-hex digests, not prefix-shaped placeholders. Source replay validation applies the same pointer-history discipline before compact proof generation: the table commit count must match the sequence-

number and commit-hash arrays, commit sequences must be positive and strictly increasing, and commit hashes must be SHA-256-shaped before pointer-history evidence can enter the compact handoff proof. Service route coverage pins the producer side too: request hashes, response hashes, idempotency-key hashes, and commit hashes are full SHA-256 digests across the route response, pointer-log outbox payload, lineage-drain summary, and graph projection. The QGLake fixture verifier also checks the management commit-history response itself, so short readable placeholders are rejected before the later lineage-drain and compact handoff checks run. Query-Graph can therefore verify the governed Sail-planned read and pointer-history inspection without parsing the full lineage payload. The core QueryGraph bundle, graph, OpenLineage, and import anchors must be SHA-256-shaped in compact verify/import/bootstrap proof before a matching summary can pass; matching strings are not enough. The bootstrap, scan, credential, view, receipt-chain, and commit-history receipt arrays must also be SHA-256-shaped before compact proof generation can consume them. The same shape check applies to accepted-view receipt evidence: bootstrap view-version receipt hashes, tombstone receipt hashes, and namespace receipt-chain hashes must be full SHA-256 digest evidence before accepted-view proof feeds the compact handoff summary. Dropped and active accepted-view source replay also compares the bootstrap view-version receipt hashes with the accepted QueryGraph verification set, so a valid-looking receipt array cannot be spliced from another bootstrap proof. The compact handoff verifier repeats the same binding against `viewReceiptChainProof.views[].acceptedReceiptHash` and `viewReceiptChainProof.views[].acceptedReceiptChainHash`, including tombstoned accepted views, catching drift even when an operator validates only the saved summary. Dropped accepted-view source replay also binds the namespace receipt-chain read back to the accepted view warehouse/namespace and rejects verified-chain count or receipt-hash coverage drift before compact handoff proof is generated. The lineage-drain summary now carries the nested chain receipts as full receipt hash coverage before that check runs. Generated replay evidence also preserves each accepted view's `acceptedReceiptChainHash` inside the namespace `receiptChains[].chainHashes` set, even when the namespace read has its own chain hash, so the compact summary can prove the accepted chain is covered by the namespace proof it verifies. The same replay now emits catalog-facing Commit graph events for the listed sequences, leaving traversal and query semantics to Grust.

For handoff testing, LakeCat can verify a saved bootstrap bundle and a saved drain response together:

```
cargo run -p lakecat-cli -- qglake-verify-replay \
  --bundle target/qglake/lakecat-bootstrap.json \
  --drain target/qglake/lineage-drain.json \
  --principal did:example:agent
```

That offline check replays the same boundary assertions used by the live QGLake fixture: the bundle manifest must verify, the QueryGraph import compatibility contract must match, and the lineage drain must carry matching bootstrap hashes, credential-denial receipts, management-list evidence, scan/fetch task evidence, and table commit-history receipt evidence, plus view receipt evidence when views are present. On success, the command prints the accepted bundle and QueryGraph import hashes, table/view counts, and compact control-plane lines such as:

[illegible]

The scan line keeps the planned and fetched credential TTL caps visible beside the task counts, while JSON mode carries the full read-restriction evidence tree. Scan planning records both requested and effective projection evidence; scan-task fetch records the server-derived required projection and mirrors it as effective-projection, so replay can compare both stages with the same policy-narrowed vocabulary. QGLake acceptance now rejects handoffs where the fetched effective projection is missing or drifts away from the fetched read restriction, which means a compact replay summary cannot quietly widen what the server actually planned. The live handoff harness performs that projection check before writing `handoff-summary.json`, so the artifact is born with the same proof shape the verifier later enforces.

```
cargo run -p lakecat-cli -- qglake-verify-handoff \
  --summary target/qglake-handoff/handoff-summary.json \
  --json
```

valid integrity anchors before the byte comparison runs. It parses the saved bootstrap bundle and reruns the tenant graph and semantic hash verifier. It also parses the saved QueryGraph import plan and requires its embedded verification, table/view stable ids, semantic hashes, standards, and graph node/edge evidence to match the compact QueryGraph import proof. The verifier requires the compact verifiedTables and verifiedViews manifests to be duplicate-free as well as count-aligned, matching service-side outbox admission, so a saved handoff cannot inflate the number of accepted tables or views by repeating an already accepted stable id. Raw lineage-drain replay summaries and compact handoff proof sections both keep replay, OpenLineage, credential prefix, view receipt, and view receipt-chain hash arrays duplicate-free, not only sha256: shaped. That covers the bootstrap, governed scan, management, table commit-history, view tombstone/receipt-chain, storage-profile upsert, and credential-vending proof sections, so a source replay or archived handoff cannot make an evidence set look larger by repeating an already accepted digest. The service applies the first version of that rule before a drain summary is returned at all: projection receipt hash arrays must match the lineage-event count and must not contain malformed or repeated replay or OpenLineage hashes. The verifier also compares those QueryGraph import-plan graph node and edge counts with the verified bootstrap bundle graph counts, so an import plan cannot keep the semantic hashes and table/view ids while silently dropping graph material. It also parses the saved lineage-drain response, reruns the typed QGLake replay verifier, and regenerates the LakeCat replay evidence that proves request identity, QueryGraph bootstrap replay, governed scan replay, pointer history, view receipt chains, storage-profile replay, and credential-vending decisions. It then compares that regenerated replay evidence to the compact lakecatReplayVerification proof. The governed scan proof includes both the requested and effective stats-field arrays from the scan-planned replay, and the verifier rejects handoffs where the effective fields no longer match the allowed columns or no longer prove policy narrowing. Credential-vending proof is not just a count: each restricted-agent and trusted-human branch carries the redacted storageProfile graph anchor and maxCredentialTtlSeconds, including profile id, provider, issuance mode, secret-reference presence, and the graph event count emitted by replay. The storage-profile upsert proof also carries its own positive graphEvents count, and captured LakeCat replay must match it. The verifier also rejects a handoff when the credential branches do not bind back to the same storage-profile upsert proof: profile id, provider, issuance mode, location-prefix hash, and secret-reference state must all match the replayed management event. A saved handoff is rejected if the archived lineage drain proves lineage receipt hashes but omits that credential TTL cap or credential-root graph projection. It also recomputes the captured LakeCat replay and QueryGraph verify/import output hashes, so terminal captures cannot drift from the compact summary. It compares the legacy string path aliases for the LakeCat replay, QueryGraph verify, and QueryGraph import captures with the hashed capturedOutputs paths they duplicate. It also hashes the service log through a full-digest serviceLogHash, so archived operational logs cannot drift behind a stable path or a short placeholder hash. The final local summary also binds the first LakeCat handoff-verifier capture with a full-digest lakecatHandoffVerifyOutputHash. Because that output can only exist after a successful verifier run, the harness performs a second sidecar self-check: first it writes target/qglake-handoff/lakecat-handoff-verify.json, then it records the file's hash in the summary, then it verifies the summary again without overwriting the declared artifact. The verifier checks that saved JSON

is a lakecat.qqlake.handoff-verification.v1 success for the same principal, catalog URL, warehouse, namespace, and table, and that its table/view counts, stable ids, standards, request-identity proof, and QueryGraph bootstrap proof still match the compact handoff summary. It also checks the saved self-verifier output's bundle, lineage-drain, QueryGraph import-plan, captured-output, and service-log hashes against the summary's artifact manifest. It also checks the saved self-verifier output's own semantic sections: captured replay semantics must match the compact LakeCat and QueryGraph proof, bundle artifact semantics must match QueryGraph verification, import-plan semantics must match QueryGraph import verification, lineage-drain semantics must match the accepted replay proof, and saved import-plan graph counts must still match the saved bundle graph counts. Then it parses the archived lineage-drain artifact and requires the saved lineage-drain semantics' delivered count, event type list, graph event count, lineage event count, and drain authorization action to match before accepting the verifier-output hash. The archived drain itself must also reconcile those same top-level counts with its replay summary array, including repeated event-type multiplicity and the exact eventTypes to replay-summary order. Then it parses those captured JSON files and checks that the replay schema/status, table/view counts, semantic hashes, standards, request-identity proof, QueryGraph bootstrap proof, governed scan proof, storage-profile upsert proof, and credential-vending proof inside the captures still match the summary. It also rejects malformed TypeDID hash slots in the request-identity and QueryGraph bootstrap proofs before a consumer has to interpret those slots. The local handoff harness runs it automatically and writes the captured verifier output to target/qqLake-handoff/lakecat-handoff-verify.json. Before the harness writes compact proof, it checks the replay and OpenLineage hash arrays it lifts from LakeCat replay evidence as full SHA-256-shaped and duplicate-free, so malformed compact proof is rejected before the archived handoff summary is treated as accepted.

The end-to-end result is a chain:

```
catalog write
-> audit event
-> outbox event
-> graph projection
-> OpenLineage projection
-> QueryGraph import evidence
```

If graph or lineage sinks are down, catalog state should not be lost or rolled back accidentally. The outbox lets LakeCat retry projection from committed state. A drain acknowledges delivery only after every projection in the batch succeeds. If OpenLineage fails after a graph event has already been emitted, the drain fails and the catalog event remains pending, so recovery starts from the committed outbox rather than from guesswork. If the graph sink fails first, LakeCat fails the drain before emitting lineage and still leaves the outbox event pending, so graph and lineage consumers recover from the same committed catalog fact instead of diverging.

Replay order is part of that contract. LakeCat selects undelivered outbox events by created_at,event_id in both embedded memory tests and the durable Turso store, so a QGLake replay does not depend on writer interleaving or database row-return quirks. Delivery acknowledgement is duplicate-safe as well: if a drainer accidentally reports the same event id twice,

LakeCat marks the event once and the receipt count remains tied to committed catalog facts. Pending batch validation happens before projection: if a store returns duplicate event IDs in the same drain batch, LakeCat fails the drain with only the duplicate event-id hash and does not emit graph, OpenLineage, or acknowledgement side effects for that batch. The same redaction rule applies to malformed pending records. If a custom or corrupted store hands the drain an event whose payload cannot be projected, LakeCat reports only the outbox event-id hash and stops before graph emission, lineage emission, or delivery acknowledgement. Malformed table and principal identity JSON decode failures, as well as unsupported event types, follow that same pattern: they carry event-hash evidence for correlation without echoing the raw event identifier into diagnostics.

1.27.9 An Agentic QGLake Flow

The agentic path is the reason LakeCat has to be more than a passive catalog. Imagine a resilience supervisor agent investigating incidents:

1. The supervisor delegates table triage to a specialist agent.
2. The specialist asks LakeCat to plan a scan over `local.default.events`.
3. LakeCat resolves the agent identity and TypeDID context.
4. TypeSec authorizes the table scan and returns a restricted capability.
5. LakeCat narrows the projection and appends the required row predicate.
6. Sail plans against the current Iceberg metadata and delete manifests.
7. LakeCat returns governed plan and fetch-task responses.
8. The specialist summarizes only the allowed result shape.
9. LakeCat records scan and credential decisions into audit/outbox.
10. QueryGraph imports graph, policy, lineage, and bootstrap evidence.

The key point is the absence of raw storage reach. The specialist agent does not need broad cloud credentials to do its job. It needs a governed plan, a bounded task set, and a receipt trail.

The local fixture compresses this story into a short artifact-producing sequence:

```
cargo run -p lakecat-cli --features qglake-fixture -- qglake-fixture \
  --output target/qglake/lakecat-bootstrap.json \
  --drain-output target/qglake/lineage-drain.json \
  --principal did:example:agent
cargo run -p lakecat-cli -- qglake-verify-replay \
  --bundle target/qglake/lakecat-bootstrap.json \
  --drain target/qglake/lineage-drain.json \
  --principal did:example:agent
```

The fixture generator opts into `lakecat-cli`'s `qglake-fixture` feature because it writes local Iceberg metadata and manifest files through Sail. Verification commands stay in the default CLI surface, which keeps ordinary catalog inspection, replay, and handoff checks available without pulling in the local fixture writer.

The one-command handoff wraps the same evidence in a live local service run and then asks QueryGraph to verify and import it:

```
scripts/qglake-handoff-local.sh
cargo run -p lakecat-cli -- qglake-verify-handoff \
  --summary target/qglake-handoff/handoff-summary.json \
  --json
cat target/qglake-handoff/handoff-summary.json
```

That fixture creates the sample table shape, installs a restricted policy, verifies governed scan planning, verifies fetch-scan-task reapplication, checks requested/effective stats-field narrowing in replay and handoff proof, exercises delete manifest handling, probes credential-vend behavior for agents and trusted humans, verifies compact table commit-history evidence, exports QueryGraph bootstrap artifacts, drains the outbox, and proves the resulting bundle through QueryGraph's Rust verifier/importer. It then asks LakeCat to verify its own compact handoff summary and recompute the raw artifact file hashes, making the summary a first-class acceptance artifact rather than an unchecked convenience file. It is small, but it is not decorative. It is the acceptance story for a catalog that participates in the user workflow from notebook to agent. The summary file gives automation a single stable place to find the accepted table/view counts, semantic hashes, bundle, lineage drain, import plan, captured verifier outputs, and raw artifact hashes without scraping terminal text.

1.28 Operating The Book's Example System

The local development posture is intentionally small:

```
cargo run -p lakecat-cli -- config
cargo run -p lakecat-cli -- storage-profile-list
cargo run -p lakecat-cli -- policy-list
cargo run -p lakecat-cli --features qglake-fixture -- qglake-fixture \
  --output target/qglake/lakecat-bootstrap.json \
  --drain-output target/qglake/lineage-drain.json \
  --principal did:example:agent
cargo run -p lakecat-cli -- qglake-verify-replay \
  --bundle target/qglake/lakecat-bootstrap.json \
  --drain target/qglake/lineage-drain.json \
  --principal did:example:agent
scripts/qglake-handoff-local.sh
cargo run -p lakecat-cli -- qglake-verify-handoff \
  --summary target/qglake-handoff/handoff-summary.json \
  --json
cargo run -p lakecat-cli -- bootstrap-export \
  --output lakecat-bootstrap.json
```

The important thing is what these commands exercise. They are not a separate product surface. They touch the same catalog, policy, bootstrap, and QueryGraph export contracts that the service uses.

1.29 Catalog Concepts In Practice

The previous chapters name the layers. This chapter follows those names through the workflows a user will actually run. The goal is to make the standard Iceberg story and the LakeCat/QueryGraph/TypeSec story visible at the same time.

1.29.1 A PySpark User Sees Iceberg

For a PySpark user, LakeCat should feel uneventful. The user configures an Iceberg REST catalog, creates a namespace, writes a table, and reads it back. The visible concepts are standard Iceberg concepts: catalog, namespace, table identifier, current metadata location, table metadata, snapshot, manifest list, manifest, data file, delete file, and optimistic commit.

```
spark = (
    SparkSession.builder
        .config("spark.sql.catalog.lakecat", "org.apache.iceberg.spark.SparkCatalog")
        .config("spark.sql.catalog.lakecat.type", "rest")
        .config("spark.sql.catalog.lakecat.uri", "http://127.0.0.1:8181/catalog/v1")
        .config("spark.sql.catalog.lakecat.warehouse", "local")
        .getOrCreate()
)

spark.sql("CREATE NAMESPACE IF NOT EXISTS lakecat.demo")
spark.sql("""
CREATE TABLE IF NOT EXISTS lakecat.demo.events (
    event_id STRING,
    event_ts TIMESTAMP,
    customer_id STRING,
    amount DOUBLE
) USING iceberg
""")
spark.sql("""
INSERT INTO lakecat.demo.events VALUES
    ('evt-1', timestamp '2026-06-22 10:00:00', 'cust-1', 42.0)
""")
spark.table("lakecat.demo.events").select("event_id", "amount").show()
```

Nothing in that workflow should require the user to know Turso, QGLake, OpenLineage, TypeSec, Grust, Croissant, CDIF, OSI, or ODRL. LakeCat's job is to make the ordinary path stronger without changing the ordinary path. Behind the REST response, LakeCat can resolve identity, choose the warehouse, validate the namespace, perform metadata-pointer CAS, store an idempotency record, write a pointer log, emit an audit row, enqueue an outbox event, and preserve replay evidence. The PySpark user still sees an Iceberg catalog.

This distinction is important for compatibility. If a PySpark user must call a QueryGraph bootstrap endpoint before reading `lakecat.demo.events`, LakeCat has failed the standard-client contract. If LakeCat records a proof-bearing outbox event after the same PySpark commit succeeds, LakeCat has improved the catalog without changing the table format.

1.29.2 A Commit Is Standard, The Proof Envelope Is LakeCat

An Iceberg commit is an optimistic metadata-pointer update. The writer prepares new table metadata and asks the catalog to advance the current metadata location only if the expected requirements still hold. That is standard Iceberg parlance.

LakeCat adds the proof envelope around that standard commit. A successful commit has several layers:

Layer	Meaning	Standard or LakeCat?
Expected metadata location	The pointer the writer believes is current.	Standard Iceberg commit requirement.
New metadata location	The new table metadata file the writer wants to publish.	Standard Iceberg table metadata.
Compare-and-swap	Advance only if the current state still matches the requirement.	Standard Iceberg catalog behavior.
Idempotency key hash	Make a client retry exact, or reject it as drift.	LakeCat catalog hardening.
Pointer log	Record the accepted pointer transition.	LakeCat proof and inspection surface.
Audit row	Bind the commit to principal, action, receipt, and redacted context.	LakeCat and TypeSec evidence.
Outbox event	Make graph and lineage delivery replayable from committed state.	LakeCat/QueryGraph integration surface.
Replay validator	Refuse malformed durable evidence before downstream projection.	LakeCat safety gate.

The current metadata pointer is the Iceberg table truth. The pointer log, audit row, and outbox event are not custom table metadata. They are catalog proof beside the table. That is why they can be useful to QueryGraph without making ordinary engines depend on QueryGraph.

A simplified commit proof can be thought of like this:

```
{
  "event-type": "table.commit",
  "warehouse": "local",
  "namespace": ["demo"],
  "table": "events",
  "expected-metadata-location": "s3://lake/demo/events/metadata/00001.json",
  "new-metadata-location": "s3://lake/demo/events/metadata/00002.json",
  "format-version": 2,
  "principal": "did:example:analyst",
  "idempotency-key-hash": "sha256:...",
  "pointer-log-sequence": 18,
  "audit-event-id": "sha256:..."
}
```

```
"outbox-event-id": "sha256:..."
}
```

The exact wire shape can evolve, but the separation should not. Iceberg owns the table metadata and optimistic pointer movement. LakeCat owns retry, redaction, auditability, pointer history, and replayable integration events. If this becomes a future Iceberg-adjacent proposal, it should be proposed as a small optional reliability profile: exact retry semantics, redacted conflict evidence, pointer-history inspection, and catalog event identity.

1.29.3 Turso Is A Store Choice, Not A Table Feature

The Turso-backed local store gives LakeCat a durable local catalog spine. It is the place where LakeCat can prove that catalog invariants survive process boundaries and retries. It persists namespaces, tables, views, projects, warehouses, storage profiles, policy bindings, idempotency rows, pointer logs, audit rows, outbox rows, and soft-delete state behind the CatalogStore trait.

That is not Iceberg parlance. Iceberg needs durable catalog state and atomic metadata-pointer movement. It does not require Turso, PostgreSQL, DynamoDB, or any other backing store. Turso is LakeCat's Rust-native local implementation choice.

The reusable lesson is the invariant shape:

Store invariant	Why it matters
Row identity must match decoded content.	A corrupted durable row should not be accepted because its JSON happens to parse.
Idempotency replay must match route and table scope.	A retry for one table must not replay a response from another table.
Pointer logs must describe actual pointer movement.	Operators and QueryGraph need history that agrees with the current table.
Audit rows must bind action and principal.	Governance proof needs authority, not just storage mutation.
Outbox events must be replay-valid.	Graph and OpenLineage projection must come from committed catalog truth.
Soft-delete and restore state must remain scoped.	Restore must not drop or revive the wrong table's evidence.

Those invariants are LakeCat catalog guarantees. The future standards material is the behavior, not the database. A catalog reliability profile could describe CAS, exact retry, pointer history, and event identity without mentioning Turso.

1.29.4 A Governed Agent Sees Receipts And Bounded Work

An agentic workflow is different from a PySpark workflow. The agent may not be trusted to receive broad object-store credentials, and the user may need proof that the agent saw only the allowed slice of the table. LakeCat keeps the table standard and changes the access envelope.

A governed request has four responsibilities:

1. LakeCat resolves the catalog context: principal, warehouse, namespace, table, current metadata pointer, requested projection, requested predicate, purpose, and credential posture.
2. TypeSec decides the authorization meaning: capability, policy match, allowed columns, mandatory row predicate, credential TTL cap, raw-credential exception status, and receipt evidence.
3. Sail interprets the table: schema field ids, projection, residual predicates, snapshot, manifest pruning, delete handling, and scan tasks.
4. LakeCat persists the proof: receipt hash, effective restriction hash, plan hash, task count, credential posture, audit row, and outbox event.

A simplified governed scan request might start as:

```
{
  "principal": "did:example:agent:revops",
  "purpose": "customer-retention-analysis",
  "warehouse": "local",
  "namespace": ["demo"],
  "table": "events",
  "requested-columns": ["event_id", "event_ts", "customer_id", "amount"],
  "requested-filter": "event_ts >= timestamp '2026-06-01 00:00:00'"
}
```

After TypeSec and Sail, the proof should be narrower and more precise:

```
{
  "principal": "did:example:agent:revops",
  "capability": "table.scan",
  "allowed-columns": ["event_id", "event_ts", "amount"],
  "mandatory-filter": "tenant_id = 'acme'",
  "credential-ttl-seconds": 900,
  "raw-credential-exception": false,
  "sail-plan": {
    "snapshot-id": 437,
    "format-version": 2,
    "effective-field-ids": [1, 2, 4],
    "task-count": 12,
    "delete-posture": "position-and-equality-deletes-considered",
    "plan-hash": "sha256:..."
  },
  "authorization-receipt-hash": "sha256:..."
}
```

The important phrase is “Sail-planned.” A TypeSec receipt saying that a column is allowed is only useful if that column is resolved against Iceberg field ids and schema evolution. A predicate is only useful if the engine knows how it interacts with partition transforms, manifest metrics, delete files, and residual filtering. LakeCat should not prove a catalog-local approximation of those facts. It should persist proof that Sail planned the actual table work. The same rule now carries through fetch-scan-tasks replay: the returned residual extension and durable table.scan-tasks-fetched

audit/outbox payload preserve requested stats fields, effective stats fields, and the compact stats-fields alias. That is LakeCat/QueryGraph proof, not standard Iceberg parlance, but it is built from the engine-owned restriction over the real Iceberg table so QGLake can prove a fetched task did not widen metrics evidence after the original Sail-planned work.

This is a LakeCat/TypeSec governance extension today. It could become an Iceberg-adjacent optional profile if the profile stays small: proof-carrying scan planning, governed credential vending, and redacted receipt binding. It should not require every Iceberg catalog to adopt TypeSec or every Iceberg client to understand QueryGraph.

1.29.5 Credentials Are Exceptions, Not The Default Agent Path

Credential vending is useful, but it is dangerous as a default answer for agents. A raw object-store credential can be broader than the intended table operation. It can outlive the request. It can be used outside the catalog's view of policy and purpose. LakeCat therefore treats raw credential vending as an audited exception for trusted principals, not as the normal governed-agent path.

The preferred constrained path is:

```
agent intent
-> LakeCat identity and table context
-> TypeSec capability decision
-> effective read restriction
-> Sail-planned scan or fetch task
-> bounded result or bounded task proof
-> audit and outbox evidence
```

The exception path is:

```
trusted principal
-> LakeCat identity and storage-profile context
-> TypeSec credential decision
-> TTL and storage-scope cap
-> redacted credential-vend proof
-> audit and outbox evidence
```

Both paths can be compatible with Iceberg. The first path is more appropriate for agents because LakeCat and Sail keep the work narrow. The second path is appropriate only when the principal is trusted enough to hold storage authority directly. A future standards conversation should be about optional proof for credential vending and scan planning, not about mandating LakeCat's local credential providers or TypeSec's exact receipt schema.

1.29.6 QueryGraph Reads The Proof, Not The Storage Layout

QueryGraph should not infer semantic truth by scraping object-storage paths. It should import replay-validated catalog proof. LakeCat is the foundation because it is the control plane that saw the namespace, table, pointer, principal, policy, view, credential posture, and commit history when each action happened.

The QGLake handoff is broad because QueryGraph is broad. It can include:

Handoff surface	What LakeCat contributes	What QueryGraph contributes
Bootstrap	Warehouse, namespace, table, view, storage, and policy anchors.	Product-level import and semantic workspace initialization.
OpenLineage	Replay-validated lineage events and receipt hashes.	Correlation with broader lineage and graph reasoning.
Graph anchors	Stable catalog facts and outbox identities.	Grust-backed graph schema, projection, traversal, and Cypher behavior.
Management inventory	Projects, warehouses, policies, storage profiles, and views.	Administrative workflows and semantic governance views.
Credential proof	Redacted storage scope, TTL, policy hash, and exception posture.	Agent risk analysis and workflow authorization.
Commit proof	Pointer transitions, idempotency, audit, and outbox evidence.	Trustworthy graph facts about table evolution.
View proof	View receipt chains and version anchors.	User-facing semantic model and dependency reasoning.

This is not standard Iceberg table access. It is an application integration surface. QueryGraph, Croissant, CDIF, OSI, ODRL, OpenLineage, TypeDID, and agent workflow semantics live above the portable table contract. LakeCat should export proof-bearing anchors. Grust should own graph mechanics. TypeSec should own governance semantics. QueryGraph should compose the user workflow.

The reason to keep this additive is practical. A standard engine can keep using the normal catalog route. QueryGraph can ask for a richer proof bundle. The same table serves both because the Iceberg metadata stays pristine.

1.29.7 Why Sail Makes The Proof Credible

The most important architectural argument is that LakeCat should push data-shaped work into Sail. This is not merely a code-organization preference. It is what makes the proof credible.

Iceberg correctness depends on details that belong to an engine:

- Field ids, not just column names.
- Schema evolution and projection after renames or reorders.
- Partition transforms and partition evolution.
- Snapshot selection and manifest-list traversal.

- Manifest metrics, lower and upper bounds, null counts, and pruning.
- Equality deletes, position deletes, sequence numbers, and delete planning.
- Metadata tables for snapshots, files, manifests, history, and partitions.
- Row lineage and future v4 metadata-tree behavior.

If LakeCat reimplements those details locally, it becomes a second partial engine. That creates two problems. First, it is slower because catalog requests start parsing and planning work that should be shared with execution. Second, it is less trustworthy because LakeCat’s proof may diverge from Sail’s actual execution semantics.

Sail is a strong engine choice because it is Rust-native and close to the lakehouse execution stack. It can share generated Iceberg model handling, Arrow/DataFusion types, table-provider integration, metadata interpretation, manifest expansion, pruning, delete handling, scan-task generation, metadata-as-data, commit validation, and typed v3/v4 behavior. LakeCat can call Sail and persist compact evidence:

Sail output	Why LakeCat records it
Snapshot id and format version	Proves which table state was interpreted.
Effective field ids	Proves policy narrowing survived schema evolution.
Predicate or restriction hash	Proves the mandatory filter was applied.
Manifest or task count	Proves planning produced bounded work.
Delete posture	Proves delete files were considered rather than ignored.
Plan hash	Gives QueryGraph and replay validators a stable anchor.
Validation result	Lets LakeCat bind commit acceptance to engine semantics.

This is faster and safer than catalog-local JSON shortcuts. LakeCat remains thin where it should be thin: identity, tenancy, pointer state, CAS, idempotency, audit, outbox, policy gates, and integration events. Sail becomes the reusable Rust home for table-format semantics. QueryGraph gets proof that connects policy to real table planning, not just to catalog intent.

That is also the right Iceberg v4 posture. LakeCat can preserve newer metadata as JSON when typed helpers are not yet available, but JSON passthrough should be labeled as a bridge. Typed v4 interpretation belongs in Sail because v4 is table-format semantics. Once Sail understands the typed shape, LakeCat can keep the same catalog role: store the pointer, authorize the action, call Sail, persist receipts, and hand the proof to QueryGraph.

1.29.8 Extension Or Future Iceberg Proposal?

The final classification is intentionally conservative:

Concept	Classification
Rust service/catalog spine	LakeCat implementation. Not an Iceberg extension.
Turso-backed store	LakeCat local store implementation. Not an Iceberg extension.
REST namespace/table paths	Standard Iceberg catalog compatibility.
Metadata-pointer CAS	Standard Iceberg optimistic commit behavior.
Idempotent commit replay	LakeCat hardening; possible optional reliability profile.
Pointer logs and commit-history proof	LakeCat management proof; possible optional pointer-history profile.
Audit and transactional outbox	LakeCat integration proof; possible optional catalog event profile.
Replay validation before projection	LakeCat safety gate; possible optional event-admission profile.
Governed scan receipts	LakeCat/TypeSec governance extension; possible optional proof-carrying scan profile.
Credential proof	LakeCat/TypeSec governance extension; possible optional governed credential-vending profile.
OpenLineage binding	LakeCat/QueryGraph lineage integration; possible optional lineage receipt profile.
QueryGraph/QGLake bootstrap	QueryGraph application integration. Not an Iceberg table feature.
Croissant, CDIF, OSI, ODRL, TypeDID	QueryGraph/TypeSec semantic and governance vocabularies. Not Iceberg metadata.
Typed Iceberg v4 interpretation	Iceberg/Sail engine work. LakeCat should consume typed Sail support.

The standardization filter is simple. If an idea requires adopting LakeCat, QueryGraph, TypeSec, Grust, Turso, or a particular application workflow, it is not an Iceberg proposal. If an idea describes a narrow optional catalog behavior that many engines and catalogs could share, it may be worth proposing as an Iceberg-adjacent profile. LakeCat should prove those ideas in practice first, then extract only the small interoperable behavior.

1.29.9 How To Name The New Pieces

The most useful naming discipline is to keep three sentences separate.

The first sentence is standard Iceberg: “this catalog resolves an Iceberg table identifier to a current metadata location, and it advances that location with optimistic commit rules.” That sentence covers the ordinary REST catalog surface, namespace and table routes, metadata files, snapshots, manifests, delete files, and table-format compatibility. LakeCat must be conservative here. A standard engine should be able to ignore every LakeCat-specific proof surface and still read and write a valid Iceberg table.

The second sentence is LakeCat: “this Rust service makes the catalog action durable, scoped, idempotent, governed, audited, and replayable.” That sentence covers the service spine, CatalogStore, Turso local store, metadata-object write posture, CAS enforcement, idempotency records, pointer logs, audit rows, transactional outbox rows, redaction, replay validation, management inventory, and QGLake handoff evidence. Those are not new table-format semantics. They are the control-plane envelope around the table-format semantics.

The third sentence is QueryGraph and TypeSec: “this governed catalog proof can be imported into a semantic graph and agent workflow without forcing normal Iceberg clients to become semantic clients.” That sentence covers TypeSec capabilities, TypeDID context, ODRL-derived restrictions, secure-agent posture, governed credential decisions, OpenLineage binding, Croissant and CDIF projection, OSI-facing workflows, QGLake acceptance, and Grust-backed graph state. LakeCat should produce proof-bearing anchors for those systems, but it should not smuggle their application model into required Iceberg metadata.

That separation answers the extension question cleanly. A feature is a LakeCat extension when it is useful beside standard Iceberg access but not required for that access. It is a QueryGraph or TypeSec extension when it interprets business semantics, policy, identity, agent risk, graph projection, or workflow meaning. It is a future Iceberg-adjacent proposal candidate only when the shape can be made small, optional, and portable across catalogs and engines that do not run LakeCat.

The following table is the proposal filter in a more operational form:

Question	If yes	If no
Does a normal engine need this to load or commit an Iceberg table?	Treat it as standard Iceberg compatibility or table-format work.	Keep it additive.
Does it depend on LakeCat's Rust service, Turso rows, or local CLI artifacts?	Treat it as LakeCat implementation.	Look for a portable behavior.
Does it describe retry, pointer history, event identity, redacted conflict evidence, or replay admission?	Consider a narrow optional catalog reliability profile after proving it locally.	Keep it local until the behavior is clearer.
Does it require TypeSec receipt shapes, TypeDID envelopes, ODRL interpretation, or secure-agent semantics?	Treat it as a governance extension, with possible future proof vocabulary only.	It may be a catalog profile candidate.
Does it require QueryGraph, QGLake, Croissant, CDIF, OSI, OpenLineage correlation, or Grust graph state?	Treat it as an application integration surface.	Extract only the catalog-neutral proof if one exists.

For example, “commit CAS” is already part of the standard Iceberg catalog story. “Commit idempotency keyed by request hash, route, table scope, and stored response” is LakeCat hardening and might later become an optional catalog reliability profile. “Pointer log exported into QueryGraph with OpenLineage and graph hashes” is an application integration surface. “The pointer log row was stored in Turso” is a LakeCat implementation detail. Those four sentences may describe one user-visible workflow, but they should not be presented as one standards claim.

The same distinction applies to governed reads. “The table has field ids, schemas, snapshots, manifests, data files, delete files, and residual scan planning” is Iceberg engine work. “A principal may read only these columns and only rows matching this purpose-bound restriction” is TypeSec governance work. “Sail planned the effective table work using field ids, delete posture, manifest metrics, and task generation” is engine proof. “LakeCat stored the receipt hash, plan hash, task count, credential posture, audit row, and outbox event” is catalog proof. “QueryGraph imported that proof into a graph and agent workflow” is application integration. The pieces fit together because each one keeps its own name.

1.29.10 The Engine-Push Argument

The argument for pushing work into Sail should be explicit enough that a reader can test future changes against it. LakeCat should push work into Sail whenever the answer depends on Iceberg table-format interpretation rather than catalog authority.

Catalog authority questions sound like this:

- Which principal is making the request?
- Which project, warehouse, namespace, table, or view is in scope?
- Which metadata pointer is current?
- Which optimistic commit requirement must hold?
- Which idempotency key is being replayed?
- Which TypeSec receipt authorized the action?
- Which audit and outbox rows prove what happened?

Engine interpretation questions sound like this:

- Which field ids correspond to a projected column after schema evolution?
- Which manifests can be skipped from partition transforms or metrics?
- Which delete files must be associated with a scan?
- Which residual filter remains after pruning?
- Which metadata tables expose snapshots, files, manifests, partitions, or history?
- Which v3 or v4 table-format feature changes planning or validation?
- Which commit requirement is valid against the table metadata shape?

LakeCat should own the first list and call Sail for the second list. If the second list is implemented in LakeCat, every governed proof risks becoming a catalog-local approximation. The catalog might prove that policy was applied to columns named `customer_id` and `amount`, while the engine knows that the current table has renamed fields, reordered fields, changed partition specs, attached equality deletes, or evolved metadata in a way the catalog shortcut did not understand. That is not a theoretical problem. It is exactly the class of drift that proof-carrying systems must avoid.

Sail is a strong engine choice because it can keep the proof close to the code that will execute or expose the table-shaped work. It is Rust-native like LakeCat, so the catalog does not have to cross a broad language boundary for hot proof paths. It sits near Arrow and DataFusion, so projections, predicates, schemas, and execution-facing metadata can use the same ecosystem types. It can own generated Iceberg REST and table-format models, so LakeCat does not have to grow local JSON interpretation for every format feature. It can expose metadata-as-data and table-provider surfaces that QueryGraph can consume through LakeCat proof and Grust graph projection. It can also become the natural home for typed Iceberg v4 work, which is table semantics rather than catalog bookkeeping.

This division is not only cleaner; it is faster. A thin LakeCat request can resolve identity, load current pointer state, ask TypeSec for authorization, call Sail for a typed plan or validation, and persist a compact proof envelope. The expensive and format-specific work is reusable engine work. The catalog does not duplicate manifest parsing, metrics decoding, delete planning, or v4 interpretation. QueryGraph gets stronger evidence because the plan hash and receipt hash attach to the engine-shaped work that actually matters.

The practical design rule for future development is therefore:

If the change needs...	Put it in...	LakeCat should keep...
Iceberg table-format interpretation, manifest logic, delete handling, metadata-as-data, scan planning, or typed v4 support	Sail	Request context, authorization receipt, pointer context, plan/validation hash, audit, and outbox evidence.
Graph schema, taxonomy, projection, traversal, stores, Cypher, or graph algorithms	Grust	Stable catalog-domain event and graph-sink boundaries.
Capability semantics, TypeDID context, policy composition, ODRL interpretation, credential risk, or secure-agent proof	TypeSec	Decision request, receipt hash, redacted context, action binding, and replay evidence.
Croissant, CDIF, OSI, OpenLineage correlation, QGLake acceptance, and user workflow composition	QueryGraph	Bootstrap, replay, and hand-off artifacts with stable hashes.
Namespace/table REST compatibility, CAS, idempotency, pointer logs, audit, outbox, and local durable store contracts	LakeCat	The catalog implementation itself.

That is the core architecture of LakeCat. It is ordinary where Iceberg needs ordinary behavior, strict where catalog authority must be strict, and ambitious where QueryGraph needs proof-rich semantic integration.

1.30 First Release Readiness

The first LakeCat release should not try to finish every idea in this book. It should release the catalog substrate that can already be proven locally. The right question is not “does LakeCat contain the whole future QueryGraph stack?” It is “does LakeCat provide a compatible, durable, governed catalog foundation that QueryGraph can trust?”

For the first release, the release-blocking behavior is the catalog spine: standard Iceberg REST config, namespace, table-load, table-create, and table-commit paths; warehouse-prefixed routing; Rust service identity handling; the CatalogStore seam; the Turso-backed local store; memory-store parity for embedded tests; metadata-pointer CAS; idempotent commit replay; pointer logs; audit rows; outbox rows; and replay admission that rejects malformed durable evidence before graph or OpenLineage projection.

The governed path is also release-blocking because it is the reason LakeCat exists for agents. A restricted agent should be able to ask for a governed read, receive a TypeSec-style receipt, get Sail-planned work instead of broad storage authority, and leave behind replayable scan, fetch, credential, management, view, and commit-history evidence. Trusted raw credential vending can exist only as an audited exception with redacted storage-scope proof.

The QueryGraph handoff is release-blocking as an acceptance proof, not as a requirement for ordinary Iceberg clients. The local QGLake workflow must keep creating a bootstrap bundle, draining LakeCat lineage/outbox evidence, verifying replay, running QueryGraph verification/import, and saving a handoff summary whose artifact hashes, table/view counts, standards, OpenLineage receipts, graph hashes, policy anchors, credential proof, view receipt chains, and commit history agree. If that handoff cannot be reproduced locally, LakeCat may still serve an Iceberg endpoint, but it has not proven the QueryGraph foundation.

The first release should explicitly defer work that belongs elsewhere or is not yet ready to claim. Typed Iceberg v4 semantics belong in Sail; LakeCat should advertise only the current JSON passthrough bridge with `typed-sail=unavailable` until Sail exposes stable typed support. Reusable graph taxonomy, traversal, Cypher, graph stores, and algorithms belong in Grust. Capability composition, TypeDID envelopes, secure-agent proof, and richer policy semantics belong in TypeSec. Croissant, CDIF, OSI, ODRL application composition, and agent-facing reasoning belong in QueryGraph. Cloud SDK secret managers beyond the current Vault and file-backed provider roots are future credential backends, not blockers for the catalog substrate.

The release evidence is concrete:

```
scripts/check-release-readiness.sh
scripts/qglake-handoff-local.sh
docs/book/build.sh
scripts/check-local-dependency-contract.sh
```

The quick check is acceptable while landing a narrow slice:

```
scripts/check-release-readiness.sh --quick
```

The full gate is the first-release claim. It runs local dependency-contract checks, workflow-trigger checks, formatting, default workspace tests, feature matrix tests, Turso rows, Sail/TypeSec/Grust integration rows, all-features workspace tests, the book build, and the QGLake handoff proof. Cloud CI remains manual-only until that local proof is boring. A release should be cut from the state those commands prove, not from a hope that a remote runner will discover the truth later.

1.31 What Comes Next

LakeCat is a direction more than a single release. The next slices should keep making the architecture more true:

1. Remove temporary Sail patch bridges when the required helpers are available upstream.
2. Keep pushing reusable Iceberg table-status, planning, metadata-as-data, and commit helpers into Sail.
3. Make the transactional outbox the only path for graph and lineage side effects.

4. Expand TypeSec-backed capability checks until every privileged path is unbypassable.
5. Keep graph taxonomy and Cypher behavior in Grust.
6. Keep OSI as a QueryGraph-owned semantic handoff rather than a LakeCat-authored business model.
7. Prove the bootstrap bundle through QueryGraph import on every meaningful public-surface change.

The payoff is a catalog foundation that feels ordinary to Iceberg clients and rich to QueryGraph. Tables remain portable. Policies become enforceable. Graph and lineage become replayable. Sail plans close to the data. QueryGraph gets a trustworthy substrate for the next version of its lakehouse intelligence.